Springer Undergraduate Texts
in Mathematics and Technology

SUMAT



George A. Anastassiou
Razvan A. Mezei

Numerical Analysis Using Sage

EXTRA MATERIALS
springerlink.com

Springer

Springer Undergraduate Texts in Mathematics and Technology

Series Editors:
J. M. Borwein, Callaghan, NSW, Australia
H. Holden, Trondheim, Norway
V. Moll, New Orleans, LA, USA

Editorial Board:
L. Goldberg, Berkeley, CA, USA
A. Iske, Hamburg, Germany
P.E.T. Jorgensen, Iowa City, IA, USA
S. M. Robinson, Madison, WI, USA

More information about this series at http://www.springer.com/series/7438

George A. Anastassiou • Razvan A. Mezei

Numerical Analysis Using Sage

Springer

George A. Anastassiou
Department of Mathematical Sciences
The University of Memphis
Memphis, TN, USA

Razvan A. Mezei
Mathematics and Computing Sciences
Lenoir-Rhyne University
Hickory, NC, USA

ISSN 1867-5506 ISSN 1867-5514 (electronic)
Springer Undergraduate Texts in Mathematics and Technology
ISBN 978-3-319-16738-1 ISBN 978-3-319-16739-8 (eBook)
DOI 10.1007/978-3-319-16739-8

Library of Congress Control Number: 2015934903

Mathematics Subject Classification (2010): 47N40, 97N40, 97N80, 97U20

Springer Cham Heidelberg New York Dordrecht London
© Springer International Publishing Switzerland 2015
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made.

Printed on acid-free paper

Springer International Publishing AG Switzerland is part of Springer Science+Business Media (www.springer.com)

I dedicate this book to my wife Koula, and my children Angela and Peggy

George A. Anastassiou

I would like to dedicate this work to my High-School Mathematics teacher, Aurel Netea, whose passion for teaching inspired me. There are no words that can completely describe my gratitude for his great support and his encouragements that helped me choose this path in my career. He was the first "giant" that I met in my search of new knowledge. I would also like to thank my wife Fernanda, and my mother Ileana, for their great love and support. I could not have done this work without their constant support.

Razvan A. Mezei.

Contents

1 Fundamentals 1
- 1.1 Why Sage 1
- 1.2 A Taste of Sage 2
- 1.3 Absolute Error vs. Relative Error 20
- 1.4 Accuracy and Precision 21
- 1.5 Rounding and Chopping 23
- 1.6 Introduction to Sage Programming Through Examples 24
  - 1.6.1 Defining a Function: Evaluating It for a Given Input 24
  - 1.6.2 Print and Range 25
  - 1.6.3 For Loop: A First Example 26
  - 1.6.4 For Loop: Another Example 27
  - 1.6.5 Conditional Statement 28
  - 1.6.6 More on Conditional Statements 29
  - 1.6.7 Working with Strings 30
  - 1.6.8 Computing a Running Total 31
  - 1.6.9 Counting 31
  - 1.6.10 While Loops 32
  - 1.6.11 Another Example on Writing Functions 33
  - 1.6.12 Nested Functions: Randomly Generated Numbers 34
  - 1.6.13 Working with Arrays 35
- 1.7 Taylor Series 37
- 1.8 Alternating Series 56
- 1.9 Sage Interacts 58
- 1.10 Exercises 60
- 1.11 Computer Exercises 61

References 63

2 Solving Nonlinear Equations 65
- 2.1 Bisection Method 65
  - 2.1.1 The Algorithm 66
  - 2.1.2 Pseudocode 68
  - 2.1.3 Sage Code 68
  - 2.1.4 Error Bounds 70
  - 2.1.5 Comments on the Bisection Method 71
  - 2.1.6 Examples 71
  - 2.1.7 Sage Library 75

2.2 Newton–Raphson's Method ... 76
2.2.1 The Algorithm ... 78
2.2.2 Pseudocode ... 79
2.2.3 Sage Code ... 80
2.2.4 Error Bounds ... 81
2.2.5 Comments on the Newton–Raphson's Method ... 81
2.2.6 Examples ... 85
2.2.7 Sage Library ... 88
2.2.8 Sage Library 2 ... 89
2.3 Secant Method ... 90
2.3.1 The Algorithm ... 93
2.3.2 Pseudocode ... 95
2.3.3 Sage Code ... 96
2.3.4 Error Bounds ... 97
2.3.5 Comments on the Secant Method ... 98
2.3.6 Examples ... 98
2.3.7 Sage Library ... 100
2.4 Fixed-Point Iteration ... 101
2.4.1 The Algorithm ... 102
2.4.2 Pseudocode ... 108
2.4.3 Sage Code ... 109
2.4.4 Examples ... 109
2.4.5 Sage Library ... 111
2.5 An Application to Business ... 112
2.6 Exercises ... 113
2.7 Computer Exercises ... 114
References ... 115

3 Polynomial Interpolation ... 117
3.1 Interpolation ... 118
3.1.1 Linear Interpolation ... 118
3.1.2 Quadratic Interpolation ... 120
3.1.3 Lagrange Formula ... 123
3.1.4 Sage Code for Lagrange Formula ... 126
3.1.5 Newton Formula ... 128
3.1.6 Sage Code for Newton Formula ... 132
3.1.7 Divided Differences ... 134
3.1.8 Sage Code for Divided Differences ... 137
3.1.9 Inverse Interpolation ... 138
3.1.10 Sage Library ... 139
3.2 Error Bounds in Polynomial Interpolation ... 143
3.3 An Application to Physics ... 153
3.4 Exercises ... 154
3.5 Computer Exercises ... 156
References ... 159

4 Numerical Differentiation ... 161
4.1 Derivative Formulae Using Taylor Series ... 161
4.1.1 First Order Derivatives ... 161
4.1.2 Second Order Derivatives ... 166
4.1.3 Method of Undetermined Coefficients ... 167

4.2 Derivative Formulae Using Interpolating Polynomials ... 169
4.3 Exercises ... 171
4.4 Computer Exercises ... 172
References ... 173

5 Numerical Integration ... 175
5.1 Trapezoidal Rule ... 178
5.1.1 Basic Trapezoidal Rule ... 178
5.1.2 Composite Trapezoidal Rule ... 180
5.1.3 Sage Code ... 181
5.1.4 Error Bounds ... 184
5.1.5 Sage Library ... 187
5.2 Simpson's Rule ... 188
5.2.1 Basic Simpson's Rule ... 188
5.2.2 Composite Simpson's Rule ... 190
5.2.3 Sage Code ... 192
5.2.4 Error Bounds ... 195
5.2.5 Sage Library ... 198
5.3 Using the Method of Undetermined Coefficients ... 200
5.3.1 Basic Trapezoidal Rule Using the Method of Undetermined Coefficients ... 200
5.3.2 Basic Simpson's Rule Using the Method of Undetermined Coefficients ... 200
5.4 Richardson Extrapolation ... 201
5.5 Romberg Method ... 203
5.5.1 Motivation ... 203
5.5.2 Algorithm ... 205
5.5.3 Sage Code ... 205
5.5.4 Error Bounds ... 208
5.6 Newton–Cotes Formulae ... 208
5.6.1 Closed Newton–Cotes Formulae ... 209
5.6.2 Open Newton–Cotes Formulae ... 209
5.7 Gauss Quadrature Formula ... 210
5.7.1 A Brief Outline ... 210
5.7.2 Algorithm by Example ... 211
5.7.3 The General Algorithm ... 214
5.7.4 Sage Code ... 216
5.8 An Application to Materials Science and Engineering ... 217
5.9 Exercises ... 218
5.10 Computer Exercises ... 222
References ... 224

6 Spline Interpolation ... 227
6.1 Linear Splines ... 230
6.1.1 Sage Code ... 232
6.1.2 Sage Library ... 233
6.2 Quadratic Splines ... 234
6.2.1 Algorithm ... 235
6.2.2 Sage Code ... 237
6.3 Natural Cubic Splines ... 240
6.3.1 Algorithm ... 243

6.3.2 Sage Code ... 247
6.3.3 Sage Library ... 250
6.4 An Application to Computer Science ... 253
6.5 Exercises ... 257
6.6 Computer Exercises ... 259
References ... 261

7 Numerical Methods for Differential Equations ... 263
7.1 Introduction ... 263
7.2 Euler's Method ... 267
7.2.1 Algorithm ... 268
7.2.2 Sage Code ... 270
7.2.3 Sage Library ... 273
7.2.4 Sage Interacts ... 276
7.2.5 Error Bounds ... 277
7.3 Taylor Series Method ... 279
7.3.1 Algorithm ... 279
7.3.2 Sage Code ... 282
7.4 Runge–Kutta's Method ... 284
7.4.1 Runge–Kutta Method of Order 2 ... 284
7.4.2 Runge–Kutta Method of Order 4 ... 285
7.4.3 Sage Code ... 286
7.5 Systems of Ordinary Differential Equations ... 288
7.5.1 Taylor Series Method ... 288
7.6 Higher-Order Ordinary Differential Equations ... 294
7.6.1 Sage Library ... 297
7.7 Systems of Higher-Order Differential Equations ... 299
7.7.1 Sage Library ... 304
7.8 An Application to Biology ... 307
7.9 Exercises ... 307
7.10 Computer Exercises ... 309
References ... 311

Erratum ... E1

Index ... 313

Preface

To the best of our knowledge, this is the first numerical analysis text that uses Sage for algorithms implementation. We chose to base our presentation of a basic numerical analysis course on Sage for the following reasons. Firstly, Sage is FREE. Secondly, it uses a Python-like syntax. This is particularly important since, in the recent years, Python became a very popular choice of script to use in introductory programming courses. Moreover, it gained the great reputation of being easy to learn. Some even claim that Python is the easiest programming language to learn by first-time programmers. An analysis made by Philip Guo in July 2014 (see http://pgbovine.net/) shows that "Python is now the most popular introductory teaching language at top U.S. universities", with eight out of the top ten Universities in the U.S. using Python in their CS 0 or CS 1 courses.

Numerical analysis has applications in virtually every applied science including computer graphics, engineering, economics, biology, physics, aerodynamics, statistics, and more. Therefore, we recommend this text to all undergraduates in mathematics, mathematics education, computer science & information technology, pre-engineering, and mathematical science students. The material included in this book was used in a one-semester course called "Introduction to Numerical Analysis" (at Lander University, Greenwood, SC) to undergraduate students in mathematics, computer science, and pre-engineering.

We need approximation techniques to deal with the basic fact that every calculator and every computer has finite memory. This implies that only a finite number of digits can be stored and hence no irrational number can be stored on any computer. Still, nowadays computers can perform very accurate computations, and even graph various functions, including $\sin(x)$, $\ln(x)$, $\sqrt[5]{x}$, and even much more elaborate functions. This is possible because of the approximation techniques developed in numerical analysis.

As an example, numerical analysis techniques are extensively used in computer science. Most image and video compression/decompression algorithms use interpolation. Using various sampling techniques, we can compress images ("lossy" compression), and then, using interpolation techniques, we can decompress them. Image restoration, digital sounds, GPS coordination, video games, etc. all use interpolation techniques. Interpolation techniques help us make the transition from discrete into continuous, and in particular, from digital into analog.

In this book, we present both: how to implement from scratch different algorithms using Sage (for this, look for sections called *Sage Code*), and how to use the existing Sage Library functions that are already implemented (look for sections called *Sage Library*).

The content is mostly self-contained. You do not need to have any Python programming background in order to read this book, although familiarity with any programming language is a plus. We present enough examples so that hopefully, the student will become proficient in writing efficient Sage code and develop algorithms on their own. This text, however, is not meant to be an introduction

to programming. For a great introduction to programming textbook using Python, the reader is invited to refer the great reference: Tony Gaddis, *Starting Out with Python*, second edition, Addison-Wesley, 2012.

With one exception (the image included on page 21), all the graphs included in this book were composed in Sage and for most of the graphs, we provide in this book its corresponding Sage code. For your convenience, all the Sage source code will also be provided at http://link.springer.com/book/10.1007/978-3-319-16739-8.

We put every effort in writing Sage code that is easy to read and understand. As such, we occasionally simplified the code for easier reading, at the expense of a few extra operations. Therefore, some of the algorithms may not contain the most optimal computational solutions. We challenge the reader to optimize these Sage programs. To help with the code, in the chapter *Solving Nonlinear Equations* we also include pseudocode in addition to its Sage implementation.

Through the use of Sage "Interacts", which are very briefly presented in this book, the reader can create great demonstrations that can be used while teaching or during presentations.

Memphis, TN, USA — George A. Anastassiou
Hickory, NC, USA — Razvan A. Mezei
December 30, 2014

The original version of the copyright page was revised. An erratum can be found at DOI 10.1007/978-3-319-16739-8_8

Chapter 1
Fundamentals

1.1 Why Sage

Most existing Numerical Analysis textbooks, are centered around some programming language (such as C, C++, etc.) or some Mathematical programs (e.g., Mathematica®, Maple®, Matlab®, etc.) see [3, 6, 7]. What we are proposing here is a Numerical Analysis book that uses Sage Math. Throughout this book we will refer to it shortly as Sage.

A first question that we will address next is: Why use Sage? What are the benefits of using Sage over the other existing options mentioned above and how is it similar to them?

1. Sage can be used as a **comprehensive Mathematics computation engine** (similarly to Mathematica®, or Maple®), by calling the functions that are already available for you. "*Whenever you use Sage you are accessing one of the world's largest collections of open source computational algorithms*" [11].
2. Sage uses a **Python-like syntax** (as an alternative to using Pascal, C, or C++, etc.). There is a reason why, in the recent years, the Python programming language became more and more popular and now many Universities are teaching Python as a first programming language (CS 0) to students that are majoring in Engineering, Computer Science, Computer Information Systems, Information Technology, even Mathematics. Some even teach CS 1 in Python and CS 2 in C++. There are several articles claiming that Python is THE favorite Introduction to Programming Language.
3. Perhaps one of the greatest advantage of using Sage is the fact that it is **FREE**. You simply can't beat that price. Moreover, there are many user manuals available on Sage website [11] and on a growing number of other websites.
4. Sage is **Open Source.** What does this imply? Among others it means that there are a lot of source codes available for everyone to see, analyze, and use for free. This gives you a tremendous potential for new research.
5. **Easy access to Sage.** You can download Sage and use it as a stand alone application, on your own computer, without any need to be connected to the Internet or you can use web-based client that can be accessible from virtually everywhere you have access to the Internet, without installing anything on your computer. We present, below, these options.

The original version of this chapter was revised. An erratum to this chapter can be found at DOI 10.1007/978-3-319-16739-8_8

Electronic supplementary material The online version of this chapter (doi: 10.1007/978-3-319-16739-8_1) contains supplementary material, which is available to authorized users.

© Springer International Publishing Switzerland 2015

G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8_1

(a) The way we prefer to access Sage and we used it for almost all the examples given in this book is through the "**Sage Cell Server**". This is available on [10]. In this web client you can simply insert your Sage code, edit it, and run it without installing anything on your computer. Moreover, you never have to worry about having the latest edition of Sage on your computer, or having a powerful enough computer. You can even use smart phones to access this website. It does however have some shortcomings: you need to have access to the Internet, and it does not save your code automatically. This can be a challenge if your browser becomes unresponsive from time to time, or if you need to refresh the page.

(b) An alternative to "Sage Cell Server" is a **Sage Notebook** (see http://sagenb.org/). Similar to the one above, you don't need to install anything on your computer. You need to create an account once (this is FREE) and then you can save, access old files, and even share your Sage code. This can successfully be used for research, homework assignments, and even lab works. For the latest trend in Sage you may want to check the SageMathCloud (https://cloud.sagemath.com) which is still under active development at the time we are writing this book.

(c) If Internet connection can be an issue, at times, then you can install Sage on your machine. This way you don't need to worry about not having access to the Internet, and you can also save your work. The challenge with this option though is that it is your responsibility to keep up with the latest version of Sage. Also, it will use the computing power of your own machine.

6. Sage Math is a great tool for many other courses not only Numerical Analysis. There are several existing books on how to use it in Calculus, Linear Algebra, Number Theory, Computational Statistics, Discrete Mathematics, Differential Equations and many others courses. There are extensive libraries that can be used for these subject areas, and the number of them grows every day.

We hope that we already convinced you on the great potential Sage has. Next we present you some basic examples and try to exhibit the great potential and the easiness to use Sage. All the examples in this book used the following Sage version: "Sage Version 6.3, Release Date: 2014-08-10". To check the version of the Sage that you are using, one can run the following code:

```
sage: version
```

to obtain

```
'Sage Version 6.3, Release Date: 2014-08-10'
```

1.2 A Taste of Sage

To get you started with some very basic examples in Sage Math, please go on [10] and try each of the examples given below. They should give you a brief idea of what Sage can do. Do not worry if you don't know all the details of the code you are using in these examples. We will give you a more detailed explanation of how to code in Sage as you read through the book. These first examples should help you get a little familiar with how to use Sage see also [12].

In Sage, as well as in Python (see [8]), everything in a line that comes after the character **#** is a **comment**, and therefore it is completely ignored by Sage. *Comments* are text that are being used to make the code easier to read. Although they are completely ignored by the compiler, you are strongly encouraged to use comments when you write code. Comments can help you read the code much faster, without focusing on every detail of the syntax, and they become very handy especially when your code becomes lengthy, or when you use some trick to shorten the code. They also help other developers

who may need to read, use, or simply check your code. You simply cannot overestimate the value of using comments in your code.

IMPORTANT: to avoid frustration, please read the following carefully and know it. In Sage, as well as in Python, leading whitespaces (spaces or tabs at the beginning of a line) are used to compute the indentation level of that line. They are used to determine the grouping of the statements. **Same level of indentation means they are part of the same group of statements**. Other programming languages (such as Java, C/C++, and C#) use the symbols { and } for the same purpose, and some programming languages, like Pascal, use the keywords *begin* and *end*. Therefore don't add leading whitespaces if they are not necessary, and DO add them if they are needed. **Wrong indentation can lead to compiling errors, or worst: wrong outputs and challenges to debug errors.**

ALSO IMPORTANT: Sage, as well as Python, is **case-sensitive**. That means, **print**, **Print**, and **PRINT** are considered different. Please make sure you use the correct letter case as indicated in the examples.

NOTE: Please note that in most programming languages, including Python (and Sage), we use $==$ when **comparing** two values or expressions, and we use $=$ for the **assignment** operator. That means $a == 7$ compares a against the value 7, while $a = 7$ assigns the value 7 to a. It is very common for novice programmers to mix these two. Do your best and try to avoid this error, as it can sometimes be very challenging to debug code containing such logical errors.

The lines below that start with "sage:" are the lines of code you should submit to the Sage interpreter. You do not need to include "sage:" when you type your code into Sage. The lines without "sage:" are the output lines. After you type in your code, you can either click on the "Evaluate" button, or press "Shift+Enter" for Sage to run the code you just inserted.

Sage Math syntax is broader than Python. For example the exponential operator in Python is **. Sage accepts both of the following exponential operators: ** and ^. Therefore, while only $2 * *3$ is a legitimate code in Python for computing 2^3, Sage accepts both of the following: $2 * *3$ as well as 2^3. We will make use of this throughout the book.

```
sage: #computes 2 to the power of 3
sage: 2**3
```

```
8
```

```
sage: #computes 2 to the power of 3
sage: 2^3
```

```
8
```

```
sage: #factors the number 2013
sage: 2013.factor()
```

```
3 * 11 * 61
```

```
sage: #also works with fractions
sage: factor(-2014/2016)
```

```
-1 * 2^-4 * 3^-2 * 7^-1 * 19 * 53
```

```
sage: #computes the square of a matrix
sage: matrix([[1,2], [3,4]])^(2)
```

```
[ 7 10]
[15 22]
```

```
sage: #computes the inverse of a matrix
sage: matrix([[1,2], [3,4]])^(-1)
```

```
[  -2    1]
[ 3/2 -1/2]
```

```
sage: #outputs the value of e with 100 bits of precision
sage: e.n(100)
```

```
2.7182818284590452353602874714
```

```
sage: #outputs the value of pi with 100 bits of precision
sage: pi.n(100)
```

```
3.1415926535897932384626433833
```

```
sage: #outputs the value of pi with 200 bits of precision
sage: pi.n(200)
```

```
3.14159265358979323846264338327950288419716939937510582097494
```

```
sage: #outputs pi with default (53) bits of precision
sage: pi.n()
```

```
2.71828182845905
```

```
sage: #outputs the value with 200 bits of precision
sage: sqrt(2).n(200)
```

```
1.41421356237309504880168872420969807856967187537694807317667
```

```
sage: #IMPORTANT: So how can we print decimal values? Use n()
sage: #If you need to output multiple items on the same line:
sage: #      you can separate them by commas
sage: print e^10, e^10.0,  (e^10).n(52), (e^10).n()
```

The previous example shows several alternatives in how to print values using decimal notation. One way is to use bit precision: simply add `.n(52)`. Instead of 52, you could use any desired bit precision. The other way is to use 10.0, which is a decimal value, rather than the whole number 10. In this case, Sage will use the implicit 52 bit precision for the output. If you use `.n()`, the default 53 bits of precision will be used. The output of the previous example will be:

```
e^10 22026.4657948067 22026.4657948067 22026.4657948067
```

```
sage: R.<x,y> = QQ[]
sage: factor(x^21 + y^21)  #factors a polynomial
```

```
(x + y) * (x^2 - x*y + y^2) * (x^6 - x^5*y + x^4*y^2
- x^3*y^3 + x^2*y^4 - x*y^5 + y^6) * (x^12 + x^11*y
- x^9*y^3 - x^8*y^4 + x^6*y^6 - x^4*y^8 - x^3*y^9 + x*y^11 + y^12)
```

```
sage: var('x,y')
sage: #simplifies an expression
sage: (4*(x-3)^3+5*(x-4)^2+8).full_simplify()
```

```
4*x^3 - 31*x^2 + 68*x - 20
```

```
sage: factorial(70)   #try factorial(1000)
```

```
11978571669969891796072783721689098736458938142546425857555362864628009582789845319680000000000000000
```

```
sage: find_root(cos(x)-x, 0,2) #only finds a root, not all!!!
```

```
0.7390851332151607
```

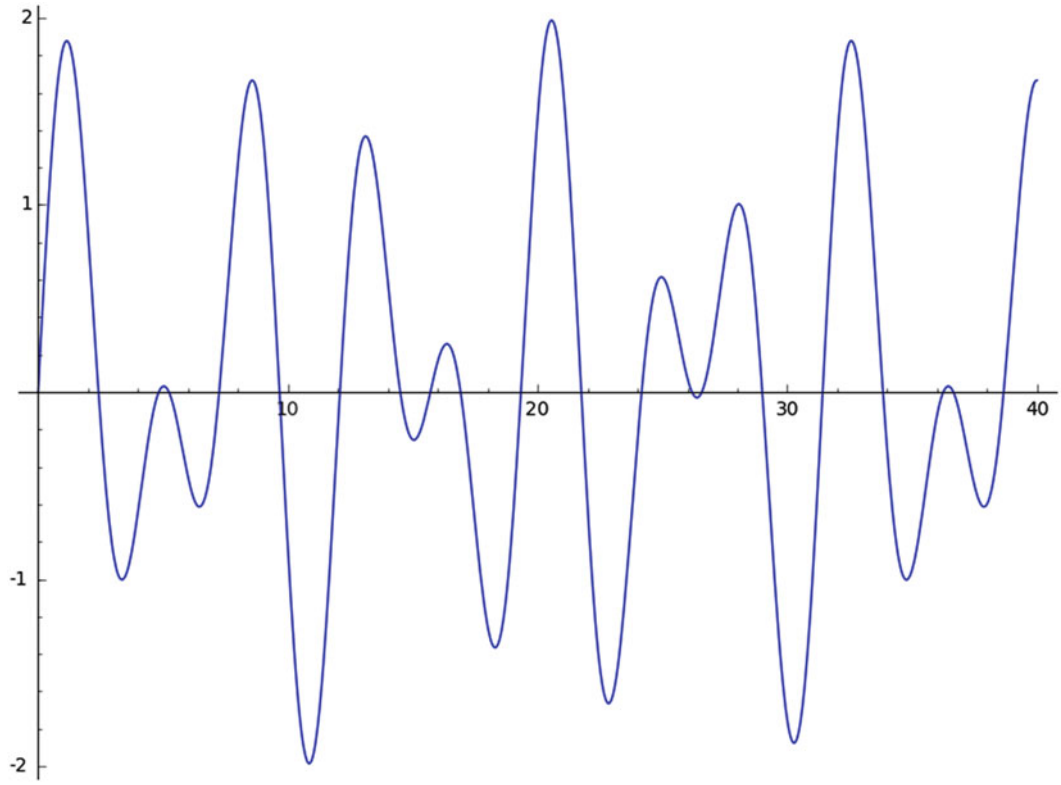
```
sage: is_prime(2015)     #checks if 2015 is a prime number
```

```
False
```

```
sage: #try something more impressive
sage: #PUT THE NEXT NUMBER IN ONLY ONE LINE!
sage: is_prime(2012314234235235252352234234234234234234234235\
4546576785543563676894568467896795789485678780789044811234124\
123132343535635234235547567512343243241)
```

```
False
```

```
sage: #plots a given function, for x in [0, 40]
sage: show(plot(sin(x) + sin(1.6*x), 0, 40))
```




```
sage: #computes the value of sine, at pi
sage: sin(pi)
```

```
0
```

```
sage: #EVALUATES the expression
sage: cos(3*pi/4)
```

```
-1/2*sqrt(2)
```

```
sage: #specifies u as a symbolic variable
sage: u = var('u')
sage: #computes the DERIVATIVE of cos(u) as a function of u
sage: diff(cos(u),u)
```

```
-sin(u)
```

```
sage: #specifies u as a symbolic variable
sage: u = var('u')
sage: #computes the FIFTH derivative of cos(u)*sin(u)
sage: diff(cos(u)*sin(u),u, 5)
```

```
16*cos(u)^2 - 16*sin(u)^2
```

```
sage: var('x')
sage: f(x) = (x^2 - 1+cos(x)) / (x^4 + 1)
sage: #an alternative to 'print' is 'show'
sage: show(derivative(f, x))
```

$$x \mapsto -\frac{4\left(x^2+\cos(x)-1\right)x^3}{\left(x^4+1\right)^2}+\frac{2\,x-\sin(x)}{x^4+1}$$

```
sage: var('x')
sage: f(x) = (x^4+3*x^2+4*x+6) / ((x+2)*(x+3)^2)
sage: #computes partial fractions of f
sage: g(x)=f.partial_fraction(x)
sage: #command show() will display the function g
sage: show(g)
```

$$x \mapsto x+\frac{20}{x+3}+\frac{26}{x+2}-\frac{102}{(x+3)^2}-8$$

```
sage: #declares u as a symbolic variable
sage: u = var('u')
sage: #computes the ANTIDERIVATIVE of cos(u)sin(u)
sage: #      as a function of u
sage: integral(cos(u)*sin(u),u)
```

```
-1/2*cos(u)^2
```

```
sage: #declares u as a symbolic variable
sage: u = var('u')
sage: #computes the DEFINITE INTEGRAL on [1,5]
sage: # of cos(u)sin(u) as a function of u
sage: integral(cos(u)*sin(u),u, 1, 5)
```

```
-1/2*cos(5)^2 + 1/2*cos(1)^2
```

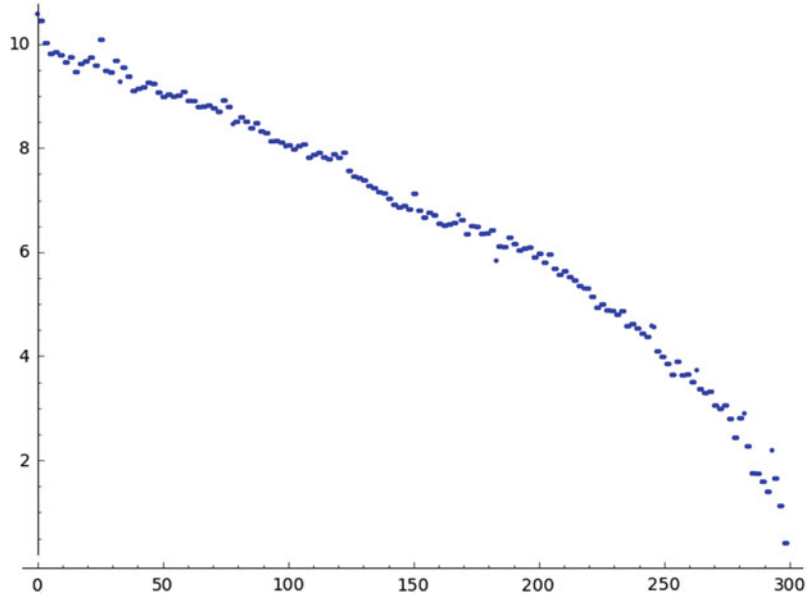
```
sage: #declares u as a symbolic variable
sage: u = var('u')
sage: #computes the DEFINITE INTEGRAL on [1,5]
```

```
sage: # of cos(u)sin(u) as a function of u
sage: # and outputs as a decimal
sage: integral(cos(u)*sin(u),u, 1, 5).n()
```

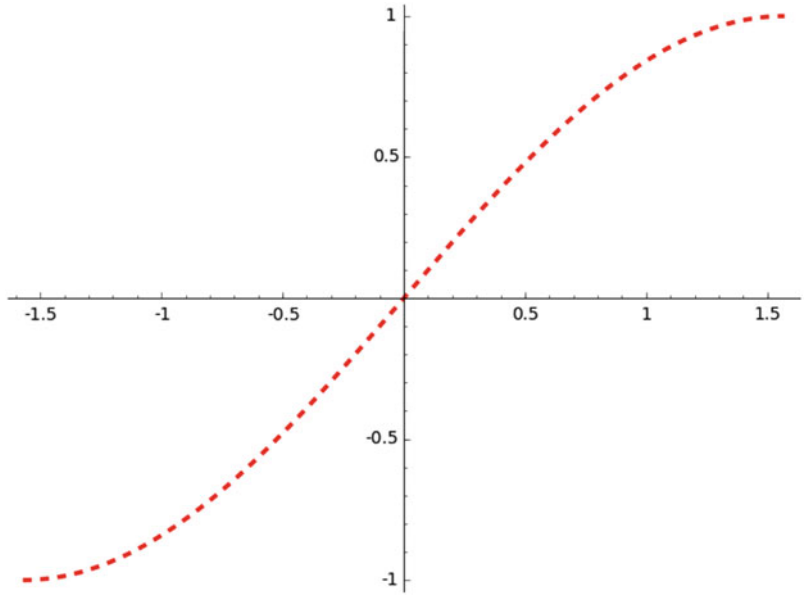
```
0.105731173132328
```

```
sage: #plot will create a graph of the given function, for x in [0,2]
sage: g = plot ( sin ( x ) + (1 - x ^2) , (x , 0 , 2))
sage: #and will create a pdf file of the plot
sage: g.save('plot.pdf')
```

```
sage: #Sage can compute the eigenvalues of the matrix and plot them
sage: myMatrix = random_matrix(RDF,300)
sage: theEigenvalue = myMatrix.eigenvalues()
sage: w=[(i,abs(theEigenvalue[i])) for i in range(len(theEigenvalue))]
sage: show(points(w))
```




```
sage: var('x')
sage: #defines f(x) to be the function sin(x)
sage: f(x) = sin(x)
sage: p=plot(f(x), (x,-pi/2, pi/2), linestyle='--',
sage:              thickness=3,color='red')
sage: #displays the plot we created above
sage: p.show()
```




```
sage: var('x')
sage: solve(x^4>8, x)  #solves an inequality
```

```
[[x < -8^(1/4)], [x > 8^(1/4)]]
```

```
sage: var('x,y')                          #specifies the variables x and y
sage: solve([x+y==7, x-y==-3], x, y)   #solves a system of two equations
```

```
[[x == 2, y == 5]]
```

```
sage: var('x,y')
sage: solve([x+y==7, x+y==-3], x, y)  #this system has no solutions
```

```
[]
```

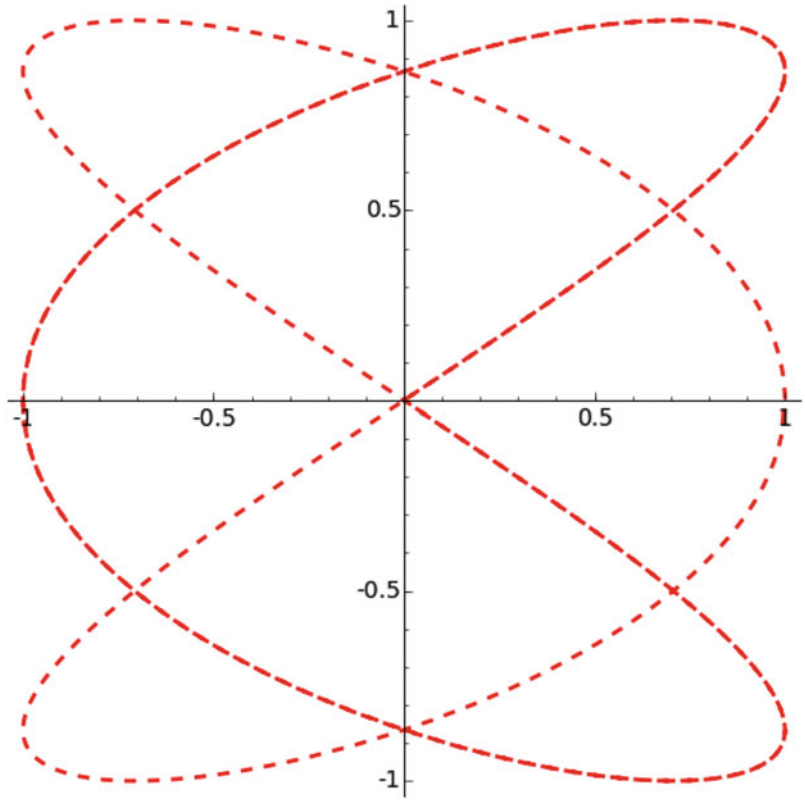
```
sage: var('x,y')
sage: solve([x+y==7, x+y==7], x, y) #this system is dependent
```

```
([x == -y + 7], [1])
```

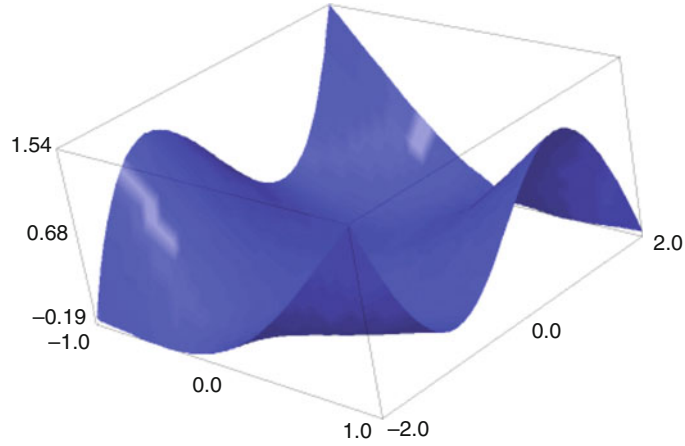
```
sage: var('x,y,z')
sage: solve([x+y==7, x+y+z==11], x, y,z)  #this is also dependent
```

```
[[x == -r1 + 7, y == r1, z == 4]]
```

```
sage: var('t')
sage: p = parametric_plot( [sin(3*t), sin(2*t)], (t, 0, 3*pi),
thickness=2, color='red', linestyle="--")
sage: p.show()    #YOU CAN ALSO USE:   show(p)
```




```
sage: f(x,y) = sin(x)^2+cos(y)*x*y
sage: #defines the function f in two variables x and y
sage: plot3d(f, (x,-1,1), (y,-2,2))
```

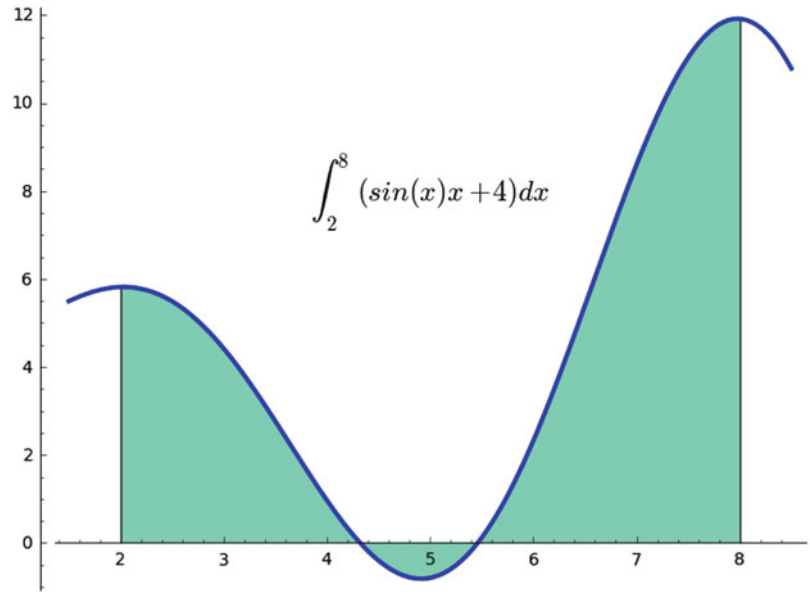



```
sage: for i in range(4):
sage: #it means i takes on the values 0, 1, 2, and 3 respectively
sage: #please note the indentation used in the next line!
sage:     print i , "   " , i^2, "   " ,i^3
```

```
0       0       0
1       1       1
2       4       8
3       9       27
```

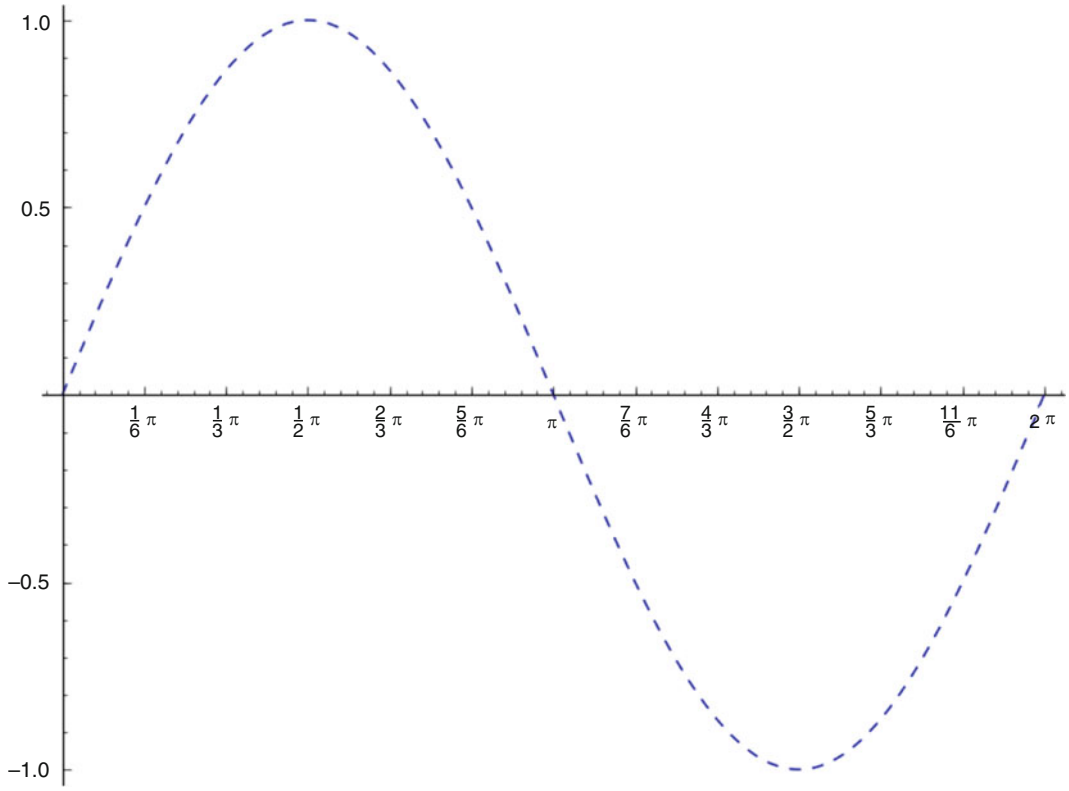
```
sage: #a bit more elaborate example!
sage: #first, define the function to be used
sage: f(x) = sin(x)*x+4
sage: #graphs the two horizontal lines
sage: P = line([(2,0),(2,f(2))], color='black')
sage: P += line([(8,0),(8,f(8))], color='black')
sage: #make the shade between the graph of f(x) and the x-axis
```

```
sage: #this will be a polygon with many points
sage: P += polygon(  [(2,0),(2,f(2))] +
sage: [(x, f(x)) for x in [2,2.1,..,8]] +
sage: [(8,0),(2,0)],
sage: rgbcolor=(0.5,0.8,0.7),aspect_ratio='automatic')
sage: #draw the text above the graph
sage: #notice how we used LaTeX style!
sage: P += text("$\\int_{2}^8 (sin(x) x+4) dx$",
sage:              (5, 8), fontsize=20, color='black')
sage: #draw the curve
sage: P += plot(f, (x, 1.5, 8.5), thickness=3)
sage: #show the plot
sage: show(P)
```



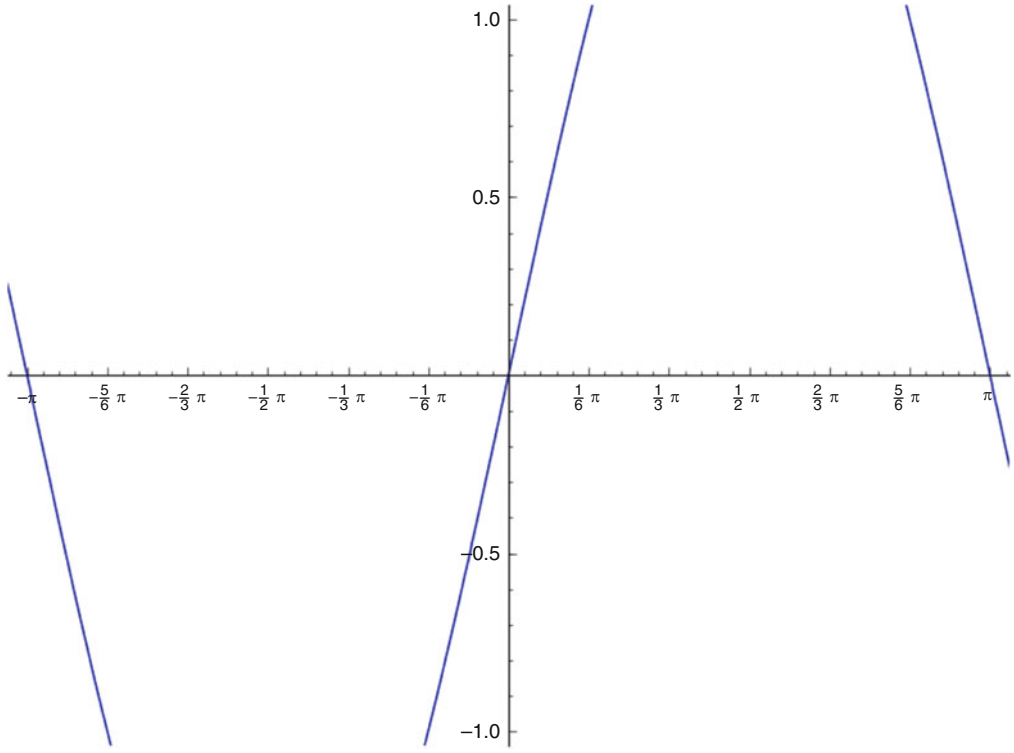
The next example shows how one can format the x-axis scale:

```
sage: #how to scale your x-axis
sage: p = plot(sin(x), 0, 2*pi, linestyle = "--")
sage: p.show(ticks=pi/6, tick_formatter=pi)
```



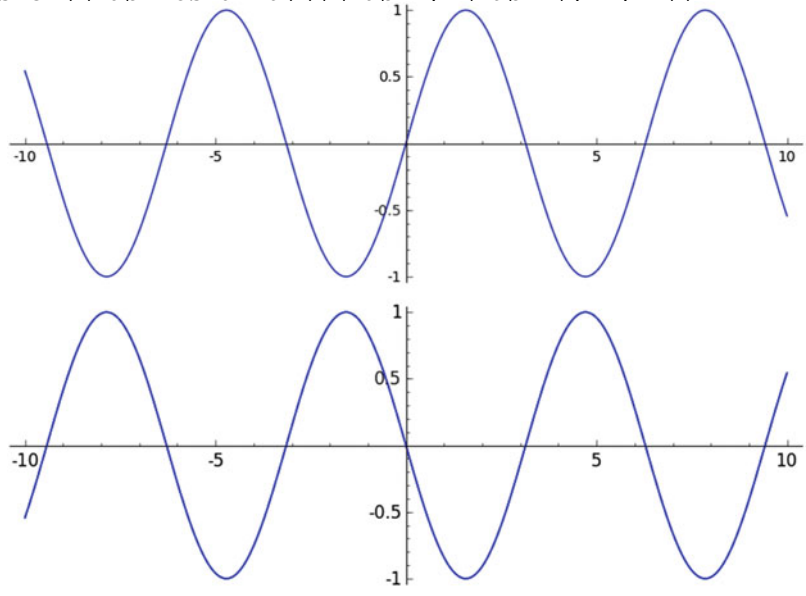
If you wish to display only a part of a graph, you can specify the desired window size in the `show()` method.

```
sage: g = plot(2*sin(x), (x, -2*pi, 2*pi) )
sage: g.show(xmin=-pi, xmax=pi, ymin=-1, ymax=1,
sage:     ticks=pi/6, tick_formatter=pi)
```



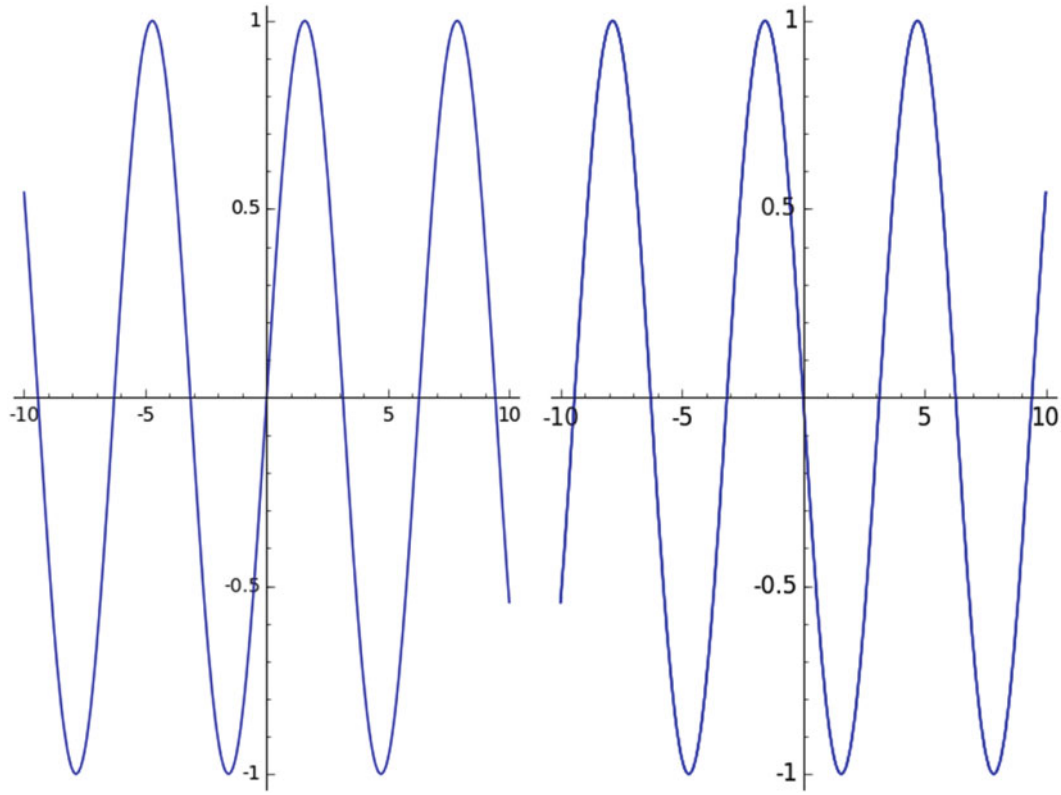
If you want to display two graphs, in two separate coordinate systems you can use code similar to:

```
sage: graph1 = plot(sin(x), (x, -10, 10))
sage: graph2 = plot(sin(x-pi), (x, -10, 10))
sage: show(graphics_array([graph1, graph2], 2, 1))
```



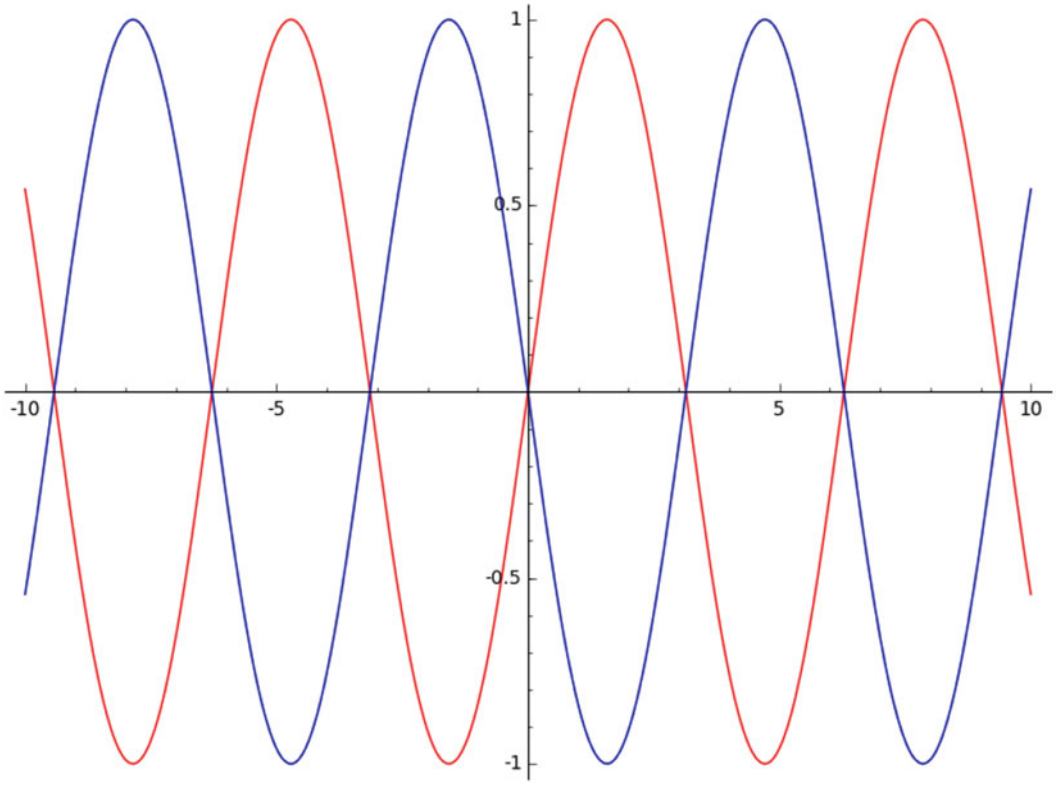
OR

```
sage: graph1 = plot(sin(x), (x, -10, 10))
sage: graph2 = plot(sin(x-pi), (x, -10, 10))
sage: show(graphics_array([graph1, graph2], 1, 2))
```



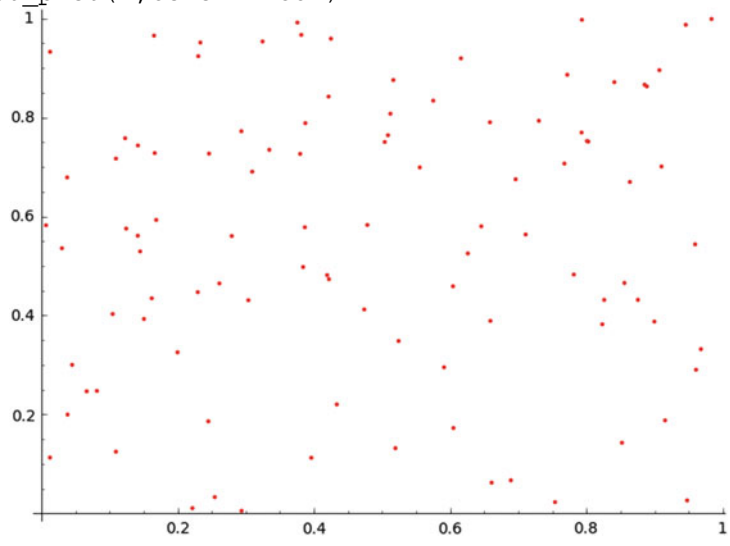
If you want both graphs to be displayed in the same coordinate system:

```
sage: graph1 = plot(sin(x), (x, -10, 10), color = 'red')
sage: graph2 = plot(sin(x-pi), (x, -10, 10))
sage: show(graph1+graph2)
```



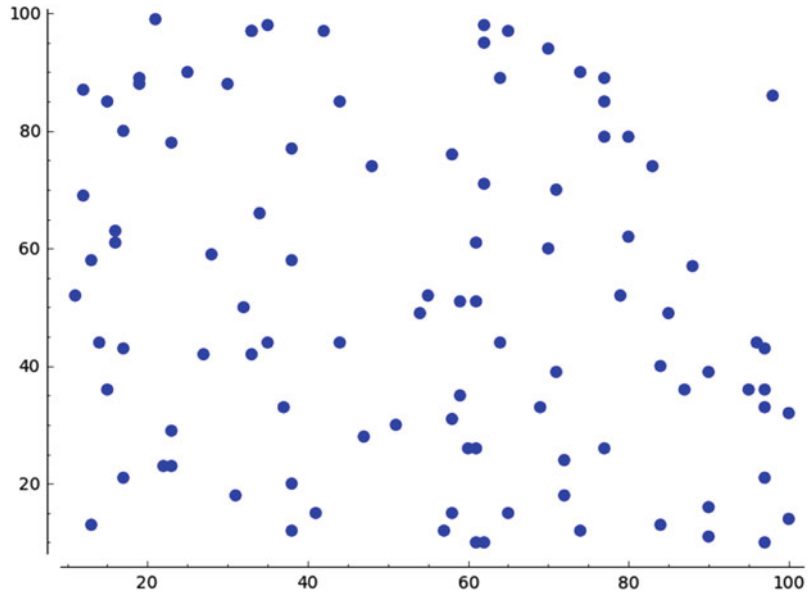
How to plot a set of points? Here is an example that plots 100 randomly generated points. The function `random()` will return a decimal value between 0 and 1. Hence `r` will be an array of 100 pairs of numbers (or 2D points, if you wish) whose values are between 0 and 1.

```
sage: r = [(random(),random()) for i in range(100)]
sage: list_plot(r,color='red')
```



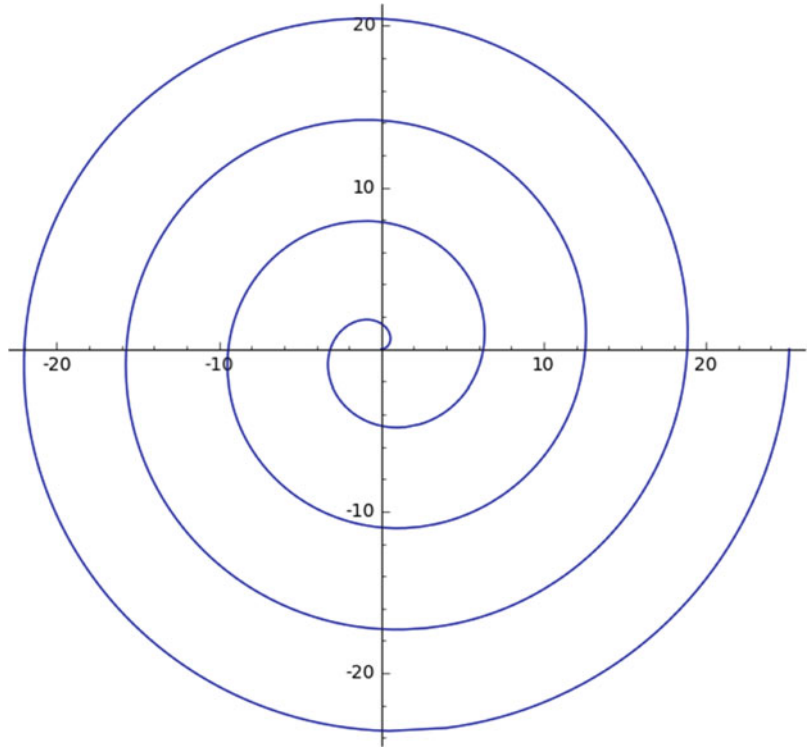
The following example will plot 100 random points with integer coordinates between 10 and 99. To obtain a random integer between 10 and 99 (inclusively), one can use the function `random.randint(10,99)`. When we display these points, were can given each point a display size. In our code we used `size = 70`.

```
sage: import random
sage: r = [(random.randint(10,99),
sage:       random.randint(10,99)) for i in range(100)]
sage: list_plot(r,color='blue', size = 70)
```



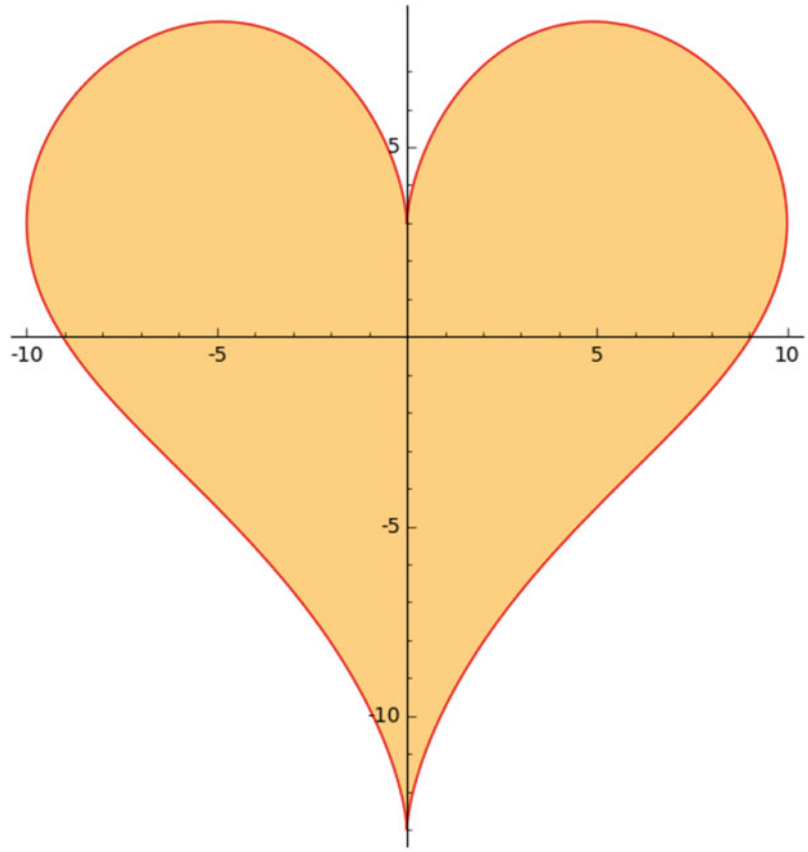
If you wish to graph parametric equations you can use code similar to:

```
sage: var('t')
sage: parametric_plot( (t*cos(t), t*sin(t)), (t, 0, 8*pi))
```



The graphs can be FILLED with a color. For example:

```
sage: var('t')
sage: parametric_plot( (10*sin(t)^3,
sage:                     9*cos(t)-4*cos(2*t)-cos(3*t)-cos(4*t)),
sage:                  (t, 0, 2*pi), color = 'red',
sage:                  fill = True, fillcolor='orange')
```

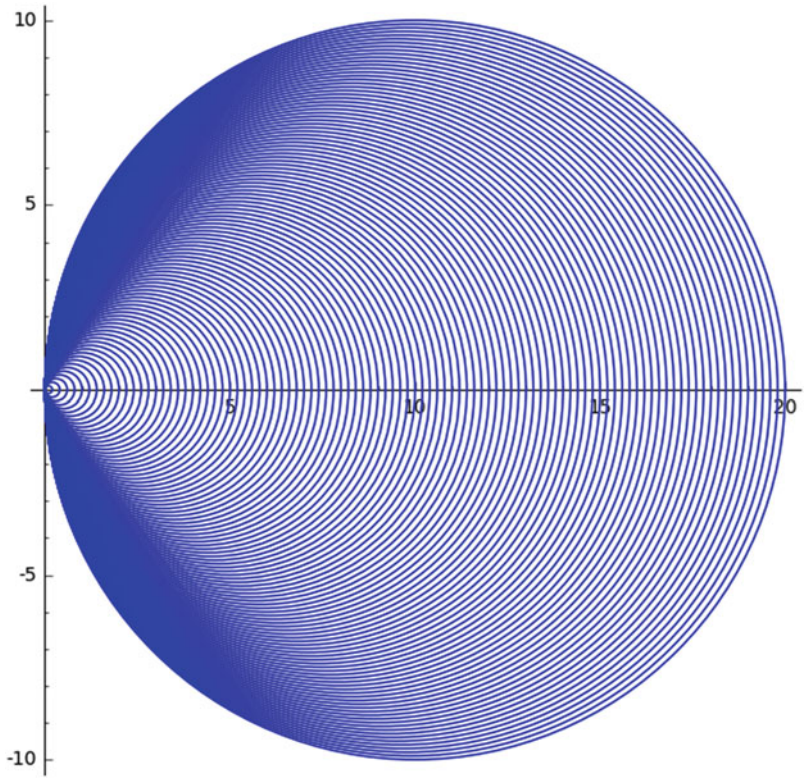


One last example that shows how one line of code in Sage can do a lot. Adding several graphs into one and displaying the result.

```
sage: show(sum(circle((i,0), i) for i in [10,9.9,..,0]))
```

The graph is displayed on the next page.

Most of the code given above should be fairly self-explanatory and the use of comments (the text in a line that follows the # symbol) should help you at least guess what the code does before you actually run the code. If there are any parts of the code that are not entirely clear, do not worry too much, we'll cover some of these examples in more details in the following sections of this book.



For a more in-depth introduction to Sage, and how to use SciPy library for numerical and scientific computing in Sage or Python, the reader may also want to check the following resource [5]. Other useful books that provide plenty of Sage code examples are: [2] for Linear Algebra and Differential Geometry, and [4] for Abstract Algebra.

1.3 Absolute Error vs. Relative Error

Irrational numbers, such as π, have an infinite number of digits. Whenever we want to write down its decimal representation, we can only include a finite part of it, and hence only write an approximation of it. This however introduces an **error**. For example, when we approximate π by 3.14159265, we are introducing an **error**. Approximating π by 3.14 will introduce a larger **error**.

As we see next, there are several ways to estimate the magnitude of the errors that result from an approximation. Depending on the targeted audience we may want to use one or another of these definitions. Here we will mention three of the most common definitions: the **error formula**, the **absolute error formula**, or the **relative error formula**. Each one of these definitions is given below.

The simplest of these three is the **error formula**. The **absolute error formula** is an alternative to the **error formula** for cases where the sign is irrelevant. A bit more elaborate, but occasionally more relevant, is the **relative error formula** which measures the error in relation to the size of the true value.

Definition 1. Let α be a number and β be an approximation for α.

The expression $\alpha - \beta$ is called the **error** of β as an approximation to α. Put another way, we have that

$$\text{error} = \text{true value} - \text{approximate value}.$$

The expression $|\alpha - \beta|$ is called the **absolute error** of β as an approximation to α.

The **relative error** of β as an approximation to α is given by $\frac{|\alpha-\beta|}{|\alpha|}$. Relative error measures the error in relation to the size of the true value.

Definition 2. The number of digits, starting with the leftmost nonzero digit, and ending with the rightmost (zero or nonzero) correct digit, is called the **number of significant digits**.

Example 3. Let $x = 0.000123$ rounded to $\overline{x} = 0.00012$ and $y = 120.128$ rounded to $\overline{y} = 120.13$. In each case, find the number of significant digits, the absolute errors, and the relative errors.

Solution. In the first case $\overline{x}$ has two significant digits. The absolute error is given by

$$|0.000123 - 0.00012| = 0.000003 = 0.3 \times 10^{-5}.$$

The relative error is given by

$$\frac{|0.000123 - 0.00012|}{|0.000123|} \simeq 0.02439 = 0.2439 \times 10^{-1}.$$

In the second case $\overline{y}$ has four significant digits. The absolute error is given by

$$|120.128 - 120.13| = 0.002 = 0.2 \times 10^{-2}.$$

The relative error in this case is given by

$$\frac{|120.128 - 120.13|}{|120.128|} \simeq 0.00001665 = 0.1665 \times 10^{-4}.$$

■

1.4 Accuracy and Precision

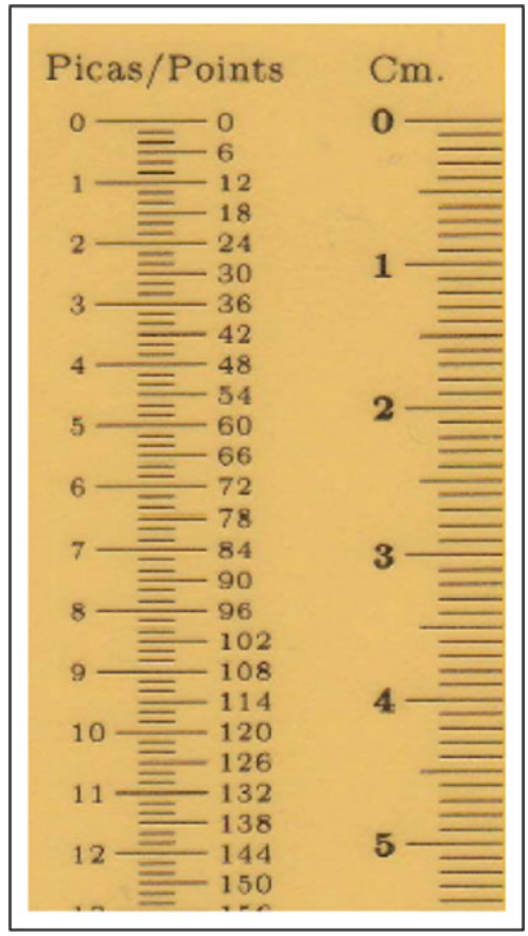
Accuracy is the degree of conformity of a measure to a true value. We have

Definition 4. Accurate to n decimal places means the n digits to the right of the decimal point are meaningful (you can trust them to be exact).

And

Definition 5. Accurate to n significant digits means that n digits, starting with leftmost nonzero digit, are meaningful (you can trust them to be exact).

We call **precision** the accuracy with which a number can be represented [14].



Example 6. Let's suppose you use a millimeter ruler to measure some lengths. This means your measurements are accurate to 1 mm. If you give the lengths in meters, then your measures are accurate to three decimal places, since 1 mm equals 0.001 m.

A measurement such as 173.1234567 m would be as accurate as 173.123 m since the ruler is accurate to 1 mm only.

In this example, the measurement of 173.123 m is accurate to three decimal places. It is accurate to six significant digits.

The measurement 0.023 m is accurate to three decimal places and to two significant digits.

As you may expect, the higher the number of significant digits used in calculations, the more accurate answers we obtain. But how much does this actually matter? Would it make a big difference if, let's say, we would use only three significant digits of precision instead of four?

The answer is, it depends—sometimes it does, other times it doesn't. In the following example, it will make quite a bit of a difference.

Example 7. Solve only for the variable y in the linear system of equations in two variables

$$\begin{cases} 0.1052x + 0.2155y = 0.7234 \\ 0.2141x + 0.4402y = 0.9562 \end{cases} \tag{1.1}$$

First only carry three significant digits of precision in all the calculations. Then repeat the work using four significant digits.

Solution. In the first part we'll use only three significant digits. We can multiply the first equation by $\alpha = -0.214/0.105 \approx -2.038$ and add it to the second equation. We should obtain $y = -0.517/0.001 = -517$.

In the second part, we'll repeat the same work, using now four significant digits. We can multiply the first equation by $\alpha = -0.2141/0.1052 \approx -2.0352$ and add it to the second equation. We should obtain $y = -0.5161/0.0016 = -322.5625$.

As you can see, there is quite a difference between the values of y, in those two cases.

If we would want to solve this system of equations using Sage, we can use the code below so solve a system of equation:

```
sage: x = var('x') #declare x a variable
sage: y = var('y') #declare y a variable
sage: solve([0.1052*x+0.2155*y==0.7234,
0.2141*x+0.4402*y==0.9562], x, y)
```

```
[[x == (3745986/5683), y == (-1809590/5683)]]
```

If we want to know the decimal form of these numbers, using let's say a 40 bit precision, just type in the following command:

```
sage: (3745986/5683).n(40),(-1809590/5683).n(40)
```

```
(659.15643146, -318.42160831)
```
■

From this example, we should learn that we should always try to carry all calculations with full precision (or as much as it is possible) and not round off prior to each of the calculations.

REMEMBER: Whenever we have a numeric expression in Sage, we can append `.n(40)` to it in order to get its decimal form, with a **40-bit precision**. Of course 40 can be replaced with any other positive integer that we want. If we simply omit the number 40, then we would use the default, 53 bits of precision.

For example, we can use the following expression to obtain the decimal form, with 50 bits of precision, of the golden ratio:

```
sage: ((1+sqrt(5))/2).n(50)
```

```
1.6180339887499
```

1.5 Rounding and Chopping

Sometimes we need to replace a number by another value that is approximately equal but has a shorter, simpler representation. For this we can use either *rounding* or *chopping*.

If we discard all the digits that follow the nth digit, and leave all the remaining digits unchanged, then we say that the number was **chopped to n digits**.

Example 8. The results of chopping 12.437, 12.460, 12.475, and 12.485, to 2 digits are: 12.43, 12.46, 12.47, and 12.48, respectively.

If we approximate a number with an n-digit number that approximates the first with the minimum error, then we say that the number was **rounded to n digits**.

Example 9. The results of rounding 12.437, 15.632, and 12.460, to 2 digits are: 12.44, 15.63, and 12.46, respectively.

One question when rounding is deciding whether we should round up or down an $(n+1)$-digit decimal number. For example, should we round up or down the following: 12.475, and 12.485?

Here is the convention we'll usually use in this book:

1. If the last digit equal 0, then we don't need to round up or down.
2. If the digit equals 1, 2, 3, or 4, we should round down.
3. Similarly, if the digit equals 6, 7, 8, or 9, we should round up.
4. The challenging question is: what should we do in the case that digit equals 5? If we would always round up, we would be biased, towards case 3. Similarly, if we would always round down, we would be biased, towards case 2. So a compromise solution would be one that rounds up roughly about half the times. The **round-to-even** method (also called **unbiased rounding**, or **bankers' rounding),** means to round to the nearest number such that the nth digit is even. **Round-to-odd** method means to round to the nearest number whose nth digit is odd.

Example 10. The results of rounding 12.475, and 12.485, to 2 digits are 12.48, and 12.48, respectively.

1.6 Introduction to Sage Programming Through Examples

We will introduce here several examples on how to develop basic Sage code. If you are not very familiar with programming, hopefully this section will help you get up to speed and write your very first Sage programs. If you are, however, familiar with Python, or Sage, then you may want to skip this section, or read it briefly.

1.6.1 Defining a Function: Evaluating It for a Given Input

Example 11. Write Sage code that will create the following function:

$$f(x) = 3\sin^2 x + \cos^3 x - 5$$

Then evaluate this function at $x = 5$.

Solution. One way you can define the function $f(x)$ and then use it to compute $f(5)$ is given below.

```
sage: f(x)=3*sin(x)+cos(x)^3-5
sage: f(5.0)
```

You should get an output similar to the one below:

```
-7.85394816310670
```

Another way to create a function in Sage is given next. This is the standard way to create a function in Python. First we need to use the ***def*** keyword. This keyword should be followed by the function's name (of our choice) and a set of parentheses. In parentheses we include any number of parameters we wish this function to have (comma separated). In the example below, function f will have only one parameter, x. Notice the use of the colon at the end of this line of code. This line is called the **header of the function**. Then, we define the **body of the function.** All the lines of code immediately

following the header of the function, that are indented relative to the header, are collectively called the body of the function. The functions may or may not return a value. In order to return a value, you should include a **return** statement (a statement that uses the keyword return).

Please note that once the return statement is executed, all the remaining lines of code within the body of the same function will be ignored. Therefore, within a body of a function, you should not have any lines of code following a `return` statement.

```
sage: def f(x):
sage:     return 3*sin(x)+cos(x)^3-5
sage: f(5.0)
```

and get:

```
-7.85394816310670
```

For more practice, let's take a look at the following function definition. We will create a function called `square()` that will square an argument. Then we will use the newly created function. In order to print something to the console, we can use the `print` function.

```
sage: def square(x):
sage:     return x*x
sage: print(square(2.5))
```

```
6.25000000000000
```

The alternative to the code above, with the same output, would be:

```
sage: square(x) = x*x
sage: print(square(2.5))
```

In future examples the body of the functions will usually be more elaborate. The above functions are just some basic examples that should help you understand how to create functions in Sage.

1.6.2 Print and Range

Example 12. Create Sage code that will print all the numbers strictly between 10 and 20.

Solution. In order to print something to the console, we can use the ***print*** method. For example, we could use the following command, to print the numbers strictly between 10 and 20:

```
sage: print 11, 12, 13, 14, 15, 16, 17, 18, 19
```

Please note how you need to use commas between the values you want to be printed out. You should get an output similar to the one below:

```
11 12 13 14 15 16 17 18 19
```

Another handy method is ***range***. The function ***range(a,b)*** returns the set of integers $\{x | a \leq x < b\}$. For example $range(10, 40)$ will return the set: $\{10, 11, 12, \ldots, 39\}$. And $range(-10, 40)$ will return the set: $\{-10, -9, -8, \ldots, 39\}$. We can also use range function with only one parameter. In this case the lower value is considered 0 implicitly. Hence $range(40)$ will return the set: $\{0, 1, 2, \ldots, 39\}$.

Therefore we could use the following code to get a similar output to the one above

```
sage: print range(11, 20)
```

Yet another way to get the same output is using arrays, as shown below:

```
sage: print [11,12,..,19]
```

Below we show another way to get the same output by using **arrays** (also called **lists**) and explicit `for` loops. We will explain loops in more details in the next subsection:

```
sage: print [x for x in [11,12,..,19]]
```

Note: in all these examples, the word `print` was not necessary, but we included it in order to make our code easier to read. Try running the code with the work `print` omitted.

1.6.3 For Loop: A First Example

Example 13. Create Sage code that will print out the square of the first ten positive integer numbers:

Solution. One way to do this is using the Sage code given below:

```
sage: print 0^2 , 1^2 , 2^2 , 3^2, 4^2, 5^2, 6^2, 7^2, 8^2, 9^2
```

You should get an output similar to the one below.

```
0 1 4 9 16 25 36 49 64 81
```

Another way to solve the same problem is using arrays. You could use something like:

```
sage: print [x^2 for x in [0,1,..,9]]
```

to get a similar output.

If you want to access the array later, you can first create it and name it (in our case we named it `arr`), then print it.

```
sage: arr =  [x^2 for x in [0,1,..,9]]
sage: print arr
```

The next example shows how we can use a **loop** to solve the same problem. **Loops** are statements that allow you to repeat a set of instructions either for a number of times, or until an expression (condition) becomes `false`. Loops are very important in programming and as such, you should invest as much time as needed in order to fully understand them. If you have experience with any other programming language, you most likely have already learned how they work, and here they should be very similar. We will make extensive use of loops throughout this book.

```
sage: for number in range(10):
sage:       print number^2
```

The output should look like:

```
0
1
4
9
16
25
36
49
64
81
```

The code `range(10)` returns the set of values (the array of values, to be more exact) $\{0, 1, 2, \ldots, 9\}$. To consider every value in this set and do something meaningful with each of them, we are using a ***for loop***. To refer to each of these values and do something useful, we need a "dummy variable" (above we chose to name it `number`). What we choose to do with **each of** the numbers is part of the code that come in **the indented line**, under the `for` loop. In our example, we square each number, and print it. Please note that the `for` statement ends with a colon ":". That is part of the Sage and Python syntax rule and therefore this is required.

Although the `range()` method is pretty common in Python, and we included it in our previous example, throughout this book we will mostly use an easier approach. Please note that this second approach works in Sage but not in Python. First we can create an array that contains the squares of the 1-digit numbers. As in the previous example, here we use the same name for the dummy variable: `number`. Then we simply print the array `squares`.

```
sage: squares = [number^2 for number in [0,1,..,9]]
sage: print squares
```

We obtain the following output

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

1.6.4 For Loop: Another Example

Example 14. Create Sage code that will print the prime factorization of all the integers strictly between 2000 and 2030:

Solution. To consider the integers strictly between 2000 and 2030, we need to use range(2001, 2030). Since we want to process each one of these integers, we'll use a for loop. The process here is to print each one of these numbers out as well as it's prime factorization. Note how we separated by commas the items we wanted to print in the second line of the Sage code below.

```
sage: for x in range(2001, 2030):
sage:       print x, "=", factor(x)
```

You should get an output similar to the one below.

```
2001 = 3 * 23 * 29
2002 = 2 * 7 * 11 * 13
2003 = 2003
2004 = 2^2 * 3 * 167
```

```
2005 = 5 * 401
2006 = 2 * 17 * 59
2007 = 3^2 * 223
2008 = 2^3 * 251
2009 = 7^2 * 41
2010 = 2 * 3 * 5 * 67
2011 = 2011
2012 = 2^2 * 503
2013 = 3 * 11 * 61
2014 = 2 * 19 * 53
2015 = 5 * 13 * 31
2016 = 2^5 * 3^2 * 7
2017 = 2017
2018 = 2 * 1009
2019 = 3 * 673
2020 = 2^2 * 5 * 101
2021 = 43 * 47
2022 = 2 * 3 * 337
2023 = 7 * 17^2
2024 = 2^3 * 11 * 23
2025 = 3^4 * 5^2
2026 = 2 * 1013
2027 = 2027
2028 = 2^2 * 3 * 13^2
2029 = 2029
```

When outputting each of the factorization, we included the number, x, the equal symbol (hence we used quotations), and the output of the ***factor*** *method applied to x.*

Note: whenever you want to output a text, you should include it between quotation marks (they are called ***strings****). For example,* ***print 2+3*** *will output* ***5****, while* ***print "2+3"*** *will output* ***2+3****.*

We can also obtain the same exact output using the following Sage code:

```
sage: for x in [2001,2002,..,2029]:
sage:       print x, "=", factor(x)
```

1.6.5 Conditional Statement

The operator % is called the **modulus operator**. The expression `a%b` gives the remainder of `a` upon division by `b`.

For example `a%2` gives the remainder of `a` upon division by 2. Hence `a%2` is equal to 0 if and only if `a` is an even number.

Another example `c%7` gives the remainder of `c` upon division by `7`. Hence `c%7` is equal to 0 if and only if c is a multiple of 7.

The next example will check if a number is even, and if so, it will print out the message `"the number is even"`. Otherwise, it will print out: `"the number is odd"`. Please note the colon that ends the line of code containing `if` and `else`.

```
sage: a = 7812
sage: if a%2 == 0:
sage:    print "the number is even"
```

```
sage: else:
sage:     print "the number is odd"
```

The obtained output is:

```
the number is even
```

The `if` statement checks if the condition (in our example `a%2==0`) is `true`. If it is, then it will execute the statement (or block of statements) that follows immediately after it, and is indented. If the condition is `false`, then it will skip to the statement (or block of statements) that follow the `else:` and is indented.

We can make the previous code more user friendly, by including the value of `a` into the output message. For example, we could use the following Sage code:

```
sage: a = 7812
sage: if a%2 == 0:
sage:     print str(a) + " is even"
sage: else:
sage:     print str(a) + " is odd"
```

to obtain

```
7812 is even
```

In the previous code, we used the `str` function to change the *number* `a` into a *string* (text) so we can then append it to the *string* "`is even`" or "`is odd`". In Sage we can "add" (append) two strings, such as `"text1" + "text2"`, to obtain a concatenation of it: `"text1text2"`.

If we plan to use this code multiple times, it would be nicer to create and use a function. Below we define a function, `isEven`, and then we call it three times.

```
sage: def isEven(a):
sage:     if a%2 == 0:
sage:         print str(a) + " is even"
sage:     else:
sage:         print str(a) + " is odd"
sage: isEven(7812)
sage: isEven(2014)
sage: isEven(2015)
```

We obtain:

```
7812 is even
2014 is even
2015 is odd
```

1.6.6 More on Conditional Statements

Example 15. Create a function, called `GetLetterGrade`**,** that will return the corresponding letter grade for a given score.

Solution. This is a very common example used in Introductory Programming courses to illustrate the use of the conditional statements. The function should return "A", for a score of at least 90, "B" for scores at least 80 but less than 90, "C" for a score at least 70 but less than 80, and so on. Please

note that in Sage, like in Python, but unlike C/C++, Java, and many others, a **string** is a text that is enclosed in either " " or in ' '. They are equivalent.

```
sage: def GetLetterGrade( score ):
sage:         if (score >= 90):
sage:                 return 'A'
sage:         elif(score >= 80):
sage:                 return 'B'
sage:         elif(score >= 70):
sage:                 return 'C'
sage:         elif(score >= 60):
sage:                 return 'D'
sage:         else:
sage:                 return 'F'
sage: print 'a score of 87 will give you a/an:', GetLetter
      Grade(87)
```

```
a score of 87 will give you a/an: B
```

1.6.7 Working with Strings

A string is printed AS IS, without being evaluated. For example the code

```
sage: a = 7812
sage: b = 1082
sage: print "a+b"
```

will output:

```
a+b
```

and the previous Sage code is equivalent to

```
sage: a = 7812
sage: b = 1082
sage: print 'a+b'
```

```
a+b
```

If you wish to output **the value** of the expression `a+b`, you need to remove the quotation marks. Therefore

```
sage: a = 7812
sage: b = 1082
sage: print a+b
```

will output:

```
8894
```

The character `'\t'` is a **tab character**, and it will be used in several examples throughout this book to format the output. Other characters like this (which are all called **escape characters**) are: `'\n'` (for a new line) and `'\\'` (if you wish to output one backslash: \).

If you wish to create a more user-friendly output, you could use code as below, where we **append ("add")** several strings together for a nicer output.

```
sage: a = 7812
sage: b = 1082
sage: print 'a+b = ', a+b
```

to obtain:

```
a+b = 8894
```

Another alternative to the code given above, with a similar output, is:

```
sage: a = 7812
sage: b = 1082
sage: print 'a+b = '+str(a+b)
```

In the last line, we used the `str` method to change the value of the number `a+b` into a string, so we can append it to the string `'a+b = '`. We could not just add the number `a+b` to the string `'a+b='`.

1.6.8 Computing a Running Total

Example 16. Create Sage code that will compute the following sum

$$\sum_{i=7}^{2015} \frac{1}{i^3}$$

Solution. To compute a sum, we first need a variable (a location where to store a value), let's call it ***answer***, that is initialized to 0. Then, using a loop, we consider every number from the required range, and add the expression $\left(\frac{1}{i^3}\right.$ in our example$\left.\right)$ to our sum. At the end we report the total using a print statement. We can report a decimal form, using the `.n()` method.

```
sage: answer = 0
sage: for i in range (7, 2015+1):
sage:     answer = answer + 1/(i^3)
sage: print answer.n(50)
```

to obtain:

```
0.011765113408142
```

An alternative to this code, with the same exact output is:

```
sage: answer = 0
sage: for i in [7,8,..,2015]:
sage:     answer = answer + 1/(i^3)
sage: print answer.n(50)
```

1.6.9 Counting

Example 17. Create Sage code that will count the number of primes less than 2015.

Solution. For this example one could use a **for loop**, to consider all nonnegative integers less than 2015. Hence the line `for i in range(2015)`. Then, using an ***if*** statement, we check if any ***i*** from the given range is prime. For this, we write the code `if(is_prime(i)==True)`. Please note that this line is indented, since we want to use this line with every i from the for loop. Also note that we use a colon at the end of this line, since this is an if statement. It is important to remember that i order to compare if two values are the same, we use ==.

```
sage: num_of_primes = 0
sage: for i in range (2015):
sage:     if( is_prime(i) == True ):
sage:         num_of_primes  = num_of_primes + 1
sage: print "we have " , num_of_primes , " primes less than 2015"
```

We get the following output:

```
we have 305 primes less than 2015
```

This example is very similar with the previous example, in the sense that we are computing a sum. Using a for loop, we are considering all nonnegative numbers less than 2015. Then, for each such number we check to see whether it is prime or not. Each time we find a prime number we add 1 to the total sum. At the end, we report this total.

1.6.10 While Loops

Example 18. Create Sage code that will output the first ten prime numbers that are larger than 2015.

Solution. `For` loops are great when we know a priori the number of times we need to repeat a statement, or group of statements. Whether this number is a constant or given by a variable.

There are cases, however, when we need to repeat a group of statements without knowing for how many times. We only know when we need to stop. In these cases, we will use an alternative to `for` loops, called a `while` loop. While loops will repeatedly execute a group of statements until a given expression (condition) becomes `false`.

In our example, we will start with the `current_number` set to 2016. Check whether it is prime or not. Then check 2017, then check 2018, and so on. We need to do this until we have found ten prime numbers. Since we don't know exactly how many numbers we need to check until we have found our tenth prime number, `for` loops are not suitable here. Therefore we'll use a `while` loop. This loop executes its body (all the statements that are indented/"inside" this loop) as long as the expression `primes_found<10` remains `true`. So as long as `primes_found < 10`, we need check if the `current_number` is prime. If it is, then we increase the counter for the number of prime numbers found, `primes_found`, and output it. Then, we increase the `current_number` by one, so we can advance and check the next one.

```
sage: primes_found = 0
sage: #the first number to check is 2016
sage: current_number = 2015+1
sage: while(primes_found<10):
sage:     if(is_prime(current_number) == True):
sage:         #we found another prime. Count it in
sage:         primes_found=primes_found+1
sage:         #Print the prime number just found
sage:         print current_number
```

```
sage:     #"move" current number to the next number
sage:     current_number=current_number+1
```

We find:

```
2017
2027
2029
2039
2053
2063
2069
2081
2083
2087
```

How would you change the previous code to print out the smallest 20 prime numbers greater than 1,000,000?

1.6.11 Another Example on Writing Functions

Example 19. Using the previous example, create a function called `print10primes` that will print the next ten primes greater than a value given as parameter, `start`. Then use this function to print the next ten prime numbers greater than 1000, and then the next ten prime numbers greater than 23456.

Solution. We can use the Sage code below

```
sage: def print10primes(start):
sage:     primes_found = 0
sage:     number = start+1
sage:     while(primes_found<10):
sage:         if(is_prime(number) == True):
sage:             primes_found=primes_found+1
sage:             print number
sage:         number=number+1

sage: print10primes(1000)
sage: print "======="
sage: print10primes(23456)
```

to obtain

```
1009
1013
1019
1021
1031
1033
1039
1049
1051
```

```
1061
=======
23459
23473
23497
23509
23531
23537
23539
23549
23557
23561
```

1.6.12 Nested Functions: Randomly Generated Numbers

Example 20. Create a function, called `GenerateQuestion`**,** that will randomly generate two integers between 10 and 99, inclusively, and will output a basic algebra question. The question should be constructed in such a way that it will randomly contain an addition, subtraction, multiplication, or division. Then create another function `GenerateTest` that prints a set of six such basic algebra questions. Then generate two such tests.

Solution. First, we need to write Sage code for a function `GenerateQuestion`. In order to be able to generate random numbers, we need to `import random` *library*. Then calling the method `random.randint(10, 99)` twice, this will generate two random numbers between 10 and 99. After that, we randomly generate a number between 0 and 3 (using `random.randint(0,3)`) and based on this number we choose a basic operation: addition, subtraction, multiplication, or division and then `print` a question.

In the body of the `GenerateTest` method, we could use a `for` loop and call the newly created function (also called **method**) `GenerateQuestion()` for six times.

We finish this program by calling the method `GenerateTest()` twice.

```
sage: import random
sage: def GenerateQuestion():
sage:     #randomly generate two numbers
sage:     number1 = random.randint(10,99)
sage:     number2 = random.randint(10,99)
sage:     #randomly choose an operation:
sage:     # 0 for +,  1 for -,   2 for  *, 3 for /
sage:     operation = random.randint(0, 3)
sage:     if(operation == 0):
sage:         print number1  , " + " , number2  ,  " =?"
sage:     elif (operation == 1):
sage:         print number1  , " - " , number2  ,  " =?"
sage:
sage:     elif (operation == 2):
sage:         print number1  , " * " , number2  ,  " =?"
sage:     else:
sage:          print number1  , " / " , number2  ,  " =?"
sage: def GenerateTest():
```

```
sage:     for x in range(6):
sage:         GenerateQuestion()
sage: print "Generating Test 1:"
sage: print "------------------"
sage: GenerateTest()
sage: print "Generating Test 2:"
sage: print "------------------"
sage: GenerateTest()
```

The output of this code is shown next.

```
Generating Test 1:
------------
55 / 95 =?
26 + 11 =?
74 + 35 =?
17 * 19 =?
81 * 37 =?
88 + 97 =?
Generating Test 2:
------------
70 * 100 =?
45 / 100 =?
80 / 24 =?
58 / 33 =?
67 - 61 =?
10 - 56 =?
```

1.6.13 Working with Arrays

Example 21. The next several examples should demonstrate how one can use arrays in Sage/Python.

Solution. One-dimensional arrays

```
sage: #creates a simple array of values
sage: mylist = [1, 3, 5, 7]
sage: #computes and outputs the length of it
sage: len(mylist)
sage: #should display 4

sage: #creates a simple array of values
sage: mylist = [1, 3, 5, 7, 9, 11]
sage: #outputs the first element
sage: mylist[0]

sage: #outputs the second element
sage: mylist[1]

sage: #outputs the last element
sage: mylist[len(mylist)-1]
```

Two-dimensional arrays

```
sage: #creates an array of pairs
sage: #   (for example, points in the plane)
sage: mylist = [(1,4),  (3,5),  (7, 11)]
sage: #computes and outputs the length of it
sage: len(mylist)
sage: #should display 3

sage: #outputs the first element of the array
sage: mylist[0]
sage: #should display (1,4)

sage: #outputs the third element of the array
sage: mylist[2]
sage: #should display (7, 11)

sage: #outputs the first coordinate of
sage: #   the third element in the array
sage: mylist[2][0]
sage: #should display 7

sage: #outputs the second coordinate of
sage: #   the third element in the array
sage: mylist[2][1]
sage: #should display  11

sage: #create an array of points
sage: myArray = [(x, 2^x) for x in [1, 2,..,6]]
sage: #display the array
sage: myArray
sage: #you should get:
[(1, 2), (2, 4), (3, 8), (4, 16), (5, 32), (6, 64)]

sage: #create arrays:
sage: myArray = range(9)
sage: #display the array
sage: myArray
sage: #you should get:
[0, 1, 2, 3, 4, 5, 6, 7, 8]

sage: #another way to create arrays:
sage: A = matrix(QQ, 2, 5, range(10))
sage: #display the array
sage: A
sage: #you should get:
```

```
[0 1 2 3 4]
[5 6 7 8 9]
```

Remark 22. Please note that in most of the Sage examples that follow we omitted to include the **optional** "`sage:` " from the beginning of each line to better display it in page.

1.7 Taylor Series

Polynomials are some of the easiest and most commonly used class of functions that we encounter in Mathematics. They can easily and efficiently be evaluated at every specific value and have very nice properties: they are continuous everywhere, differentiable everywhere, and integrable on any bounded interval. Finding the derivatives or antiderivatives of a polynomial are very easy tasks too. Also, we can easily write computer programs that can evaluate a polynomial at any specific value: it only involves the four basic arithmetic operators—addition, subtraction, multiplication, and division.

It is this and other very useful analytical properties (e.g., the *Weierstrass Approximation Theorem* given next) that make polynomials a great choice in many approximation problems.

Most of the functions that do occur in Mathematics, however, do not have all those nice properties. It is hard, if not impossible, to evaluate exactly functions like $\cos(x)$, $\arctan(x)$, $\ln(x)$, e^x, $\sqrt[5]{x}$, etc. For example, how would you evaluate these functions, if you were not allowed to use any library or built-in functions already created for you? Could you easily compute $\ln(1.01)$, or say $e^{0.1}$, or even $\sqrt[5]{3}$?

In 1885, Karl Weierstrass [15] proved the famous *Weierstrass Approximation Theorem* (also known by the name of *The Fundamental Theorem of Approximation*) which, in a simplified form, states that *any continuous function on a closed and bounded interval can be uniformly approximated on that interval by polynomials to any degree of accuracy*. This powerful statement gives Numerical Analysis a very strong foundation.

A next step, knowing it is possible to approximate a continuous function by a polynomial, would be to find/construct such a polynomial. Given a function $f(x)$, we would like to find some *polynomial* $T(x)$ so that we can easily approximate $f(a)$ by computing $T(a)$ and the error of approximation is small enough.

Perhaps the most widely used polynomial approximation are the **Taylor polynomials**, which students should have already encounter in their study of Calculus (see [13] for a review of Calculus). Without going into all the details about all the conditions that are needed to be satisfied (they will be described further down in this Chapter!), let us be reminded the following formulae:

$$T_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(c)}{k!}(x-c)^k \tag{1.2}$$

is called the n**th order Taylor polynomial**, and

$$T(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(c)}{k!}(x-c)^k \tag{1.3}$$

is called the **Taylor series,** for the function f at x, around c.

In particular (see [1, 9]), some familiar examples of Taylor series (again, we are anticipating the $=$ sign included in the next formulae!)

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \cdots \tag{1.4}$$

$$= \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k}}{(2k)!}, \qquad \text{for } -\infty < x < \infty. \tag{1.5}$$

$$\arctan(x) = 1 - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \frac{x^9}{9} - \cdots \tag{1.6}$$

$$= \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{2k+1}, \qquad \text{for } -1 \le x \le 1. \tag{1.7}$$

$$\ln(1+x) = 1 - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \frac{x^5}{5} - \cdots \tag{1.8}$$

$$= \sum_{k=1}^{\infty} (-1)^{k+1} \frac{x^k}{k}, \qquad \text{for } -1 < x \le 1. \tag{1.9}$$

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \cdots \tag{1.10}$$

$$= \sum_{k=0}^{\infty} \frac{x^k}{k!}, \qquad \text{for } -\infty < x < \infty. \tag{1.11}$$

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + x^4 + x^5 + \cdots \tag{1.12}$$

$$= \sum_{k=0}^{\infty} x^k, \qquad \text{for } -1 < x < 1. \tag{1.13}$$

Remark 23. The Taylor series around $c = 0$ are also called **Maclaurin Series**. All of the above examples are Maclaurin Series.

For the purpose of approximation, we could, intuitively for now, approximate $f(x) = e^x$ by the Taylor polynomial $T_4(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!}$ and hence $e^{0.1}$ could be approximated by $T_4(0.1) = 1 + 0.1 + \frac{0.1^2}{2!} + \frac{0.1^3}{3!} + \frac{0.1^4}{4!} \approx 1.10517091$. Therefore one could say $e^{0.1} = \sqrt[10]{e} \approx 1.10517091$. The error we committed by this approximation is less that $\frac{1}{10^8}$ but we'll see later in this chapter how one can estimate the error of approximation. For now, we'll use a rather intuitive approach.

All those formulae can be derived using (1.3). Similarly, for the partial sum, one could use (1.3). But one can also use Sage to get the previous formulae (the partial sums only!). For this we'll use the code like `f(x).taylor(x, c, n)`, where $f(x)$ is the function under consideration, x is the variable of the function, c is the point around which the Taylor series are developed, with n being the degree up to which the terms from the Taylor series are displayed.

For example, $\cos(x).taylor(x, 0, 5)$ will display the Taylor polynomial of degree at most 5, around $c = 0$, for the function $\cos(x)$.

```
var('x')
print "cos(x) = " , cos(x).taylor(x, 0, 5)
print "arctan(x) = " , arctan(x).taylor(x, 0, 5)
print "ln(x) = " , ln(x).taylor(x, 0, 5)
print "e^x = " , (e^x).taylor(x, 0, 5)
print "1/(1-x) = " , (1/(1-x)).taylor(x, 0, 5)
```

we obtain the following:

```
cos(x) = 1/24*x^4 - 1/2*x^2 + 1
arctan(x) = 1/5*x^5 - 1/3*x^3 + x
ln(x) = log(x)
e^x = 1/120*x^5 + 1/24*x^4 + 1/6*x^3 + 1/2*x^2 + x + 1
1/(1-x) = x^5 + x^4 + x^3 + x^2 + x + 1
```

Example 24. Use the first few (up to degree five) terms of the Taylor series approximate the value of the function e^x at $x = 0.5$. That is, approximate $\sqrt{e}$.

Solution. Using the formula (1.10) we obtain the following approximation:

$$e^x \approx 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!}. \tag{1.14}$$

Therefore we obtain

$$e^{0.5} \approx 1 + 0.5 + \frac{0.5^2}{2} + \frac{0.5^3}{3!} + \frac{0.5^4}{4!} + \frac{0.5^5}{5!} \tag{1.15}$$

$$\approx 1.64869791666667 \tag{1.16}$$

which is accurate to four decimal places. The symbol $\approx$ means approximately equal.

If we want to use Sage, we could use the following Sage code:

```
var('x')
f(x)=e^x
T5(x)=(e^x).taylor(x, 0, 5)
print T5(0.5)
#we used    print T5(x)   to output the Taylor polynomial
```

to get the same result:

```
1.64869791666667
```

Next we give an algorithm in Sage that will create a table of values that can be used to compare partial sums up to degree nine (rather than five, as requested by the problem) against the "true" value. Please note that the "true" value is itself an approximation, one in which we expect the error to be small (and which is truncated to the default number of digits!). First we declare the variable x and we define the function f that we'll use in this example. In the third line of code we print the first line of the table. Please note that strings need to be enclosed in double quotation marks. The "$\backslash t$" is a special string and it will print a **tab space**, so the results are nicely aligned. Then, instead of using a statement for each partial sum requested, we are using a **for loop**. Hence in a for loop we are requesting Sage to compute the Taylor's partial sum of degree i (x is the variable, 0 is the point about which the Taylor's series is computed, and i is the power of this partial sum), and `print` the line number, the obtained approximation and the "true" value.

```
var('x')
f(x) = e^x
print "#", "\tUsing Taylor Approx",  "\tTrue Value"
for i in range(1, 9+1):
    partial(x)=f(x).taylor(x, 0, i)
    print  i, "\t", partial(0.5), "\t", f(0.5)
```

will generate the output:

```
#     Using Taylor Approx     True Value
1     1.50000000000000     1.64872127070013
2     1.62500000000000     1.64872127070013
3     1.64583333333333     1.64872127070013
4     1.64843750000000     1.64872127070013
5     1.64869791666667     1.64872127070013
6     1.64871961805556     1.64872127070013
7     1.64872116815476     1.64872127070013
8     1.64872126503596     1.64872127070013
9     1.64872127041825     1.64872127070013
```

From this table we can easily see that using merely 4–5 terms from the Taylor series, we get quite a good approximation (although, of course, "quite good" is a relative expression).

While the previous example worked quite well, the next example shows why we need to be careful with the intuition, and follow a more Mathematical approach. It is very important that we know the conditions that need to be satisfied in order to guarantee that such an approach works. Just because an intuitive approach worked in one example it doesn't imply it will always work, for every example.

Example 25. Using the first few (up to degree five) terms of the Taylor series approximate the value of the function e^x at $x = 10$. That is, approximate e^{10}.

Solution. Using (1.14) we obtain:

$$e^{10} \approx 1 + 10 + \frac{10^2}{2} + \frac{10^3}{3!} + \frac{10^4}{4!} + \frac{10^5}{5!} \tag{1.17}$$

$$\approx 1477.66666667 \tag{1.18}$$

which is quite far from the more accurate value of 22026.4657948067.

For a better view of what's going on, we could use the following Sage code in order to create a table of values using partial sums up to degree twenty:

```
var('x')
f(x) = e^x
print "#", "\tUsing Taylor Approx",  "\tTrue Value"
for i in range(1, 20+1):
    partial(x)=f(x).taylor(x, 0, i)
    print  i, "\t", partial(10.0), "\t", f(10.0)
```

We obtain:

```
#     Using Taylor Approx     True Value
1     11.0000000000000     22026.4657948067
2     61.0000000000000     22026.4657948067
3     227.666666666667     22026.4657948067
4     644.333333333333     22026.4657948067
5     1477.66666666667     22026.4657948067
6     2866.55555555556     22026.4657948067
7     4850.68253968254     22026.4657948067
8     7330.84126984127     22026.4657948067
9     10086.5731922399     22026.4657948067
10     12842.3051146384     22026.4657948067
```

```
11     15347.5159531826     22026.4657948067
12     17435.1916519694     22026.4657948067
13     19041.0960356516     22026.4657948067
14     20188.1705954246     22026.4657948067
15     20952.8869686065     22026.4657948067
16     21430.8347018453     22026.4657948067
17     21711.9804272798     22026.4657948067
18     21868.1724969657     22026.4657948067
19     21950.3788494319     22026.4657948067
20     21991.4820256651     22026.4657948067
```

From this table we can easily see that we need more than just a few terms from Taylor series in order to get a good enough approximation. We should also note that the point $x = 10$ is further away from $c = 0$ than $x = 0.5$, which may indicate that further away points from $x = c$ would usually yield less accurate results. This is actually a true statement as we will see later in this chapter.

Remark 26. **Taylor's series will converge much faster (to the true value) near the point of expansion and slower (if at all) at more remote points.** This result is a direct consequence of the Taylor's Theorem with Remainder (given on page 48).

Following the expression "*a picture is worth a thousand words*" it may be worth seeing the graphs of Taylor polynomials compared to the true function. Let's graph, below, the function $f(x) = e^x$, along with few partial sums of the Taylor series for f, about the point $c = 0$. For this we can write the following Sage code

```
x = var('x')
f(x) = e^x  #this is the function we'll use

#the original plot, for x in [-1.6]
p=plot(f(x),-1,6, color= 'black', linestyle='--',
                  legend_label='f(x)')

#plot for one partial sum - of degree 1
g1(x) = f(x).taylor(x, 0, 1)
p1=plot(g1(x),-1,6, color= 'red', legend_label='dgr 1')


#plot for one partial sum - of degree 4
g4(x) = f(x).taylor(x, 0, 4)
p4=plot(g4(x),-1,6, color= 'green', legend_label='dgr 4')


#plot for one partial sum - of degree 5
g5(x) = f(x).taylor(x, 0, 5)
p5=plot(g5(x),-1,6, color= 'brown', legend_label='dgr 5')


#plot for one partial sum - of degree 7
g7(x) = f(x).taylor(x, 0, 7)
p7=plot(g7(x),-1,6, color= 'blue', legend_label='dgr 7')

#graph all the plots together
show(p+p1+p4+p5+p7)
```

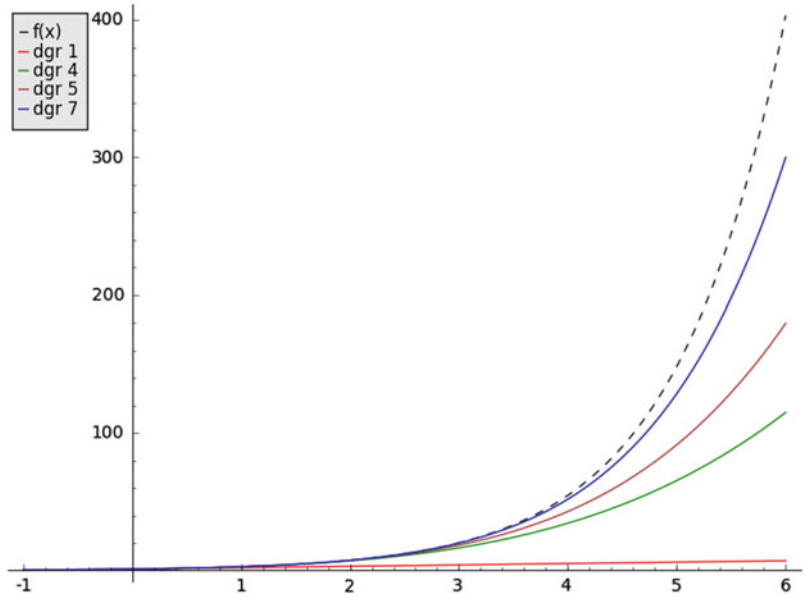
Above, $q1(x), q2(x), \ldots, q7(x)$, are the partial sums from the Taylor polynomial expansion of the function $f(x) = e^x$, around $c = 0$, of degrees 1, 4, 5, and 7 respectively. Then, p, $p1$, $p4$, $p5$, and $p7$, are the plots (graphs) for each of these functions, graphed in the interval $[-1, 6]$. To show all these plots together in one graph, we use the command: `show(p+p1+p4+p5+p7)`.

This code will produce the following graph.

Please note, from the graph, that:

- the partial sums graph become less accurate (further away from $f(x)$) for points that are further away from $c = 0$.
- for any given point, usually, the higher the degree of the partial sum, the more accurate it gets (the closer it gets to the original function).

Another (similar) example would be for the function $f(x) = \arctan(x)$. Using the following Sage code we obtain the graph:

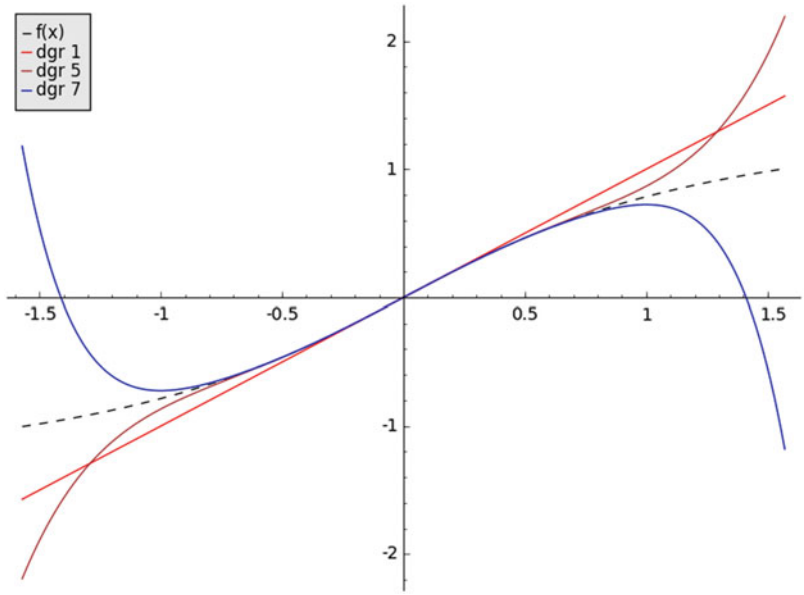



```
x = var('x')
f(x) = arctan(x) #this is the function we'll use

#the original plot
p=plot(f(x),-pi/2,pi/2, color= 'black', linestyle='--',
       legend_label='f(x)')
#plot for one partial sum - of degree 1
g1(x) = f(x).taylor(x, 0, 1)
p1=plot(g1(x),-pi/2,pi/2, color= 'red', legend_label='dgr 1')

#plot for one partial sum - of degree 5
g5(x) = f(x).taylor(x, 0, 5)
p5=plot(g5(x),-pi/2,pi/2, color= 'brown', legend_label='dgr 5')

#plot for one partial sum - of degree 7
```

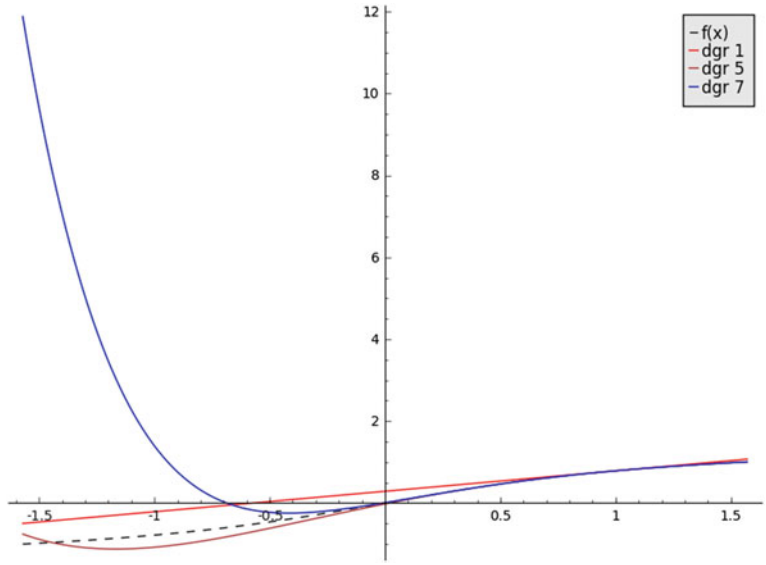



```
g7(x) = f(x).taylor(x, 0, 7)
p7=plot(g7(x),-pi/2,pi/2, color= 'blue', legend_label='dgr 7')

#graph all the plots together
show(p+p1+p5+p7)
```

We observe again that for points close to $c = 0$, the partial sums from the Taylor series are providing quite good estimates. For more remote points, however, this does not hold anymore.

Changing the value of c from the previous example to $c = 1$, we obtain the following graph:



```
x = var('x')
f(x) = arctan(x) #this is the function we'll use

#the original plot
p=plot(f(x),-pi/2,pi/2, color= 'black', linestyle='--',
       legend_label='f(x)')

#plot for one partial sum - of degree 1
g1(x) = f(x).taylor(x, 1, 1)
p1=plot(g1(x),-pi/2,pi/2, color= 'red', legend_label='dgr 1')

#plot for one partial sum - of degree 5
g5(x) = f(x).taylor(x, 1, 5)
p5=plot(g5(x),-pi/2,pi/2, color= 'brown', legend_label='dgr 5')

#plot for one partial sum - of degree 7
g7(x) = f(x).taylor(x, 1, 7)
p7=plot(g7(x),-pi/2,pi/2, color= 'blue', legend_label='dgr 7')

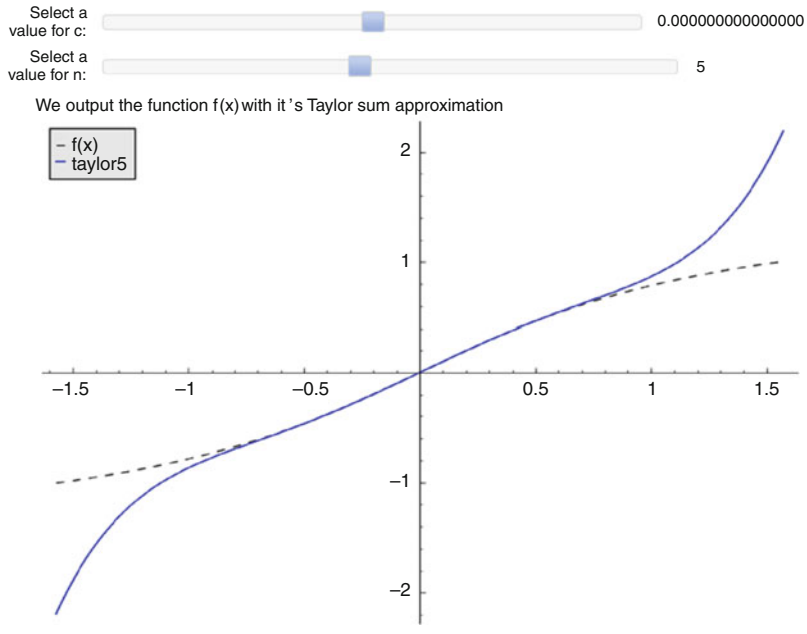
#graph all the plots together
show(p+p1+p5+p7)
```

Notice that in this case the partial sums of the Taylor series provide results that are more accurate for values of x that are closer to 1, rather than values close to 0.

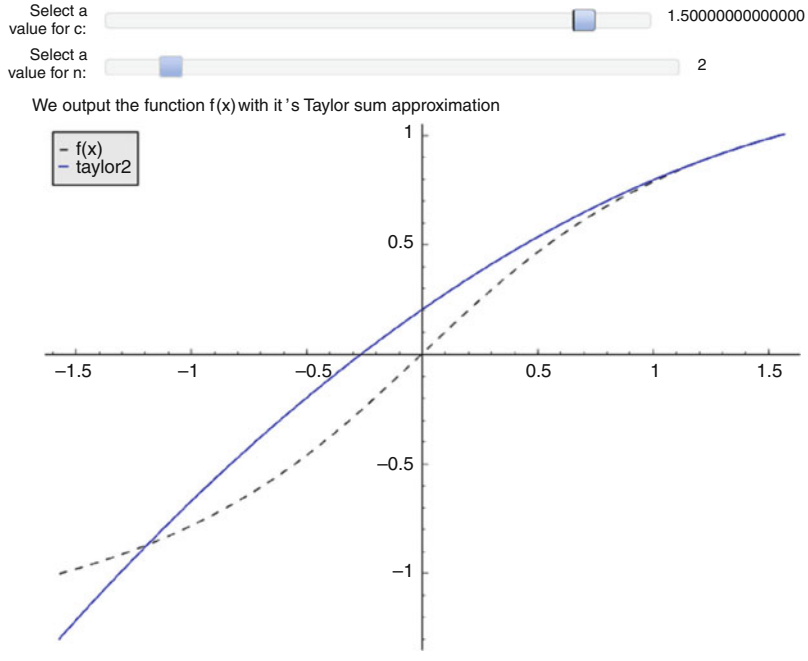
Below, we give the Sage code that will provide **Sage interacts** that can be used for an interactive visualization of the graphs seen above.

```
x = var('x')
f(x) = arctan(x) #this is the function we'll use
@interact
def echo_input(c = slider(vmin=-2, vmax=2,
                          step_size=0.5, default=0,
                          label="Select a value for c: "),
               n = slider(vmin=1, vmax=10,
                          step_size=1, default=5,
                          label="Select a value for n: ")):
    print "We output the function f(x)\
with it's Taylor sum approximation"
    #the original plot
    p1=plot(f(x),-pi/2,pi/2,
            color= 'black',
            linestyle='--',
            legend_label='f(x)')
    #compute the Taylor sum of degree n
    taylorN(x) = f(x).taylor(x, c, n)
    p2=plot(taylorN(x),-pi/2,pi/2,
            color= 'blue',
            legend_label="taylor"+str(n))
    (p1+p2).show()
```

and obtain:



Please feel free to try this code and move the two sliders to select different values of x and c.



It's time now to see the Mathematical reasoning behind Taylor series and the conditions that need to be satisfied in order for us to get a "good" approximation.

Let's start with a definition. As mentioned above, the general formula for the **Taylor series for a function f, about the point c** is given by

$$T(x) = f(c) + f'(c)(x-c) + \frac{f''(c)}{2!}(x-c)^2 + \cdots + \frac{f^{(n)}(c)}{n!}(x-c)^n + \cdots$$
$$= \sum_{k=0}^{\infty} \frac{f^{(k)}(c)}{k!}(x-c)^k, \tag{1.19}$$

provided that f, f', f'', f''', $f^{(4)}$, ... all exist at the point c.

Example 27. Find the first five partial sums of the Taylor series for the function $f(x) = \ln(x)$ around $c = 1$. Then use them to (intuitively) approximate value for $\ln(1.2)$.

Solution. Using (1.19) we obtain:

$$f(x) = \ln(x) \Longrightarrow f(1) = \ln(1) = 0$$
$$f'(x) = \frac{1}{x} \Longrightarrow f'(1) = 1$$
$$f''(x) = -\frac{1}{x^2} \Longrightarrow f''(1) = -1$$
$$f'''(x) = \frac{2}{x^3} \Longrightarrow f'''(1) = 2$$
$$f^{(4)}(x) = -\frac{6}{x^4} \Longrightarrow f^{(4)}(1) = -6$$

Hence

$$\ln(1.2) \approx T_0(1.2) \tag{1.20}$$
$$= f(1) = 0$$

$$\ln(1.2) \approx T_1(1.2) \tag{1.21}$$
$$= f(1) + f'(1)(1.2-1) = 0.2$$

$$\ln(1.2) \approx T_2(1.2) \tag{1.22}$$
$$= f(1) + f'(1)(1.2-1) + \frac{f''(1)}{2!}(1.2-1)^2 = 0.18$$

$$\ln(1.2) \approx T_3(1.2) \tag{1.23}$$
$$= f(1) + f'(1)(1.2-1) + \frac{f''(1)}{2!}(1.2-1)^2 + \frac{f'''(1)}{3!}(1.2-1)^3$$
$$\simeq 0.1826666667$$

$$\ln(1.2) \approx T_4(1.2) \tag{1.24}$$
$$= f(1) + f'(1)(1.2-1) + \frac{f''(1)}{2!}(1.2-1)^2$$

$$+\frac{f'''(1)}{3!}(1.2-1)^3+\frac{f^{(4)}(1)}{4!}(1.2-1)^4$$
$$\simeq 0.1822666667$$

The last answer is correct to three decimal places.

Using the following Sage code:

```
var('x')
f(x) = ln(x)
c  = 1


T0(x) =  f(x).taylor(x, c, 0)
print T0(1.2), " from " , T0(x)

T1(x) =  f(x).taylor(x, c, 1)
print T1(1.2), " from " , T1(x)

T2(x) =  f(x).taylor(x, c, 2)
print T2(1.2), " from " , T2(x)

T3(x) =  f(x).taylor(x, c, 3)
print T3(1.2), " from " , T3(x)

T4(x) =  f(x).taylor(x, c, 4)
print T4(1.2), " from " , T4(x)
```

we get similar results:

```
0 from 0
0.200000000000000 from x - 1
0.180000000000000 from -1/2*(x - 1)^2 + x - 1
0.182666666666667 from 1/3*(x - 1)^3 - 1/2*(x - 1)^2 + x - 1
0.182266666666667 from -1/4*(x - 1)^4 + 1/3*(x - 1)^3 - 1/2*(x -
1)^2 + x - 1
```

Note: loops are smarter! The same exact output could be achieved using the following alternative code:

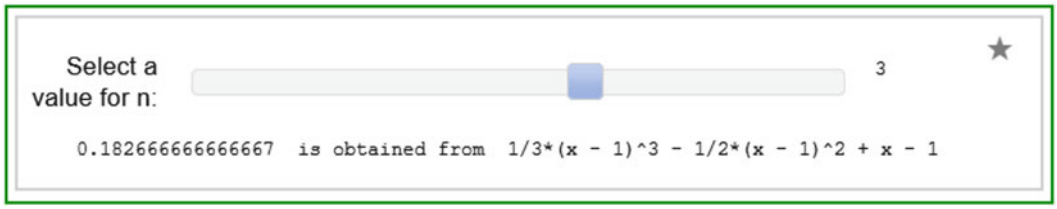
```
var('x')
f(x) = ln(x)
c = 1
for i in range(5):
   T(x)=f(x).taylor(x, c, i)
   print T(1.2), " from " , T(x)
```

Using the Sage interact:

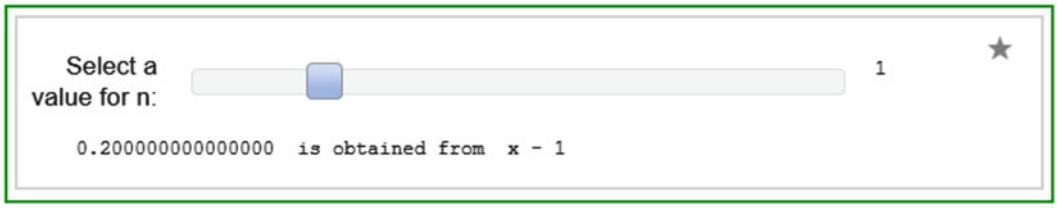
```
var('x')
f(x) = ln(x)
c = 1
@interact
def echo_input(i = slider(vmin=0, vmax=5,
                          step_size=1, default=2,
```

```
                                label="Select a value for n: ")):
    T = f(x).taylor(x, c, i)
    print T(1.2) , " is obtained from " , T(x)
```

one can obtain the following interactive representation:



Moving the slider to the left or right, one can obtain all the values shown above.



In the previous examples we used partial sums of the Taylor series for a function $f(x)$ to approximate the value of that function, at a given value of x. When can we say that $f(x)$ is the same or approximately the same as $T(x)$? And how large will the error be when we approximate $f(x)$ by $T(x)$? The answers to these questions are given in the next theorem:

Theorem 28 (Taylor's Theorem with Remainder). *If the function f is continuous and has continuous derivatives of order 1, 2, ..., (n+1) in some closed interval $[a, b]$, then for any x and c from $[a, b]$ we have that*

$$f(x) = \sum_{k=0}^{n} \frac{f^{(k)}(c)}{k!}(x-c)^k + E_n(x) \tag{1.25}$$

*where the **error term** E_n given by:*

$$E_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x-c)^{n+1},$$

for some ξ between c and x.

It is important to note that ξ depends on both values of c and x.
The polynomial

$$T_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(c)}{k!}(x-c)^k \tag{1.26}$$

is called the n**th order Taylor polynomial**. With this notation, the formula (1.25) becomes $f(x) = T_n(x) + E_n(x)$.

Proof. Let x, c from $[a, b]$ be fixed. The case $x = c$ is trivial (since it reduces to $f(x) = f(c)$). So let's assume that $x \neq c$.

There must be a K, depending on x such that

$$f(x) = T_n(x) + \frac{K}{(n+1)!}(x-c)^{n+1}$$
$$= \sum_{k=0}^{n} \frac{f^{(k)}(c)}{k!}(x-c)^k + \frac{K}{(n+1)!}(x-c)^{n+1}.$$

In order to prove the theorem, it remains to show that $K = f^{(n+1)}(\xi)$, for some ξ between c and x.

Let's define the helper function

$$H(t) = f(x) - \sum_{k=0}^{n} \frac{f^{(k)}(t)}{k!}(x-t)^k - \frac{K}{(n+1)!}(x-t)^{n+1}$$

with t between c and x inclusively.

Since $f(t), f'(t), \ldots, f^{(n)}(t)$ are all continuous we conclude that H is continuous on its domain.

Since f has continuous derivatives of order 1, 2, …, (n+1) we conclude that also H is differentiable and we have that

$$H'(t) = -\sum_{k=0}^{n} \frac{f^{(k+1)}(t)}{k!}(x-t)^k + \sum_{k=1}^{n} \frac{f^{(k)}(t)}{(k-1)!}(x-t)^{k-1} + \frac{K}{n!}(x-t)^n$$
$$= -\frac{f^{(n+1)}(t)}{n!}(x-t)^n + \frac{K}{n!}(x-t)^n$$

Furthermore, we have that $H(c) = H(x) = 0$.

Then, applying Rolle's Theorem, there must some ξ strictly between c and x such that $H'(\xi) = 0$. This implies that for some ξ strictly between c and x we have that

$$H'(\xi) = 0.$$

Hence,

$$-\frac{f^{(n+1)}(\xi)}{n!}(x-\xi)^n + \frac{K}{n!}(x-\xi)^n = 0$$

and since $x - \xi \neq 0$ we obtain

$$K = f^{(n+1)}(\xi).$$

■

The following examples are direct applications of Taylor's Theorem with Remainder:

Example 29. Prove formula (1.10), that is prove: $e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$, for $-\infty < x < \infty$.

Solution. Let $f(x) = e^x$, and $c = 0$. Let's consider x from some interval $[a, b]$. Then we have that $f^{(k)}(x) = e^x$, for $k = 1, 2, \ldots, n, n+1$. Since all the conditions needed in Taylor's Theorem with Remainder are satisfied, we can apply it and get

$$\begin{aligned} f(x) &= \sum_{k=0}^{n} \frac{e^0}{k!}(x-0)^k + E_n(x) \\ &= \sum_{k=0}^{n} \frac{x^k}{k!} + E_n(x) \end{aligned} \tag{1.27}$$

where $E_n(x) = \frac{e^\xi}{(n+1)!}x^{n+1}$. Since

$$0 \le |E_n(x)| = \left|\frac{e^\xi}{(n+1)!}x^{n+1}\right| \le \frac{e^b}{(n+1)!}|x|^{n+1} \to 0, \text{ as } n \to \infty,$$

we obtain that $E_n(x) \to 0$, as $n \to \infty$. If we take limit as $n \to \infty$ on both sides of (1.27) we obtain formula (1.10). ■

Example 30. Let's assume we want to use the following approximation

$$e^x \approx T_5(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!}, \text{ for } x \text{ in } [-1, 1].$$

Find an upper bound (worst case scenario) for the error resulted from this approximation.

Solution. Using Taylor's Theorem with Remainder, with $f(x) = e^x$, and $c = 0$ we obtain

$$0 \le |E_5(x)| = \left|\frac{e^\xi}{6!}x^6\right| \le \frac{e}{6!} \cdot 1 < 0.004$$

■

Example 31. Let's assume we want to use the following approximation

$$\begin{aligned} e^x \approx T_{10}(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} \\ + \frac{x^6}{6!} + \frac{x^7}{7!} + \frac{x^8}{8!} + \frac{x^9}{9!} + \frac{x^{10}}{10!}, \text{ for } x \text{ in } [-1, 1]. \end{aligned}$$

Find an upper bound (worst case scenario) for the error resulted from this approximation.

Solution. Using Taylor's Theorem with Remainder, with $f(x) = e^x$, and $c = 0$ we obtain

$$0 \le |E_{10}(x)| = \left|\frac{e^\xi}{11!}x^{11}\right| \le \frac{e}{11!} \cdot 1 < .00000007.$$

■

Example 32. Find a polynomial that approximates e^x on the interval $[-1, 1]$ with an error no greater than 10^{-5}, for any x in $[-1, 1]$.

Solution. Using Taylor's Theorem with Remainder, with $f(x) = e^x$, and $c = 0$ we obtain

$$|E_n(x)| = \left| \frac{e^{\xi}}{(n+1)!} x^{n+1} \right| \leq \frac{e}{(n+1)!} \leq 10^{-5}.$$

This implies that $n \geq 8$. Hence we would use the following approximation

$$e^x \approx 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \frac{x^6}{6!} + \frac{x^7}{7!} + \frac{x^8}{8!}.$$

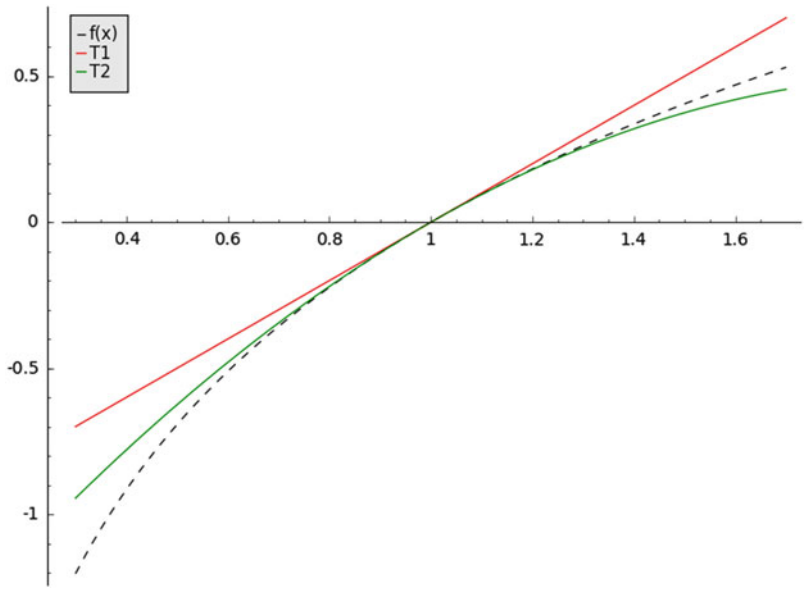
■

Example 33. Produce linear and quadratic Taylor polynomial approximations for $f(x) = \ln x$, in the interval $[0.3, 1.7]$. Graph the function and its polynomial approximations.

Solution. Using Taylor's Theorem with Remainder, with $f(x) = \ln x$, and $c = 1$ we obtain

$$\begin{aligned} T_1(x) &= 0 + 1 \cdot (x-1) \\ &= x - 1 \\ T_2(x) &= 0 + 1 \cdot (x-1) + \frac{-1}{2}(x-1)^2 \\ &= -0.5x^2 + 2x - 1.5. \end{aligned}$$

We can obtain the same results using Sage. Below you can find the graph as well as the Sage code:



■

```
#this part of the code computes the Taylor polynomials
var('x')
f(x) = ln(x)
```

```
c  = 1

T1(x) =  f(x).taylor(x, c, 1)
T2(x) =  f(x).taylor(x, c, 2)

T1(x), T2(x).full_simplify()
```

to obtain

```
(x - 1, -1/2*x^2 + 2*x - 3/2)
```

```
#next part of the code will generate the graph
p=plot(f(x),0.3,1.7, color= 'black', linestyle='--',
       legend_label='f(x)')
p1=plot(T1(x), 0.3,1.7, color = 'red', legend_label='T1')
p2=plot(T2(x), 0.3,1.7, color = 'green', legend_label='T2')
show(p+p1+p2)
```

We can also obtain the previous results using Sage interacts:

```
var('x')
f(x) = ln(x)
c = 1
@interact
def echo_input(i = slider(vmin=0, vmax=5,
                          step_size=1, default=2,
                          label="Select a value for n: ")):
    print "T(x) = " , (f(x).taylor(x, c, i)).full_simplify()
```

for:

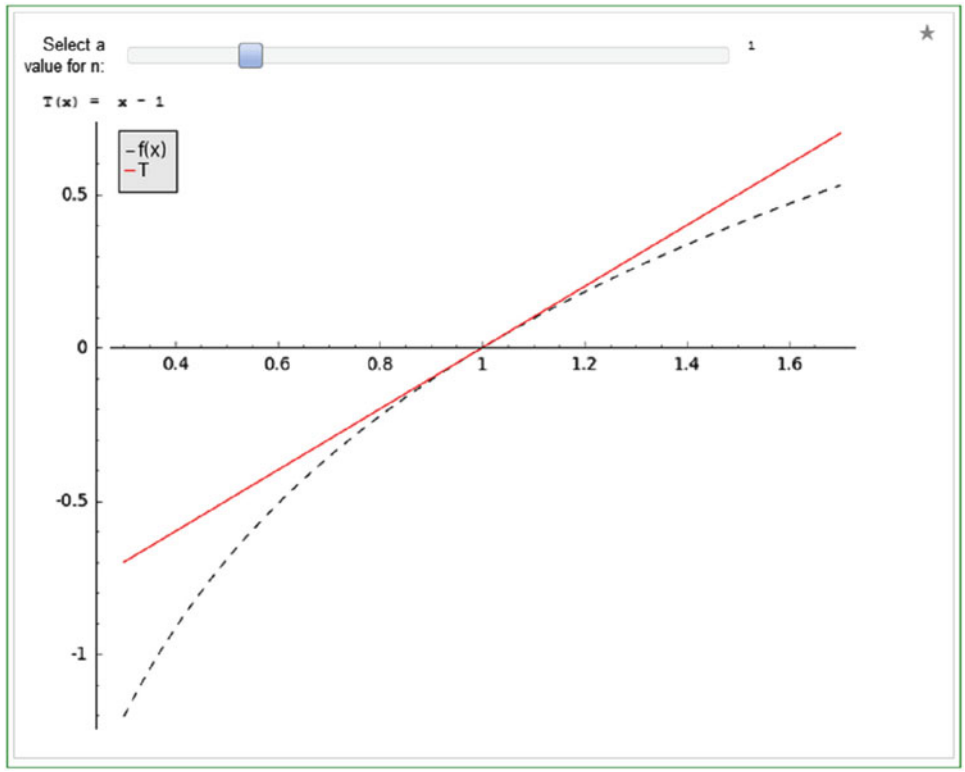


and

```
var('x')
f(x) = ln(x)
c = 1
@interact
def echo_input(i = slider(vmin=0, vmax=5,
                          step_size=1, default=2,
                          label="Select a value for n: ")):
    print "T(x) = " , (f(x).taylor(x, c, i)).full_simplify()
    p=plot(f(x),0.3,1.7, color= 'black', linestyle='--',
       legend_label='f(x)')
    T1(x) =  f(x).taylor(x, c, i)
```

```
T=plot(T1(x), 0.3,1.7, color = 'red', legend_label='T')
show(p+T)
```

for:



An important application of Taylor's Theorem with Remainder is the Mean Value Theorem, which is obtained when we let $n = 0$:

Theorem 34 (Mean-Value Theorem). *If the function f is continuous in some closed interval $[a, b]$, and differentiable on (a, b) then*

$$f(b) = f(a) + f'(\xi)(b - a) \tag{1.28}$$

for some ξ in (a, b).

This theorem can be used when we want to obtain an approximation for the value of $f'(x)$, with x in (a, b). We could use that

$$f'(x) \approx \frac{f(b) - f(a)}{b - a}, \text{ for any } x \text{ in } (a, b).$$

And this value is exact for at least one value, ξ, in (a, b).

Of course, the smaller the interval (a, b) is, the more accurate answers we expect to get for $f'(x)$. This should not be surprising, given the definition of $f'(x)$.

Taylor's Theorem is frequently written in the following *equivalent* form (by changing x into $x + h$, and c into x):

Theorem 35 (Taylor's Theorem with Remainder (2)). *If the function f is continuous and has continuous derivatives of order 1, 2, ..., (n+1) in some closed interval $[a, b]$, then for any x and $x + h$ from $[a, b]$ we have that*

$$f(x+h) = \sum_{k=0}^{n} \frac{f^{(k)}(x)}{k!} h^k + E_n(x+h) \tag{1.29}$$

*where the **error term** E_n given by:*

$$E_n(x+h) = \frac{f^{(n+1)}(\xi)}{(n+1)!} h^{n+1}, \tag{1.30}$$

for some ξ between x and $x + h$.

The n**th order Taylor polynomial** can be rewritten as:

$$T_n(x+h) = \sum_{k=0}^{n} \frac{f^{(k)}(x)}{k!} h^k. \tag{1.31}$$

Instead of using values of c that are close to x, in the new form one could use values of h that are close to 0, which may be more intuitive in some applications (although they are equivalent!). In many applications, this form provides a much more convenient notation. We will use this form quite frequently throughout this book.

We can easily see, from formula (1.30), that as $h \to 0$, so does the error term, $E_n \to 0$. But this is just a part of the information given by that formula. That formula says so much more than that. It also gives us information about the **rate of convergence**. It tells us how fast it converges to 0. It gives us the "speed", the "rapidity" with which the error term approaches 0, as $h \to 0$.

For this, we introduce the following definition of the *big O notation*.

Definition 36 (Big O Notation). We say that a quantity

$$E = \mathcal{O}(h^n), \tag{1.32}$$

(read it as: "E is big Oh h^n") if you can find some constant C, independent of h, such that, for all small enough values of $|h|$

$$|E| \leq C \cdot h^n. \tag{1.33}$$

From this definition, it follows that $E, \frac{E}{h}, \frac{E}{h^2}, \ldots, \frac{E}{h^{n-1}} \to 0$, as $h \to 0$. It also tells us that $E \to 0$ at least as fast as $h^n \to 0$. This definition is very handy when we do error estimation.

Using Taylor's Theorem and the Big O Notation we have

$$\begin{aligned} f(x+h) &= f(x) + f'(\xi)h \\ &= f(x) + \mathcal{O}(h) \end{aligned}$$

$$\begin{aligned} f(x+h) &= f(x) + f'(x)h + \frac{f''(\xi)}{2!}h^2 \\ &= f(x) + f'(x)h + \mathcal{O}(h^2) \end{aligned}$$

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(\xi)}{3!}h^3$$
$$= f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \mathcal{O}(h^3)$$

We can see how Big O Notation simplifies our notation, without giving up the essential part, the rate of convergence.

Using this notation and (1.30) we have

$$E_n = \mathcal{O}(h^{n+1}). \tag{1.34}$$

A simplified notation for $\mathcal{O}(h^n)$ that is being used in some textbooks is $\mathcal{O}(n)$. We consider this notation somewhat misleading and therefore we will try avoiding it throughout this book.

Both notations imply that as you decrease h into half, the quantity (usually the error) that is $\mathcal{O}(h^n)$ will get decreased by a factor of 2^n. If you decrease h by a factor of 10, then the quantity will be decreased by a factor of 10^n.

Example 37. Expand $ln(1+h)$ into powers of h. Then compute ln(1.00001). Estimate the maximum error involved from using a third order Taylor polynomial.

Solution. Let $f(x) = \ln(x)$. Then we have that $f'(x) = 1/x$, $f''(x) = -1/x^2$, $f'''(x) = 2/x^3$, $f^{(4)}(x) = -6/x^4$. Using

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \frac{f^{(4)}(\xi)}{4!}h^4$$

we get

$$\ln(1+h) = 0 + 1\cdot h - \frac{1}{2!}h^2 + \frac{2}{3!}h^3 - \frac{6}{4!\xi^4}h^4$$
$$= h - \frac{h^2}{2} + \frac{h^3}{3} - \frac{h^4}{4\xi^4},$$

with ξ unknown, between 1 and $1+h$.

If we let $h = 0.00001$, we obtain that

$$\ln(1.00001) \approx 0.00001 - \frac{0.00001^2}{2} + \frac{0.00001^3}{3}$$
$$= 0.000009999950000333333333\ldots..$$

and for $h = -0.00001$, we obtain that

$$\ln(0.99999) \approx -0.00001 - \frac{0.00001^2}{2} - \frac{0.00001^3}{3}$$
$$= -0.000010000050000000.$$

In the first case, the error cannot exceed:

$$\left|\frac{h^4}{4\xi^4}\right| = \left|\frac{0.00001^4}{4\xi^4}\right|$$

$$< \left| \frac{0.00001^4}{4} \right|$$
$$= 0.00000\ 00000\ 00000\ 00000\ 25.$$
$$= 2.5 * 10^{-21}$$

Hence, our approximation should be correct to at least 20 decimal places.

A quick check on Sage seems to disagree with this:

```
sage: ln(1.00001)-0.00000999995000033333333
```

It displays

```
6.55078340747531e-17
```

This is because of the default rounding used by Sage.

But the following code:

```
sage: ln(1.00001.n(500))-0.00000999995000033333333
```

displays a more accurate answer:

```
-2.4989126305226966 1e-21
```

■

1.8 Alternating Series

We recall from Calculus that a series

$$\sum_{k=0}^{\infty} a_k = a_0 + a_1 + \cdots + a_n + \cdots$$

is **convergent** to some value V if the sequence of partial sums $\{S_n\}_n$, where

$$S_n = \sum_{k=0}^{n} a_k = a_0 + a_1 + \cdots + a_n$$

converges to V. That is, if

$$\lim_{n\to\infty} S_n = V.$$

Theorem 38 (Alternating Series Theorem). *If $a_0 \geq a_1 \geq a_2 \geq \cdots \geq a_n \geq \cdots \geq 0$ and $a_n \to 0$, as $n \to 0$ then the following alternating series*

$$\sum_{k=0}^{\infty} (-1)^k a_k = a_0 - a_1 + a_2 - a_3 + \cdots \tag{1.35}$$

converges. Let $V := \sum_{k=0}^{\infty} (-1)^k a_k$. Then we have the following error estimation:

$$|V - S_n| \leq a_{n+1}. \tag{1.36}$$

That means that if we sum only terms up to a_n, and omit all the remaining terms, then the error of approximation is at most as large as a_{n+1}.

Example 39. How many terms are needed if we want to compute cos(1) with an error strictly less that 10^{-6}.

Solution. From (1.4) we have that

$$\cos(1) = 1 - \frac{1}{2!} + \frac{1}{4!} - \frac{1}{6!} + \frac{1}{8!} - \cdots$$

If we stop at the term $\frac{1}{(2n)!}$, then the error will be at most $\frac{1}{(2n+2)!}$. Solving the problem means solving the following inequality

$$\frac{1}{(2n+2)!} < 10^{-6}.$$

We can solve this problem either by trying several values for n, until the previous inequality is satisfied, or using the Sage code below. There are other Mathematical ways to solve this inequality but having Sage available, we decided use a very simple method to solve it. In either case, we should get that $n = 4$ is the smallest positive integer value that satisfies our inequality, and conclude that

$$\cos(1) \approx 1 - \frac{1}{2!} + \frac{1}{4!} - \frac{1}{6!} + \frac{1}{8!}$$

with an error less that 10^{-6}. The error is actually at most $\frac{1}{10!}$ which is less than $2.756 * 10^{-7}$.

The Sage code below starts with $n = 1$, and keep increasing n as long as the previous inequality is not yet satisfied. Once we find a satisfying n, we stop and print it. ■

```
n=1;     #we start with n=1
         #then try all values until our inequality is satisfied
while (1/factorial(2*n+2) >= 10^-6 ):
   n = n+1
print n;
```

Example 40. How many terms are needed if we want to compute arctan(1) with an error strictly less that 10^{-6}.

Solution. From (1.6) we have that

$$\cos(1) = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots$$

If we stop at the term $\frac{1}{2n+1}$, then the error will be at most $\frac{1}{2n+3}$. Solving the problem means solving the following inequality

$$\frac{1}{2n+3} < 10^{-6}.$$

Hence

$$2n + 3 > 1,000,000$$

or

$$n > 499998.5.$$

Therefore we need to include the term with $n = 499,999$ (which is 500,000th term) if we need to approximate arctan 1 with an error less than 10^{-6}. ■

1.9 Sage Interacts

A very useful tool that we did not fully explore in this book is "Sage interact" (also called Sage Interactions).

The following Sage code will create a Sage interact for computing and graphing Taylor polynomials of degrees up to 10. The order can be chosen using a slider. We prepared a slider that can take any value between 1 and 10, inclusively. Then, it will compute, display, and graph the original function, as well as the corresponding Taylor polynomial, or the order selected by the user.

```
html('<h2>Taylor Polynomials using Sage Interactions</h2>')
#select the interval [a,b]
a = -3
b =  3

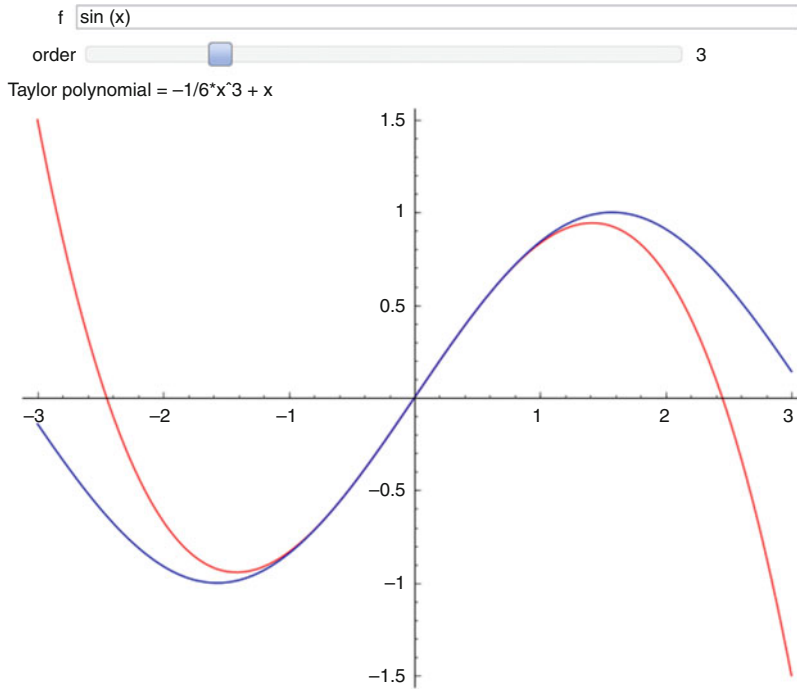
@interact
#default function is sin(x)
#order:  interval=[1,10], step = 1, default = 3
def taylor_through_sage_interaction(f = input_box(default=sin(x)),
                                    order = slider(1,10,1,3) ):
    var('x')

    #compute the taylor polynomial of order given by the slider
    taylor_pol(x) =f.taylor(x,0, order )

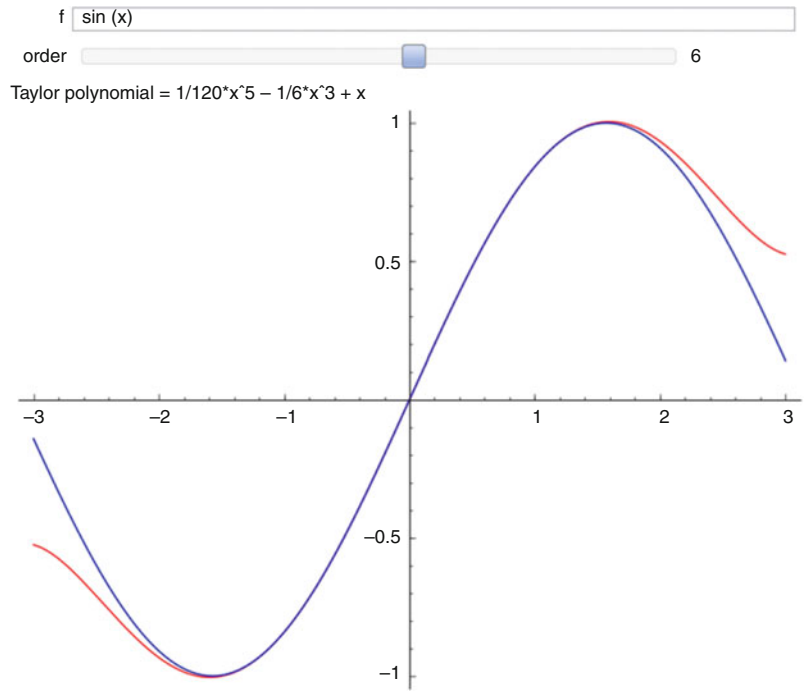
    #plot the taylor polynomial
    p = plot( taylor_pol(x) , (x, a, b), color = 'red' )

    #plot the original function
    p+= plot( f , (x, a, b), color = 'blue' )
    print 'Taylor polynomial = ' + str(taylor_pol(x))
    show(p)
```

The output should look similar to:



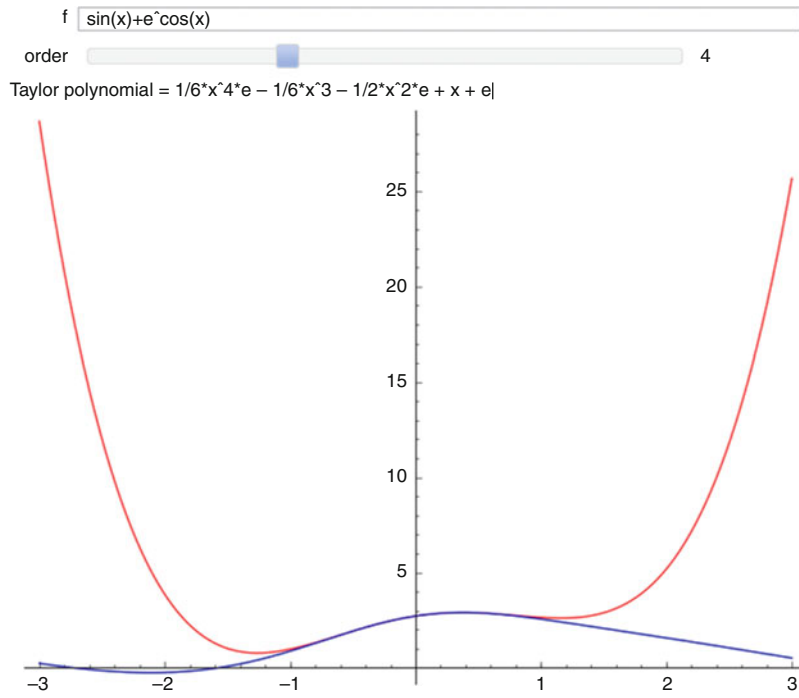
Turning the slider to the value 6, we obtain



Changing the function f to $\sin(x) + e^{\cos(x)}$, and the order to 4, we obtain the Taylor polynomial:

```
Taylor polynomial = 1/6*x^4*e - 1/6*x^3 - 1/2*x^2*e + x + e
```

and the graph



1.10 Exercises

1. Consider $x = 0.000147$ rounded to $\hat{x} = 0.00015$. Find the number of significant digits, absolute errors, and relative errors?
2. Consider $x = 0.00237$ rounded to $\hat{x} = 0.0024$. Find the number of significant digits, absolute errors, and relative errors?
3. Consider $x = 123.4567$ rounded to $\hat{x} = 123.46$. Find the number of significant digits, absolute errors, and relative errors?
4. Consider $x = 314.57823$ rounded to $\hat{x} = 314.59$. Find the number of significant digits, absolute errors, and relative errors?
5. For each of the following numbers, what are the results of rounding them? 0.0418, 0.0459, 0.0451, 0.0455, 0.0465.
6. For each of the following numbers, what are the results of chopping them? 0.0418, 0.0459, 0.0451, 0.0455, 0.0465.
7. Find the linear and quadratic Taylor polynomials for the function $f(x) = \sqrt{x}$, using $c = 1$ as the point of expansion. Graph the function and these Taylor polynomials on the same set of axes.
8. Find the linear and quadratic Taylor polynomials for the function $f(x) = e^{\cos x}$, using $c = 0$ as the point of expansion. Graph the function and these Taylor polynomials on the same set of axes.

9. Find the linear and quadratic Taylor polynomials for the function $f(x) = e^{\sin x}$, using $c = 0$ as the point of expansion. Graph the function and these Taylor polynomials on the same set of axes.
10. Find the linear and quadratic Taylor polynomials for the function $f(x) = \log(1 + e^x)$, using $c = 0$ as the point of expansion. Graph the function and these Taylor polynomials on the same set of axes.
11. Find the Taylor series expansion for the function $\sin(x)$, around $c = 0$.
12. Find the Taylor series expansion for the function $\cos(x)$, around $c = 0$.
13. Write the Taylor series expansion for the function $f(x) = x^3 + 3x^2 - 2x + 3$, using $c = 2$ as the point of expansion.
14. Write the Taylor series expansion for the function $f(x) = x^3 + 3x^2 - 2x + 3$, using $c = 3$ as the point of expansion.
15. Find the

    a. Taylor polynomial of degrees 1 through 5, for the function $f(x) = e^{x^2}$ using $c = 0$ as the point of expansion
    b. Taylor polynomial of degrees 1 through 5, for the function $f(x) = e^{x^2}$ using the Taylor polynomials for $f(x) = e^x$. Compare the results with part a).

16. Find the

    a. Taylor polynomial of degrees 1 through 5, for the function $f(x) = \cos(x^2)$ using $c = 0$ as the point of expansion
    b. Taylor polynomial of degrees 1 through 5, for the function $f(x) = \cos(x^2)$ using the Taylor polynomials for $f(x) = \cos(x)$. Compare the results with part a).

17. The function $f(x) = \frac{\sin x}{x}$ is undefined at $x = 0$. Use Taylor series for $\sin(x)$ to determine a natural definition for $f(0)$.
18. The function $f(x) = \frac{e^x - 1}{x}$ is undefined at $x = 0$. Use Taylor series for $\sin(x)$ to determine a natural definition for $f(0)$.
19. How can values of the function $f(x) = \sin(1 + x)$ be accurately computed for values of x close to 0?
20. How can values of the function $f(x) = \cos(2 + x)$ be accurately computed for values of x close to 0?
21. If the sine series is to be used in computing $\sin 1$ with an error less than 10^{-7}, find the number of terms needed to be used from the series.
22. If the ln series is to be used in computing $\ln 2$ with an error less than 10^{-7}, find the number of terms needed to be used from the series.
23. If the arctan series is to be used in computing $\arctan 2$ with an error less than 10^{-8}, find the number of terms needed to be used from the series.
24. If the e^x series is to be used in computing e^{-1} with an error less than 10^{-9}, find the number of terms needed to be used from the series.

1.11 Computer Exercises

1. Use Sage to solve the following system of equations:

$$\begin{cases} x - 2y = 7 \\ 3x + y = 0 \end{cases}.$$

2. Use Sage to solve the following system of equations:

$$\begin{cases} 2x - 5y = 17 \\ 3x + 7y = 21 \end{cases}.$$

3. Create Sage code that will define the function $f(x) = 2e^{3x} - 5\cos(x) + 3$, and evaluate it at the points $x = 0$ and $x = \pi$.
4. Create Sage code that will define the function $f(x) = e^{x^2} - 3\sin(x^2) + 2$, and evaluate it at the points $x = 0$ and $x = \sqrt{\frac{\pi}{2}}$.
5. Use Sage to graph the function $f(x) = 2e^{3x} - 5\cos(x) + 3$.
6. Use Sage to graph the function $f(x) = e^{x^2} - 3\sin(x^2) + 2$.
7. Use Sage to compute the following sum $f(x) = \sum_{i=0}^{2015} \frac{1}{i^2+i+1}$.
8. Use Sage to compute the following sum $f(x) = \sum_{i=0}^{512} \frac{1}{i^3+2i^2+8}$.
9. Use Sage to find the number of primes between 1945 and 2015.
10. Use Sage to find the number of primes between 123,456 and 123,456,789.
11. Use Sage to smallest 10 prime numbers larger than 2016.
12. Use Sage to smallest 20 prime numbers larger than 123,456,789.
13. Use Sage to print the function $f(x) = \cos(x)$, for x in $[-2\pi, 2\pi]$.
14. Use Sage to print the function $f(x) = \sin(x)$, for x in $[-2\pi, 2\pi]$.
15. Use Sage to print the function $f(x) = \sin(\sqrt[3]{x})$, for x in $[-20, 20]$.
16. Use Sage to print the function $f(x) = \cos(\sqrt[3]{x})$, for x in $[-20, 20]$.
17. Use Sage to find 10th order Taylor polynomial of the function $f(x) = \cos(x)$. Then use this polynomial to find an approximate value for $\cos(0.001)$.
18. Use Sage to find 8th order Taylor polynomial of the function $f(x) = \sin(x)$. Then use this polynomial to find an approximate value for $\sin(0.003)$.
19. Use Sage to find 8th order Taylor polynomial of the function $f(x) = \sin(x)$. Then use this polynomial to find an approximate value for $\sin(7.001)$.
20. Use Sage to find 10th order Taylor polynomial of the function $f(x) = \cos(x)$. Then use this polynomial to find an approximate value for $\cos(8.003)$.
21. Use Sage to find 5th order Taylor polynomial for the function $f(x) = \cos(\sqrt[3]{x})$. Then use this polynomial to find an approximate value for $\cos(\sqrt[3]{2})$.
22. Use Sage to plot on the same graph, the function $f(x) = \sin(x)$, and the Taylor's polynomials up to order 5, around $c = 0$.
23. Use Sage to plot on the same graph, the function $f(x) = \cos(x)$, and the Taylor's polynomials up to order 5, around $c = 0$.
24. Use Sage to plot on the same graph, the function $f(x) = \cos(x)$, and the Taylor's polynomials up to order 5, around $c = 7$.
25. Use Sage to plot on the same graph, the function $f(x) = \ln(x)$, and the Taylor's polynomials up to order 5, around $c = 7$.
26. Use Sage to find the linear and quadratic Taylor polynomials for the function $f(x) = \sqrt{x}$, using $c = 1$ as the point of expansion. Graph the function and these Taylor polynomials on the same set of axes.
27. Use Sage to find the linear and quadratic Taylor polynomials for the function $f(x) = e^{\cos x}$, using $c = 0$ as the point of expansion. Graph the function and these Taylor polynomials on the same set of axes.
28. Use Sage to find the linear and quadratic Taylor polynomials for the function $f(x) = e^{\sin x}$, using $c = 0$ as the point of expansion. Graph the function and these Taylor polynomials on the same set of axes.

29. Use Sage to find the linear and quadratic Taylor polynomials for the function $f(x) = \log(1+e^x)$, using $c = 0$ as the point of expansion. Graph the function and these Taylor polynomials on the same set of axes.
30. If the sine series is to be used in computing $\sin 1$ with an error less than 10^{-7}, use Sage to find the number of terms needed to be used from the series.
31. If the ln series is to be used in computing $\ln 2$ with an error less than 10^{-7}, use Sage to find the number of terms needed to be used from the series.
32. If the arctan series is to be used in computing $\arctan 2$ with an error less than 10^{-8}, use Sage to find the number of terms needed to be used from the series.
33. If the e^x series is to be used in computing e^{-1} with an error less than 10^{-9}, use Sage to find the number of terms needed to be used from the series.

References

1. Abramowitz, M., Stegun, I.A. (eds.): Handbook of Mathematical Functions with Formulae, Graphs, and Mathematical Tables. Dover, New York (1972)
2. Anastassiou, G.A., Iatan, I.F.: Intelligent Routines II: Solving Linear Algebra and Differential Geometry with Sage. Springer, Cham (2014)
3. Atkinson, K., Han, W.: Elementary Numerical Analysis, 3rd edn. Wiley, New York (2003)
4. Beezer, R.A.: Sage for abstract algebra. http://abstract.pugetsound.edu/Sage-aata.html (2013)
5. Blanco-Silva, F.J.: Learning SciPy for Numerical and Scientific Computing. Packt Publishing, Birmingham/Mumbai (2013)
6. Cheney, W., Kincaid, D.: Numerical Mathematics and Computing, 7th edn. Brooks/Cole: Cengage Learning, Boston (2013)
7. Epperson, J.F.: An Introduction to Numerical Methods and Analysis, revised edition. Wiley, Hoboken (2007)
8. Gaddis, T.: Starting Out with Python, 2nd edn. Addison-Wesley, Boston (2012)
9. Rivlin, T.J.: An Introduction to the Approximation of Functions. Dover, New York (2003)
10. Sage Cell Server: http://sagecell.sagemath.org/. Retrieved 2014
11. Stein, W.A., et al.: Sage mathematics software (Version 6.3). The sage development team. Sage official webpage: http://www.sagemath.org (2014)
12. Stein, W.A.: Sage for power users. http://www.wstein.org/books/sagebook/sagebook.pdf (2012)
13. Sullivan, M., Miranda, K.: Calculus: Early Transcendentals. W. H. Freeman and Company, New York (2014)
14. Webster's American English Dictionary: http://www.merriam-webster.com/dictionary. Retrieved 2014
15. Weierstrass, K.: Uber die analytische Darstellbarkeit sogenannter willkurlicher Functionen einer reellen Veranderlichen. Sitzungsberichte der Akademie zu Berlin **2**, 633–639 (1885)

Chapter 2
Solving Nonlinear Equations

In this chapter we'll deal with finding (or rather approximating) the solution of an equation $f(x) = 0$. This is a very common problem in Applied Mathematics, but finding the exact solution is not always an easy task.

A real number r that satisfies the relation $f(r) = 0$ is called a **root** (or **a zero**, or **a solution**) for the equation $f(x) = 0$.

Solving for **linear equations** of type $ax + b = 0$ is a rather trivial job. We have formulae to deal will solving **quadratic** and **cubic equations**, such as $ax^2 + bx + c = 0$ or $ax^3 + bx^2 + cx + d = 0$. But what about equations such as: $2^{x^2} - 10x + 1 = 0$? Or $3.7x^8 - 7.5x^3 + 3x - 10.3 = 0$? Or $2^x - 3x^e = 0$? How would you solve these equations?

In this chapter we will learn a few methods that will help us "solve" any nonlinear equation. Although "*to solve*" means to find every possible solution of a given equation, the methods presented in this chapter will help us obtain only one solution at a time, and although usually not exact, we can get these answers to any desired degree of accuracy.

We will focus here on both the Mathematical reasoning of these methods and the corresponding algorithms that we will implement in Sage.

Notation: in this chapter, by r we will denote the exact solution of the equation we are trying to solve. If there are more than one solutions, then it should be clear from the context which one r refers to.

2.1 Bisection Method

In this section we will present the first method, which although not the fastest, it is one of the simplest, and its convergence is guaranteed.

So let us start by reminding you the Intermediate Value Theorem, a result that can be found in virtually any Calculus textbook (see for example [9]):

The original version of this chapter was revised. An erratum to this chapter can be found at DOI 10.1007/978-3-319-16739-8_8

Electronic supplementary material The online version of this chapter (doi: 10.1007/978-3-319-16739-8_2) contains supplementary material, which is available to authorized users.

© Springer International Publishing Switzerland 2015

G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8_2

Theorem 41 (Intermediate Value Theorem). *Let f be a continuous function on the closed interval $[a, b]$ such that $f(a) \neq f(b)$. Then, for any value K strictly between $f(a)$ and $f(b)$, there is at least one r in (a, b) such that $f(r) = K$.*

In particular, we obtain the foundation for the Bisection Method

Corollary 42. *Let f be a continuous function on the closed interval $[a, b]$ such that $f(a) * f(b) < 0$. Then there is at least one r in (a, b) such that $f(r) = 0$.*

Given the previous Corollary, it should come at no surprise that in this section we only consider real-valued functions that are continuous on a closed and bounded interval $[a, b]$ such that $f(a) * f(b) < 0$. These will guarantee that we have at least one solution in the interval (a, b).

The algorithm for the Bisection Method, which is described below, simply means using the previous Corollary repeatedly until we obtain a solution with the desired degree of accuracy.

2.1.1 The Algorithm

Let f be a continuous function on the closed interval $[a, b]$ such that $f(a) * f(b) < 0$. We are trying to find a solution of the equation $f(x) = 0$ in the interval $[a, b]$. Here is how we do it:

We first compute $f\left(\frac{a+b}{2}\right)$. Then, exactly one of the following is true:

- If $f\left(\frac{a+b}{2}\right) = 0$ then we are done. We found a solution, $r = \frac{a+b}{2}$.
- Otherwise, if $f(a) * f\left(\frac{a+b}{2}\right) < 0$ then we repeat this process and search again for a solution, in the interval $\left[a, \frac{a+b}{2}\right]$.
- Otherwise, $f\left(\frac{a+b}{2}\right) * f(b) < 0$, we repeat this process and search again for a solution, in the interval $\left[\frac{a+b}{2}, b\right]$.

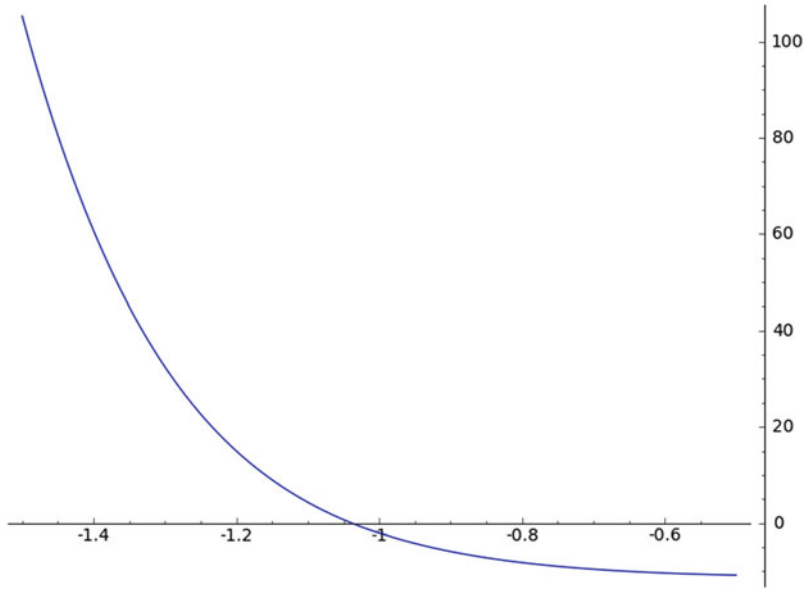
We then keep repeating these iterations until either we found a solution, or the length of the interval is less than some predefined error bound.

So as we could see above, this method starts with an interval $[a, b]$. If a solution is not found in the middle, at $x = \frac{a+b}{2}$ it then "bisects" the interval $[a, b]$ into two equal subintervals: $\left[a, \frac{a+b}{2}\right]$ and $\left[\frac{a+b}{2}, b\right]$. Of these two, it is guaranteed that at least one of them contains a solution. You discard the other one. And keep on halving these intervals until you find a solution (either exactly, or within a predefined error bound).

For example, let's try to find a solution for the following equation: $3.7x^8 - 7.5x^3 + 3x - 10.3 = 0$. In this case, $f(x) = 3.7x^8 - 7.5x^3 + 3x - 10.3$. We can either try two random values for a and b or we can use a graphing calculator to get a starting interval $[a, b]$. Let's try using a graph. In Sage (see [6–8]) (and adjusting the graphing interval as needed), we could use the following code:

```
sage: var('x')
sage: f(x) = 3.7*x^8-7.5*x^3+3*x-10.3
sage: p=plot(f(x), (x,-1.5, -0.5))
sage: p.show()
```

And obtain



From this graph, we can see that a solution exists in the interval $[-1.5, -0.5]$.

Since f is continuous on $[-1.5, -0.5]$, $f(-1.5) > 0$ and $f(-0.5) < 0$, we can see that all the conditions for the Bisection Method are satisfied and we can use it to estimate the zero in this interval. Let's consider the following error bound: 0.05. If we stop once the interval has the length less than 0.05, then taking the midpoint of this interval will yield an estimation of the solution with an absolute error less than $0.05/2 = 0.025$. Below we apply the Bisection Method:

Since $f\left(\frac{-1.5+(-0.5)}{2}\right) < 0$, we next consider the first half of the interval $[-1.5, -0.5]$, which is $[-1.5, -1]$.

Then, since $f(\frac{-1.5+(-1)}{2}) > 0$, we next consider the second half of $[-1.5, -1]$, that is $[-1.25, -1]$.

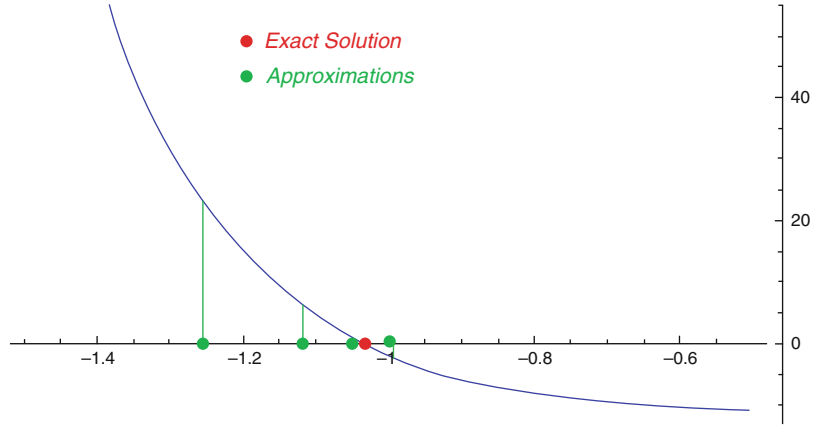
Then, since $f(\frac{-1.25+(-1)}{2}) > 0$, we next consider the second half of $[-1.25, -1]$, that is $[-1.125, -1]$.

Since $f(\frac{-1.125+(-1)}{2}) > 0$, we next consider the second half of $[-1.125, -1]$, that is $[-1.0625, -1]$.

Since $f(\frac{-1.0625+(-1)}{2}) < 0$, we next consider the second half of $[-1.0625, -1]$, that is $[-1.0625, -1.03125]$.

But since the length of the interval $[-1.0625, -1.03125]$ is less than 0.05 we can stop and estimate the solution to be it's midpoint: $r = \frac{-1.0625+(-1.03125)}{2} = -1.046875$. Obviously r is less than 0.05/2 away from any point in this interval, and hence it is less than 0.025 away from the true (exact) solution of the given equation.

We could visually see this example in the image below.



Note from this illustration that not every value of r is closer to the exact solution of the equation.

2.1.2 Pseudocode

We give below the pseudocode for Bisection Method

```
function Bisection(f, lower, upper, errorBound)
{
    r = (lower+upper)/2
```

while(f(r)≠0 and (upper-lower)≥errorBound)

```
    {
      if(f(lower)*f(r)<0)
              upper = r
      else if(f(upper)*f(r)<0)
              lower = r
      r = (lower+upper)/2
    }
    return r
}
```

2.1.3 Sage Code

This algorithm implemented in Sage as a function would look like

```
def Bisection(f, lower, upper, errorBound):
    r = (lower+upper)/2
    while( f(r)!=0 and  (upper-lower) >= errorBound
           ):
        if(f(lower)*f(r)<0):
            upper = r
        else:            #(f(upper)*f(r)<0)
            lower = r
```

```
        r = (lower+upper)/2
    return r
```

Another way to implement the same method is using recursivity. It is a less efficient, but maybe more clear:

```
def Bisection(f, lower, upper, errorBound):
    r = (lower+upper)/2
    if( f(r)!=0 and  (upper-lower) >= errorBound  ):
        if(f(lower)*f(r)<0):
            return Bisection(f, lower, r, errorBound)
        else:
            return Bisection(f, r, upper, errorBound)
    return r
```

The previous example can be solved, using Sage, by the following code:

```
def Bisection(f, lower, upper, errorBound):
    r = (lower+upper)/2
    while( f(r)!=0 and  (upper-lower) >= errorBound
    ):
        if(f(lower)*f(r)<0):
            upper = r
        else:
            lower = r
        r = (lower+upper)/2
    return r

def f(x):
    return 3.7*x^8-7.5*x^3+3*x-10.3

print "solution = ", Bisection(f, -1.5, -0.5, 0.05).n()
```

And we obtain the output:

```
solution =  -1.04687500000000
```

If we wish, we can include some `print` statements to output all the intermediate steps, then we could modify the previous code as it is shown below:

```
def Bisection(f, lower, upper, errorBound):
    print 'Starting with the interval [', lower,',', upper,']'
    print 'f(', lower,'  )=', f(lower)
    print 'f(', upper, ' )=', f(upper)
    print '-----------------------------------------'
    print '-----------------------------------------'
    #computing the middle of the interval
    #this is our first approximation of r
    r = (lower+upper)/2
    while( f(r)!=0 and  (upper-lower) >= errorBound  ):
        print  'in the interval [' , lower,  ',' , upper , ']'
        print  '      midpoint = ' , r,  ' f(r)= ', f(r)
        print
        if(f(lower)*f(r)<0):
```

```
            upper = r
        else:
            lower = r
        r = (lower+upper)/2
    return 'the approximate solution is: '+str(r)
def f(x):
    return 3.7*x^8-7.5*x^3+3*x-10.3
Bisection(f, -1.5, -0.5, 0.05)
```

And we obtain the output:

```
Starting with the interval [ -1.50000000000000 ,
  -0.500000000000000 ]
f( -1.50000000000000   )= 105.339453125000
f( -0.500000000000000  )= -10.8480468750000
-----------------------------------------
-----------------------------------------
in the interval [ -1.50000000000000 , -0.500000000000000 ]
      midpoint =  -1.00000000000000  f(r)=  -2.10000000000000

in the interval [ -1.50000000000000 , -1.00000000000000 ]
      midpoint =  -1.25000000000000  f(r)=  22.6521560668945

in the interval [ -1.25000000000000 , -1.00000000000000 ]
      midpoint =  -1.12500000000000  f(r)=  6.49711363911629

in the interval [ -1.12500000000000 , -1.00000000000000 ]
      midpoint =  -1.06250000000000  f(r)=  1.51790103104431

in the interval [ -1.06250000000000 , -1.00000000000000 ]
      midpoint =  -1.03125000000000  f(r)=  -0.435675070687559

'the approximate solution is: -1.04687500000000'
```

2.1.4 Error Bounds

At the first iteration of this algorithm, we estimate the solution of the equation $f(x) = 0$ with the midpoint of the interval $[a, b]$, that is we estimate $r = \frac{a+b}{2}$. Since this value of r is less than $\frac{b-a}{2}$ away from any value in the interval (a, b) (and in particular from the *exact solution* of the equation), the error of approximation is bounded above by $\frac{b-a}{2}$. Hence the absolute error committed is less than $\frac{b-a}{2}$.

At the second iteration of this algorithm, we look either at $[a, \frac{a+b}{2}]$ or at $[\frac{a+b}{2}, b]$, depending on the signs of $f(a)$, $f(b)$, and $f(r)$. Since the midpoint of either one of these intervals is at a distance less than half the size of that interval, in this case $\frac{b-a}{4}$, from all the other points within the same interval, the error is bounded above by $\frac{b-a}{4}$.

If we continue to apply the Bisection Method in the same fashion over and over again, we would get the following upper bounds for the errors: $\frac{b-a}{8}, \frac{b-a}{16}, \ldots, \frac{b-a}{2^{n+1}}, \ldots$.

It can be proven, using Mathematical Induction, that after n iterations of applying the Bisection Method,

$$\text{the error of approximation } < \frac{b-a}{2^{n+1}}. \tag{2.1}$$

Note that the sequence $\left\{\frac{b-a}{2^{n+1}}\right\}_{n=1,2,\ldots}$ will always converge to 0, hence this algorithm always converges. Therefore (if we ignore the limitations of computers or calculators) we can always find an approximation of the solution with any desired level of accuracy.

2.1.5 Comments on the Bisection Method

As we can observe from the previous subsection, each new iteration gives us a lower error of approximation. The upper bound of this error gets cut into half with each iteration. In terms of bits (base 2 digits) we can say that with each iteration we gain one bit of accuracy in our solution. Therefore this method will always converge to a solution. However, as we will see once we introduce other methods, the Bisection Method converges significantly slower than the other methods. In particular, this method makes no use of the magnitudes of values of the function, it only uses their signs (positive or negative).

It is very important to note one of the limitations of this method. It cannot be used to find zero's whose multiplicity is 2 (or any other even number). Can you elaborate on this issue? Can you see why this is the case?

Another note that is worth being mentioned: if you ever took any Data Structures related course (usually also taught in a Computer Science 2 course), you should observe that this method uses the same exact idea as in a Binary Search method (which is used to search for a given element, in a sorted list/container of items).

2.1.6 Examples

Example 43. Using Sage, find a solution of the equation $x^2 = 4\sin(x)$ in $[1, 3]$, with an error less than 0.000005.

Solution. Let $f(x) = x^2 - 4\sin(x)$, on the interval $[1, 3]$. Since $-1 \leq \sin(x) \leq 1$ we can easily check that $f(1) < 0$, and $f(3) > 0$. Also, f is a continuous function, being a linear sum of two continuous functions (a polynomial function and a trigonometric function). Given all these conditions that are satisfied, we know f has a root in $(1, 3)$, and we can apply the Bisection Method. The Sage code below does the job of finding the root, with an error of approximation less than 0.000005, that is an error bound (for the interval) of 0.00001. For completeness, we include code that will also output a table with all intermediate calculations.

```
def Bisection(f, lower, upper, errorBound):
       print 'a \t\t\t\t f(a) \t\t\t b \t\t\t f(b)'
       print '-----------------------------------------\
 -----------------------------------------'
       r = (lower+upper)/2
       while( f(r)!=0 and (upper-lower) >= errorBound ):
              print lower,'\t',f(lower),'\t', upper,'\t',f(upper)
```

```
                if(f(lower)*f(r)<0):
                        upper = r
                else:
                        lower = r
                r = (lower+upper)/2
        return 'approximate solution: ' + str(r)
def f(x):
        return x^2-4*sin(x)
Bisection(f, 1.0, 3.0, 0.00001)
```

We obtained the approximate solution $r = 1.93375015258789$ and the following output

```
a                                f(a)                 b                                f(b)
------------------------------------------ ------------------------------------------
1.00000000000000       -2.36588393923159       3.00000000000000       8.43551996776053
1.00000000000000       -2.36588393923159       2.00000000000000       0.362810292697273
1.50000000000000       -1.73997994641622       2.00000000000000       0.362810292697273
1.75000000000000       -0.873443787495748      2.00000000000000       0.362810292697273
1.87500000000000       -0.300718126438775      2.00000000000000       0.362810292697273
1.87500000000000       -0.300718126438775      1.93750000000000       0.0198491265504952
1.90625000000000       -0.143255172243018      1.93750000000000       0.0198491265504952
1.92187500000000       -0.0624056513559537     1.93750000000000       0.0198491265504952
1.92968750000000       -0.0214535898071695     1.93750000000000       0.0198491265504952
1.93359375000000       -0.000846021499982808   1.93750000000000       0.0198491265504952
1.93359375000000       -0.000846021499982808   1.93554687500000       0.00949061035242948
1.93359375000000       -0.000846021499982808   1.93457031250000       0.00431955821900587
1.93359375000000       -0.000846021499982808   1.93408203125000       0.00173608422489169
1.93359375000000       -0.000846021499982808   1.93383789062500       0.000444860318459028
1.93371582031250       -0.000200623353052709   1.93383789062500       0.000444860318459028
1.93371582031250       -0.000200623353052709   1.93377685546875       0.000122107792292159
1.93374633789062       -0.0000392604530032870  1.93377685546875       0.000122107792292159
1.93374633789062       -0.0000392604530032870  1.93376159667969       0.0000414230014911254
'approximate solution: 1.93375015258789'
```

Since the error of approximation is less than 0.000005, only the first five decimal places are accurate, and therefore we should only report the following answer: $r = 1.93375$.

If we wish to use Sage library methods to compute the solution to this equation, one can use the following:

```
find_root(x^2-4*sin(x), 1,3)
```

to obtain

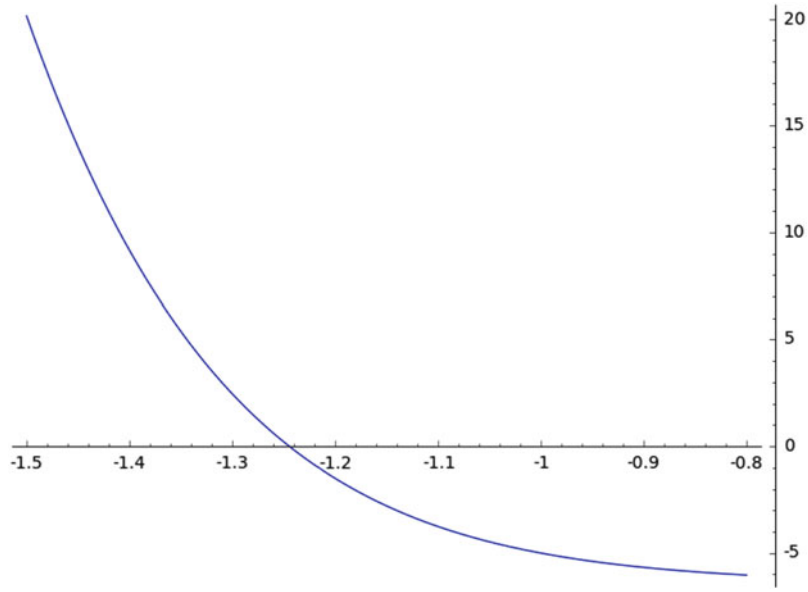
```
1.9337537628270214
```

Example 44. Find the largest root of $f(x) = x^8 - x - 7$, accurate to within 0.00001.

Solution. In order to have an idea where could the largest root be, we should first graph the function. This will help us find an interval $[a, b]$ that can be used in the Bisection Method. The function is continuous everywhere, since it is a polynomial. Running the Sage code below and adjusting the interval $[a, b]$ until we get a satisfactorily graph, we obtain

```
sage: var('x')
sage: f(x) = x^8-x-7
sage: p=plot(f(x), (x,-1.5, -0.8))
sage: p.show()
```

This will produce the following graph:



Then we apply the Bisection method for function f on $[-1.5, -0.8]$. We can do the computation manually, using a calculator, or using Sage code.

```
def Bisection(f, lower, upper, errorBound):
       print 'a \t\t\t\t f(a) \t\t\t b \t\t\t f(b) '
       print '-----------------------------------------\
-----------------------------------------'
       r = (lower+upper)/2
       while( f(r)!=0 and (upper-lower) >= errorBound ):
             print lower,'\t',f(lower), '\t', upper,'\t',f(upper)
             if(f(lower)*f(r)<0):
                   upper = r
             else:
                   lower = r
             r = (lower+upper)/2
       return 'approximate solution: ' + str(r)
def f(x):
       return  x^8-x-7
Bisection(f, -1.5, -0.8, 0.00002)
#NOTE: accurate to 0.00001 means
# the size of the interval is at most 0.00002
```

In either case, we would obtain something similar to:

```
a                              f(a)                 b                              f(b)
-------------------------------------------------------------------------------------------
-1.50000000000000       20.1289062500000       -0.800000000000000      -6.03222784000000
-1.50000000000000       20.1289062500000       -1.15000000000000       -2.79097713746094
-1.32500000000000       3.82507483083511       -1.15000000000000       -2.79097713746094
-1.32500000000000       3.82507483083511       -1.23750000000000       -0.262513027024752
-1.28125000000000       1.54349718993853       -1.23750000000000       -0.262513027024752
-1.25937500000000       0.586997221817224      -1.23750000000000       -0.262513027024752
-1.24843750000000       0.149557452176373      -1.23750000000000       -0.262513027024752
-1.24843750000000       0.149557452176373      -1.24296875000000       -0.0595660658616204
-1.24570312500000       0.0442134050362473     -1.24296875000000       -0.0595660658616204
-1.24570312500000       0.0442134050362473     -1.24433593750000       -0.00787061634904340
-1.24501953125000       0.0181226626491746     -1.24433593750000       -0.00787061634904340
-1.24467773437500       0.00511386028712657    -1.24433593750000       -0.00787061634904340
-1.24467773437500       0.00511386028712657    -1.24450683593750       -0.00138141624211929
-1.24459228515625       0.00186546215677108    -1.24450683593750       -0.00138141624211929
-1.24454956054687       0.000241833030023564   -1.24450683593750       -0.00138141624211929
-1.24454956054687       0.000241833030023564   -1.24452819824219       -0.000569839082981360
'approximate solution: -1.24454421997070'
```

Hence our answer would be -1.24454421997070 (or rather -1.24454 since the other digits many not be accurate).

Like in the previous example, one could use Sage library methods to compute/approximate the solution to this equation:

```
find_root(x^8-x-7, -1.5, -0.8)
```

to obtain

```
-1.2445431960326296
```

Example 45. In the previous example, starting with the same interval $[a, b]$, how many iterations are needed in order to find the largest root accurate to within 0.000000000001?

Solution. Using the formula (2.1) we need to find n large enough so that

$$\text{the error of approximation } < \frac{b-a}{2^{n+1}} < 0.000000000001.$$

Hence

$$\frac{-0.8-(-1.5)}{2^{n+1}} < 0.000000000001$$

or

$$2^{n+1} > 0.7 * 10^{12}$$

hence

$$\begin{aligned} n &> \frac{\ln(0.7 * 10^{12})}{\ln 2} - 1 \\ &\approx 38.3485639658 \end{aligned}$$

hence n should be at least 39. Therefore, if we use at least 39 iterations, then it is guaranteed that the Bisection Method will produce an approximate solution with an absolute error less than 0.000000000001. Check this value also using Sage (Use *Bisection(f, −1.5, −0.8, 0.000000000002)*.)

Example 46. How many iterations are needed to compute the root of $x^3 + 100 - e^{x^2}$ from the interval $[0, 4]$ with an error of approximation less than 10^{-7}?

Solution. Using the formula (2.1) we need to find n large enough so that

$$\text{the error of approximation } < \frac{b-a}{2^{n+1}} < 10^{-7}.$$

Therefore

$$\frac{4-0}{2^{n+1}} < 10^{-7}$$

or

$$2^{n-1} > 10^7$$

hence

$$\begin{aligned} n &> \frac{\ln(10^7)}{\ln 2} + 1 \\ &\approx 24.2534966 \end{aligned}$$

hence n should be at least 25. Try to check this result using a Sage example.

2.1.7 Sage Library

Below we show how one can use the `scipy` library to use the Bisection Method already implemented there (see [3]). Using that method we will solve the examples given above.

Example 47. Using Sage, find a solution of the equation $x^2 = 4\sin(x)$ in [1,3].

Solution. We can use the following Sage code

```
import scipy.optimize
f(x) = x^2-4*sin(x)
print "solution = " , scipy.optimize.bisect(f, 1, 3)
```

to obtain

```
solution =  1.93375376283
```

Example 48. Find the largest root of $f(x) = x^8 - x - 7$.

Solution. Using a graph, as before, we can see that the largest root is inside the interval $[-1.5, -0.8]$. Then we can use the following Sage code

```
import scipy.optimize
f(x) = x^8-x-7
print "solution = " , scipy.optimize.bisect(f, -1.5, -0.8)
```

to obtain

```
solution =  -1.24454319603
```

2.2 Newton–Raphson's Method

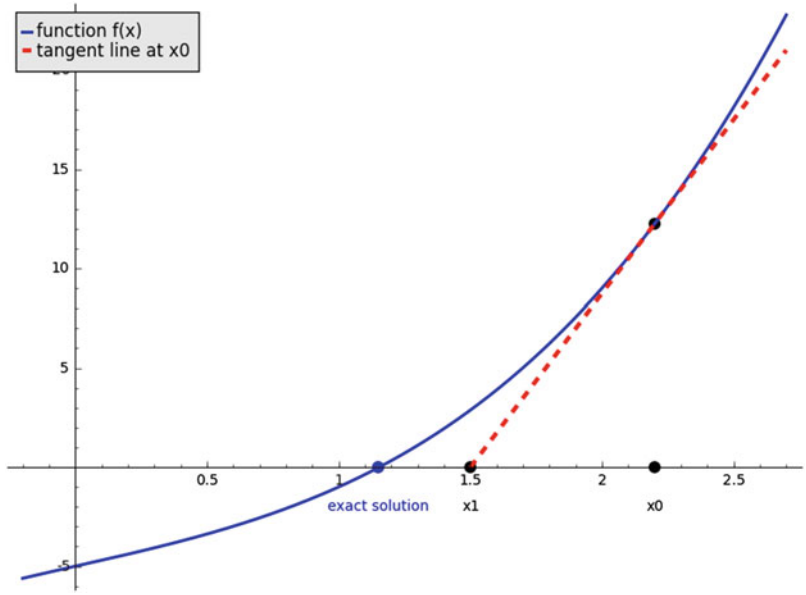
Newton–Raphson's Method is also known as Newton's Method (see also [4, 5]).

In this section we assume that the function f is differentiable (and hence it is also continuous).

As we will see below, this method usually converges much faster than the Bisection Method. We say *usually*, because there are some cases when this method may converge very slowly, or not converge at all.

As opposed to Bisection method, which will always lead to a solution, there are cases when the Newton's Method will diverge. This is one of the reasons why in practice some hybrid methods, that combines both methods for an optimal result, are being used.

Let's have a look at the following graph. For completeness, we included it's Sage code come below, without giving too many details other than the comments embedded in it. Please refer to the official Sage Math Documentation to find more details about "2D Plotting" [6].



Here comes a basic outline of the Newton–Raphson's Method: As with the previous section, we are trying to solve the equation $f(x) = 0$. This is the same as finding the point where the graph of $f(x)$ crosses the x-axis (See the "exact solution" plotted in the previous graph.) Let's start with some approximation to the exact solution (or "guess" if you wish), and call it x_0 (we will need that $f'(x_0) \neq 0$). From the given graph it seems reasonable to approximate the function $f(x)$ with the tangent line to $f(x)$ at x_0. That is

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

Doing so, we can easily compute the zero of the tangent line (this zero is "usually" is a better approximation to the exact solution than x_0). Since, $f(x_0) + f'(x_0)(x - x_0) = 0$ can be easily solved, we obtain our approximate solution:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

We can see from the graph that we expect x_1 to be a more accurate approximation of the exact solution than x_0. Now repeat this process and compute

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)},$$

and so on. Our hope (for now) is that we will obtain better and better approximations with each new iteration.

Below we give the Sage code for the above graph, and then we will give the outline of the Newton–Raphson's Method. For now, we assume $f'(x_i) \neq 0$, for every $i = 0, 1, \ldots, n, \ldots$. The special case when $f'(x_i) = 0$, for some value of i, is treated towards the end of this section.

```
#the function we considered is
var('x')
f(x)=x^3+3*x-5

#next we compute its derivative
fp(x)=diff(f(x),x)

#the particular choice of x0
x0 = 2.2

#the interval [a,b] is
a = -0.2
b = 2.7

#preparing the plots
#first comes the function
function = plot(f(x), (x,a,b), linestyle='-', thickness=2,
                color='blue', legend_label='function f(x)')
#then comes the tangent line
tangent = plot( (fp(x0)*(x-x0)+f(x0)) , (x,1.5,b),
               linestyle='--', thickness=3, color='red',
               legend_label='tangent line at x0')
#we include some text on the graph
textt = text('x1', (1.5,-2), color='black')

#we plot some of the points, and some text
points = list_plot( [(x0, f(x0)), (x0, 0), (1.5,0)],
                                color='black', size=70)
textp = text('x0', (x0,-2), color='black')

#then we plot the exact solution, with text
Exact = list_plot( [(1.15,0)] , color='blue', size=70)
textExact = text('exact solution', (1.15,-2), color='blue')

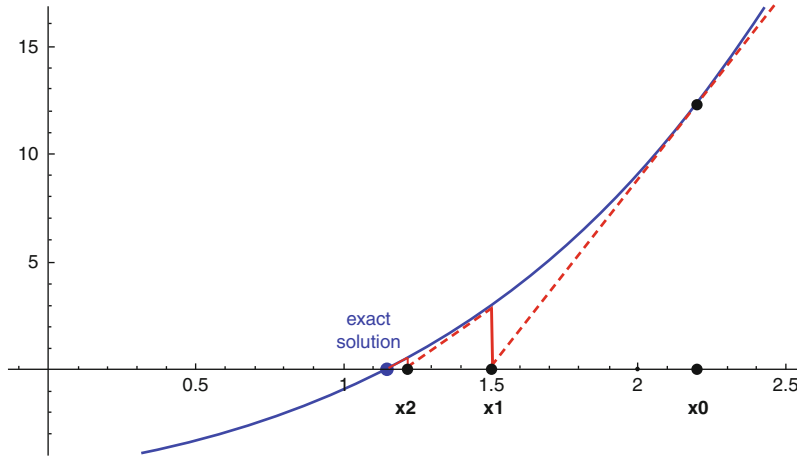
#next we display all the plots we created above
show(function+tangent+textt+points+textp+textExact+Exact)
```

2.2.1 The Algorithm

Newton–Raphson's method is as follows: start with some approximate value to the exact solution, let's say x_0. Then, using formula

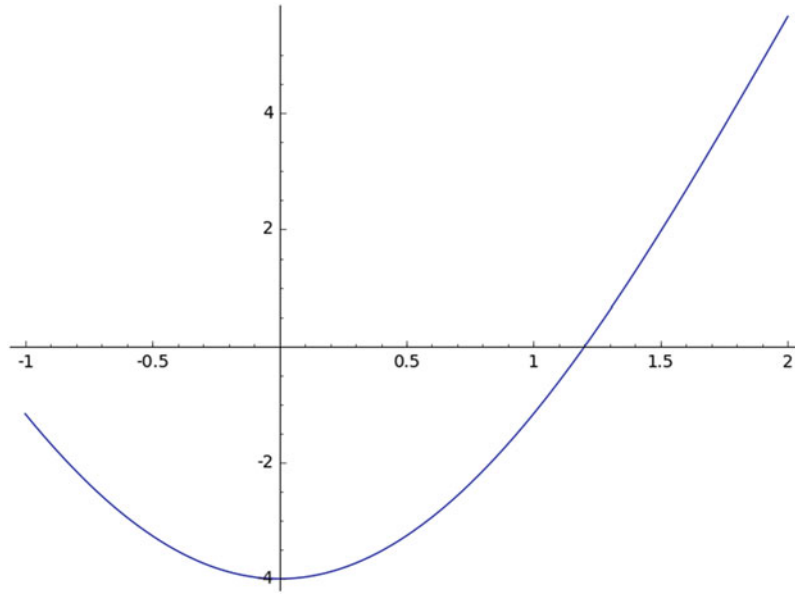
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)},\ n = 0, 1, 2, \ldots \tag{2.2}$$

we successively compute $x_1, x_2, x_3, \ldots$. Under special circumstances (to be discussed below), this sequence of points will converge to the exact solution, r.



For example, suppose we wish to find a positive solution for the equation $x^2 - 4\cos(x) = 0$. Here $f(x) = x^2 - 4\cos(x)$ and $f'(x) = 2x + \sin(x)$. Using Sage to graph this function we can see that a candidate for x_0 could be 1.

```
plot( x^2-4*cos(x) , (x,-1,2))
```



Then, we get the following successive approximations: $x_0 = 1$, $x_1 = 1 - \frac{f(1)}{f'(1)} \simeq 1.216405952$, $x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} \simeq 1.201599182$, $x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} \simeq 1.20153830$. Therefore, the exact solution is approximately $r \simeq 1.20153830$. Below is the Sage code we used to carry these computations.

```
var('x')
f(x)= x^2-4*cos(x)
fp(x)=diff(f(x),x)

x0=1.0
x1=x0-f(x0)/fp(x0)
x2=x1-f(x1)/fp(x1)
x3=x2-f(x2)/fp(x2)

print x0, x1, x2, x3
```

to obtain:

```
1.00000000000000 1.21640595223885 1.20159918212198 1.20153830038108
```

2.2.2 Pseudocode

Before we write the pseudocode and Sage code, we should be aware of some potential challenges when running the algorithm.

First issue is that the iterations may not converge. Because of this, we want to make sure the program will "know" to eventually give up. So for this, we need to introduce a parameter *maxSteps* that will tell the algorithm when to give up. In the example below, we chose this value to be 100. That guarantees that the algorithm will stop after at most 100 iterations.

Then, we also need to decide when do we consider that our approximation is "good" enough. What does "good" mean for us? Or, "how good is good enough"? In the example below, we chose $almostZero = 0.000000001$. That means any *answer*, such that $|f(answer)| < 0.000000001$ will be considered accurate enough, and the Newton–Raphson's iterations should stop there.

One challenge we will not take into account here (in order to keep the algorithm simple) is the case when $f'(x_i)$ is zero for some i. What happens if $f'(x_i)$ is so close to zero that when represented on computer, it appears as a zero (an obvious example that would be a challenge for most computers today is: $10^{-10^{2014}}$ which would be represented as a 0). What do we do when an **underflow** occurs?

Here comes the pseudocode that we will use for Newton–Raphson algorithm:

```
function printNewtonRaphson(f, f', x0, maxSteps, almostZero )
{
    output x0, '\t', f(x0)

    #we compute the iterations and stop when we get
    #a "good" approximation, or tried too many steps
    i = 0
    while(  i < maxSteps and  |f(x_i)| > almostZero )
    {
          x_{i+1} = x_i - f(x_i)/f'(x_i)
          print x_i , '\t', f(x_i)
          i = i + 1
    }
}
```

2.2.3 Sage Code

We give, below, our Sage implementation for the Newton–Raphson's Method.

```
def printNewtonRaphson(f, fp, x0, maxSteps, almostZero ):
    print '\t', 'xi', '\t\t\t','f(xi)'

    #we start with x0, and print it
    approx = x0
    print x0, '\t', f(x0)

    #then we compute the iterations and
    #stop when we get a "good" approximation
    #or when tried too many steps
    steps = 0
    while(  steps<maxSteps and  abs(f(approx))>almostZero ) :
        approx = approx - f(approx)/fp(approx)
        print approx, '\t', f(approx)
        steps = steps + 1
```

To apply this code to out previous example, we simply need to add the Sage code below:

```
#define f(x)
var('x')
f(x)= x^2-4*cos(x)

#compute f'(x)
fp(x)=diff(f(x),x)

#choose a starting point
x0=1.0

#choose the max number of iterations
maxSteps = 100

#choose what makes an approx acceptable
almostZero = 0.000000001

#call the function
printNewtonRaphson(f, fp, x0, maxSteps, almostZero)
```

The example above could be equivalently simplified to

```
#define f(x)
var('x')
f(x)= x^2-4*cos(x)

#compute f'(x)
fp(x)=diff(f(x),x)

#call the function
printNewtonRaphson(f, fp, 1.0, 100, 0.000000001)
```

Either one will yield the following output:

```
          xi              f(xi)
1.00000000000000     -1.16120922347256
1.21640595223885     0.0915686905445798
1.20159918212198     0.000373428376449381
1.20153830038108     6.38189456836358e-9
1.20153829934058     6.66133814775094e-16
```

2.2.4 Error Bounds

The error of approximation formula for Newton–Raphson Method is not as nice and clear as the one given in the Bisection Method. For example, in the Bisection Method we could compute ahead of time the number of iterations needed to achieve a certain accuracy. Such an exact computation is not available for Newton–Raphson's Method. For Bisection Method we had a clear upper bound formula but for Newton's Method we can only estimate it.

We will give below the standard approach for the error of approximation in the case of Newton–Raphson's Method:

$$\text{error} = \text{exact solution} - x_n.$$

Let's denote by r the exact solution. Since r is a zero for the f, using the Mean Value Theorem (see page 53), we obtain:

$$\begin{aligned} f(x_n) &= f(x_n) - f(r) \\ &= f'(\xi_n)(x_n - r) \end{aligned}$$

for some ξ_n between x_n and r. Hence

$$r - x_n = -\frac{f(x_n)}{f'(\xi_n)}.$$

Therefore, if x_n is close to r, so must be ξ_n, and we have

$$r - x_n \simeq -\frac{f(x_n)}{f'(x_n)} = x_{n+1} - x_n.$$

So the error of approximation formula use for Newton–Raphson's Method is given by

$$\text{exact solution} - x_n \simeq x_{n+1} - x_n$$

which in practice seems to be quite accurate.

2.2.5 Comments on the Newton–Raphson's Method

As mentioned above these iterations, under certain conditions, converge much faster than in Bisection Method. The number of correct figures nearly doubles with each new iteration and therefore about five

iterations will usually give us a pretty accurate approximation. In those cases, we say that Newton–Raphson's Method has a quadratic convergence.

Theorem 49 (Newton's Method Theorem). *If r is such that $f(r) = 0$, and f, f', f'' are continuous in a neighborhood of r, and $f'(r) \neq 0$, then, for any x_0 sufficiently close r, we have that x_0, x_1, x_2, ..., x_n, ... will converge to r and for some C*

$$|r - x_{n+1}| \leq C\,|r - x_n|^2\,, \quad n = 0, 1, 2, \ldots.$$

Proof. An outline only!
Using Taylor's Theorem with Remainder (see page 48) we obtain that for any x_n close enough to r we have

$$f(r) = f(x_n) + f'(x_n)(r - x_n) + \frac{f''(\xi)}{2}(r - x_n)^2,$$

for some ξ between r and x_n. Therefore

$$0 = \frac{f(x_n)}{f'(x_n)} + (r - x_n) + \frac{f''(\xi)}{2f'(x_n)}(r - x_n)^2$$

hence

$$0 = (x_n - x_{n+1}) + (r - x_n) + \frac{f''(\xi)}{2f'(x_n)}(r - x_n)^2.$$

From here we deduce

$$r - x_{n+1} = \frac{-f''(\xi)}{2f'(x_n)}(r - x_n)^2$$

therefore

$$|r - x_{n+1}| = \left|\frac{f''(\xi)}{2f'(x_n)}\right| |r - x_n|^2\,.$$

This last equation can be used to deduce the statement of the Theorem. ■

But there are cases when the convergence may be slower, about linear (for example when the exact solution has multiplicity more than one). And there are yet other cases when the sequence will not converge at all. These cases are discussed next.

If, for some n, the value of $f'(x_n)$ is so small that $f(x_n)/f'(x_n)$ cannot be represented on the computer, then the method will fail. We will not be able to compute the iterations past that point x_n ($x_{n+1}, x_{n+2}, \ldots$). To address this issue, we should use an improved version of the Pseudocode given above, so that we stop the algorithm and report this incident. Here is what we could do: if at any time $f'(x_n)$ becomes less than a certain value (let's say 0.00000001) then we simply stop the algorithm and output an error message.

```
function printNewtonRaphson(f, f', x0, maxSteps, almostZero )
{
    output x0, '\t', f(x0)

    #we compute the iterations and stop when we get
```

```
    #a "good" approximation, or tried too many steps
    i = 0
```

while($i <$ *maxSteps* and $|f(x_i)| >$ *almostZero* and $|f'(x_i)| \geq 0.00000001$)

{

$x_{i+1} = x_i - f(x_i)/f'(x_i)$
print x_i, '\t', $f(x_i)$
i = i + 1

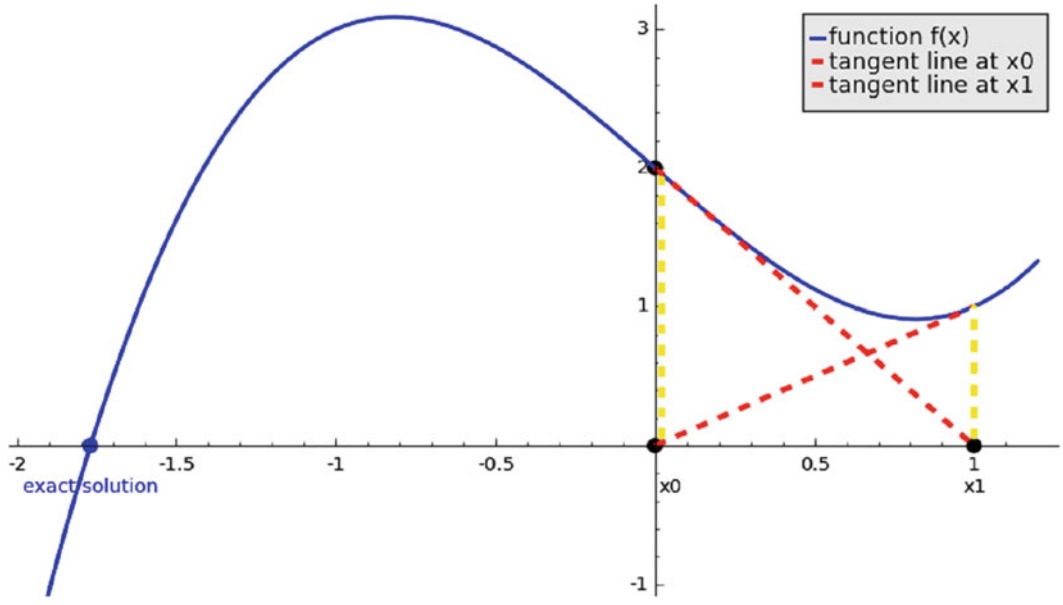
}

if($|f'(x_i)| >$ *smallestPositiveValue*|)
print "ERROR: Max. number of steps reached"
if($f'(x_i)| < 0.00000001$)
print "ERROR: derivative has a very small value, or it is 0"

}

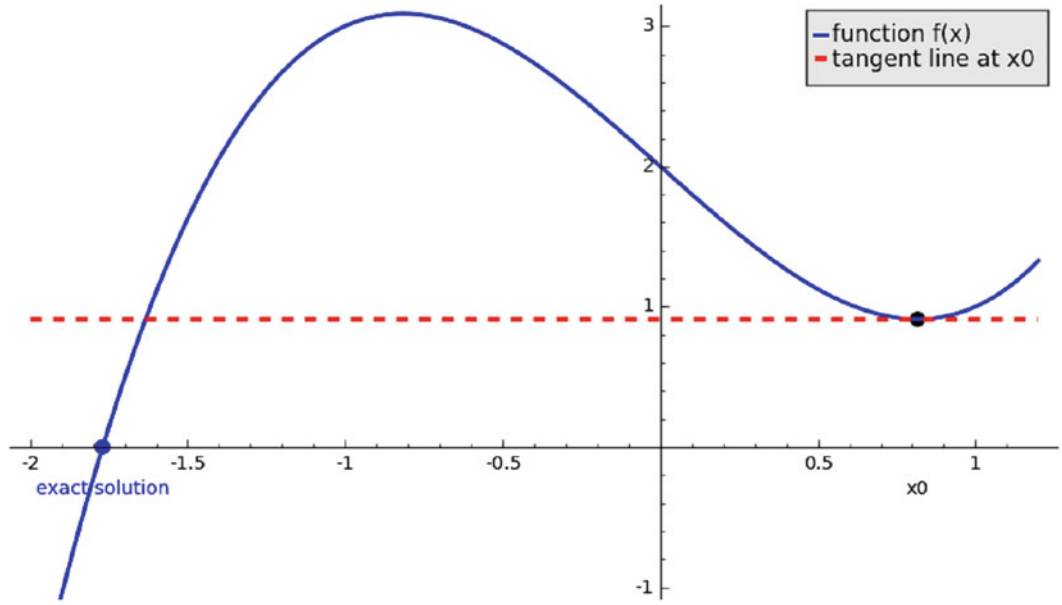
One way to try avoiding this issue, and that will work in many cases, would be to choose a closer value of x_0 to the exact solution, r. As an exercise, modify the Sage code given earlier so it can address the challenge just discussed here.

Another possible challenge is when the sequence of iterations, $x_0, x_1, \ldots, x_n, \ldots$ does not converge. The iterations can get into a cycle, by repeatedly taking on the same values over and over again. Here is an example: let $f(x) = x^3 - 2x + 2$, and $x_0 = 0$. Then, using Newton's Method, we obtain $x_1 = 1, x_2 = 0, x_3 = 1, \ldots$. Hence we get a cycle, not a convergent sequence.



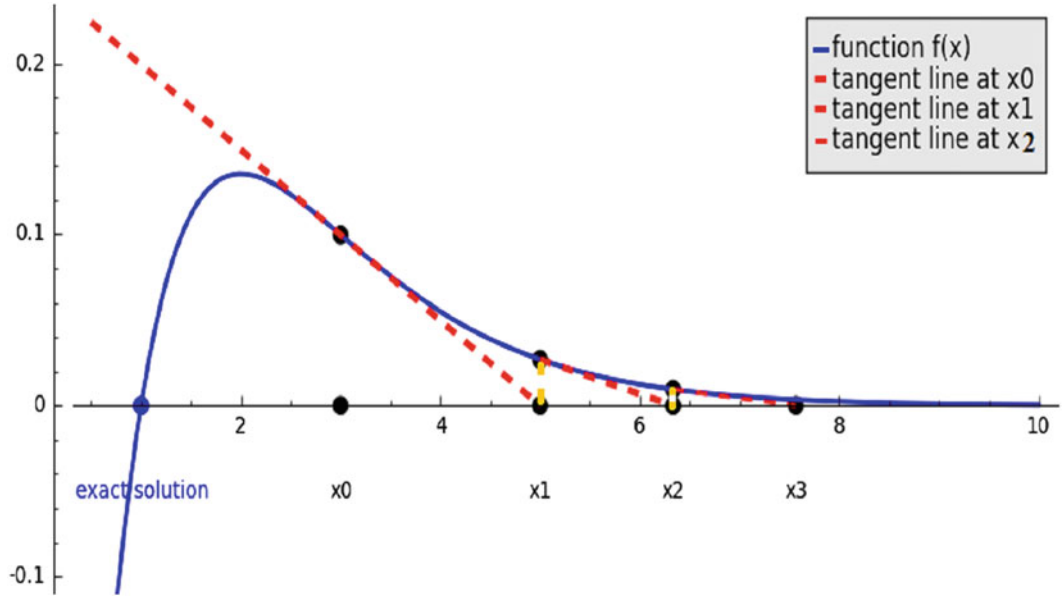
Iterations Enter an Infinite Cycle

There are cases when we might get $f'(x_n) = 0$, for some n. As an example, take the previous function, $f(x) = x^3 - 2x + 2$, but choose $x_0 = \sqrt{\frac{2}{3}}$. Then, $f'(x_0) = 0$ and we cannot compute x_1 and hence we cannot make use of Newton–Raphson's Method.



Horizontal Tangent Line

In some cases the iterations diverge by moving further and further away from the exact solution (instead of approaching it). In the case of, $f(x) = (x-1)e^{-x}$, if we choose $x_0 = 3$, we will see that the iterations are going further and further away from the only zero of the function $r = 1$. In this case, the iterations are not converging to 1, and the Sage will stop only because the value of $f(x_n)$ becomes very small, making the algorithm believe it found a solution.



xi are getting further and further away from the exact solution

Usually all these situations can be avoided by selecting a value of x_0 closer to the exact solution.

2.2.6 Examples

Example 50. Compute the first five iterations of Newton–Raphson's method to find a solution of the equation $x^2 = 4\sin(x)$.

Solution. Let $f(x) = x^2 - 4\sin(x)$. Then $f'(x) = 2x - 4\cos(x)$. Using $x_0 = 1.5$, we obtain:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} \simeq 2.14039277238801,$$

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} \simeq 1.95200894640563,$$

$$x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} \simeq 1.93393057392984,$$

$$x_4 = x_3 - \frac{f(x_3)}{f'(x_3)} \simeq 1.93375377978974,$$

and

$$x_5 = x_4 - \frac{f(x_4)}{f'(x_4)} \simeq 1.93375376282702.$$

Example 51. Using Sage, find a solution of the equation $x^2 = 4\sin(x)$ in [1,3].

Solution. Let $f(x) = x^2 - 4\sin(x)$, on the interval $[1, 3]$. Let's apply Newton–Raphson's Method using $x_0 = 1.5$.

```
#define f(x)
var('x')
f(x)= x^2-4*sin(x)

#compute f'(x)
fp(x)=diff(f(x),x)

#call the function
printNewtonRaphson(f, fp, 1.0, 100, 0.000000001)
```

Then we obtain:

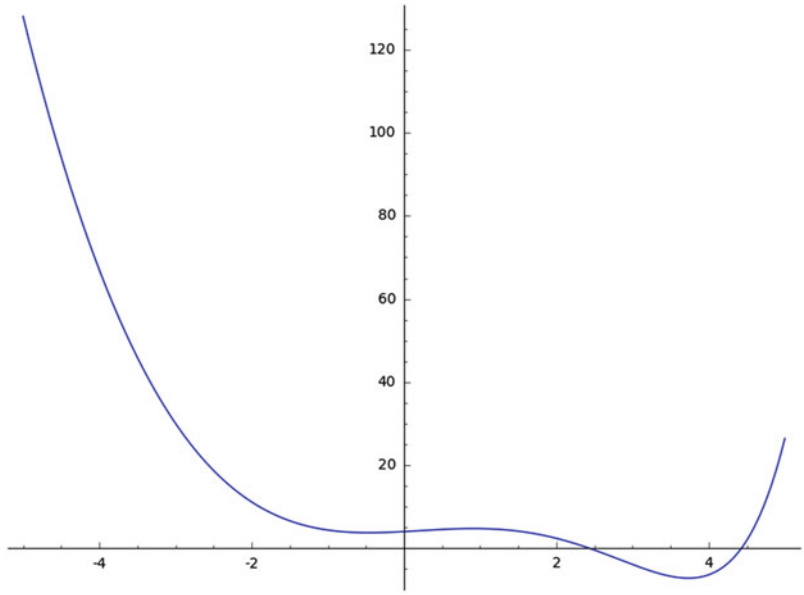
xi	$f(xi)$
1.00000000000000	−2.36588393923159
−13.6758596578334	190.611022845009
−7.13289172861309	53.8824908755625
−3.94582296731645	12.6883310840735
−1.46617193667756	6.12778758471310
0.362975604615735	−1.28847882452474
−0.0646036910653746	0.262408683769386
−0.000925601534061141	0.00370326234577898
$-2.13821352164768e-7$	$8.55285454378637e-7$
$-1.14298881865264e-14$	$4.57195527461057e-14$

If you compare these results with the corresponding example that used Bisection Method (see page 43), you will see that this method converges faster.

Example 52. Using Sage, find the largest solution of the equation $e^x = x^3 - 3$.

Solution. Let $f(x) = e^x - x^3 + 3$. If we graph this function, we can see that the largest solution is around 4.5. Therefore we can apply Newton–Raphson's Method using $x_0 = 4.5$.

```
plot(e^x-x^3+3, (x, -5,5))
```



Then, we can apply Newton–Raphson's Method using the code

```
#define f(x)
var('x')
f(x)= e^x-x^3+3


#compute f'(x)
fp(x)=diff(f(x),x)
#call the function
printNewtonRaphson(f, fp, 4.5, 100, 0.000000001)
```

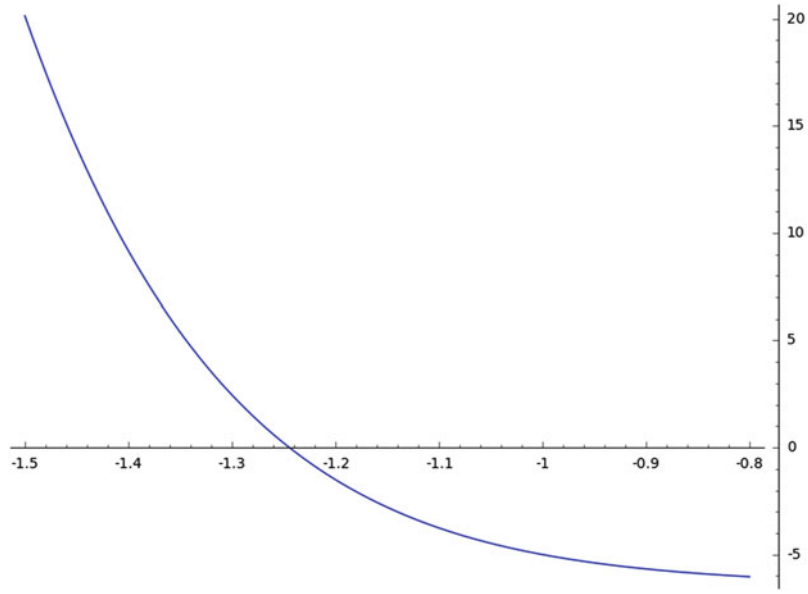
To obtain:

```
       xi                f(xi)
4.50000000000000   1.89213130052181
4.43534961520168   0.127976347740542
4.43030415283240   0.000733634812831951
4.43027489423740   2.45581048829990e-8
4.43027489325791   0.000000000000000
```

Example 53. Using Newton–Raphson, find the largest root of $f(x) = x^8 - x - 7$.

Solution. In order to have an idea where could the largest root be, we should provide a graph of the function.

```
plot(x^8-x-7, (x, -1.5,-0.8))
```



Since the solution seems to be close to -1.2 we choose this value for x_0. Using the code

```
#define f(x)
var('x')
f(x)= x^8-x-7


#compute f'(x)
fp(x)=diff(f(x),x)
#call the function
printNewtonRaphson(f, fp, -1.2, 100, 0.000000001)
```

we obtain:

```
    xi                  f(xi)
-1.20000000000000     -1.50018304000000
-1.25057004771720     0.232814860486689
-1.24464131599207     0.00372920995296955
-1.24454322238949     1.00146604165019e-6
-1.24454319603264     7.37188088351104e-14
```

For comparison purposes, if we choose $x_0 = -0.8$, which is further away from the exact solution, we would need more iterations

```
    xi                  f(xi)
-0.800000000000000     -6.03222784000000
-3.05274645429906     7538.73623499328
-2.67137213793097     2589.10499016581
```

```
-2.33805087708853     888.297339207666
-2.04741544726537     303.826449940133
-1.79580193886599     102.956003751903
-1.58257000864920     33.9288292520208
-1.41283933426179     10.2888806076826
-1.29964510559836     2.43915405062124
-1.25191491788326     0.285820586171777
-1.24468954991890     0.00556315419511222
-1.24454325466533     2.22783322012532e-6
-1.24454319603265     3.60600438398251e-13
```

You may want to try other values, to compare the speed of convergence. Try to relate your answers to the shape of the graph.

Example 54. Use Newton's Method to compute $\sqrt[3]{5}$

Solution. Since $\sqrt[3]{5}$ is the solution of the equation $x^3 - 5 = 0$, we will apply Newton's Method to $f(x) = x^3 - 5$. Let's start with $x_0 = 2$. We have $f'(x) = 3x^2$ and:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = 1.75$$

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = 1.7108843537415$$

$$x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} = 1.70997642891697$$

$$x_4 = x_3 - \frac{f(x_3)}{f'(x_3)} = 1.70997594667683.$$

Since the last two iterations are very close to each other we can stop and, we conclude that

$$\sqrt[3]{5} \simeq 1.70997.$$

If we want more digits of precision, we simply compute a few more iterations.

2.2.7 Sage Library

Sage already has a Newton–Raphson's Method included in its libraries. Below is an example on how to solve the following problem: use Newton–Raphson's Method to compute $\sqrt[3]{5}$.

```
sage: x = PolynomialRing(RealField(), 'x').gen()
sage: f = x^3 - 5
sage: #if we want n = 4 iterations
sage: #and start with x0 = 2 then use:
sage: f.newton_raphson(4, 2)
```

The output obtained is:

```
[1.75000000000000, 1.71088435374150, 1.70997642891697,
  1.70997594667683]
```

2.2.8 Sage Library 2

We present, below, another Sage function that implements the Newton–Raphson's Method. This method is included in the `scipy` library. We will apply this method to all the examples given above in this section.

Example 55. Use Newton–Raphson's method to find a solution of the equation $x^2 = 4\sin(x)$.

Solution. Using $x_0 = 1.5$, and the following Sage code

```
#import the following library
import scipy.optimize

#the function f(x) from our problem
f(x) = x^2-4*sin(x)

#compute the derivative of f
fp(x)=diff(f(x),x)

#call the Newton's method and output the result
print "solution = " , scipy.optimize.newton(f, 1.5, fp)
```

we obtain:

```
solution =  1.93375376282702
```

Example 56. Using Sage, find the largest solution of the equation $e^x = x^3 - 3$.

Solution. Using $x_0 = 4.5$ and the following Sage code

```
#import the following library
import scipy.optimize

#the function f(x) from our problem
f(x) = e^x-x^3+3

#compute the derivative of f
fp(x)=diff(f(x),x)

#call the Newton's method and output the result
print "solution = " , scipy.optimize.newton(f, 4.5, fp)
```

we obtain:

```
solution =  4.43027489325791
```

Example 57. Find the largest root of $f(x) = x^8 - x - 7$.

Solution. Using $x_0 = -1.2$ and the following Sage code

```
#import the following library
import scipy.optimize

#the function f(x) from our problem
f(x) = x^8-x-7
```

```
#compute the derivative of f
fp(x)=diff(f(x),x)

#call the Newton's method and output the result
print "solution = " , scipy.optimize.newton(f, -1.2, fp)
```

we obtain:

```
solution =  -1.24454319603264
```

Example 58. Use Newton's Method to compute $\sqrt[3]{5}$

Solution. Using $x_0 = 2$ and the following Sage code

```
#import the following library
import scipy.optimize

#the function f(x) from our problem
f(x) = x^3-5

#compute the derivative of f
fp(x)=diff(f(x),x)

#call the Newton's method and output the result
print "solution = " , scipy.optimize.newton(f, 2, fp)
```

we obtain:

```
solution =  1.709975946676697
```

2.3 Secant Method

In this section we only require that f is a continuous function.

As opposed to Newton's Method, this method does not need f to be differentiable.

As opposed to Bisection Method, this method does not require that the initial two points, a, b be chosen such that the function $f(a)$ and $f(b)$ have opposite signs.

The idea behind the Secant Method is similarly to Newton's Method: it approximates the function f with a line. Rather than using a *tangent* line, here we will be using a *secant* line. Then, the zero of the secant line would give us an approximation for the exact zero of the original function.

We start by choosing two initial points, x_0 and x_1. Then, we can easily compute the line that passes through $(x_0, f(x_0))$ and $(x_1, f(x_1))$. This line is called a *secant line*. The equation of this line is

$$y = f(x_1) + (x - x_1) \cdot \frac{f(x_1) - f(x_0)}{x_1 - x_0}.$$

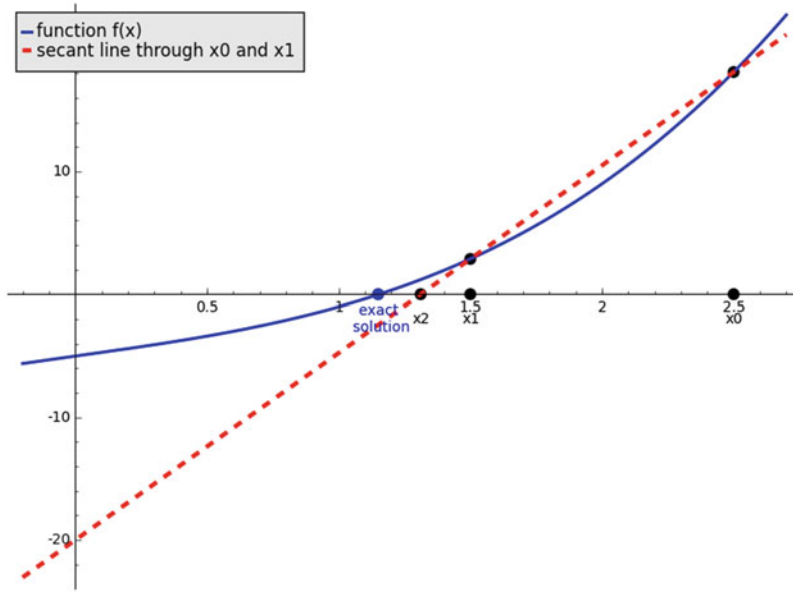
Solving for the zero of this line

$$0 = f(x_1) + (x - x_1) \cdot \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

we obtain

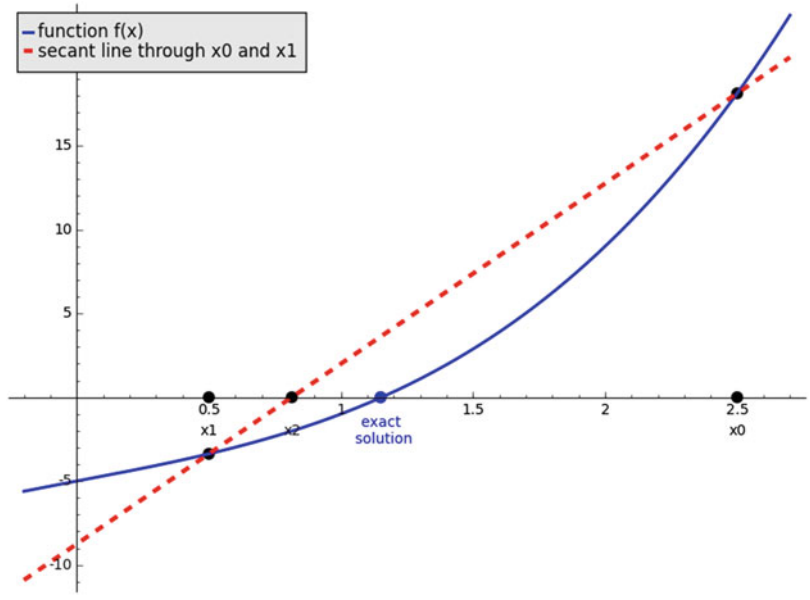
$$x_2 = x_1 - f(x_1) \cdot \frac{x_1 - x_0}{f(x_1) - f(x_0)}.$$

As mentioned above, the two initial points can be on either side of the exact solution of f. The next two graphs illustrate both of these cases. We will also give their Sage code below.



Both initial points are to the left of the exact solution

and



The two initial points are on either side of the exact solution

Using x_1 and the newly computed x_2, we can repeat this process and get

$$x_3 = x_2 - f(x_2) \cdot \frac{x_2 - x_1}{f(x_2) - f(x_1)}$$

and then compute x_4, x_5, so on.

The Sage code for the above graphs is given below:

```
##############################
######   FIRST GRAPH   #######
##############################
#the function we considered is
var('x')
f(x)=x^3+3*x-5

#the particular choice of x0, and x1
x0 = 2.5
x1 = 1.5
x2 = x1-f(x1)*((x1-x0)/(f(x1)-f(x0)))

#the interval [a,b]
a = -0.2
b = 2.7

#first comes the function
function = plot(f(x), (x,a,b), linestyle='-', thickness=2,
                color='blue', legend_label='function f(x)')
#then comes the secant line
secant = plot( f(x1)+(x-x1)*(((f(x1)-f(x0)))/(x1-x0)) , (x,a,b),
               linestyle='--', thickness=3, color='red',
               legend_label='secant line through x0 and x1')
#plotting the points at x0, x1, x2
points = list_plot( [(x0, f(x0)),
     (x0, 0), (x1,f(x1)), (x1,0), (x2,0)],
     color='black', size=70)

#plotting the exact solution
Exact = list_plot( [(1.15,0)] , color='blue', size=70)

#we include some labels on the graph
textx0 = text('x0', (x0,-2), color='black')
textx1 = text('x1', (x1,-2), color='black')
textx2 = text('x2', (x2,-2), color='black')
textExact = text('exact \n solution', (1.15,-2), color='blue')

#displaying all the plots
show(function+secant+textx0+textx1+textx2+points+textExact+Exact)

##############################
######   SECOND GRAPH   ######
##############################
```

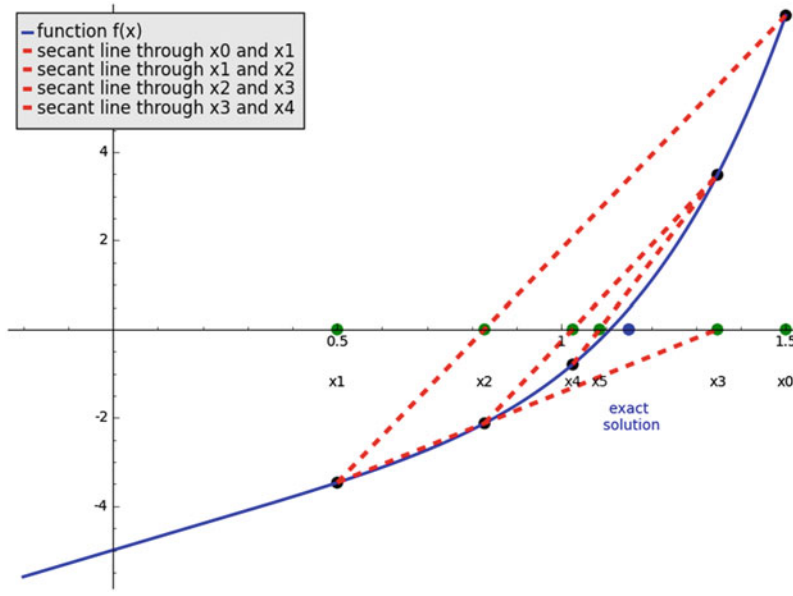
```
# all the code remains the same except for the next line
x1 = 0.5
```

2.3.1 The Algorithm

Starting with two initial points, x_0 and x_1, and using the recursive formula

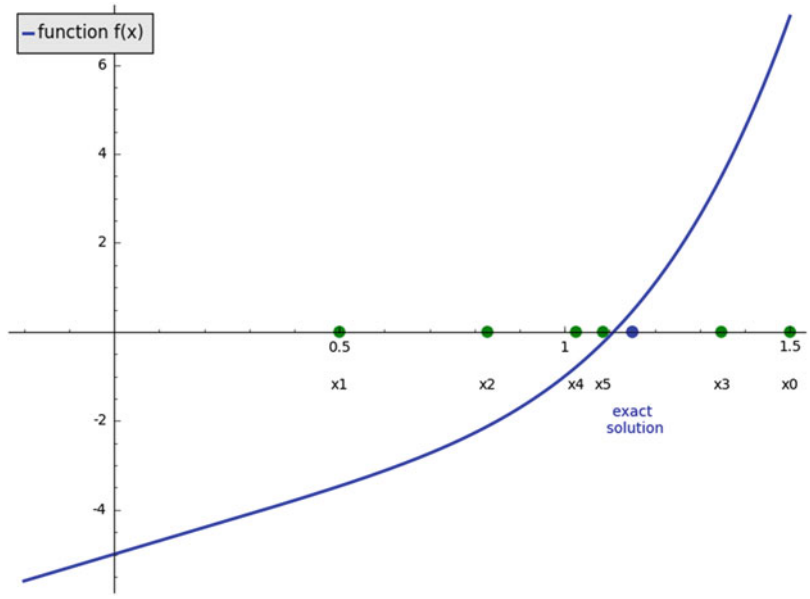
$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}, \quad n = 1, 2, 3, \ldots \tag{2.3}$$

we obtain a sequence of approximations $x_0, x_1, x_2, \ldots, x_n, \ldots$. Like in the case of Newton's Method, the sequence converges to an exact solution if x_0 and x_1 are chosen sufficiently close to that exact solution.



As an exercise, write a Sage code to reproduce this graph. The function used was $f(x) = x^5 + 3x - 5$.

If we redraw the previous graph, this time without graphing the secant lines, we would obtain:



Note how the points $x_0, x_1, x_2, x_3, x_4, x_5$ are approaching the exact solution r.

For example, for $f(x) = x^5 + 3x - 5$ and $x_0 = 1.5, x_1 = 0.5$, we would obtain the following values

$$
\begin{aligned}
x_1 &= 0.828402366863905 \\
x_2 &= 1.34752525292843 \\
x_3 &= 1.02499744774395 \\
x_4 &= 1.08481190418994 \\
x_5 &= 1.11124254590685 \\
x_6 &= 1.10849301757437 \\
x_7 &= 1.10857522427947 \\
x_8 &= 1.10857550808747.
\end{aligned}
$$

Hence, we can conclude that

$$\text{exact solution} \simeq 1.108575.$$

Another way to derive the Secant Method recurrence formula is using the definition of derivative and Newton's Method formula. Starting with Newton's Method formula we obtain:

$$x_{n+1} = x_n - f(x_n) \cdot \frac{1}{f'(x_n)}$$

$$= x_n - f(x_n) \cdot \left(\lim_{h \to 0} \frac{x_n - (x_n - h)}{f(x_n) - f(x_n - h)} \right)$$
$$\approx x_n - f(x_n) \cdot \left(\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \right),$$

for $n = 0, 1, 2, \ldots$.

2.3.2 Pseudocode

```
function printSecant(f, x0, x1, maxSteps, almostZero )
{
    #output the initial values
    output '#','\t', 'xi', '\t\t\t','f(xi)'
    output 0, x0, '\t', f(x0)
    output 1, x1, '\t', f(x1)

    #then we compute the iterations
    #and stop when we get a "good" approximation
    #or tried too many steps
    x2 = x1-f(x1)*((x1-x0)/(f(x1)-f(x0)))
    steps = 1

    while(  steps<maxSteps and  abs(f(x2))>almostZero )
    {
        steps = steps + 1
        output steps, x2, '\t', f(x2)

        #save the values of x1 and x2 into x0 and x1
        x0=x1
        x1=x2

        #then compute the new value of x2
        x2 = x1-f(x1)*((x1-x0)/(f(x1)-f(x0)))
    }
    if( abs(f(x2))<=almostZero )
        output steps, x2, '\t', f(x2)
        output 'solution = ', x2
    else if (steps<maxSteps)
        output 'ERROR: max number of steps!'
}
```

2.3.3 Sage Code

The code below uses the Secant Method Iteration formula repeatedly, until either it reaches a max number of steps allowed, or $f(x_k)$ is essentially 0 ($|f(x_k)|$ is less than some predefined value "*almostZero*").

```
def printSecant(f, x0, x1, maxSteps, almostZero ):
    #output the initial values
    print '#','\t', 'xi', '\t\t\t','f(xi)'
    print 0, x0, '\t', f(x0)
    print 1, x1, '\t', f(x1)

    #then we compute the iterations
    #and stop when we get a "good" approximation
    #or tried too many steps
    x2 = x1-f(x1)*((x1-x0)/(f(x1)-f(x0)))
    steps = 1
    while(  steps<maxSteps and  abs(f(x2))>almostZero ) :
        steps = steps + 1
        print steps, x2, '\t', f(x2)
        #save the values of x1 and x2 into x0 and x1
        x0=x1
        x1=x2
        #compute the new value of x2
        x2 = x1-f(x1)*((x1-x0)/(f(x1)-f(x0)))
    if( abs(f(x2))<= almostZero ) :
        print steps+1, x2, '\t', f(x2)
        print 'solution = ', x2
    elif (steps<maxSteps):
        print 'ERROR: max number of steps!'
```

For example, if you wish to find a zero for the function $f(x) = x^2 - 4\cos(x)$ you would add the following Sage code

```
#define f(x)
var('x')
f(x)= x^2-4*cos(x)

#choose two starting points
x0 = 1.0
x1 = 1.3

#choose the max number of iterations
maxSteps = 100

#choose an error of approximation
almostZero = 0.000000001

#call the function
printSecant(f, x0, x1, maxSteps, almostZero)
```

And obtain the following output:

```
#        xi                f(xi)
0 1.00000000000000     -1.16120922347256
1 1.30000000000000     0.620004685501651
2 1.19557604243186     -0.0365079126687797
3 1.20138293895457     -0.000952854869568753
4 1.20153856040398     1.60122160486331e-6
5 1.20153829932919     -6.98432423007489e-11
solution = 1.20153829932919
```

Another way to write this algorithm is using the Secant Method repeatedly, until two consecutive values, let's say x_n and x_{n+1}, have the first k decimal places equal. In this case, you would conclude that your answer would have k accurate digits. For this, simply replace the lines of code:

```
while(  steps<maxSteps and  abs(f(x2))>almostZero ) :
```

and

```
    if( abs(f(x2))<= almostZero ) :
```

with

```
while(  steps<maxSteps and  abs(x2-x1)>almostZero ) :
```

and

```
    if( abs(x2-x1)<=almostZero ) :
```

respectively.

Running this new version of code, on the previous example, we would obtain:

```
#        xi                f(xi)
0 1.00000000000000     -1.16120922347256
1 1.30000000000000     0.620004685501651
2 1.19557604243186     -0.0365079126687797
3 1.20138293895457     -0.000952854869568753
4 1.20153856040398     1.60122160486331e-6
5 1.20153829932919     -6.98432423007489e-11
6 1.20153829934058     6.66133814775094e-16
solution = 1.20153829934058
```

2.3.4 Error Bounds

It can be shown that for values of x_n close enough to the exact solution, the rate of convergence of Secant Method is equal to the Golden Ratio $\frac{1+\sqrt{5}}{2} \simeq 1.61803398874989$. Hence:

$$|r - x_{n+1}| \leq C\,|r - x_n|^{1.618},\ \text{for } n \geq 1. \tag{2.4}$$

Therefore, for x_n close enough to the exact solution, we could use again the following error estimate:

$$r - x_{n-1} \approx x_n - x_{n-1}. \tag{2.5}$$

2.3.5 Comments on the Secant Method

Although the rate of convergence of the Secant Method is slower than the one for Newton's Method (and therefore it requires more steps in order to get the same precision), the Secant Method doesn't require the computation of the derivative, f', for each point x_i. After the first step, each new iteration requires only one function evaluation.

Overall, the Secant Method is usually comparable to (but slightly slower than) Newton's Method, and it is usually faster than Bisection Method. But, similar to Newton's Method, under certain circumstances, the iterations may not converge.

2.3.6 Examples

Example 59. Using Secant Method, find the solution of $x^2 = 4\sin x$, accurate to within 0.00001.

Solution. Graphing this function we can see that it has two roots. One is $r_1 = 0$, and the other one is around 2. For the second one, we can choose $x_0 = 2.2$ and $x_1 = 2$. Then, using Secant Method we obtain:

$$\begin{aligned} x_2 &= x_1 - f(x_1)\cdot\frac{x_1 - x_0}{f(x_1) - f(x_0)} \\ &= 1.94163302791153 \\ x_3 &= 1.93402441194247 \\ x_4 &= 1.93375491430541 \\ x_5 &= 1.93375376299613 \end{aligned}$$

and since the last two iterations have the first five decimal places identical, we conclude that

$$r_2 \simeq 1.93375$$

We could also get the same result using the Sage algorithm given above. We include the complete code below

```
def printSecant(f, x0, x1, maxSteps, almostZero ):
    #output the initial values
    print '#','\t', 'xi', '\t\t\t','f(xi)'
    print 0, x0, '\t', f(x0)
    print 1, x1, '\t', f(x1)
    #then we compute the iterations
    #and stop when we get a "good" approximation
    #or tried too many steps
    x2 = x1-f(x1)*((x1-x0)/(f(x1)-f(x0)))
    steps = 1
    while(  steps<maxSteps and  abs(x2-x1)>almostZero ) :
        steps = steps + 1
        print steps, x2, '\t', f(x2)
        #save the values of x1 and x2 into x0 and x1
        x0=x1
```

```
        x1=x2
        #compute the new value of x2
        x2 = x1-f(x1)*((x1-x0)/(f(x1)-f(x0)))
    if( abs(x2-x1)<= almostZero ) :
        print steps+1, x2, '\t', f(x2)
        print '----------------------'
        print 'solution = ', x2
    elif (steps<maxSteps):
        print 'ERROR: max number of steps!'


#define f(x)
f(x)= x^2-4*sin(x)

#choose two starting points
x0 = 2.2
x1 = 2.0

#choose the max number of iterations
maxSteps = 100

#choose what makes an approx acceptable
almostZero = 0.00001

#call the main function
printSecant(f, x0, x1, maxSteps, almostZero)
```

We get the following output:

```
#        xi                f(xi)
0 2.20000000000000    1.60601438472164
1 2.00000000000000    0.362810292697273
2 1.94163302791153    0.0418409940408484
3 1.93402441194247    0.00143131333084412
4 1.93375491430541    6.08864118234465e-6
5 1.93375376299613    8.94181173549669e-10
- - - - - - - - - - - - - - - - - - - - - -
solution = 1.93375376299613
```

Example 60. Find all the solutions of the equation $e^x = x^4 - x^3 - 100$, accurate to within 0.00001.

Solution. Graphing the function $f(x) = e^x - x^4 + x^3 + 100$, we can observe that there are three solutions. One of them is around -3, the other one is around 4, and the last one is a little over 8. To find these solutions accurately, we could run the Secant Method three times, once for each of the solutions.

For the first solution, we could use the following Sage code:

```
#define f(x)
f(x)= e^x-x^4+x^3+100
#choose two starting points
x0 = -3.0
x1 = -2.8
```

```
#choose the max number of iterations
maxSteps = 100

#choose what makes an approx acceptable
almostZero = 0.00001

#call the main function
printSecant(f, x0, x1, maxSteps, almostZero)
```

to obtain $r_1 \simeq -2.93944$:

```
#     xi            f(xi)
0 -3.00000000000000     -7.95021293163214
1 -2.80000000000000     16.6432100626252
2 -2.93534683696947     0.521443126747268
3 -2.93972450091570     -0.0359812249035273
4 -2.93944192676814     0.0000701070880942956
5 -2.93944247627490     9.39725806892167e-9
---------------
solution = -2.93944247627490
```

For r_2, we simply change $x_0 = 3.8$ and $x_1 = 4.0$, to obtain $r_2 \simeq 3.73004$:

```
#       xi              f(xi)
0 3.80000000000000     -8.94041550669917
1 4.00000000000000     -37.4018499668558
2 3.73717522903341     -0.888108078153977
3 3.73078265489117     -0.0917692766445555
4 3.73004598112390     -0.000270493391042237
5 3.73004380333116     -8.27927522095706e-8
-----------------------
solution = 3.73004380333116
```

For the last solution we could use $x_0 = 8.0$ and $x_1 = 8.2$, to obtain $r_3 \simeq 8.32527$:

```
#       xi              f(xi)
0 8.00000000000000     -503.042012958272
1 8.20000000000000     -228.899292667646
2 8.36699279296928     87.5023802602154
3 8.32081014565299     -9.02152589511661
4 8.32512656803232     -0.306694702937421
5 8.32527847273938     0.00113321161097701
6 8.32527791353026     -1.41546479426324e-7
-----------------------
solution = 8.32527791353026
```

2.3.7 *Sage Library*

We present, below, a Sage Library function that implements the Secant Method. We will apply this to the examples given above in this section. The difference between Newton–Raphson's Method and Secant Method is that the method `scipy.optimize.newton` is not given any derivative function.

Example 61. Use Secant method to find a solution of the equation $x^2 = 4\sin(x)$.

Solution. Using $x_0 = 1.5$, and the following Sage code

```
#import the following library
import scipy.optimize

#the function f(x) from our problem
f(x) = x^2-4*sin(x)

#if no derivative of f is given
#then newton becomes secant method!
####################################
#call the secant method and output the result
print "solution = " , scipy.optimize.newton(f, 2.2)
```

we obtain:

```
solution =  1.93375376282702
```

Example 62. Using Sage, find all the solutions of the equation $e^x = x^4 - x^3 - 100$.

Solution. Using $x_0 = -3$, $x_0 = 3.8$, and $x_0 = 8$ and the following Sage code

```
#import the following library
import scipy.optimize

#the function f(x) from our problem
f(x) = e^x-x^4+x^3+100

#if no derivative of f is given
#then newton becomes secant method!
####################################
#call the secant method and output the result
print "solution1 = " , scipy.optimize.newton(f, -3.0)
print "solution2 = " , scipy.optimize.newton(f, 3.8)
print "solution3 = " , scipy.optimize.newton(f, 8.0)
```

we obtain the first solution:

```
solution1 = -2.93944247634856
solution2 = 3.73004380266438
solution3 = 8.32527791360010
```

2.4 Fixed-Point Iteration

We start with the following definition

Definition 63. An element c from the domain of a function f is called a **fixed point of** f if $f(c) = c$.

Graphically, the *fixed points* of a function f are the x-coordinates (or the y-coordinates) of the points where the graph of f intersects the graph of $y = x$.

In this subsection, rather than trying to find a zero of an equation of the form $g(x) = 0$, we will try to solve an equation of the form $f(x) = x$. That is, we are trying to find a *fixed point* of f.

Any equation of the form $g(x) = 0$ can be translated into an equation of the form $f(x) = x$ and vice versa. In fact, there are multiple equivalent ways one can do so, and some of these are "better" (their corresponding iterations will have a faster convergence) than others.

The following two results provide us the theoretical background for the Fixed-Point Iteration.

Theorem 64. *If f is a continuous function on the interval $[a, b]$ such that $a \leq f(x) \leq b$, then it has at least one fixed point in that interval.*

Proof. *We use Intermediate Value Theorem (see page 66) for the function $g(x) = f(x) - x$ defined on $[a, b]$.*

If $g(a) = 0$, then we are done. a is a fixed point for f.
If $g(b) = 0$, then we are done. b is a fixed point for f.
If both $g(a), g(b) \neq 0$ then it must be the case that $g(a) = f(a) - a > 0$ and $g(b) = f(b) - b < 0$. Therefore, by Intermediate Value Theorem, there must be a value r in (a, b) such that $g(r) = 0$, which means $f(r) = r$. ■

The following result (see [1]) is a consequence of the Banach Fixed Point Theorem that was published in 1922.

Theorem 65. *Let f be a function such that both f, f' are continuous on the interval $[a, b]$, $a \leq f(x) \leq b$, and*

$$\max_{a \leq x \leq b} \left| f'(x) \right| < 1. \tag{2.6}$$

Then f has a unique fixed point in $[a, b]$ and for any initial value x_0 from $[a, b]$, the sequence $x_0, x_1 = f(x_0), x_2 = f(x_1), \ldots$ will converge to that fixed point.

In practice, however, these theorems are often skipped (since finding such an interval $[a, b]$ may prove challenging!), and we usually use a less restrictive result (such as the Theorem below) or even go straight and use the computing power of Sage (or other Software packages).

The following is a more practical result that can be used

Theorem 66 (See Corollary 3.4.3 in [1]). *If f, f' are continuous on some interval $[c, d]$ containing the fixed point, r, such that*

$$\left| f'(r) \right| < 1$$

then there must be some interval $[a, b]$ containing r such that the previous theorem can be applied to. If, $|f'(r)| > 1$ then the Fixed-Point Iteration method will not converge.

We will use this Theorem in our next examples.

2.4.1 The Algorithm

Start by choosing an initial value x_0. Then, use the following iteration formula

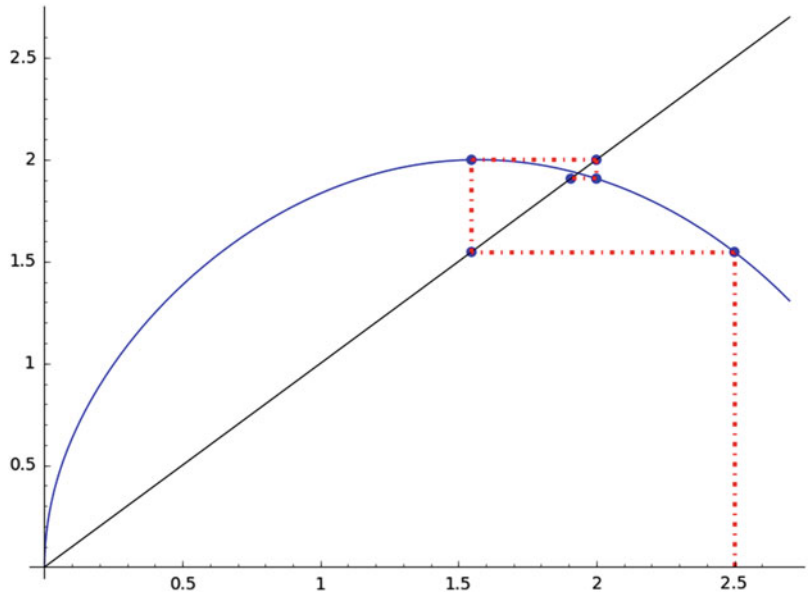
$$x_n = f(x_{n-1}), \quad n \geq 1,$$

then compute enough iterations until you get convergence. In practice, that means, until the difference between two consecutive values becomes sufficiently small.

For example, if we try to find the largest root of the equation $x^2 = 4\sin(x)$, one could choose to the transform this equation into $x = 2\sqrt{\sin x}$, and hence use $f(x) = 2\sqrt{\sin x}$. Then choosing, let's say $x_0 = 2.\mathbf{5}$, we would obtain:

```
#          xi                  f(xi)
0 2.50000000000000     1.54721962772446
1 1.54721962772446     1.99972206319223
2 1.99972206319223     1.90726301723324
3 1.90726301723324     1.94311820681003
4 1.94311820681003     1.93026962424369
5 1.93026962424369     1.93502686037230
6 1.93502686037230     1.93328543686329
7 1.93328543686329     1.93392562006977
8 1.93392562006977     1.93369064094332
9 1.93369064094332     1.93377693933650
10 1.93377693933650     1.93374525205397
11 1.93374525205397     1.93375688797486
12 1.93375688797486     1.93375261525684
13 1.93375261525684     1.93375418421809
————————————————————————
solution = 1.93375261525684
```

It may be useful to see a graph with the first three iterations.



The Sage code for getting this image is given below, at the end of this subsection.

Please note that from the previous graph, we could see that the fixed point r is inside the interval $\left[\frac{\pi}{2}, \frac{3\pi}{4}\right]$. And since $f'(x) = \frac{\cos x}{\sqrt{\sin x}}$, we have that

$$\left|f'(r)\right| \leq \max_{\frac{\pi}{2} \leq x \leq \frac{3\pi}{4}} \left|\frac{\cos x}{\sqrt{\sin x}}\right| = \frac{\cos \frac{3\pi}{4}}{\sqrt{\sin \frac{3\pi}{4}}} = \frac{1}{\sqrt[4]{2}} < 1$$

hence, using the previous theorem, we expected to be able to use the Fixed-Point Iteration method.

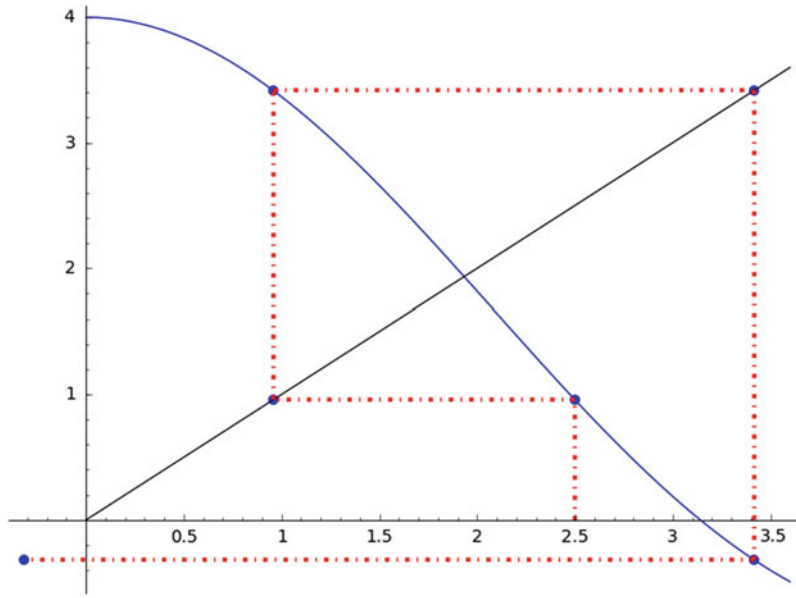
We could also transform $x^2 = 4\sin(x)$, into $x = \frac{4\sin(x)}{x}$, and hence use $f(x) = \frac{4\sin(x)}{x}$. Then choosing, the same $x_0 = 2.5$, we could not obtain convergence in less than 100 steps

```
#          xi                 f(xi)
0 2.50000000000000    0.957555430566331
1 0.957555430566331    3.41614526513983
2 3.41614526513983    -0.317452931107809
3 -0.317452931107809    3.93315347572487
4 3.93315347572487    -0.723542560382597
5 -0.723542560382597    3.66001328626969
6 3.66001328626969    -0.541537830270961
7 -0.541537830270961    3.80733802699290
8 3.80733802699290    -0.648900101565583
9 -0.648900101565583    3.72513690203326
10 3.72513690203326    -0.591640129711840
11 -0.591640129711840    3.77069165174771
12 3.77069165174771    -0.624200055562205
13 -0.624200055562205    3.74526311018305
14 3.74526311018305    -0.606278389468001
15 -0.606278389468001    3.75941547072754
16 3.75941547072754    -0.616331796398259
17 -0.616331796398259    3.75152338144045
18 3.75152338144045    -0.610749950572025
19 -0.610749950572025    3.75592003835844
20 3.75592003835844    -0.613867187445809
21 -0.613867187445809    3.75346923799843
22 3.75346923799843    -0.612131926781433
23 -0.612131926781433    3.75483493883959
24 3.75483493883959    -0.613099629027355
25 -0.613099629027355    3.75407376948671
26 3.75407376948671    -0.612560510820985
27 -0.612560510820985    3.75449796288972
28 3.75449796288972    -0.612861027717393
29 -0.612861027717393    3.75426155025950
30 3.75426155025950    -0.612693564794332
31 -0.612693564794332    3.75439330432377
32 3.75439330432377    -0.612786899632573
33 -0.612786899632573    3.75431987578976
34 3.75431987578976    -0.612734884834889
35 -0.612734884834889    3.75436079823005
36 3.75436079823005    -0.612763873848995
37 -0.612763873848995    3.75433799163164
38 3.75433799163164    -0.612747718105886
```

```
39 -0.612747718105886     3.75435070200217
40 3.75435070200217     -0.612756721944439
        [  ... OMITTED ...]
87 -0.612753499771556     3.75434615336968
88 3.75434615336968     -0.612753499778813
89 -0.612753499778813     3.75434615336397
90 3.75434615336397     -0.612753499774768
91 -0.612753499774768     3.75434615336715
92 3.75434615336715     -0.612753499777022
93 -0.612753499777022     3.75434615336537
94 3.75434615336537     -0.612753499775766
95 -0.612753499775766     3.75434615336636
96 3.75434615336636     -0.612753499776466
97 -0.612753499776466     3.75434615336581
98 3.75434615336581     -0.612753499776076
99 -0.612753499776076     3.75434615336612
100 3.75434615336612     -0.612753499776293
```

In fact, we could see that the sequence seems to diverge.

The corresponding graph would look like:



Similar to the previous case, note from the graph that the fixed point r is inside the interval $\left[\frac{\pi}{2}, \frac{3\pi}{4}\right]$. And since $f'(x) = \frac{4x\cos(x) - 4\sin x}{x^2}$, we have that

$$\left|f'(r)\right| \geq \min_{\frac{\pi}{2} \leq x \leq \frac{3\pi}{4}} \left|\frac{4x\cos(x) - 4\sin x}{x^2}\right| = \left|\frac{4x\cos(\frac{\pi}{2}) - 4\sin\frac{\pi}{2}}{(\frac{\pi}{2})^2}\right| = \frac{16}{\pi^2} > 1$$

hence the iteration method would not converge.

Here come the Sage code of obtaining the previous graphs. For the first one,

```
var('x')
f(x) = 2*sqrt(sin(x))

#interval is (a,b)
a = 0
b = 2.7

#plot the functions y=f(x) and y=x
allPlots = plot( f(x), (x, a, b) )
allPlots += plot( x, (x, a, b), color = 'black')

x0=2.5
#plot the first vertical line
allPlots += line([(x0,0),(x0,f(x0))], color='red',
              linestyle = "-.", thickness=3)
#plot the point of intersection with  y = f(x)
allPlots += list_plot(  [(x0,f(x0))]  , size = 50)

#plot the first horizontal  line
allPlots += line([(x0,f(x0)), (f(x0),f(x0))],
              color='red', linestyle = "-.", thickness=3)
#plot the point of intersection with  y = x
allPlots += list_plot(  [ (f(x0) , f(x0)) ]  , size = 50)

x1=f(x0)
#plot the second vertical line
allPlots += line([(x1,f(x0)),(x1,f(x1))],
              color='red', linestyle = "-." , thickness=3)
#plot the point of intersection with  y = f(x)
allPlots += list_plot(  [(x1,f(x1))]  , size = 50)

#plot the second horizontal  line
allPlots += line([(x1,f(x1)), (f(x1),f(x1))],
              color='red', linestyle = "-.", thickness=3)

#plot the point of intersection with  y = x
allPlots += list_plot(  [(f(x1),f(x1))]  , size = 50)


x2=f(x1)
#plot the second vertical line
allPlots += line([(x2,f(x1)),(x2,f(x2))],
              color='red', linestyle = "-." , thickness=3)

#plot the point of intersection with  y = f(x)
allPlots += list_plot(  [(x2,f(x2))]  , size = 50)
```

```
#plot the second horizontal  line
allPlots += line([(x2,f(x2)), (f(x2),f(x2))],
              color='red', linestyle = "-.", thickness=3)

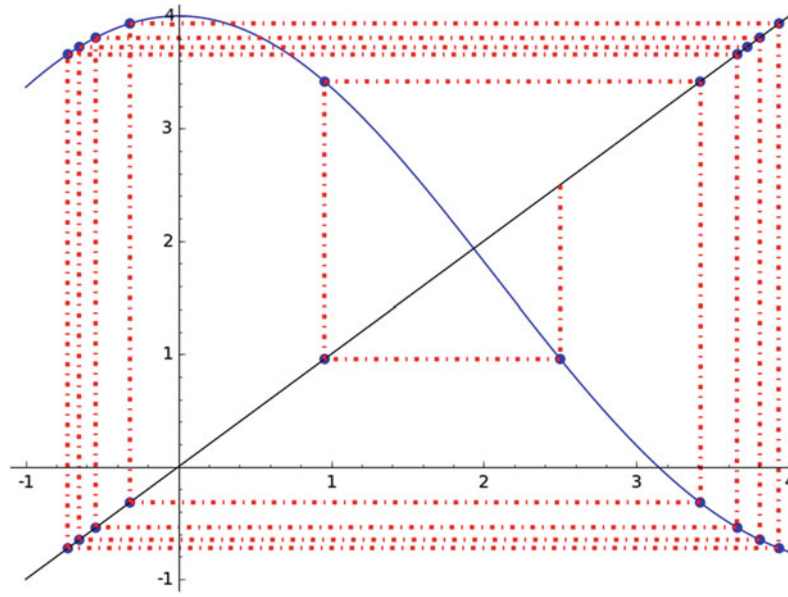
#plot the point of intersection with  y = x
allPlots += list_plot(  [(f(x2),f(x2))]  , size = 50)

#display the graph
show(allPlots)
```

For the second graph, we could just change the previous code with the corresponding function. Below, we included more than three steps (we used 10); hence we felt the need to redesign our code. And hence, we used the following Sage code,

```
var('x')
f(x) = 4*sin(x)/x
#interval is (a,b)
a = -1
b = 4
#plot the functions y=f(x) and y=x
allPlots = plot( f(x), (x, a, b) )
allPlots += plot( x, (x, a, b), color = 'black')
x0=2.5
steps = 0
while steps<10:
    #plot the first vertical line
    allPlots += line([(x0,x0),(x0,f(x0))], color='red',
              linestyle = "-.", thickness=3)
    #plot the point of intersection with  y = f(x)
    allPlots += list_plot(  [(x0,f(x0))]  , size = 50)
    #plot the first horizontal  line
    allPlots += line([(x0,f(x0)), (f(x0),f(x0))],
              color='red', linestyle = "-.", thickness=3)
    #plot the point of intersection with  y = x
    allPlots += list_plot(  [ (f(x0) , f(x0)) ]  , size = 50)
    steps = steps+1
    x0=f(x0)
#display the graph
show(allPlots)
```

we would obtain:



2.4.2 Pseudocode

```
function printFixedPoint(f, x0, maxSteps, almostZero )
{
    #output the initial values
    output '#','\t', 'xi', '\t\t\t','f(xi)'
    output 0, x0, '\t', f(x0)

    #then we compute the iterations and stop when we either
    #get a "good" approximation or tried too many steps
    steps = 0
    while(  steps<maxSteps and  |x0-f(x0)|>almostZero )
    {
        steps = steps + 1
        x0 = f(x0)
        print steps, x0, '\t', f(x0)
    }
    if( |x0-f(x0)| <= almostZero )
    {
        output steps+1, x0, '\t', f(x0)
        output '----------------------'
        output 'solution = ', x0
    }
    else if (steps<maxSteps):
        print 'ERROR: max number of steps!'
}
```

2.4.3 Sage Code

```
def printFixedPoint(f, x0, maxSteps, almostZero ):
    #output the initial values
    print '#','\t', 'xi', '\t\t\t','f(xi)'
    print 0, x0, '\t', f(x0)

    #then we compute the iterations and stop when we
    # either get a "good" approximation or tried too many steps
    x1 = f(x0)
    steps = 0
    while(  steps<maxSteps and  abs(x1-x0)>almostZero ) :
        steps = steps + 1
        print steps, x1, '\t', f(x1)

        x0=x1
        x1=f(x0)

    if( abs(x1-x0)<= almostZero ) :
        print steps+1, x1, '\t', f(x1)
        print '----------------------'
        print 'solution = ', x1
    elif (steps<maxSteps):
        print 'ERROR: max number of steps!'
```

For the example given above, one would only need to add

```
#define f(x)
f(x)= 2*sqrt(sin(x))

#choose a starting point
x0 = 2.0

#choose the max number of iterations
maxSteps = 100

#choose what makes an approx acceptable
almostZero = 0.00001

#call the main function
printFixedPoint(f, x0, maxSteps, almostZero)
```

2.4.4 Examples

Example 67. Solve $x^2 - x - 7 = 0$, correct to three decimal places.

Solution. The problem is equivalent to $x = \frac{x+7}{x}$. Hence we use $f(x) = 1 + \frac{7}{x}$. Let's choose some starting value, say $x_0 = 3.0$. Then we have

$$\begin{aligned}
x_1 &= 1 + \frac{7}{x_0} \simeq 3.33333333333333 \\
x_2 &= 1 + \frac{7}{x_1} = 3.1 \\
x_3 &= 1 + \frac{7}{x_2} \simeq 3.25806451612903 \\
x_4 &= 1 + \frac{7}{x_3} \simeq 3.14851485148515 \\
&\cdots \\
x_{16} &= 1 + \frac{7}{x_{15}} = 3.19209328550934 \\
x_{17} &= 1 + \frac{7}{x_{16}} = 3.19291836857552
\end{aligned}$$

and hence

$$x \simeq 3.192.$$

A similar work can be done to find the second solution.

Compare the previous answer with the exact solution, obtained from using quadratic formula

$$\begin{aligned}
x &= \frac{1 - \sqrt{29}}{2} \simeq -2.19258240357, \\
x &= \frac{1 + \sqrt{29}}{2} \simeq 3.19258240357.
\end{aligned}$$

Using the Sage code above and adding

```
#define f(x)
f(x)= 1+7/x

#choose a starting point
x0 = 3.0

#choose the max number of iterations
maxSteps = 100

#choose what makes an approx acceptable
almostZero = 0.001

#call the main function
printFixedPoint(f, x0, maxSteps, almostZero)
```

or simply

```
f(x)= 1+7/x
printFixedPoint(f, 3.0, 100, 0.001)
```

we obtain

```
#     xi              f(xi)
0 3.00000000000000    3.33333333333333
1 3.33333333333333    3.10000000000000
2 3.10000000000000    3.25806451612903
3 3.25806451612903    3.14851485148515
4 3.14851485148515    3.22327044025157
5 3.22327044025157    3.17170731707317
6 3.17170731707317    3.20701322669948
7 3.20701322669948    3.18271628620756
8 3.18271628620756    3.19937920019287
9 3.19937920019287    3.18792445721283
10 3.18792445721283    3.19578603381337
11 3.19578603381337    3.19038443936350
12 3.19038443936350    3.19409294805755
13 3.19409294805755    3.19154549157906
14 3.19154549157906    3.19329475906566
15 3.19329475906566    3.19209328550934
16 3.19209328550934    3.19291836857552
17 3.19291836857552    3.19235169583210
- - - - - - - - - - - - - - - - - - -
solution = 3.19291836857552
```

Before we run Sage, we should first verify the conditions of Theorem 65 (see page 102). Note that both, $f(x) = 1 + \frac{7}{x}$, and $f'(x) = -\frac{7}{x^2}$ are continuous on $[3, 3.5]$, and $3 \leq f(x) \leq 3.5$. Also

$$\max_{a \leq x \leq b} \left| f'(x) \right| = \max_{3 \leq x \leq 3.5} \frac{7}{x^2} = \frac{7}{9} < 1$$

hence any value from the interval $[3, 3.5]$ would lead to a convergent sequence of iterations.

2.4.5 *Sage Library*

If we wish to solve the previous problem using Sage Library functions, we could use the code given below

Example 68. Solve $x^2 - x - 7 = 0$, correct to three decimal places.

Solution. The problem is equivalent to $x = \frac{x+7}{x}$. Hence we use $f(x) = 1 + \frac{7}{x}$. Let's choose some starting value, say $x_0 = 3.0$. Then using

```
#import the following library
import scipy.optimize

#the function f(x) from our problem
def f(x):
    return  1+7/x

#if no derivative of f is given
#then newton becomes secant method!
```

```
###################################
#call the fixed_point method and output the result
print "solution = " , scipy.optimize.fixed_point(f,[3.0])
```

we obtain

```
solution =  [ 3.1925824]
```

If we wish to find both solutions, using the Fixed-Point Iteration, then we could use the following code

```
#import the following library
import scipy.optimize

#the function f(x) from our problem
def f(x):
    return  1+7/x

#if no derivative of f is given
#then newton becomes secant method!
###################################
#call the fixed_point method and output the result
print "solutions: " , scipy.optimize.fixed_point(f,[3.0, -3.0])
```

to obtain

```
solutions:  [ 3.1925824 -2.1925824]
```

2.5 An Application to Business

Suppose we need to borrow $200,000 in order to be able to buy a new home. But our monthly salary only allows us to pay around $800 per month. Assume a 30-year mortgage, and the following formula

$$\text{PV} = \text{PMT}\left(\frac{1-(1+i)^{-n}}{i}\right), \tag{2.7}$$

given in [2, p. 159], where

$$\begin{aligned} \text{PV} &= \text{present value of all payments,} \\ \text{PMT} &= \text{periodic payment,} \\ i &= \text{rate per period,} \\ n &= \text{number of periods.} \end{aligned}$$

What **annual interest rates** can we afford to pay?

Solution. Using the information given above, we would first need to solve for i—the **monthly interest rate**, from the equation

$$200{,}000 = 800\left(\frac{1-(1+i)^{-30*12}}{i}\right),$$

or

$$250i - 1 + (1 + i)^{-360} = 0.$$

How would you solve for i in this equation?

Using one of the methods we learned in this chapter, solving this equation should not be too challenging. For example, if we choose to use the Newton–Raphson's Method, we would use the following Sage code:

```
#define f(x)
var('x')
f(x)= 250*x-1+(1+x)^(-360)

#compute f'(x)
fp(x)=diff(f(x),x)

#choose a starting point
x0=3.0

#choose the max number of iterations
maxSteps = 100

#choose what makes an approx acceptable
almostZero = 0.000000001

#call the function
printNewtonRaphson(f, fp, x0, maxSteps, almostZero)
```

and obtain the following **monthly interest rate** $i = 0.00216126116310383$.

Hence, we would conclude that the **maximum annual interest rate** r that we can afford would be

$$r = 0.00216126116310383 * 12 = 0.02593513395724596,$$

or approximately 2.59 %.

2.6 Exercises

1. Do five iterations (by hand) of the Bisection Method to compute a root of $f(x) = x^3 - 3$ in the interval $[0, 3]$. Then give an estimate of the error of approximation.
2. Using Bisection Method find the largest root of $f(x) = x^6 - x - 7$, accurate to within 0.001.
3. Using Bisection Method find all the root of $f(x) = x^3 - 2x + 2$, accurate to within 0.0001.
4. Find all the roots of $f(x) = 2 - x^2 e^{-\pi x}$, accurate to within 0.0001.
5. Find the points of intersection of the curves $y = 5x^2$ and $y = e^x$ by finding the zeros of $e^x - 5x^2 = 0$ correct to five decimal places.
6. How many steps of the Bisection Method are needed to compute a root of $3.7x^8 - 7.5x^3 + 3x - 10.3$ in the interval $[-1.5, -0.5]$ with precision of 10 decimal places?
7. How many steps of the Bisection Method are needed to compute a root of $3.7x^8 - 7.5x^3 + 3x - 10.3$ in the interval $[-1.5, -0.5]$ with an error of approximation less than 0.00005?

8. How many steps of the Bisection Method are needed to compute a root of $3.7x^8-7.5x^3+3x-10.3$ in the interval $[-1.5,-0.5]$ with an error of approximation less than 10^{-17}?
9. How many steps of the Bisection Method are needed to compute a root of the equation $x = e^{-x}$ in the interval $[-1, 1]$ with an error tolerance of 10^{-4}?
10. Using Newton Method, solve the equation $f(x) = x^6 - x - 7$.
11. Find the three points of intersection of the following graphs: $y = 3 - x^4$ and $y = \frac{1}{x}$.
12. Use Newton Method to compute $\sqrt{7}$.
13. Using previous example, find the Newton's Iteration formula for computing $\sqrt{a}$.
14. Using Newton method, develop a strategy to find the value of $\ln 7$, correct to five decimal places.
15. Find the zeros of $f(x) = x^3 + 4x^2 - 7$ using Newton's Method, correct to two significant figures.
16. Find the solution of $x = 2\sin x$ using four steps of the Newton's Method.
17. Using Secant Method, find the solution of $\cos x = x$, accurate to within 0.0001.
18. Using Secant method, develop a strategy to find the value of $\ln 7$, correct to five decimal places.
19. Using Fixed-Point Iteration, develop a strategy to solve $x^2 - x - 2 = 0$, correct to three decimal places.
20. Using Fixed-Point Iteration, solve $x^2 - 4\sin x = 0$, correct to three decimal places.
21. Decide whether or not you can find the solution of $x^2 = 5$ using the Fixed Point Iteration method applied to $f(x) = -\frac{x^2}{5} + x + 1$. Include a complete argument for your answer.
22. Decide whether or not you can find the solution of $x^2 = 5$ using the Fixed Point Iteration method applied to $f(x) = -x^2 + x + 5$. Include a complete argument for your answer.
23. Decide whether or not you can find the solution of $x^2 = 5$ using the Fixed Point Iteration method applied to $f(x) = \frac{5}{x}$. Include a complete argument for your answer.

2.7 Computer Exercises

For all these problems, you are expected to use Sage. Make sure to include a conclusion with your answer.

1. Using Bisection Method find the largest root of $f(x) = x^6 - x - 7$, accurate to within 0.000001.
2. Using Bisection Method, find a solution of the equation $x^3 = 3x - 1$ in [0, 1], with an error less than 0.00000025.
3. Using Bisection Method, find a solution of the equation $x^3 = 2\sin(x)$ in [0.5, 2], with an error less than 0.00000025.
4. Using Bisection Method, find a solution of the equation $x = e^x$ in [0, 1], with accuracy within 10^{-6}.
5. Find all the root of $f(x) = 2 - x^2e^{-\pi x}$, accurate to within 0.0000001.
6. Find the points of intersection of the curves $y = x^4 - 10$ and $y = \ln(x)$ by finding the zeros of $x^4 - 10 - \ln(x) = 0$ correct to seven decimal places.
7. Using Bisection Method find all the root of $f(x) = x^3 - 2x + 2$, accurate to within 0.0001.
8. Using Newton Method find the largest root of $f(x) = x^6 - x - 7$.
9. Find the three points of intersection of the following graphs: $y = 3 - x^4$ and $y = \frac{1}{x}$. Include a graph.
10. Use Newton Method to compute $\sqrt{7}$, correct to seven decimal places.
11. Find the zeros of $f(x) = x^3 + 4x^2 - 7$ using Newton's Method, correct to seven significant figures.
12. Find the solution of $x = 2\sin x$ using ten steps of the Newton's Method.
13. Find the solution of $\sin x = e^x - 2$ using ten steps of the Newton's Method.
14. Find the root for $x^3 - x\sin x + 6$ using ten steps of the Newton's Method.

15. Use Newton's Method to compute $\sqrt{3}$ with 100 decimal places of accuracy. Note: you may need to modify the Sage code given so that it will carry all the operations with sufficient significant digits.
16. Using Secant Method, find the solution of $\cos x = x$, accurate to within 0.0001.
17. Using Secant Method, find both solutions of $\cos x = x^2$, accurate to within 0.0000001.
18. Find all solutions of the equation $e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3}$, accurate to within 0.0000001.
19. Using Fixed-Point Iteration, develop a strategy to solve $x^2 - x - 2 = 0$, correct to six decimal places.
20. Using Fixed-Point Iteration, solve $x^2 - 4\sin x = 0$, correct to six decimal places.
21. Using Fixed Point Iteration method, compute the first ten iterations for $f(x) = 1 + \frac{2}{x}$, with $x_0 = 1$.
22. Try to solve these problems using Sage Library functions.
23. Suppose you need to borrow \$150,000 in order to be able to buy a new home. But your monthly salary only allows us to pay around \$650 per month. Assuming a 30-year mortgage, and the formula (2.7) given on page (112), what **annual interest rates** could you afford to pay?
24. Suppose you need to borrow \$150,000 in order to be able to buy a new home. But your monthly salary only allows us to pay around \$800 per month. Assuming a 20-year mortgage, and the formula (2.7) given on page (112), what **annual interest rates** could you afford to pay?
25. Suppose you need to borrow \$450,000 in order to be able to buy a new home. But your monthly salary only allows us to pay around \$1,500 per month. Assuming a 30-year mortgage, and the formula (2.7) given on page (112), what **annual interest rates** could you afford to pay?

References

1. Atkinson, K., Han, W.: Elementary Numerical Analysis, 3rd edn. Wiley, New York (2003)
2. Barnett, R., Ziegler, M., Byleen, K. Finite Mathematics for Business, Economics, Life Sciences, and Social Sciences (11th ed.). Pearson Prentice Hall, Boston, MA (2008)
3. Blanco-Silva, F.J.: Learning SciPy for Numerical and Scientific Computing. Packt Publishing, Birmingham (2013)
4. Cheney, W., Kincaid, D.: Numerical Mathematics and Computing, 7th edn. Brooks/Cole: Cengage Learning, Boston (2013)
5. Epperson, J.F.: An Introduction to Numerical Methods and Analysis, revised edn. Wiley, Hoboken (2007)
6. Stein, W.A., et al.: Sage Mathematics Software (Version 6.3), The Sage Development Team (2014). Sage official webpage: http://www.sagemath.org
7. Sage Cell Server: http://sagecell.sagemath.org/
8. Stein, W.A.: Sage for Power Users (2012). http://www.wstein.org/books/sagebook/sagebook.pdf
9. Sullivan, M., Miranda, K.: Calculus – Early Transcendentals. W.H. Freeman and Company, New York (2014)

Chapter 3
Polynomial Interpolation

In this chapter we will deal with the following type of problems (see also [1, 4, 5]): suppose you are given a set of data points,

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}, \ n \geq 1.$$

Can you find a function whose graph passes through every one of the points? That is, can you find a function p such that $p(x_0) = y_0, \dots, p(x_n) = y_n$?

The process of finding and evaluation such a function is what we call **interpolation**. $x_0, x_1, \dots, x_n$ are called **nodes**.

The process of finding a polynomial that passes through a given set of data points is called **polynomial interpolation**. This is the topic of the current chapter.

There are many reasons why we may want to use interpolation. For example, if you measure the outside temperature every hour, then using interpolation you can *estimate* the outside temperature for any minute of the day.

We can also apply this technique when we are missing some data and need some values to fill in.

They are extensively used in Computer Science. Most image and video compression/decompression algorithms use interpolation. Using various sampling techniques, we can compress images ("lossy" compression), and then, using interpolation techniques, we can decompress them. Image restoration, digital sounds, GPS coordination, video games, fonts, etc. all use interpolation techniques.

Interpolation techniques help us make the transition from discrete into continuous, and in particular, from digital into analog.

[0]The original version of this chapter was revised. An erratum to this chapter can be found at DOI 10.1007/978-3-319-16739-8_8

Electronic supplementary material The online version of this chapter (doi: 978-3-319-16739-8_3) contains supplementary material, which is available to authorized users.

© Springer International Publishing Switzerland 2015

G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8_3

3.1 Interpolation

In this chapter, for simplicity, by **polynomial of degree** $n, n \geq 1$, we mean a function $p(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$ where $a_0, a_1, \ldots, a_n$ are constants, and some of them (if not all!) can be zero. This is somewhat different than most of the other Mathematics subjects, where such a polynomial is said to be of **degree at most** n, but this is a common notation in Numerical Analysis language.

IMPORTANT NOTE: Throughout this chapter, we will assume that all the nodes $x_i's$ are distinct.

3.1.1 Linear Interpolation

Perhaps the simplest polynomial interpolation is the *linear interpolation*. Given two points (x_0, y_0) and (x_1, y_1), we need to find the linear function $p(x) = a_1 x + a_0$ that passes through these two points. That means, $p(x_0) = y_0$ and $p(x_1) = y_1$.

Since $p(x)$ is, in this case, a linear function, and since we are given two points, (x_0, y_0) and (x_1, y_1), the polynomial $p(x)$ is the unique line that passes through the given points. Therefore $a_1 = \frac{y_1 - y_0}{x_1 - x_0}$ is the slope of the line, and $a_0 = y_1 - a_1 x_1$. Hence

$$\begin{aligned} p(x) &= a_1 x + a_0 \qquad (3.1)\\ &= \frac{y_1 - y_0}{x_1 - x_0} \cdot x + y_1 - \frac{y_1 - y_0}{x_1 - x_0} \cdot x_1 \\ &= \left(\frac{x - x_1}{x_0 - x_1}\right) \cdot y_0 + \left(\frac{x - x_0}{x_1 - x_0}\right) \cdot y_1. \qquad (3.2) \end{aligned}$$

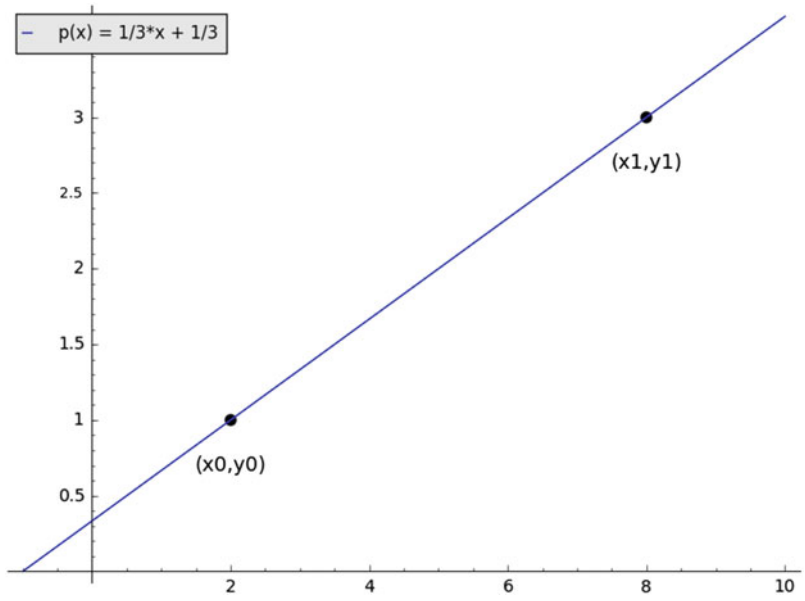
At this point, formula (3.1) may seem more intuitive than its equivalent form (3.2), but the latter will be easier to use and memorize once we go to quadratic and higher degree interpolation. To anticipate, the form in (3.2) is called *Lagrange Form* (see the subsection *Lagrange Form* below).

Example 69. You are given the following data points $\begin{array}{c||c|c} x & 2 & 8 \\ \hline\hline y & 1 & 3 \end{array}$. Find the linear function that interpolates this table.

Solution. Using formula (3.2) we obtain

$$\begin{aligned} p(x) &= \left(\frac{x-8}{2-8}\right) \cdot 1 + \left(\frac{x-2}{8-2}\right) \cdot 3 \\ &= \frac{1}{3}x + \frac{1}{3}. \end{aligned}$$

Hence, if you wish to estimate the value of y for $x = 7.5$, we get $p(7.5) = \frac{1}{3} \cdot 7.5 + \frac{1}{3} = \frac{8.5}{3} \simeq 2.83333$.



The graph above was given by the Sage code (see also [2, 3, 6–8]):

```
#given points
x0 = 2
y0 = 1
x1 = 8
y1 = 3
#compute the linear interpolation
p(x)=(((x-x1)/(x0-x1)))*y0+(((x-x0)/(x1-x0)))*y1

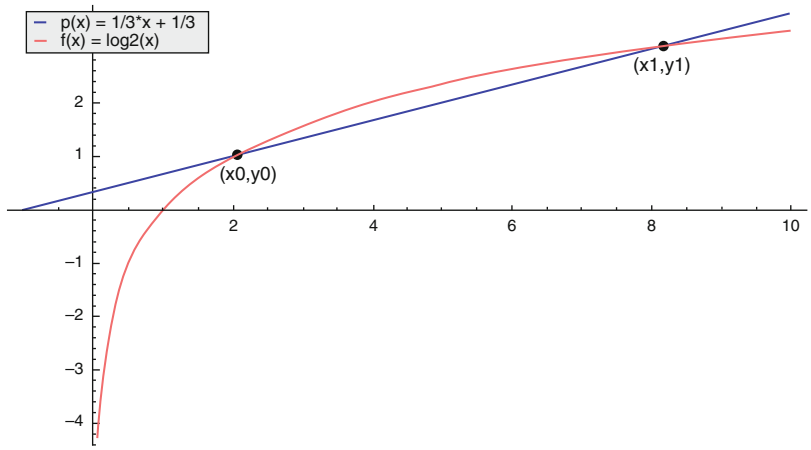
#graph the linear interpolation
pl=plot(p(x),  (x,-1,10),
       legend_label =   "    p(x) = "+str(p(x)))

pl += list_plot([(x0,p(x0)),(x1,p(x1))],
               color = 'black', size = 70)

pl+= text( '(x0,y0)' , ( x0, p(x0)-0.3 ),
         fontsize = 14, color = 'black')

pl+= text( '(x1,y1)' , ( x1, p(x1)-0.3 ),
         fontsize = 14, color = 'black')
show(pl)
```

In the next graph, we will include both the polynomial of interpolation, $p(x)$, and the function $f(x) = \log_2 x$ (this is where our data points came from!).



Please note that the linear function will provide fairly good estimations for values of x close to the nodes (x_0 or x_1). For values of x further away from these nodes, the estimations can be quite bad. In our example, we obtained that $p(7.5) \simeq 2.83333$, while $\log_2(7.5) \simeq 2.90689$. On the other side, $p(5) = 2$, and $\log_2(5) \simeq 2.321928$, hence the error of approximation is about five times higher in this case.

Another note, which may be obvious: if the original function f is a linear function, then the linear interpolation will be equal to f, $p(x) = f(x)$. We say that linear interpolation is **exact** for linear functions.

Example 70. Knowing that $\sqrt{109} \simeq 10.4403$ and $\sqrt{111} \simeq 10.5356$, estimate $\sqrt{110}$.

Solution. Using formula (3.2) we obtain

$$\begin{aligned} p(x) &= \left(\frac{x-111}{109-111}\right) \cdot 10.4403 + \left(\frac{x-109}{111-109}\right) \cdot 10.5356 \\ &= 0.04765x + 5.24645 \end{aligned}$$

hence

$$\begin{aligned} \sqrt{110} &\simeq 0.04765 \cdot 110 + 5.24645 \\ &= 10.48795. \end{aligned}$$

For comparison, the true value is $\sqrt{110} \simeq 10.48808$.

3.1.2 Quadratic Interpolation

For this type of interpolation, we are given three points (x_0, y_0), (x_1, y_1), and (x_2, y_2), and we'll need to find the quadratic function $p(x) = a_2x^2 + a_1x + a_0$ that passes through all of these three points. That means, $p(x_0) = y_0$, $p(x_1) = y_1$ and $p(x_2) = y_2$.

Note again that here, by *quadratic function*, we mean a function of the form $f(x) = ax^2 + bx + c$, where a, b, c are constants and they can all be zero. In particular, if the given data points are on a straight line, then the "quadratic function" turns out to be a line.

To find the coefficients a_0, a_1, and a_2, one can solve the following system of equations

$$\begin{cases} a_2x_0^2 + a_1x_0 + a_0 = y_0 \\ a_2x_1^2 + a_1x_1 + a_0 = y_1 \\ a_2x_2^2 + a_1x_2 + a_0 = y_2 \end{cases}.$$

Then the interpolating function is given by: $p(x) = a_2x^2 + a_1x + a_0$.

Next we'll give you another way to find $p(x) = a_2x^2 + a_1x + a_0$ and these formulae will be consistent to the following subsection, Lagrange Formula. The motivation for these formulae will be presented in the next subsection and hence many of the details will be included there, and omitted here. We have that

$$p(x) = y_0L_0(x) + y_1L_1(x) + y_2L_2(x), \tag{3.3}$$

where

$$\begin{aligned} L_0(x) &= \left(\frac{x - x_1}{x_0 - x_1}\right)\left(\frac{x - x_2}{x_0 - x_2}\right) \\ L_1(x) &= \left(\frac{x - x_0}{x_1 - x_0}\right)\left(\frac{x - x_2}{x_1 - x_2}\right) \\ L_2(x) &= \left(\frac{x - x_0}{x_2 - x_0}\right)\left(\frac{x - x_1}{x_2 - x_1}\right) \end{aligned} \tag{3.4}$$

are called **Lagrange Basis Polynomials**.

It should be easy to check that $p(x)$ is quadratic function (since it is a linear combination of such polynomials) and that it interpolates the given data: $p(x_0) = y_0$, $p(x_1) = y_1$, and $p(x_2) = y_2$.

Let's see some examples.

Example 71. You are given the following data points $\begin{array}{c|c|c|c} x & 2 & 4 & 8 \\ \hline y & 1 & 2 & 3 \end{array}$. Find the quadratic function that interpolates this table.

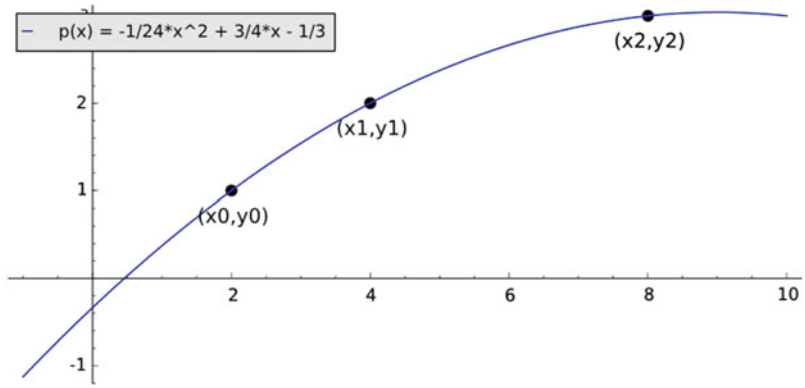
Solution. Using formula (3.4) we get

$$\begin{aligned} L_0(x) &= \left(\frac{x-4}{2-4}\right)\left(\frac{x-8}{2-8}\right) = \frac{1}{12}(x^2 - 12x + 32) \\ L_1(x) &= \left(\frac{x-2}{4-2}\right)\left(\frac{x-8}{4-8}\right) = -\frac{1}{8}(x^2 - 10x + 16) \\ L_2(x) &= \left(\frac{x-2}{8-2}\right)\left(\frac{x-4}{8-4}\right) = \frac{1}{24}(x^2 - 6x + 8) \end{aligned}$$

and hence, by (3.3) we obtain

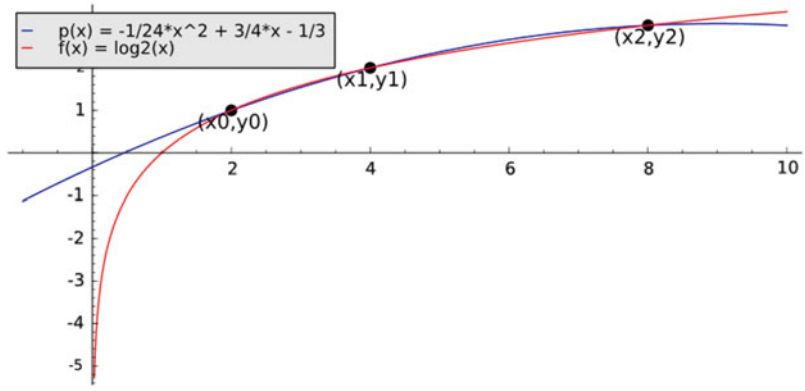
$$\begin{aligned} p(x) &= L_0(x) + 2L_1(x) + 3L_2(x) \\ &= -\frac{1}{24}x^2 + \frac{3}{4}x - \frac{1}{3}. \end{aligned}$$

Below we include a graph for our interpolating polynomial.



Hence, if you wish to estimate the value of y, for $x = 7.5$, we get $p(7.5) = -\frac{1}{24}\cdot 7.5^2 + \frac{3}{4}\cdot 7.5 - \frac{1}{3} = \frac{283}{96} \simeq 2.94791$.

In the next graph, we will include both, the quadratic polynomial of interpolation, $p(x)$, and the function $f(x) = \log_2 x$ (this is where our data points came from!). As you can see, the quadratic polynomial of interpolation seems to follow more closely the function $f(x) = \log_2 x$, than the linear polynomial obtained in the previous subsection. Hence we will expect that in the case of quadratic interpolation to obtain better estimates.



In this case we have: $p(7.5) \simeq 2.94791$, compared to the correct value of $\log_2(7.5) \simeq 2.90689$, and $p(5) = \frac{19}{8} = 2.375$, compared to $\log_2(5) \simeq 2.3219$. Compare these results with the ones obtained in Example 69.

Later in this chapter we will discuss more about the error estimates.

Example 72. Knowing that $\sqrt{109} \simeq 10.4403$, $\sqrt{111} \simeq 10.5356$, and $\sqrt{112} \simeq 10.5830$, estimate $\sqrt{110}$.

Solution. Using formula (3.3) we obtain

$$p(x) \simeq -0.00008333x^2 + 0.06598333x + 4.2382$$

hence

$$\sqrt{110} \simeq 10.48807.$$

For comparison, the true value is $\sqrt{110} \simeq 10.48808$. As we can see, we obtained a much better estimate than in the Example 70.

3.1.3 Lagrange Formula

We consider here the general case. Given the set of $n+1$ data points,

x_0	x_1	x_2	...	x_n
y_0	y_1	y_0	...	y_n

we want to find the polynomial function of degree (at most) n,

$$p(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$$

such that $p(x)$ interpolates these points. That is:

$$p(x_0) = y_0, p(x_1) = y_1, \ldots, p(x_n) = y_n.$$

For this, let us start by defining the **Lagrange Basis Polynomials**, $L_0(x)$, $L_1(x)$, ..., $L_n(x)$, each of degree (exactly) n:

$$\begin{aligned} L_i(x) &= \prod_{j=0, j\neq i}^{n} \frac{(x - x_j)}{(x_i - x_j)}, \text{ for } \quad i = 0, 1, \ldots, n. \\ &= \frac{(x-x_0)\cdot(x-x_1)\cdot\cdots\cdot(x-x_{i-1})\cdot(x-x_{i+1})\cdot\cdots\cdot(x-x_n)}{(x_i-x_0)\cdot(x_i-x_1)\cdot\cdots\cdot(x_i-x_{i-1})\cdot(x_i-x_{i+1})\cdot\cdots\cdot(x_i-x_n)}. \end{aligned} \tag{3.5}$$

Please note that the denominators given in the previous formula, are product of constant numbers, hence they are constant. Therefore the fact that each one of these Lagrange Basis Polynomials has degree exactly n should be trivial. They also have the following very nice property:

$$L_i(x_j) = \delta_{ij} = \begin{cases} 0, \text{ for } i \neq j \\ 1, \text{ for } i = j \end{cases}, \ 0 \leq i, j \leq n. \tag{3.6}$$

Hence, the polynomial

$$p(x) = y_0 L_0(x) + y_1 L_1(x) + \cdots + y_n L_n(x), \tag{3.7}$$

will interpolate the data points given, and is called the **interpolation polynomial of degree n, in the Lagrange formula.**

Since $p(x)$ is a linear combination of polynomials of degree n, it implies that $p(x)$ has degree at most n (since some cancellation may occur when we add these terms).

To check that the polynomial $p(x)$ given in formula (3.7) interpolates the given set of data points, we have:

$$\begin{aligned} p(x_0) &= y_0 L_0(x_0) + y_1 L_1(x_0) + \cdots + y_n L_n(x_0) \\ &= y_0 \cdot 1 + y_1 \cdot 0 + \cdots + y_n \cdot 0 \\ &= y_0, \end{aligned}$$

$$\begin{aligned} p(x_1) &= y_0 L_0(x_1) + y_1 L_1(x_1) + \cdots + y_n L_n(x_1) \\ &= y_0 \cdot 0 + y_1 \cdot 1 + \cdots + y_n \cdot 0 \\ &= y_1, \end{aligned}$$

and in general:

$$\begin{aligned} p(x_i) &= y_0 L_0(x_i) + y_1 L_1(x_i) + \cdots + y_n L_n(x_i) \\ &= y_i \cdot 1 \\ &= y_i. \end{aligned}$$

In particular, for $n = 1$, we obtain the linear interpolation formula given in (3.2). For $n = 2$, we get the quadratic interpolation formula given in (3.3). Hence, each of the previous two subsections is a particular case of this current subsection.

Example 73. Find the polynomial that interpolates the following data points:

x	2	4	8	16
y	1	2	3	4

Solution. We have

$$\begin{aligned} L_0(x) &= \frac{(x-4)\cdot(x-8)\cdot(x-16)}{(2-4)\cdot(2-8)\cdot(2-16)} \\ &= -\frac{1}{168}x^3 + \frac{1}{6}x^2 - \frac{4}{3}x + \frac{64}{21}. \end{aligned}$$

We could also use Sage to simplify this, using the following code:

```
f(x) = (((x-4)*(x-8)*(x-16))/((2-4)*(2-8)*(2-16)))
f.full_simplify()
```

to obtain:

```
x |--> -1/168*x^3 + 1/6*x^2 - 4/3*x + 64/21
```

Similarly,

$$\begin{aligned} L_1(x) &= \frac{(x-2)\cdot(x-8)\cdot(x-16)}{(4-2)\cdot(4-8)\cdot(4-16)} \\ &= \frac{1}{96}x^3 - \frac{13}{48}x^2 + \frac{11}{6}x - \frac{8}{3}, \end{aligned}$$

$$\begin{aligned} L_2(x) &= \frac{(x-2)\cdot(x-4)\cdot(x-16)}{(8-2)\cdot(8-4)\cdot(8-16)} \\ &= -\frac{1}{192}x^3 + \frac{11}{96}x^2 - \frac{13}{24}x + \frac{2}{3}, \end{aligned}$$

$$L_3(x) = \frac{(x-2)\cdot(x-4)\cdot(x-8)}{(16-2)\cdot(16-4)\cdot(16-8)}$$
$$= \frac{1}{1344}x^3 - \frac{1}{96}x^2 + \frac{1}{24}x - \frac{1}{21}.$$

Hence, the polynomial of interpolation is given by

$$p(x) = L_0(x) + 2L_1(x) + 3L_2(x) + 4L_3(x)$$
$$= \frac{1}{448}x^3 - \frac{7}{96}x^2 + \frac{7}{8}x - \frac{10}{21}.$$

Example 74. Find the polynomial that interpolates the following data points:

x	1	2	3	4
y	2	4	6	8

Solution. We have

$$L_0(x) = \frac{(x-2)\cdot(x-3)\cdot(x-4)}{(1-2)\cdot(1-3)\cdot(1-4)}$$
$$= -\frac{1}{6}x^3 + \frac{3}{2}x^2 - \frac{13}{3}x + 4$$

$$L_1(x) = \frac{(x-1)\cdot(x-3)\cdot(x-4)}{(2-1)\cdot(2-3)\cdot(2-4)}$$
$$= \frac{1}{2}x^3 - 4x^2 + \frac{19}{2}x - 6$$

$$L_2(x) = \frac{(x-1)\cdot(x-2)\cdot(x-4)}{(3-1)\cdot(3-2)\cdot(3-4)}$$
$$= -\frac{1}{2}x^3 + \frac{7}{2}x^2 - 7x + 4$$

$$L_3(x) = \frac{(x-1)\cdot(x-2)\cdot(x-3)}{(4-1)\cdot(4-2)\cdot(4-3)}$$
$$= \frac{1}{6}x^3 - x^2 + \frac{11}{6}x - 1$$

Hence, the polynomial of interpolation is given by

$$p(x) = 2L_0(x) + 4L_1(x) + 6L_2(x) + 8L_3(x)$$
$$= 2x.$$

Notice how every one of the Lagrange Basis Polynomials have degree 3, but in the end, once we compute the linear combination, the polynomial of interpolation $p(x)$ has degree 1.

By construction we can see the **existence** of a polynomial of interpolation, of degree (at most) n, for any given set of $n+1$ data points,

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}.$$

This can also be proven by induction (which will lead to another formula that can be used for the polynomial of interpolation: Newton Formula).

The **uniqueness** can also be proven: given set of $n+1$ data points, there is one, and only one, polynomial of degree at most n that interpolates them.

Both, the existence and the uniqueness will be proven below.

3.1.4 Sage Code for Lagrange Formula

Below we give the Sage implementation for computing the Lagrange formula of the polynomial interpolation given a set of $n+1$ data points:

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}.$$

We start with some explanation of the code. First, we put those values into a two-dimensional array (or, an array of pairs in this case). For more details on how to work with arrays, please check the examples we provided on page 35.

To simplify the code, we chose to use *points* and *n* as global variables, rather than parameters of the functions that use them. If you are somewhat experienced with programming, you may choose to follow the better programming practice and send these values as parameters. We hope that simplifying the code will help you better understand the code and focus more on the Mathematical model.

Next we compute *n,* from this array, so that everything that comes next will be computed automatically from the given array. That is, you only need to change the values in the array *points*, and the algorithm will work without having to change anything else.

Then we implemented the function L that computes the Lagrange Basis Polynomials. For example, you want to get $L_0(x)$, you only need to call the function $L(0, x)$.

Last function that we implemented, $Lagrange_Interpolation(x)$, will compute the interpolation polynomial $p(x)$ using Lagrange formula.

For example, let's assume we are given the following data

$$\begin{array}{c||c|c|c|c} x & 1 & 2 & 3 & 4 \\ \hline y & 3 & 6 & 9 & 12 \end{array}.$$

```
#the array of data points given
points = [(1,3), (2,6), (3,9), (4,12)]

#we compute n from the size of the array
n = len(points)-1

#compute the Lagrange Basis Polynomials
def L(i, x):
    #initial value before a repeated product
```

```
    Li=1
    for j in range(n+1) :
        if(i!=j):
           #we repeatedly multiply by (x-xj)/(xi-xj)
           Li=Li*(x-points[j][0])/(points[i][0]-points[j][0])
    return Li

#compute the Lagrange interpolation polynomial
def Lagrange_Interpolation(x):
    #initial value before a repeated sum
    p=0
    for i in range(n+1):
        #we repeatedly add by yi*Li(x)
        p = p + points[i][1]*L(i,x)
    #simplify the answer before we return it
    return p.full_simplify()

#output the Lagrange interpolation polynomial
Lagrange_Interpolation(x)
```

Then the output should be:

```
3*x
```

In the previous example, if you wanted to change the previous code, so that it will print all the Lagrange Basis Polynomials, you would only have to include one line of code

```
print "L"+ str(i) +"(x) = "+ str(Li)
```

inside the definition of the *L(i,x)* function like below:

```
#compute the Lagrange Basis polynomials
def L(i, x):
    #initial value before a repeated product
    Li=1
    for j in range(n+1) :
        if(i!=j):
           #we repeatedly multiply by (x-xj)/(xi-xj)
           Li=Li*(x-points[j][0])/(points[i][0]-points[j][0])
    print "L"+ str(i) +"(x) = "+ str(Li)
    return Li
```

and, leaving everything else unchanged. Then you would get the following output:

```
L0(x) = -1/6*(x - 2)*(x - 3)*(x - 4)
L1(x) = 1/2*(x - 1)*(x - 3)*(x - 4)
L2(x) = -1/2*(x - 1)*(x - 2)*(x - 4)
L3(x) = 1/6*(x - 1)*(x - 2)*(x - 3)
3*x
```

In the previous example, instead of printing the polynomial of interpolation $p(x)$, let's say you wanted to compute $p(2.75)$. Then you could, for example, replace the last line of Sage code

```
Lagrange_Interpolation(x)
```

with

```
p(x) = Lagrange_Interpolation(x)
p(2.75)
```

You would get:

```
8.25000000000000
```

You could also simply use

```
Lagrange_Interpolation(2.75)
```

to get the same output, but you would have to remove the part of code `.full_simplify()` from the definition of the function `Lagrange_Interpolation()`.

3.1.5 Newton Formula

The Lagrange's formula of finding the interpolation polynomial is a fairly intuitive, and an easy to understand and apply formula. For any given table of data points

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}.$$

we could easily compute the polynomial of interpolation.

Lagrange Formula has a challenge. Suppose you want to add a new data point to the given table of points. How would you find the new polynomial of interpolation? Using Lagrange form you would have to do the entire work again, to find this new polynomial. You could not take advantage of the previous work done to find the polynomial of interpolation for the old list. Newton's formula will be able to overcome this challenge and take advantage of knowing the polynomial of interpolation found before adding the new data point. That is: Newton Formula can easily extend the polynomial of interpolation when adding more data points. Also, Newton Formula will be more suitable in actual computations, since it generally uses less operations.

It should be noted that both *Newton formula* and *Lagrange formula* give us the very same polynomial of interpolation (an obvious truth given its uniqueness).

The proof of the next theorem gives us the motivation behind the Newton Formula.

Theorem 75. *Given any set of data points*

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}.$$

with all x_is distinct, there is a unique polynomial $p(x)$, of degree at most n, that interpolates the given set. That is $p(x_0) = y_0$, $p(x_1) = y_1, \dots, p(x_n) = y_n$, $n \geq 0$.

Proof. **Part 1: Existence.**

We can easily prove this by induction, over n.

For verification step, for $n = 0$ we have that $p(x) = y_0$ is a polynomial of degree 0 (constant function) that interpolates the given set.

Although not needed, we also include the case $n = 1$. Here

$$p(x) = \left(\frac{x - x_1}{x_0 - x_1}\right) \cdot y_0 + \left(\frac{x - x_0}{x_1 - x_0}\right) \cdot y_1$$

is a polynomial of degree at most 1 (linear function) that interpolates the given set.
For the induction step, let's assume that given any set of $k + 1$ data points, $k \geq 0$, we can always find an interpolation function $p(x)$ of degree at most k. Let's prove that we can also find one for any given set of $k + 2$ data points.
Let's suppose we are given the following $k + 2$ data points:

$$\begin{array}{c|c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_k & x_{k+1} \\ \hline y_0 & y_1 & y_0 & \dots & y_k & y_{k+1} \end{array}.$$

Then, by the assumption of the induction step, we can find a polynomial $p(x)$ of degree at most k such that $p(x_0) = y_0$, $p(x_1) = y_1, \dots, p(x_k) = y_k$. We need to construct polynomial $q(x)$ such that $q(x_0) = y_0, q(x_1) = y_1, \dots, q(x_{k+1}) = y_{k+1}$ and $q(x)$ has degree at most $k + 1$. For this, let

$$q(x) = p(x) + c(x - x_0)(x - x_1) \cdots (x - x_k) \tag{3.8}$$

where c is a constant to be determined such that $q(x_{k+1}) = y_{k+1}$. Using basic algebra we can easily find that:

$$c = \frac{y_{k+1} - p(x_{k+1})}{(x_{k+1} - x_0)(x_{k+1} - x_1) \cdots (x_{k+1} - x_k)}. \tag{3.9}$$

Then, it is an easy task to check that $q(x)$ interpolates all the $k + 1$ data points. Since $p(x)$ is a polynomial of degree at most k, and $(x - x_0)(x - x_1) \cdots (x - x_k)$ is a polynomial of degree $k + 1$, we can see that $q(x) = p(x) + c(x - x_0)(x - x_1) \cdots (x - x_k)$ has degree at most $k + 1$.

Part 2: Uniqueness.
This part is left as an exercise. ■

The following formula is what we will call **Newton interpolation formula**

$$\begin{aligned} p(x) = c_0 &+ c_1(x - x_0) + c_2(x - x_0)(x - x_1) \\ &+ c_3(x - x_0)(x - x_1)(x - x_2) + \cdots \\ &+ c_n(x - x_0)(x - x_1)(x - x_2) \cdots (x - x_{n-1}) \end{aligned} \tag{3.10}$$

Let's see through an example, how we can obtain a polynomial of interpolation for a given set of data points. We'll use the same technique as in the proof of the previous Theorem. This is known as the **Newton Algorithm**.

Example 76. Using Newton Algorithm, find the polynomial of least degree that interpolates the following data points:

$$\begin{array}{c||c|c|c|c} x & 2 & 4 & 8 & 16 \\ \hline y & 1 & 2 & 3 & 4 \end{array}.$$

Proof. We will repeatedly use the formulae (3.8) and (3.9) to obtain our answer.

Looking at the first data point, $(2, 1)$, we can easily construct a polynomial that interpolates it: $p_0(x) = 1$.

Next step is to find a polynomial that interpolates the first two data points, $(2, 1)$ and $(4, 2)$. For this, we first compute

$$\begin{aligned} c_1 &= \frac{y_1 - p_0(x_1)}{x_1 - x_0} \\ &= \frac{2-1}{4-2} = \frac{1}{2}, \end{aligned}$$

and obtain the new interpolating function

$$\begin{aligned} p_1(x) &= p_0(x) + c_1(x - x_0) \\ &= 1 + \frac{1}{2}(x-2). \end{aligned}$$

Then, we will look for the polynomial that interpolates the first three data points. For this, we first find

$$\begin{aligned} c_2 &= \frac{y_2 - p_1(x_2)}{(x_2 - x_0)(x_2 - x_1)} \\ &= \frac{3-4}{(8-2)(8-4)} = -\frac{1}{24}, \end{aligned}$$

and then we have that

$$\begin{aligned} p_2(x) &= p_1(x) + c_2(x - x_0)(x - x_1) \\ &= 1 + \frac{1}{2}(x-2) - \frac{1}{24}(x-2)(x-4). \end{aligned}$$

If we apply this procedure one more step we get

$$\begin{aligned} c_3 &= \frac{y_3 - p_2(x_3)}{(x_3 - x_0)(x_3 - x_1)(x_3 - x_2)} \\ &= \frac{4 - p_2(16)}{(16-2)(16-4)(16-8)} = \frac{3}{1344} \end{aligned}$$

and conclude that

$$\begin{aligned} p_3(x) &= p_2(x) + c_3(x - x_0)(x - x_1)(x - x_2) \\ &= 1 + \frac{1}{2}(x-2) - \frac{1}{24}(x-2)(x-4) \\ &\quad + \frac{3}{1344}(x-2)(x-4)(x-8). \end{aligned} \tag{3.11}$$

Although this answer looks different than

$$p(x) = \frac{1}{448}x^3 - \frac{7}{96}x^2 + \frac{7}{8}x - \frac{10}{21},$$

obtained in Example 73 (page 124), after simplifying it you can see that they are actually the same.

If you want to let Sage simplify for you the polynomial given in (3.11) you can use:

```
p(x)= 1+(1/2)*(x-2)-(1/(24))*(x-2)*(x-4) +(3/(1344))*(x-2)
    *(x-4)*(x-8)
p(x).full_simplify()
```

and obtain

```
1/448*x^3 - 7/96*x^2 + 7/8*x - 10/21
```

Let's see what happens when the polynomial of interpolation for a given set of $n+1$ data points has a degree strictly less than n.

Example 77. Using Newton Algorithm, find the polynomial of least degree that interpolates the following data points:

$$\begin{array}{c|c|c|c|c} x & 2 & 3 & 5 & 8 \\ \hline y & 8 & 12 & 20 & 32 \end{array}.$$

Proof. We will, again, repeatedly use the formulae (3.8) and (3.9) to obtain our answer.

First, the constant polynomial that interpolates the point $(2, 8)$ is given by

$$p_0(x) = 8.$$

Then, adding the data point $(3, 12)$ we have:

$$c_1 = \frac{12-8}{3-2} = 4$$

hence

$$\begin{aligned} p_1(x) &= 8 + 4(x-2) \\ &= 4x. \end{aligned}$$

At this point, we could observe that $p_1(x)$ interpolates all the other points, and hence (by uniqueness) is **the** polynomial of interpolation for the entire table.

We will compute one more step to see what would happen in this special case, once we found our polynomial of interpolation. The next step would be

$$c_2 = \frac{12-12}{(5-2)(5-3)} = 0$$

and hence

$$\begin{aligned} p_2(x) &= p_1(x) + 0 \\ &= 4x. \end{aligned}$$

and so on for the last data point ($c_3 = 0$).

3.1.6 Sage Code for Newton Formula

We give, below, our Sage implementation for the Newton's algorithm. We start with some explanations first.

Given a set of data points, we will compute the polynomial of interpolation using Newton Formula. As with the Sage code for Lagrange Formula, the code below is quite generic, in the sense that one only needs to change the set of given data points, and the rest of the code will work unchanged.

For more details on how to use arrays, please check the examples we provided on page 35.

Like in the case of Lagrange formula, we again use a global array, called `points`, and a global variable `n`. We challenge you to improve this code by using parameters rather than global variables.

In the implementation of `Newton_Interpolation()` function we started with the polynomial that interpolates the very first data point, $p_0(x) = y_0$. Hence we used `p(x) = points[0][1]`.

Remember that `points[0]` gives you the very first data point (which in this case is a pair (x, y)), `points[1]` gives you the second data point, `points[2]` gives you the third data point, etc. For the third (or any other) data point, you can access the x-coordinate using `points[2][0]`, and the y-coordinate using `points[2][1]`.

When using the Newton formula, we'll need to use an expression of the form $(x - x_0)(x - x_1)\ldots(x - x_k)$. We'll build it in `newTerm(x)`. At first, `newTerm(x)=x-x0`, hence we initialize it using the code `newTerm(x) = x-points[0][0]`. Then, inside the for loop, we repeatedly multiply it by $(x - x_k)$, hence we used `newTerm(x) = newTerm(x)*(x-points[k][0])`.

The Newton Formula is applied by using the `for` loop, which will repeatedly add a new data point to our interpolation polynomial, and hence compute a new value for c, and updating the expression of $p(x)$.

Let's suppose we want to find the polynomial of interpolation of least degree for the following data set

$$\begin{array}{c||c|c|c|c} x & 1 & 2 & 3 & 4 \\ \hline\hline y & 3 & 6 & 9 & 12 \end{array}.$$

then we would use the following Sage code:

```
#the array of data points given
points = [(2,1), (4,2), (8,3), (16,4)]

#we compute n from the size of the array
n = len(points)-1


#compute the Newton interpolation polynomial
def Newton_Interpolation(x):
    #the very first step is computed here
    #p0(x)=y0
    p(x) = points[0][1]

    #new term will be x-x0
    #it will eventually store (xk-x0)(xk-x1)...(xk-xk-1)
    newTerm(x) = x-points[0][0]

    #for k in range(1,n+1):
    for k in [1, 2, .., n]:
```

```
        #we repeatedly add the new data point
        #first compute the new c
        c = ((points[k][1]-p(points[k][0])) / newTerm(points
            [k][0]))

        #and then add it to the interpolating function
        p(x) = p(x) + c*newTerm(x)

        #prepare for next iteration
        newTerm(x) = newTerm(x)*(x-points[k][0])
    #simplify the answer before we return it
    return p

#call the function and find the final answer
Newton_Interpolation(x)
```

This will give you

```
x |--> 1/448*(x - 2)*(x - 4)*(x - 8) - 1/24*(x - 2)*(x - 4) + 1/2*x
```

If you want your answer to be fully simplified, you could replace the last line with

```
Newton_Interpolation(x).full_simplify()
```

to get

```
x |--> 1/448*x^3 - 7/96*x^2 + 7/8*x - 10/21
```

For a better visual representation, one can use the `show` method, and hence replace the last line with

```
show( Newton_Interpolation(x).full_simplify() )
```

to obtain:

$$x \mapsto \frac{1}{448}\,x^3 - \frac{7}{96}\,x^2 + \frac{7}{8}\,x - \frac{10}{21}$$

If one wants to also include a graph of the interpolating polynomial, then Sage code similar to the following can be useful:

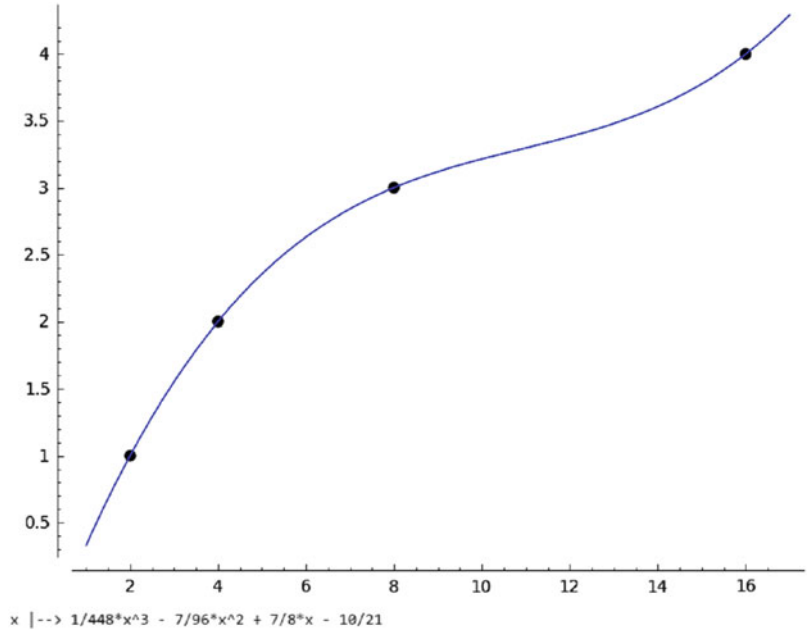
```
#plot the interpolating function
p = plot( Newton_Interpolation(x) , x, points[0][0]-1,points[n]
  [0]+1)

#plot the points of the table
p += list_plot( points, color = 'black', size = 70)

#display the plot
show(p)

#display the interpolating polynomial
Newton_Interpolation(x).full_simplify()
```

to obtain the graph and the interpolating polynomial:



3.1.7 Divided Differences

Using Newton formula (see formula (3.10), on page 129), one can observe that the polynomial of least degree that interpolates

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}$$

is given by

$$\begin{aligned} p(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots \\ + c_n(x - x_0)(x - x_1)\cdots(x - x_{n-1}) . \end{aligned}$$

In this section, we'll see how we can use *divided differences* to efficiently find the coefficients $c_0, c_1, \dots, c_n$, and hence efficiently find the polynomial of interpolation $p(x)$. For notation purposes, let's assume that $y_0 = f(x_0), y_1 = f(x_1), \dots, y_n = f(x_n)$, for some function f. Then we can use the following notation $c_0 = f[x_0], c_1 = f[x_0, x_1]$, $c_2 = f[x_0, x_1, x_2]$, ..., $c_n = f[x_0, x_1, x_2, \dots, x_n]$. The quantity $f[x_0, x_1, \dots, x_k]$ will be called **divided difference of order k** for f.

How would you normally solve for $c_0, c_1, \dots, c_n$? Using the fact that $p(x_0) = y_0$, $p(x_1) = y_1$, ..., $p(x_n) = y_n$ we could solve the following system of $n + 1$ equations

$$\begin{cases} p(x_0) = y_0 \\ p(x_1) = y_1 \\ p(x_2) = y_2 \\ \quad \dots \\ p(x_n) = y_n \end{cases}, \tag{3.12}$$

which is equivalent to

$$\begin{cases} c_0 = y_0 \\ c_0 + c_1(x_1 - x_0) = y_1 \\ c_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1) = y_2 \\ \cdots \\ c_0 + c_1(x_n - x_0) + c_2(x_n - x_0)(x_n - x_1) + \cdots \\ +c_n(x_n - x_0)(x_n - x_1)\cdots(x_n - x_{n-1}) = y_n \end{cases}, \tag{3.13}$$

hence

$$\begin{cases} c_0 = y_0 \\ c_1 = \frac{y_1 - c_0}{x_1 - x_0} = \frac{y_1 - y_0}{x_1 - x_0} \\ c_2 = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0} \\ \cdots \end{cases}, \tag{3.14}$$

or

$$\begin{cases} f[x_0] = y_0 \\ f[x_0, x_1] = \frac{y_1 - y_0}{x_1 - x_0} = \frac{f[x_1] - f[x_0]}{x_1 - x_2} \\ f[x_0, x_1, x_2] = \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0} = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_1} \\ \cdots \end{cases}. \tag{3.15}$$

Using Mathematical Induction, it can be shown that the divided differences of order k have the following properties

1. $f[x_0, x_1, \ldots, x_k]$ stays the same if we interchange (permute) any of $x_0, x_1, \ldots, x_k$.
2. it satisfies the following recursive formula

$$\begin{aligned} f[x_0, x_1, \ldots, x_k] &= \frac{f[x_1, x_2, \ldots, x_k] - f[x_0, x_1, \ldots, x_{k-1}]}{x_k - x_0}, \text{ for } k \geq 1 \\ f[x_i] &= f(x_i), \text{ for } i = 0, 1, \ldots, n. \end{aligned} \tag{3.16}$$

A proof for each of these properties an be found in [1, p. 126] and [4, pp. 162–163].

To compute each of these divided differences, we could use a table such as the one below, and formula (3.16)

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$\boxed{f[x_0] = f(x_0)}$			
		$\boxed{f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0}}$		
x_1	$f[x_1] = f(x_1)$		$\boxed{f[x_0, x_1, x_2]}$	
		$f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1}$		$\boxed{f[x_0, x_1, x_2, x_3]}$
x_2	$f[x_2] = f(x_2)$		$f[x_1, x_2, x_3]$	
		$f[x_2, x_3] = \frac{f[x_3] - f[x_2]}{x_3 - x_2}$		
x_3	$f[x_3] = f(x_3)$			

Example 78. Use Divided Differences to find the Newton form of the polynomial that interpolates the following data points:

$$\begin{array}{c||c|c|c|c} x & 2 & 4 & 8 & 16 \\ \hline y & 1 & 2 & 3 & 4 \end{array}.$$

Proof. We will need to find each of the following $f[x_0]$, $f[x_0, x_1]$, $f[x_0, x_1, x_2]$, and $f[x_0, x_1, x_2, x_3]$. For this, using a table like the one above, we would get:

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
2	$\boxed{1}$			
		$\boxed{\frac{2-1}{4-2} = \frac{1}{2}}$		
4	2		$\boxed{\frac{\frac{1}{4}-\frac{1}{2}}{8-2} = -\frac{1}{24}}$	
		$\frac{3-2}{8-4} = \frac{1}{4}$		$\boxed{\frac{(-\frac{1}{96})-(-\frac{1}{24})}{16-2} = \frac{1}{448}}$
8	3		$\frac{\frac{1}{8}-\frac{1}{4}}{16-4} = -\frac{1}{96}$	
		$\frac{4-3}{16-8} = \frac{1}{8}$		
16	4			

hence the Newton formula for the polynomial of interpolation of the given data points is given by

$$p(x) = 1 + \frac{1}{2}(x-2) - \frac{1}{24}(x-2)(x-4) + \frac{1}{448}(x-2)(x-4)(x-8)$$

which is consistent with the result we obtained in the previous subsections using the other methods.

Example 79. Use Divided Differences to find the Newton form of the polynomial that interpolates the following data points:

$$\begin{array}{c||c|c|c|c} x & 1 & 2 & 3 & 4 \\ \hline y & 3 & 6 & 9 & 12 \end{array}.$$

Proof. We will need to find each of the following $f[x_0]$, $f[x_0, x_1]$, $f[x_0, x_1, x_2]$, and $f[x_0, x_1, x_2, x_3]$. For this, using a table like the one above, we would get:

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	$\boxed{3}$			
		$\boxed{\frac{6-3}{2-1} = 3}$		
2	6		$\boxed{\frac{3-3}{3-1} = 0}$	
		$\frac{9-6}{3-2} = 3$		$\boxed{\frac{0-0}{4-1} = 0}$
3	9		$\frac{3-3}{4-2} = 0$	
		$\frac{12-9}{4-3} = 3$		
4	12			

hence the Newton formula for the polynomial of interpolation of the given data points is given by

$$p(x) = 3 + 3(x-1) + (x-1)(x-2) + 0(x-1)(x-2)(x-3)$$
$$3x.$$

which is consistent with the result we obtained in the previous subsections using the other methods.

3.1.8 Sage Code for Divided Differences

We present, below, the Sage code that makes use of the divided differences presented above to compute the coefficients $c_0, c_1, \ldots, c_n$.

```
#the array of data points given
points = [(2,1),(4,2),(8,3), (16,4)]

#we compute n from the size of the array
n = len(points)-1

#computes the set of divided differences
def DividedDifferences():
    #create an array n+1 by n+1
    divDiff = [[0 for i in range(n+1)] for i in range(n+1)]

    #first column will contain the y values
    #this is column 0!
    for row in [0,1,2,..,n]:
        divDiff[row][0]=points[row][1]

    #compute, recursively the div differences
    #one column at a time
    #in second column will have f[.]
    #in third column will have f[.,.], etc
    for column in [1,2,..,n]:
        #for a column, compute the value in each row
        for row in [0,1, 2, .., n-column]:
            divDiff[row][column]=(divDiff[row+1][column-1]-
divDiff[row][column-1])/(points[row+column][0]-points[row][0])
    return [divDiff[0][i] for i in [0,1,..,n]]

#call the function and find the final answer
DividedDifferences()
```

The output should be

```
[1, 1/2, -1/24, 1/448]
```

If you wish, you can modify the previous code as follows, so it will print the entire table of divided differences.

```
#the array of data points given
points = [(2,1),(4,2),(8,3), (16,4)]
```

```
#we compute n from the size of the array
n = len(points)-1

#computes the set of divided differences
def PrintDividedDifferencesTable():
    #create an array n+1 by n+1
    divDiff = [[0 for i in range(n+1)] for i in range(n+1)]

    #first column will contain the y values
    #this is column 0!
    for row in [0,1,2,..,n]:
        divDiff[row][0]=points[row][1]

    #compute, recursively the div differences
    #one column at a time
    #in second column will have f[.]
    #in third column will have f[.,.], etc
    for column in [1,2,..,n]:
        #for a column, compute the value in each row
        for row in [0,1, 2, .., n-column]:
            divDiff[row][column]=(divDiff[row+1][column-1]
                -divDiff[row][column-1])/(points[row+
                column][0]-points[row][0])
    for row in [0,1, 2, .., n]:
        print points[row][0],"\t|", [ divDiff[row][column]  for
        column in [0,1,2,..,n-row]]

#call the function
PrintDividedDifferencesTable()
```

This will output

```
2   | [1, 1/2, -1/24, 1/448]
4   | [2, 1/4, -1/96]
8   | [3, 1/8]
16  | [4]
```

3.1.9 Inverse Interpolation

An interesting application of interpolation is approximating the inverse of a function. Suppose we are given a set of $n + 1$ data points

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}.$$

Let's assume they come from a function $f(x)$. That is $y_0 = f(x_0)$, $y_1 = f(x_1)$, etc.

Using the formulae we have seen in this chapter, we can find the polynomial $q(y)$, of degree at most n that interpolates the table

$$\begin{array}{|c|c|c|c|c|}\hline y_0 & y_1 & y_2 & \dots & y_n \\ \hline x_0 & x_1 & x_0 & \dots & x_n \\ \hline \end{array}$$

This polynomial, $q(y)$, can be used as an approximation for the inverse of the function $f(x)$.

Another application for the inverse interpolation is finding an approximate value for the root of a function. Once you find $q(x)$, under favorable conditions, computing $q(0)$ can give you a good estimation of the solution of the equation $f(x) = 0$.

Example 80. Find the inverse interpolation for the following set of data points

$$\begin{array}{||c|c|c|c|}\hline x & 2 & 4 & 8 & 16 \\ \hline y & 1 & 2 & 3 & 4 \\ \hline \end{array}.$$

Solution. We can use Sage and apply the Newton's Method to find the polynomial of interpolation for

$$\begin{array}{||c|c|c|c|}\hline y & 1 & 2 & 3 & 4 \\ \hline x & 2 & 4 & 8 & 16 \\ \hline \end{array}.$$

Then we get $q(y) = \frac{1}{3}x^3 - x^2 + \frac{8}{3}x$.

3.1.10 Sage Library

If you need to find the polynomial of interpolation without willing to write your own code, the you can use Sage library, which already contains a function that can be used to find Lagrange Polynomial. For example, one can write code similar to:

```
R = PolynomialRing(QQ, 'x')
R.lagrange_polynomial([(0,  1),(1,  2.4),(2,  2.001),(3, 3.45)])
```

And obtain

```
0.607833333333333*x^3 - 2.72300000000000*x^2
   + 3.51516666666667*x + 1.00000000000000
```

Example 81. Use the Sage library to find the polynomial of interpolation, $p(x)$, for

$$\begin{array}{||c|c|c|c|c|c|}\hline x & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline y & 0 & 3 & 6 & 9 & 12 & 15 \\ \hline \end{array}.$$

Solution. We could use the following Sage code

```
R = PolynomialRing(QQ, 'x')
R.lagrange_polynomial([(0,  0),(1,  3),(2,  6),(3, 9), (4,12),
   (5,15)])
```

This would yield the following result:

```
3*x
```

Example 82. Use Sage library functions to find the polynomial of interpolation, $p(x)$, for the following set of data points

$$\begin{array}{c|c|c|c|c|c|c} x & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline y & 0 & 3 & 6.000001 & 9 & 12 & 15 \end{array}.$$

Solution. We could use the following Sage code

```
R = PolynomialRing(QQ, 'x')
R.lagrange_polynomial([(0,0),(1,3),(2, 6.000001),(3, 9),(4,12),
    (5,15)])
```

This would yield the following result:

```
4.16666666667546e-6*x^5 - 0.000058333333334564*x^4
    + 0.00029583333333958*x^3 - 0.00064166666668021*x^2
    + 3.00050000000000*x
```

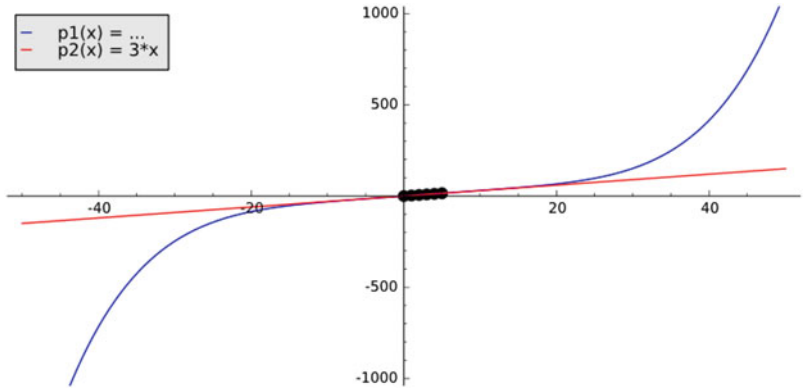
As you can see, a very small change in values can have a very high impact in the polynomial of interpolation. For all the values of x between 0 and 3, the two graphs are relatively the same. But for larger values, they will get significantly different.

We give below the graph of the previously two functions:

$$p_1(x) = 3x$$

and

$$\begin{aligned} p_2(x) = {} & 4.16666666667546 \cdot 10^{-6}x^5 - 0.000058333333334564x^4 \\ & +0.00029583333333333958x^3 - 0.00064166666668021x^2 + 3.0005x \end{aligned}$$



Example 83. Use the Sage library to find the polynomial of interpolation, $p(x)$, for

$$\begin{array}{c|c|c|c|c} x & 2 & 4 & 8 & 16 \\ \hline y & 1 & 2 & 3 & 4 \end{array},$$

and then graph both, $p(x)$ and $f(x) = \log_2 x$.

Solution. We could use the following Sage code

```
#create  the table of data points
size = 4
points = [  ( 2^x,x ) for x in  [1, 2,.., size] ]
```

```
#find the polynomial of interpolation, p(x)
R = PolynomialRing(QQ, 'x')
p(x) = R.lagrange_polynomial(points)

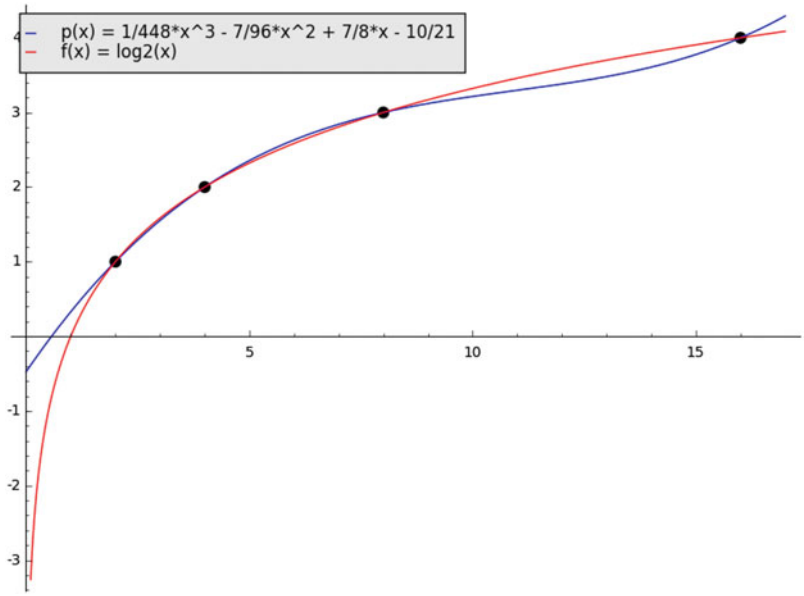
#plot the polynomial of interpolation
pl=plot(p(x),  (x,0,2^size+1),
       legend_label =   "    p(x) = "+str(p(x).full_simplify()))

#plot the points of the table
pl += list_plot( points, color = 'black', size = 70)

#plot the function f(x)
f(x)=ln(x)/ln(2)
pl +=plot(f(x),  (x,0,2^size+1),
       legend_label =   "    f(x) = log2(x)", color = 'red')

#display the plot
show(pl)
```

We obtain



As you can see, in this case the polynomial of interpolation provides a fairly accurate approximation for $f(x) = \log_2 x$.

Example 84. Modify the previous code to only include the first three points of the table given above, namely,

$$\begin{array}{c|c|c|c} x & 2 & 4 & 8 \\ \hline y & 1 & 2 & 3 \end{array},$$

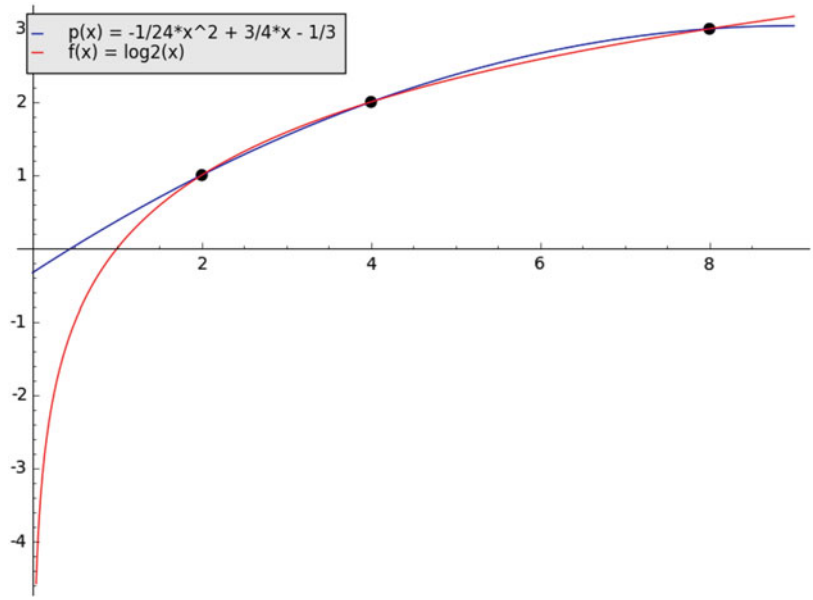
Solution. We would only need to change one line:

```
size = 4
```

into

```
size = 3
```

and obtain:



To get the divided differences, using Sage library, one can use code similar to the one in the next example.

Example 85. Use the Sage library to obtain the divided differences for

$$\begin{array}{c|c|c|c|c} x & 2 & 4 & 8 & 16 \\ \hline y & 1 & 2 & 3 & 4 \end{array}.$$

Solution. We could use the following

```
points = [(2,1),(4,2),(8,3), (16,4)]
R = PolynomialRing(QQ, "x")
R.divided_difference(points)
```

and obtain

```
[1, 1/2, -1/24, 1/448]
```

If you modify the last line and into

```
show( R.divided_difference(points) )
```

then you obtain output similar to:

$$\left[1, \frac{1}{2}, -\frac{1}{24}, \frac{1}{448}\right]$$

To get the full divided differences table, one can use the following code

Example 86. Use the Sage library to obtain the full table of Newton divided differences for

$$\begin{array}{c|c|c|c|c} x & 2 & 4 & 8 & 16 \\ \hline y & 1 & 2 & 3 & 4 \end{array}.$$

Solution. We could use the following

```
points = [(2,1),(4,2),(8,3), (16,4)]
R = PolynomialRing(QQ, "x")
R.divided_difference(points, full_table=True)
```

and obtain

```
[[1], [2, 1/2], [3, 1/4, -1/24], [4, 1/8, -1/96, 1/448]]
```

Using the `show` method, you can get a more appealing output:

3.2 Error Bounds in Polynomial Interpolation

Suppose we are given a set of $n+1$ data points

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}.$$

Let's assume they come from a function $f(x)$. That is $y_0 = f(x_0)$, $y_1 = f(x_1)$, etc. Let $p(x)$ be the polynomial that interpolates these data points.

A natural question regarding the error of approximation would be: how well does the polynomial $p(x)$ approximate $f(x)$? In this section, we'll **try** to answer it.

To simplify things, in this section, by **error of approximation** of a function f by another function p we will mean

$$\sup_{x \text{ in domain of } f} |f(x) - p(x)|. \tag{3.17}$$

In particular, if the domain of f is $[a,b]$, and both, f and p are continuous, then the previous expression can be rewritten as

$$\max_{x \in [a,b]} |f(x) - p(x)|. \tag{3.18}$$

We might expect that the worst case scenario to be when f is not continuous. In this case, we really cannot say much about the error of approximating f by p. One thing we can count on is the fact that $f(x) = p(x)$ for $x = x_0, x_1, \dots, x_n$ since p interpolates the data points. For any other value of x, the value of $|f(x) - p(x)|$ can be anything, from very small, even 0, to very large.

Next, we will focus our analysis on continuous functions that are defined on some closed interval $[a, b]$. In this case, one might expect that the larger the set of data points, the better we can estimate f by p. That is, we might expect (or guess!) that the larger the value of n, the smaller the error of approximation. As we will see next, this conjecture turns out to be false.

A classical example to show divergence in the case of interpolating polynomials is given by the **Runge Function**

$$f(x) = \frac{1}{1 + 25x^2}$$

defined on $[-1, 1]$. If we let p_n be the polynomial that interpolates the values of f at the $n + 1$ equally spaced nodes between -1 and 1,

$$\begin{aligned} x_0 &= -1, \\ x_1 &= -1 + \frac{2}{n}, \\ x_2 &= -1 + 2 \cdot \frac{2}{n}, \\ &\cdots \\ x_{n-1} &= -1 + (n-1) \cdot \frac{2}{n}, \\ x_n &= 1 \end{aligned} \tag{3.19}$$

we would have that

$$\lim_{n \to \infty} \max_{x \in [-1,1]} |f(x) - p_n(x)| = \infty.$$

We can see this idea using Sage.

```
#runge function
f(x) = 1/(1+25*x^2)

#plot the function
myPlot = plot(f(x), (x, -1,1), color = 'black')

#plot 4 polynomials of interpolation using colors
colors = ['orange',  'green']
step = 0
for n in [4, 8]:
    #compute the data points
    points = [(-1+i*(2/n),  f(-1+i*2/n)) for i in [0,1,..,n]]

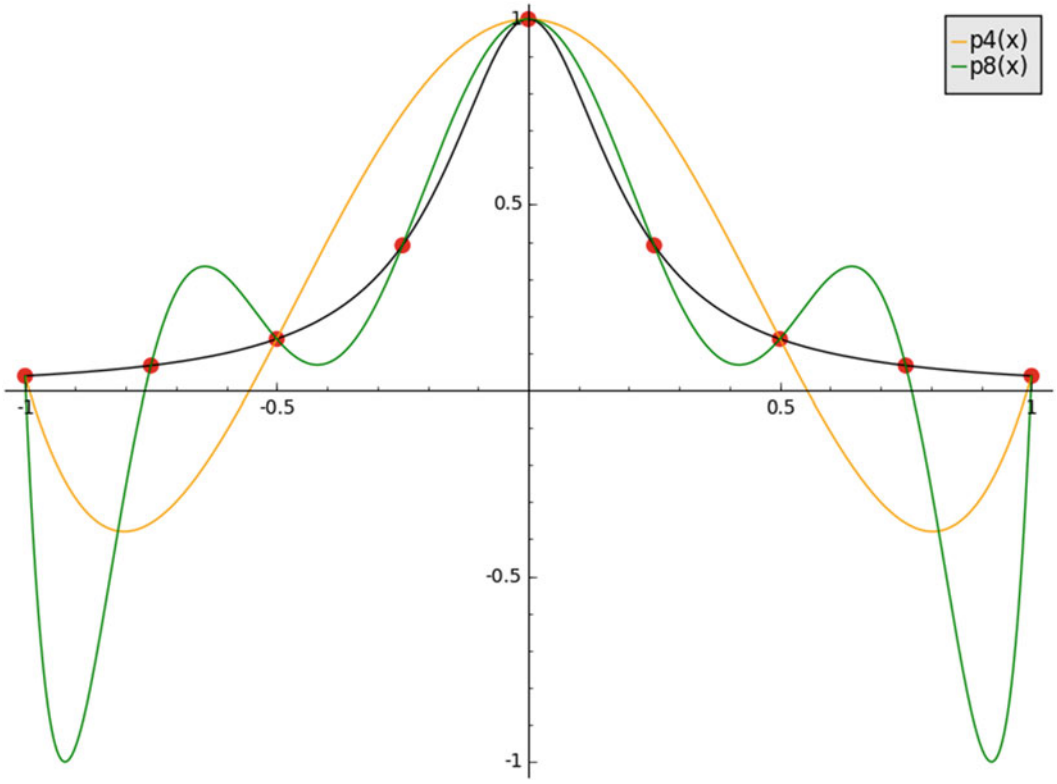
    #compute the polynomial of interpolation
    R = PolynomialRing(QQ, 'x')
    p(x) = R.lagrange_polynomial(points)

    #plot it
    myPlot += plot(p(x), (x, -1,1), color = colors[step],
                          legend_label = "p"+str(n)+"(x)")
```

```
    step = step + 1
#plot the points too
myPlot += list_plot(points, color = 'red' , size = 70)

#display the plot
myPlot.show()
```

We would obtain



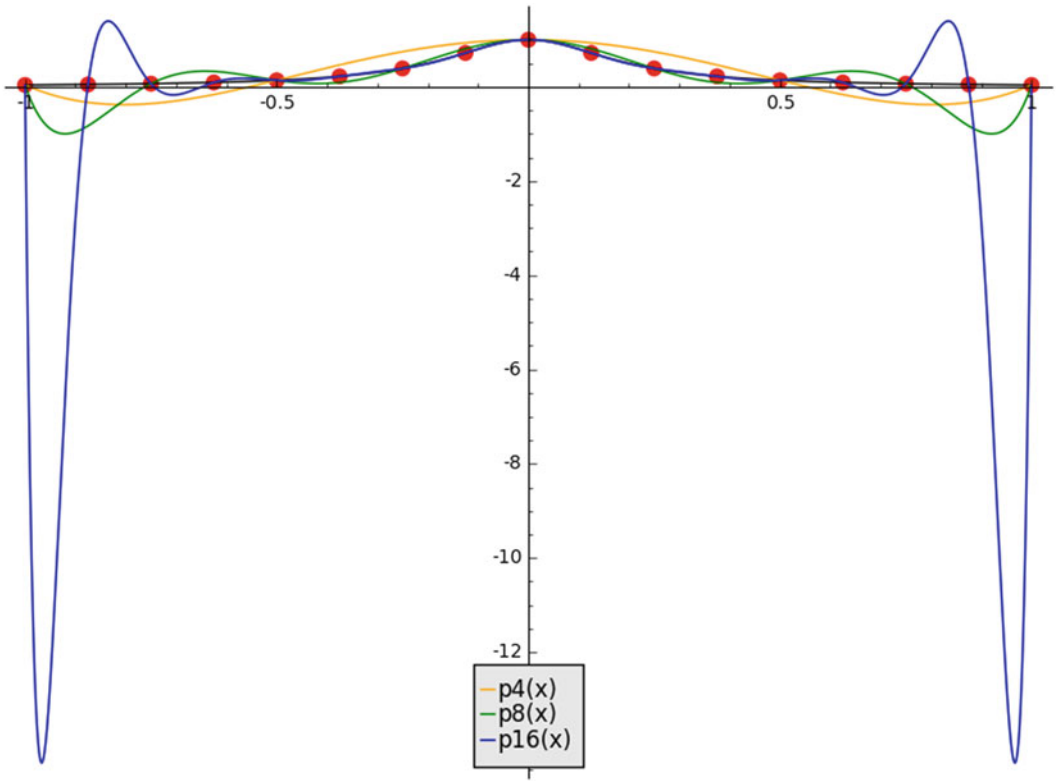
We can notice here how $p_8(x)$ has a higher error of approximation than $p_4(x)$.
If we wish to include $p_{16}(x)$, then we only need to change the following two lines

```
colors = ['orange',  'green',  'blue']
```

and

```
for n in [4, 8, 16]:
```

to obtain



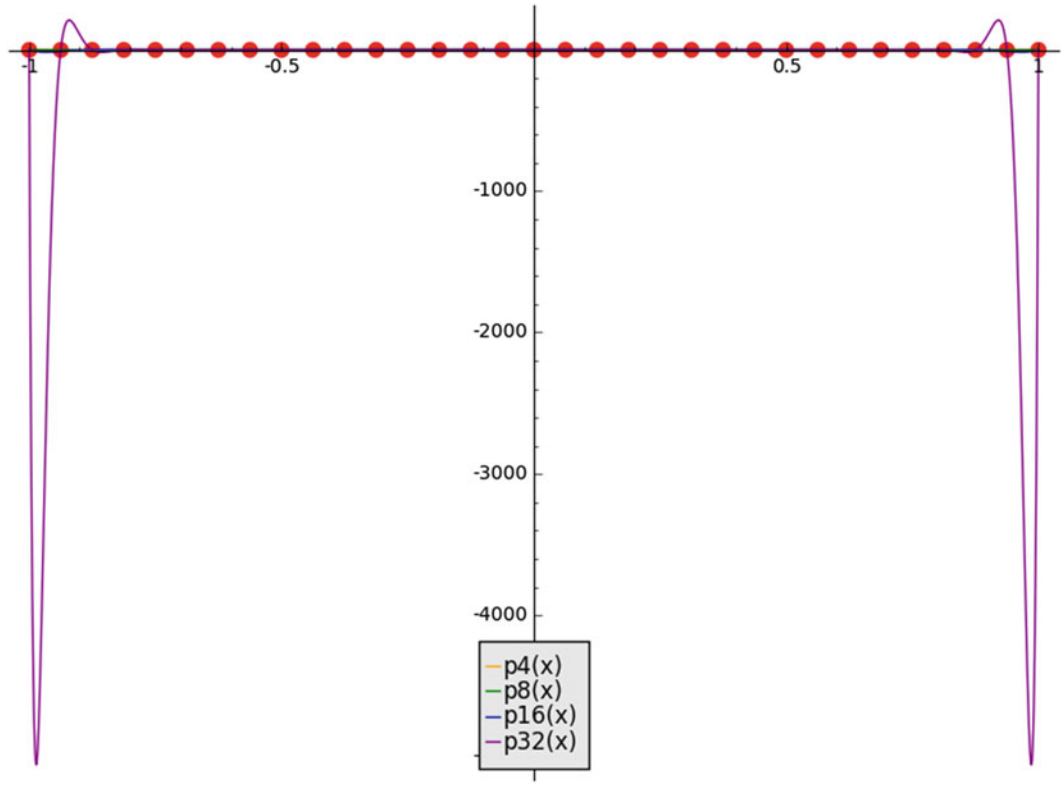
Last but not least, let's also include p_{32}. For this we make the following two changes:

```
colors = ['orange',  'green',  'blue', 'purple']
```

and

```
for n in [4, 8, 16, 32]:
```

to obtain



We give below a Sage interact that allows the user to select the number of points for interpolating the Runge's function.

```
#runge function
f(x) = 1/(1+25*x^2)

#plot the function
myPlot = plot(f(x), (x, -1,1), color = 'black')

@interact
def echo_input(j = slider(vmin=1, vmax=5,
                          step_size=1, default=2,
                          label="Select a degree: ")):
```

```
    print "You selected: " + str(2^j) + " points!"
    step = 0
    n = 2^j
    #compute the data points
    points = [(-1+i*(2/n),  f(-1+i*2/n)) for i in [0,1,..,n]]

    #compute the polynomial of interpolation
    R = PolynomialRing(QQ, 'x')
    p(x) = R.lagrange_polynomial(points)

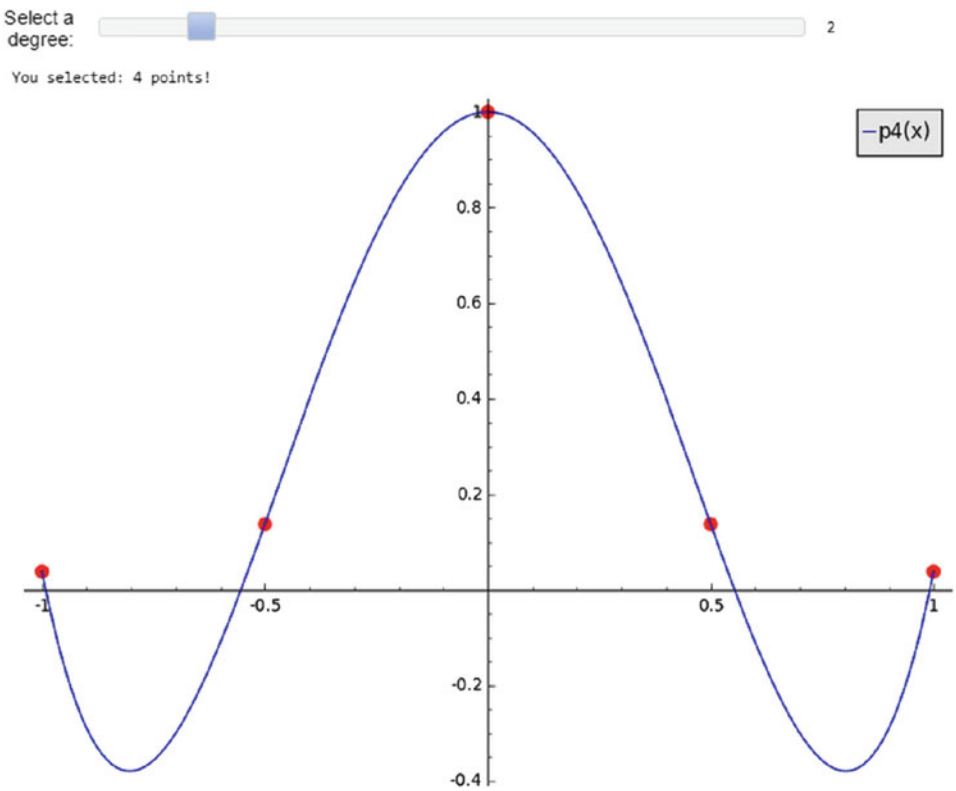
    #plot it
    myPlot = plot(p(x), (x, -1,1),  legend_label = "p"+str(n)
       +"(x)")
    #plot the points too
    myPlot += list_plot(points, color = 'red' , size = 70)



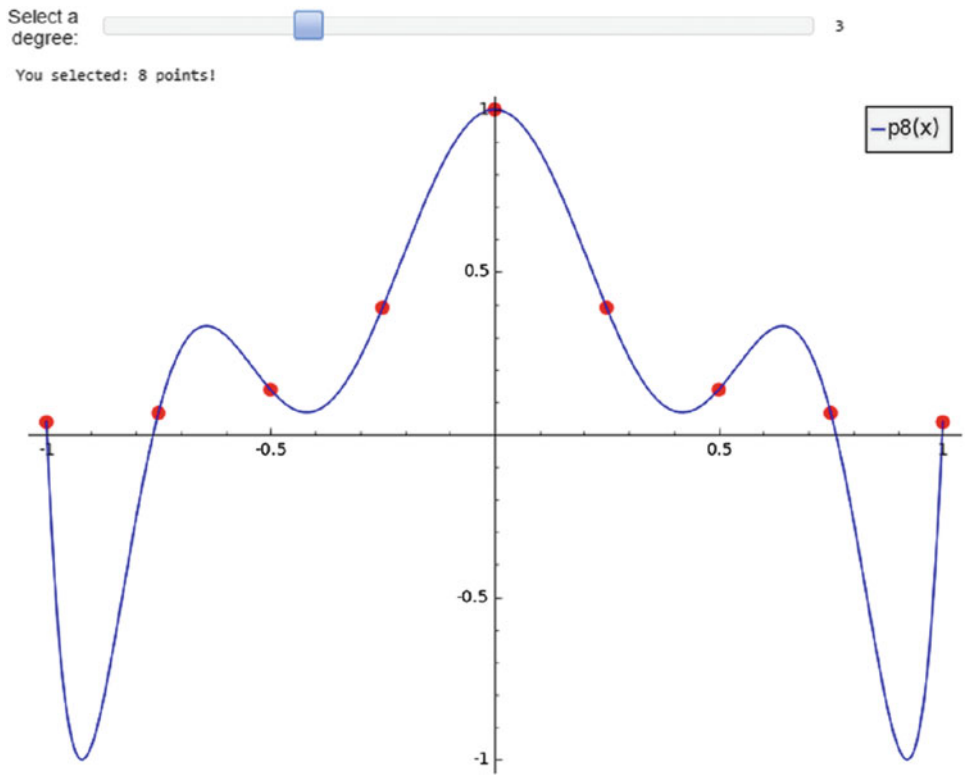
    #display the plot
    myPlot.show()
```

This will output the following graphs:

For $n = 2$:



For $n = 3$:



What you should see in these graphs is what is commonly called as **Runge's Phenomenon.** This is a proof that higher order polynomials do not always improve the accuracy.

One way to deal with this phenomenon in this case is to choose points that are not evenly spaced. For this, one can choose the following nodes, called **Chebyshev nodes**,

$$x_i = \cos\left(\frac{2i+1}{2n+2}\pi\right), \quad i = 0, 1, \ldots, n. \tag{3.20}$$

Using Chebyshev's nodes, and the Sage code we wrote for Newton Formula:

```
def Newton_Interpolation(x):
 #copy here the code given on page 132!


#runge function
f(x) = 1/(1+25*x^2)

#plot the function
myPlot = plot(f(x), (x, -1,1), color = 'black')

#plot 4 polynomials of interpolation using colors
colors = ['orange', 'green', 'blue', 'purple']
step = 0
```

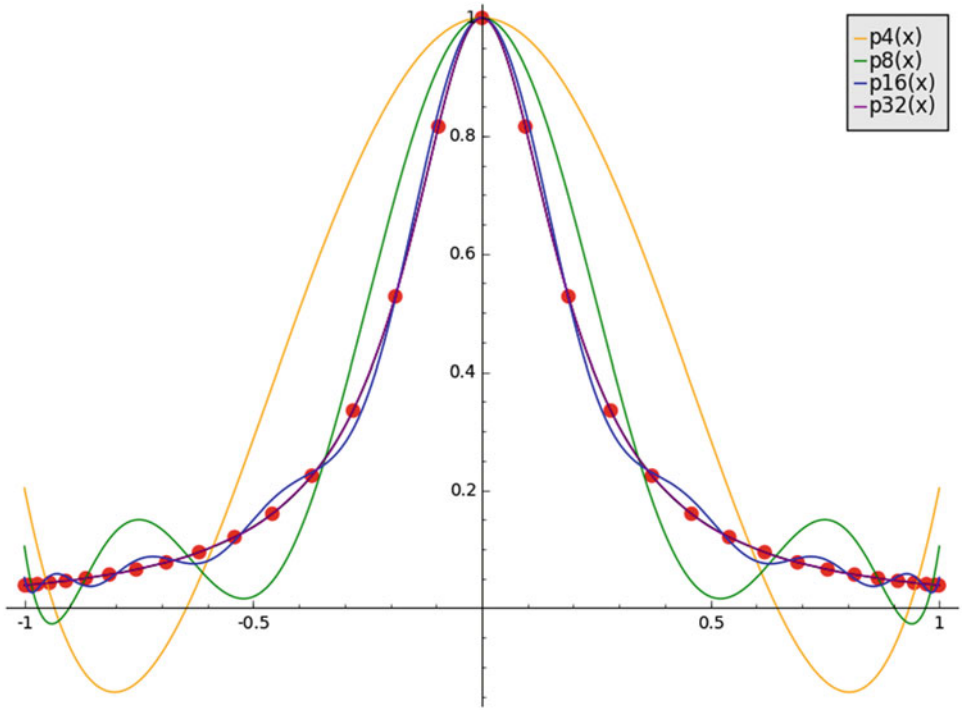
```
for n in [4,8,16, 32]:
    #compute the data points
    #xi=cos( (2*i+1)*pi/(2*n+2))
    #xi=-1+i*(2/n)
    points = [(  (cos( (2*i+1)*pi/(2*n+2))).n(),
              f(  ( cos( (2*i+1)*pi/(2*n+2)))).n()) for
              i in [0,1,..,n]]
    #compute the interpolation polynomial
    p(x) = Newton_Interpolation(x)

    #plot it
    myPlot += plot(p(x), (x, -1,1), color = colors[step],
              legend_label = "p"+str(n)+"(x)")
    step = step + 1

#plot the points too
myPlot += list_plot(points, color = 'red' , size = 70)

#display the plot
myPlot.show()
```

we obtain the following graph:



which "shows" (although without a proof!) a better behavior for $p_n(x)$ as n increases.

Despite these challenges, for a certain class of continuous functions, defined on a closed and bounded interval $[a, b]$, we can give an upper bound for the error of approximation. For this, let's consider the case of evenly spaced nodes in $[a, b]$:

$$\begin{aligned} x_0 &= a, \\ x_1 &= a + h, \\ x_2 &= a + 2h, \\ &\cdots \\ x_n &= a + nh = b, \end{aligned}$$

where

$$h = \frac{b-a}{n}.$$

We give the following theorem without a proof. Interested reader may refer to [4, pp. 181–183].

Theorem 87. *If f is a continuous function on $[a, b]$ with $f^{(n+1)}$ also continuous, such that $\left|f^{(n+1)}(x)\right| \leq M$, for some M, then for the polynomial $p_n(x)$ that interpolates the evenly spaced nodes $x_0, x_1, \ldots, x_n$ we have that*

$$|f(x) - p_n(x)| \leq \frac{1}{4(n+1)} M h^{n+1} \tag{3.21}$$

for all x in $[a, b]$.

Example 88. Obtain a polynomial that interpolates $\cos(x)$ on $[-\pi, \pi]$, using 6 evenly spaced nodes, and estimate $\cos(2.5)$. Then give an upper bound for the error of approximation.

Solution. Using the following Sage code, and the `Newton_Interpolation()` function we wrote earlier in this chapter:

```
def Newton_Interpolation(x):

#copy the code given on page 132

#function f
f(x) = cos(x)

#interval [a,b]
a = -pi
b = pi

#plot function f on [a,b]
myPlot = plot(f(x), (x, a, b), color = 'red',
              linestyle='--',
              legend_label = "f(x)")

#we want 6 evenly spaced nodes, hence n=5
n = 5

#the evenly spaced nodes are
points = [ (a + (b-a)/n*i ,   f(a + (b-a)/n*i)) for
          i in [0,1,..,n]]
```

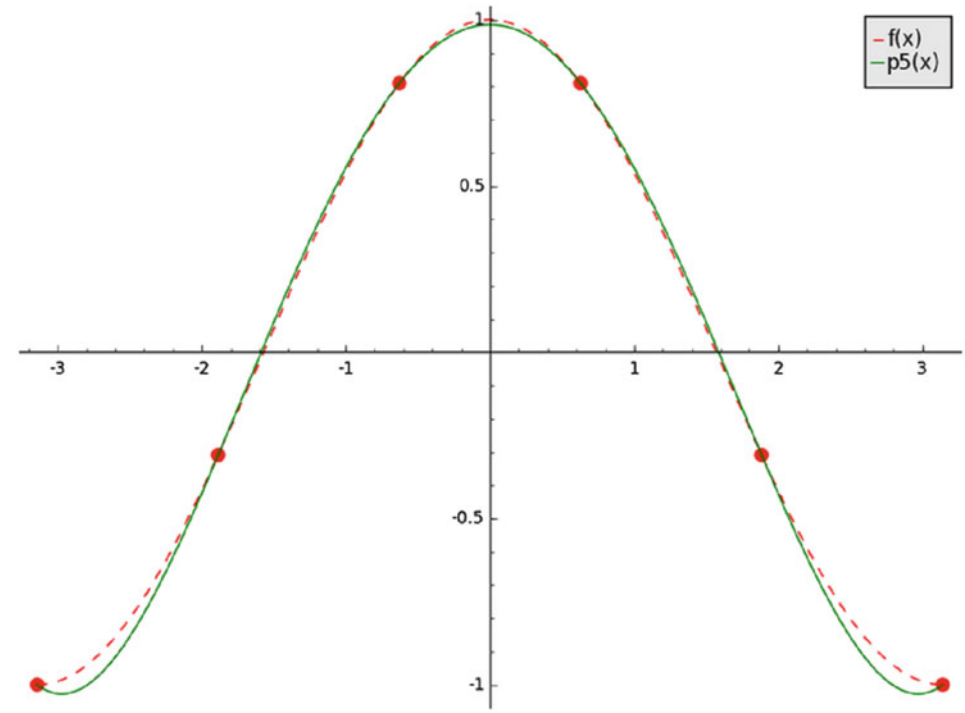
```
#compute the polynomial of interpolation
p(x) = Newton_Interpolation(x)

#plot it
myPlot += plot(p(x), (x, a,b), color = 'green',
               legend_label = "p"+str(n)+"(x)")


#plot the points too
myPlot += list_plot(points, color = 'red' , size = 70)


#display the plot, the function, and p(2.5)
myPlot.show(), p(x).full_simplify(), p(2.5).n()
```

we obtained the following graph,




```
(None,
 1/1536*(3*pi^4*(175*sqrt(5) + 113) - 50*pi^2*x^2*(73*sqrt(5) - 25) + 3125*x^4*(sqrt(5) - 1))/pi^4,
 -0.856062012732587)
```

and

$$p_5(x) = \frac{1}{1536\pi^4}(3\pi^4(175\sqrt{5}+113) - 50\pi^2x^2(73\sqrt{5}-25) + 3125x^4(\sqrt{5}-1)),$$

$$\cos(2.5) \simeq -0.856062012732586.$$

In order to give an upper bound for the error of approximation we will use formula (3.21) of the previous theorem. We have

$$|f(x) - p_5(x)| \leq \frac{1}{24} \cdot 1 \cdot \left(\frac{2\pi}{5}\right)^6 \simeq 0.164077089036852. \tag{3.22}$$

Using a calculator, we can estimate the actual error of approximation for $x = 2.5$:

$$\cos(2.5) - (-0.856062012732586) \simeq 0.0549183971856523.$$

3.3 An Application to Physics

Secretariat (March 30, 1970–October 4, 1989) is considered to be one of the greatest Thoroughbred racehorses of all time. On the First Saturday of May 1973, he made history by finishing the race of the Kentucky Derby in less than 2 min (119.4 s), and its record is still unsurpassed (as of 2014). Another surprising thing is that the Big Red Horse ran faster every quarter of mile of the race.

Below, we give the quarter mile fractions recorded at Kentucky Derby

distance (in miles)	0.00	0.25	0.50	0.75	1.00	1.25
time (in seconds)	0.0	25.2	49.2	73.0	96.4	119.4

1. Find the polynomial p that interpolates the data given above, treating time as the independent variable.
2. Use this polynomial to estimate Secretariat's speed at the finish of the race. That is, find $p'(119.4)$.

Solution.

1. Using the Sage code we developed for Newton Interpolation,

```
#the array of data points given
points=[(0,0),(25.2,0.25),(49.2,0.5),(73,0.75),(96.4,1),(119.4,1.25)]

#we compute n from the size of the array
n = len(points)-1

#call the function and find the final answer
Newton_Interpolation(x).full_simplify()
```

we obtain the polynomial:

```
x |--> -(1.44369270474e-11)*x^5 + (4.97960558388e-09)*x^4
       - (6.29658403201e-07)*x^3 + (3.88206642935e-05)*x^2
       + 0.00926834584814*x
```

2. Replacing the last line of Sage code given above with

```
#call the Newton Interpolation Function
p(x) = Newton_Interpolation(x).full_simplify()
#compute the derivative of this polynomial
```

```
p_prime(x) = diff( p(x), x )

#evaluate it at x = 119.4
p_prime(119.4)
```

we obtain 0.0108430556766 miles per second, which means 39.035 miles per hour.

3.4 Exercises

1. Knowing that $e^{0.72} \simeq 2.054433$ and $e^{0.73} \simeq 2.075080$, estimate $e^{0.722}$.
2. Knowing that $e^{0.72} \simeq 2.054433$ and $e^{0.73} \simeq 2.075080$, estimate $e^{0.727}$.
3. Find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c} x & 1.4 & 1.7 \\ \hline\hline y & 4.7 & 3.3 \end{array}.$$

   Use your answer to estimate the y value for $x = 1.5$.
4. Find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c} x & 1.4 & 1.7 \\ \hline\hline y & 4.7 & 4.7 \end{array}.$$

   Use your answer to estimate the y value for $x = 1.5$.
5. Find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c} x & 1.4 & 1.2 & 1.7 \\ \hline\hline y & 4.7 & 2.3 & 4.7 \end{array}.$$

   Use your answer to estimate the y value for $x = 1.5$.
6. Find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c} x & 1.4 & 1.7 & 2.1 \\ \hline\hline y & 4.7 & 7 & 10 \end{array}.$$

   Use your answer to estimate the y value for $x = 1.5$.
7. Find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c} x & 1.4 & 1.7 & 2.1 \\ \hline\hline y & 7 & 7 & 7 \end{array}.$$

   Use your answer to estimate the y value for $x = 1.5$.
8. Describe the steps you would use to estimate the value of $\sqrt[3]{7}$ using linear interpolation. Then estimate this value.
9. Describe the steps you would use to estimate the value of $\sqrt[3]{7}$ using quadratic interpolation. Then estimate this value.
10. Describe the steps you would use to estimate the value of $\sqrt[3]{7}$ using cubic interpolation. Then estimate this value.

11. Describe the steps you would use to estimate the value of $\sqrt[4]{213}$ using linear interpolation. Then estimate this value.
12. Prove the following: given a set of $n+1$ data points, there is only one polynomial of degree at most n that interpolates them.
13. Use Lagrange interpolation formula to obtain a polynomial of least degree that interpolates the table

$$\begin{array}{c|c|c|c|c|c} x & 0 & 2 & 3 & 4 & 7 \\ \hline y & 7 & 10 & 20 & 32 & 1 \end{array}.$$

14. Verify that both polynomials $p(x) = -\frac{1}{2}x^3 + \frac{13}{2}x^2 - 8x$, and $q(x) = -\frac{1}{40}x^4 - \frac{9}{40}x^3 + \frac{111}{20}x^2 - 7x$, interpolate the table

$$\begin{array}{c|c|c|c|c} x & 0 & 2 & 4 & 5 \\ \hline y & 0 & 6 & 40 & 60 \end{array}.$$

Does this violate the "*uniqueness*" part of the statement given on page 3.1.3?

15. Use Newton Algorithm to find the polynomial of least degree that interpolates the table

$$\begin{array}{c|c|c|c|c|c} x & 0 & 2 & 3 & 4 & 7 \\ \hline y & 7 & 10 & 20 & 32 & 1 \end{array}.$$

16. Use Newton Algorithm to find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c|c} x & 0 & 2 & 4 & 5 \\ \hline\hline y & 0 & 6 & 40 & 60 \end{array}.$$

17. Use Newton Algorithm to find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c} x & 1.4 & 1.7 & 2.1 \\ \hline\hline y & 7 & 7 & 7 \end{array}.$$

18. Use Newton Algorithm to find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c} x & 2 & 7 & 9 \\ \hline\hline y & 10 & 35 & 45 \end{array}.$$

19. Construct a divided-differences table and then write the Newton formula for the polynomial that interpolates the table

$$\begin{array}{c|c|c|c|c|c} x & 0 & 2 & 3 & 4 & 7 \\ \hline y & 7 & 10 & 20 & 32 & 1 \end{array}.$$

20. Construct a divided-differences table and then write the Newton formula for the polynomial that interpolates the table

$$\begin{array}{c||c|c|c|c} x & 0 & 2 & 4 & 5 \\ \hline\hline y & 0 & 6 & 40 & 60 \end{array}.$$

21. Construct a divided-differences table and then write the Newton formula for the polynomial that interpolates the table

$$\begin{array}{c||c|c|c} x & 1.4 & 1.7 & 2.1 \\ \hline\hline y & 7 & 7 & 7 \end{array}.$$

22. Give an upper bound for the error of approximation, if $\sin x$ is replaced by the polynomial of interpolation $p(x)$, for 11 evenly spaced nodes in $[-2\pi.2\pi]$.
23. Give an upper bound for the error of approximation, if $\sin x$ is replaced by the polynomial of interpolation $p(x)$, for 6 evenly spaced nodes in $[-\pi.\pi]$.
24. Give an upper bound for the error of approximation, if $\cos x$ is replaced by the polynomial of interpolation $p(x)$, for 11 evenly spaced nodes in $[-2\pi.2\pi]$.
25. Give an upper bound for the error of approximation, if $\cos x$ is replaced by the polynomial of interpolation $p(x)$, for 6 evenly spaced nodes in $[-\pi.\pi]$.

3.5 Computer Exercises

Use Sage to solve each of the following (include Sage code and a conclusion in your answers)

1. Knowing that $e^{0.72} \simeq 2.054433$ and $e^{0.73} \simeq 2.075080$, estimate $e^{0.722}$. What is the error of approximation committed using this value? Use Sage to compute this error.
2. Find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c} x & 1.4 & 1.7 \\ \hline\hline y & 4.7 & 3.3 \end{array}.$$

   Using Sage, graph the interpolation polynomial.
3. Find the polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c} x & 1.4 & 1.7 & 2.1 \\ \hline\hline y & 4.7 & 7 & 10 \end{array}.$$

   Using Sage, graph the interpolation polynomial.
4. Describe the steps you would use to estimate the value of $\sqrt[3]{17}$ using linear interpolation. Then estimate this value. Use Sage to compute the error of approximation.
5. Describe the steps you would use to estimate the value of $\sqrt[4]{213}$ using linear interpolation. Then estimate this value. Use Sage to compute the error of approximation.
6. Describe the steps you would use to estimate the value of $\sqrt[3]{17}$ using quadratic interpolation. Then estimate this value. Use Sage to compute the error of approximation.
7. Describe the steps you would use to estimate the value of $\sqrt[4]{213}$ using quadratic interpolation. Then estimate this value. Use Sage to compute the error of approximation.
8. Describe the steps you would use to estimate the value of $\sqrt[3]{17}$ using cubic interpolation. Then estimate this value. Use Sage to compute the error of approximation.
9. Describe the steps you would use to estimate the value of $\sqrt[4]{213}$ using cubic interpolation. Then estimate this value. Use Sage to compute the error of approximation.

10. Use the Sage library to find the polynomial of interpolation, $p(x)$, for

$$\begin{array}{c||c|c} x & 2.4 & 4.5\ 7.1\ 8.2 \\ \hline\hline y & 12 & 2.5\ 3.7\ 4 \end{array},$$

and then use if to find the value of y for $x = 6.7$.

11. Use the Sage library to find the polynomial of interpolation, $p(x)$, for the following stock market values

$$\begin{array}{c||c|c} x & 2009 & 2010 \quad 2011 \quad 2012 \quad 2013 \\ \hline\hline y & 10,428 & 11,577\ 12,657\ 13,437\ 16,469 \end{array},$$

and then use if to find the value of y for the year 2015.

12. Use the Sage library to find the polynomial of interpolation, $p(x)$, for

$$\begin{array}{c||c|c} x & 2 & 4\ 9\ 16 \\ \hline\hline y & 1 & 2\ 3\ 4 \end{array},$$

and then graph both, $p(x)$ and $f(x) = \sqrt{x}$.

13. Use the Sage library to find the polynomial of interpolation, $p(x)$, for

$$\begin{array}{c||c|c} x & 2 & 4\ 9\ 16\ 25 \\ \hline\hline y & 1 & 2\ 3\ 4\ \ 5 \end{array},$$

and then graph both, $p(x)$ and $f(x) = \sqrt{x}$.

14. Use the Sage library to find the polynomial of interpolation, $p(x)$, for

$$\begin{array}{c||c|c} x & 2 & 4\ 9\ 16 \qquad 25 \\ \hline\hline y & 1 & 2\ 3\ 4.0001\ 5 \end{array},$$

and then graph both, $p(x)$ and $f(x) = \sqrt{x}$. Compare this result with the one given in the previous problem.

15. Use Lagrange interpolation formula to obtain a polynomial of least degree that interpolates the table

$$\begin{array}{c|c|c|c|c} x & 0 & 2 & 4 & 5 \\ \hline y & 7 & 9 & 40 & 61 \end{array}.$$

Graph each of the Lagrange Basis Polynomials, and the interpolation polynomials on the same set of axes.

16. Use Lagrange interpolation formula to obtain a polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c|c} x & 1 & 2 & 3 & 5 \\ \hline\hline y & 3 & 5 & 5 & 7 \end{array}.$$

Graph each of the Lagrange Basis Polynomials, and the interpolation polynomials on the same set of axes.

17. Use Lagrange interpolation formula to obtain a polynomial of least degree that interpolates the table

$$\begin{array}{c||c|c|c|c|c|c|c} x & 1 & 1.5 & 2 & 2.5 & 3 & 3.4 & 4 \\ \hline y & 3 & 5.1 & 7 & 6.5 & 4 & 3.2 & 2 \end{array}.$$

Graph each of the Lagrange Basis Polynomials, and the interpolation polynomials on the same set of axes. Also approximate the y value for $x = 3.75$.

18. Construct a divided-differences table and then write the Newton formula for the polynomial that interpolates the table

$$\begin{array}{c|c|c|c|c|c} x & 0 & 2 & 3 & 4 & 7 \\ \hline y & 7 & 10 & 20 & 32 & 1 \end{array}.$$

19. Construct a divided-differences table and then write the Newton formula for the polynomial that interpolates the table

$$\begin{array}{c||c|c|c|c} x & 0 & 2 & 4 & 5 \\ \hline y & 0 & 6 & 40 & 60 \end{array}.$$

20. Construct a divided-differences table and then write the Newton formula for the polynomial that interpolates the table

$$\begin{array}{c||c|c|c} x & 1.4 & 1.7 & 2.1 \\ \hline y & 7 & 7 & 7 \end{array}.$$

21. Obtain a polynomial that interpolates $\cos(x)$ on $[-2\pi, 2\pi]$, using 6 evenly spaced nodes, and estimate $\cos(2.5)$. Then give an upper bound for the error of approximation. Include a graph showing both $f(x)$ and $p(x)$.
22. Obtain a polynomial that interpolates $\sin(x)$ on $[-\pi, \pi]$, using 6 evenly spaced nodes, and estimate $\sin(2.5)$. Then give an upper bound for the error of approximation. Include a graph showing both $f(x)$ and $p(x)$.
23. Obtain a polynomial that interpolates $\sin(x)$ on $[-\pi, \pi]$, using 16 evenly spaced nodes, and estimate $\sin(1.2)$. Then give an upper bound for the error of approximation. Include a graph showing both $f(x)$ and $p(x)$.
24. Obtain a polynomial that interpolates $\cos(x)$ on $[-\pi, \pi]$, using 10 evenly spaced nodes, and estimate $\cos(1.5)$. Then give an upper bound for the error of approximation. Include a graph showing both $f(x)$ and $p(x)$.
25. Obtain a polynomial that interpolates $\cos(x)$ on $[-2, 2]$, using 10 evenly spaced nodes, and estimate $\cos(1.5)$. Then give an upper bound for the error of approximation. Include a graph showing both $f(x)$ and $p(x)$.
26. Obtain the polynomial $p(x)$ that interpolates $f(x) = \frac{1}{1+25x^2}$ on $[-1, 1]$, using 11 evenly spaced nodes. Then, for each of the 31 evenly spaced points, output the values of f and p. Observe the largest difference between these values.
27. Obtain the polynomial $p(x)$ that interpolates $f(x) = \frac{1}{1+25x^2}$ on $[-1, 1]$, using 11 Chebyshev nodes. Then, for each of the 31 evenly spaced points, output the values of f and p. Observe the difference between these values. Are they significantly different than the ones obtained in the previous example?

28. Obtain the polynomial $p(x)$ that interpolates $f(x) = \frac{1}{1+25x^2}$ on $[-1, 1]$, using 25 evenly spaced nodes. Then, for each of the 49 evenly spaced points, output the values of f and p. Observe the largest difference between these values.
29. Obtain the polynomial $p(x)$ that interpolates $f(x) = \frac{1}{1+25x^2}$ on $[-1, 1]$, using 25 Chebyshev nodes. Then, for each of the 49 evenly spaced points, output the values of f and p. Observe the difference between these values. Are they significantly different than the ones obtained in the previous example?

References

1. Atkinson, K., Han, W.: Elementary Numerical Analysis, 3rd edn. Wiley, New York (2003)
2. Beezer, R.A.: Sage for Abstract Algebra (2013). http://abstract.pugetsound.edu/sage-aata.html
3. Blanco-Silva, F.J.: Learning SciPy for Numerical and Scientific Computing. Packt Publishing, Birmingham (2013)
4. Cheney, W., Kincaid, D.: Numerical Mathematics and Computing, 7th edn. Brooks/Cole: Cengage Learning, Boston (2013)
5. Epperson, J.F.: An Introduction to Numerical Methods and Analysis, revised edn. Wiley, Hoboken (2007)
6. Stein W.A., et al.: Sage Mathematics Software (Version 6.3), The Sage Development Team (2014). Sage official webpage: http://www.sagemath.org
7. Sage Cell Server: http://sagecell.sagemath.org/
8. Stein, W.A.: Sage for Power Users (2012). http://www.wstein.org/books/sagebook/sagebook.pdf

Chapter 4
Numerical Differentiation

In this chapter we discuss different numerical approximation techniques for estimating the derivatives of functions.

4.1 Derivative Formulae Using Taylor Series

4.1.1 First Order Derivatives

Let us recall the definition of the derivative of a function f (see [7]):

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}. \tag{4.1}$$

This motivates the following approximation formula

$$f'(x) \simeq \frac{f(x+h) - f(x)}{h}, \tag{4.2}$$

for h close enough to 0.

The expression $\frac{f(x+h)-f(x)}{h}$ is called the **numerical derivative** of f at x, with stepsize h.

The same formula (4.2) can also be derived using Taylor Series. But in here, we will be able to derive an estimate for the error of approximation, assuming that f is twice differentiable, with the second order derivative continuous around x. Using Theorem 35 (Taylor's Theorem with Remainder (2), given on page 54), with $n = 1$, we can derive the following formula:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2} f''(\xi), \tag{4.3}$$

The original version of this chapter was revised. An erratum to this chapter can be found at DOI 10.1007/978-3-319-16739-8_8

Electronic supplementary material The online version of this chapter (doi: 978-3-319-16739-8_4) contains supplementary material, which is available to authorized users.

© Springer International Publishing Switzerland 2015

G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8_4

for some ξ, between x and $x+h$. Therefore, by solving for $f'(x)$ we obtain

$$f'(x) = \frac{f(x+h)-f(x)}{h} - \frac{h}{2}f''(\xi) \tag{4.4}$$

$$= \frac{f(x+h)-f(x)}{h} + \mathcal{O}(h). \tag{4.5}$$

This formula shows that the error of approximating $f'(x)$ by $\frac{f(x+h)-f(x)}{h}$ is given by $-\frac{h}{2}f''(\xi)$, for some ξ between x and $x+h$. From it we can see that the rate of convergence is linear. It means that when h is halved, so is the error of approximation. In this estimation we did not take into account the change of the term $f''(\xi)$, which can speed up the convergence.

Example 89. Approximate the first derivative of the function $f(x) = \cos x$, at $x = 0$, and assess the error of approximation.

Solution. In this example, $f(x) = \cos x$. Hence, for small (close to 0!) values of h, we approximate

$$f'(x) \simeq \frac{\cos(x+h)-\cos x}{h}.$$

Doing so, the error of approximation is given by

$$-\frac{h}{2}f''(\xi) = \frac{h}{2}\cos(\xi),$$

for some ξ between x and $x+h$. Since $|\cos(\xi)| \leq 1$, we estimate our error to be at most $\frac{h}{2}$.

Using the Sage code (see also [2, 4–6])

```
#the given function
f(x)= cos(x)

#computing its derivative
fp(x) = diff(f(x),x)

#the given point x
x0 = 0

print "values of h",'\t', "numerical approx",'   \t' ,
               "error estimate",'\t' , "actual error"
for step in [1,2,..10]:
    #getting h
    h = 0.1/(2^step)

    #computing the numerical derivative
    df(x)=(f(x+h)-f(x))/h

    #actual error
    error = fp(x0)-df(x0)

    #displaying the results
    print h.n(30),'\t',df(x0).n(30),'   \t',h/2.n(30),'\t' ,error
```

we obtain the following table displaying the values of h used, the numerical approximations we obtained for the derivative of $f(x) = \cos x$ at $x = 0$, the estimation for the error (which was supposed to be at most $\frac{h}{2}$), and the actual error:

```
values of h     numerical approx  error estimate    actual error
0.050000000     -0.024994792      0.025000000       0.0249947921006743
0.025000000     -0.012499349      0.012500000       0.0124993489718932
0.012500000     -0.0062499186     0.0062500000      0.00624991862021318
0.0062500000    -0.0031249898     0.0031250000      0.00312498982748366
0.0031250000    -0.0015624987     0.0015625000      0.00156249872844683
0.0015625000    -0.00078124984    0.00078125000     0.000781249841111276
0.00078125000   -0.00039062498    0.00039062500     0.000390624980127541
0.00039062500   -0.00019531250    0.00019531250     0.000195312497453415
0.00019531250   -0.000097656250   0.000097656250    0.000097656562496362021
0.000097656250 -0.000048828126    0.000048828125    0.000048828125727595 8
```

Example 90. Approximate the first derivative of the function $f(x) = \sin x$, at $x = 2.67$, and assess the error of approximation.

Solution. In this example, $f(x) = \sin x$. Hence, for small values of h, we can approximate

$$f'(x) \simeq \frac{\sin(x+h) - \sin x}{h}.$$

Doing so, the error of approximation is given by

$$-\frac{h}{2} f''(\xi) = \frac{h}{2} \sin(\xi),$$

for some ξ between x and $x + h$. Since $|\sin(\xi)| \leq 1$, we again estimate our error to be at most $\frac{h}{2}$.

Using the Sage code

```
#the given function
f(x)= sin(x)

#computing its derivative
fp(x) = diff(f(x),x)

#the given point x
x0 = 2.67

print "values of h",'\t', "numerical approx",'   \t' ,\
                "error estimate",'\t' , "actual error"
for step in [1,2,..10]:
    #getting h
    h = 0.1/(2^step)

    #computing the numerical derivative
    df(x)=(f(x+h)-f(x))/h

    #actual error
    error = fp(x0)-df(x0)

    #displaying the results
    print h.n(30),'\t  ',df(x0).n(30),'   \t',h/2.n(30),
      '\t',error
```

we obtain

```
values of h    numerical approx error estimate  actual error
0.050000000     -0.90183000     0.025000000     0.0109841363851974
0.025000000     -0.89643160     0.012500000     0.00558573156250508
0.012500000     -0.89366204     0.0062500000    0.00281617453528005
0.0062500000    -0.89225977     0.0031250000    0.00141390083034698
0.0031250000    -0.89155427     0.0015625000    0.000708402087643001
0.0015625000    -0.89120043     0.00078125000   0.000354563746217784
0.00078125000   -0.89102324     0.00039062500   0.000177372521822483
0.00039062500   -0.89093458     0.00019531250   0.0000887089198083890
0.00019531250   -0.89089023     0.000097656250  0.0000443601230170065
0.000097656250 -0.89086805     0.000048828125  0.0000221814767126238
```

In order to improve the speed of convergence, one can use Theorem 35 (Taylor's Theorem with Remainder (2), given on page 54), with $n = 2$, to obtain

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(\xi_1) \tag{4.6}$$

for some ξ_1 between x and $x+h$. Hence, we have

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(\xi_2) \tag{4.7}$$

for some ξ_2 between x and $x-h$. Subtracting these two last formulae, we obtain

$$f(x+h) - f(x-h) = 2hf'(x) + \frac{h^3}{6}\left[f'''(\xi_1) + f'''(\xi_2)\right] \tag{4.8}$$

hence

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{12}\left[f'''(\xi_1) + f'''(\xi_2)\right] \tag{4.9}$$

$$= \frac{f(x+h) - f(x-h)}{2h} + \mathcal{O}(h^2). \tag{4.10}$$

Therefore, one could use the following numerical approximation for the first order derivative of f,

$$f'(x) \simeq \frac{f(x+h) - f(x-h)}{2h}, \tag{4.11}$$

for h close to zero. The right-hand side of the previous expression is called the **central difference formula**. Notice that the error of approximation here, given by $-\frac{h^2}{12}\left[f'''(\xi_1) + f'''(\xi_2)\right]$, converges to 0 quadratically, as h goes to zero.

Example 91. Use formula (4.11) to approximate the first derivative of the function $f(x) = \cos x$, at $x = 0$, and assess the error of approximation.

Solution. In this example, $f(x) = \cos x$. Hence, for small values of h, we approximate

$$f'(x) \simeq \frac{\cos(x+h) - \cos(x-h)}{2h}.$$

Doing so, the error of approximation is given by

$$-\frac{h^2}{12}\left[f'''(\xi_1)+f'''(\xi_2)\right]=\frac{h^2}{12}\left(\cos(\xi_1)+\cos(\xi_2)\right),$$

for some ξ_1,ξ_2 between $x-h$ and $x+h$. Since $|\cos(x)|\leq 1$, we estimate our error to be at most $\frac{h^2}{6}$.

Using the Sage code

```
#the given function
f(x)= cos(x)

#computing its derivative
fp(x) = diff(f(x),x)

#the given point x
x0 = 0

print "values of h",'\t', "numerical approx",'   \t' ,\
               "error estimate",'\t' , "actual error"
for step in [1,2,..10]:
    #getting h
    h = 0.1/(2^step)

    #computing the numerical derivative
    df(x)=(f(x+h)-f(x-h))/(2*h)

    #actual error
    error = fp(x0)-df(x0)

    #displaying the results
    print h.n(30),'\t',df(x0).n(30),'   \t\t',(h^2/6).n(30),'\t',error
```

we obtain the following table of values

```
values of h   numerical approx  error estimate  actual error
0.050000000    0.00000000       0.00041666667   0
0.025000000    0.00000000       0.00010416667   0
0.012500000    0.00000000       0.000026041667  0
0.0062500000   0.00000000       6.5104167e-6    0
0.0031250000   0.00000000       1.6276042e-6    0
0.0015625000   0.00000000       4.0690104e-7    0
0.00078125000  0.00000000       1.0172526e-7    0
0.00039062500  0.00000000       2.5431315e-8    0
0.00019531250  0.00000000       6.3578288e-9    0
0.000097656250 0.00000000       1.5894572e-9    0
```

Example 92. Use formula (4.11) to approximate the first derivative of the function $f(x)=\sin x$, at $x=2.67$, and assess the error of approximation.

Solution. In this example, $f(x)=\sin x$. Hence, for small values of h, we approximate

$$f'(x)\simeq\frac{\sin(x+h)-\sin(x-h)}{2h}.$$

Doing so, the error of approximation is given by

$$-\frac{h^2}{12}\left[f'''(\xi_1)+f'''(\xi_2)\right]=\frac{h^2}{12}\left(\sin(\xi_1)+\sin(\xi_2)\right),$$

for some ξ_1, ξ_2 between $x-h$ and $x+h$. Since $|\sin(x)| \leq 1$, we estimate our error to be at most $\frac{h^2}{6}$.

Using the Sage code

```
#the given function
f(x)= sin(x)

#computing its derivative
fp(x) = diff(f(x),x)

#the given point x
x0 = 2.67

print "values of h",'\t', "numerical approx",'   \t' ,\
               "error estimate",'\t' , "actual error"
for step in [1,2,..10]:
    #getting h
    h = 0.1/(2^step)

    #computing the numerical derivative
    df(x)=(f(x+h)-f(x-h))/(2*h)

    #actual error
    error = fp(x0)-df(x0)

   #displaying the results
   print h.n(30),'\t',df(x0).n(30),'   \t\t',(h^2/6).n(30),'\t',error
```

we obtain

```
values of h    numerical approx  error estimate  actual error
0.050000000    -0.89047473       0.00041666667   -0.000371139382367880
0.025000000    -0.89075307       0.00010416667   -0.000092735446132233
0.012500000    -0.89082267       0.000026041667  -0.000023198298594693
0.0062500000   -0.89084007       6.5104167e-6    -5.79976643744207e-6
0.0031250000   -0.89084442       1.6276042e-6    -1.44994377715424e-6
0.0015625000   -0.89084550       4.0690104e-7    -3.62486101246340e-7
0.00078125000  -0.89084578       1.0172526e-7    -9.06213660778477e-8
0.00039062500  -0.89084584       2.5431315e-8    -2.26552249182888e-8
0.00019531250  -0.89084586       6.3578288e-9    -5.66472702079324e-9
0.000097656250 -0.89084587       1.5894572e-9    -1.41693201616278e-9
```

4.1.2 Second Order Derivatives

We can do a similar work as above to derive a formula for the second derivative. Here, we use Theorem 35 (Taylor's Theorem with Remainder (2), given on page 54), with $n=3$, to obtain

$$f(x+h)+f(x-h)=2f(x)+h^2f''(x)+\frac{h^4}{24}\left(f^{(4)}(\xi_1)-f^{(4)}(\xi_2)\right)$$

for some ξ_1, ξ_2 between $x-h$ and $x+h$. Hence, we get

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} - \frac{h^2}{24}\left(f^{(4)}(\xi_1) - f^{(4)}(\xi_2)\right)$$
$$= \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + \mathcal{O}(h^2). \tag{4.12}$$

From the last formula, we see that the rate of convergence is quadratic. That is, as h is halved, the error is approximately quartered.

Therefore, for h close enough to zero, we have that

$$f''(x) \simeq \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}.$$

Example 93. Approximate the second derivative of the function $f(x) = \cos x$, at $x = 0$, and assess the error of approximation.

Solution. In this example, $f(x) = \cos x$. Hence, for small values of h, we approximate

$$(\cos(x))'' \simeq \frac{\cos(x+h) - 2\cos(x) + \cos(x-h)}{h^2}.$$

Doing so, the error of approximation is at most

$$\left|\frac{h^2}{24}\left(f^{(4)}(\xi_1) - f^{(4)}(\xi_2)\right)\right| \leq \frac{h^2}{12}.$$

Using similar Sage code as the one above, we obtain the following table

```
values of h     numerical approx  error estimate  actual error
0.050000000     -0.99979168       0.00020833333   -0.000208315973054596
0.025000000     -0.99994792       0.000052083333  -0.0000520822482030781
0.012500000     -0.99998698       0.000013020833  -0.0000130207663460169
0.0062500000    -0.99999675       3.2552083e-6    -3.25520522892475e-6
0.0031250000    -0.99999919       8.1380208e-7    -8.13801307231188e-7
0.0015625000    -0.99999980       2.0345052e-7    -2.03377567231655e-7
0.00078125000   -0.99999995       5.0862630e-8    -5.07570803165436e-8
0.00039062500   -0.99999999       1.2715658e-8    -1.30385160446167e-8
0.00019531250   -0.99999999       3.1789144e-9    -7.45058059692383e-9
0.000097656250 -1.0000000         7.9472860e-10    0.000000000000000
```

4.1.3 Method of Undetermined Coefficients

Other formulae can be derived using the **Method of Undetermined Coefficients**. We will present this method by means of an example.

Suppose we wish to find a formula for $f''(x)$ in terms of $f(x+2h)$, $f(x+h)$, $f(x)$, and $f(x-h)$. That is, we want to find the constants A, B, C, and D, such that

$$f''(x) \simeq Af(x+2h) + Bf(x+h) + Cf(x) + Df(x-h). \tag{4.13}$$

By Theorem 35 (Taylor's Theorem with Remainder (2), given on page 54), with $n = 3$, to obtain

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(x) + \cdots .$$

Hence we have

$$f(x+h) \simeq f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(x) + \cdots ,$$
$$f(x-h) \simeq f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(x) + \cdots ,$$

and

$$f(x+2h) \simeq f(x) + 2hf'(x) + 2h^2 f''(x) + \frac{4h^3}{3}f'''(x) + \frac{2h^4}{3}f^{(4)}(x) + \cdots .$$

Then, denoting $DD_h(x) = Af(x+2h) + Bf(x+h) + Cf(x) + Df(x-h)$, we have

$$\begin{aligned} DD_h(x) \simeq\ & Af(x) + 2Ahf'(x) + 2Ah^2 f''(x) + \frac{4Ah^3}{3}f'''(x) + \frac{2Ah^4}{3}f^{(4)}(x) + \cdots \\ & + Bf(x) + Bhf'(x) + \frac{Bh^2}{2}f''(x) + \frac{Bh^3}{6}f'''(x) + \frac{Bh^4}{24}f^{(4)}(x) + \cdots \\ & + Cf(x) \\ & + Df(x) - Dhf'(x) + \frac{Dh^2}{2}f''(x) - \frac{Dh^3}{6}f'''(x) + \frac{Dh^4}{24}f^{(4)}(x) + \cdots \end{aligned}$$

hence

$$\begin{aligned} DD_h(x) \simeq\ & (A + B + C + D)\, f(x) \\ & + (2A + B - D)\, hf'(x) \\ & + \left(2A + \frac{B}{2} + \frac{D}{2}\right) h^2 f''(x) \\ & + \left(\frac{4A}{3} + \frac{B}{6} - \frac{D}{6}\right) h^3 f'''(x) \\ & + \left(\frac{2A}{3} + \frac{B}{24} + \frac{D}{24}\right) h^4 f^{(4)}(x) \ldots . \end{aligned} \tag{4.14}$$

Since we want our formula to work for arbitrary functions f, we need to solve

$$\begin{cases} A + B + C + D = 0 \\ (2A + B - D)\, h = 0 \\ \left(2A + \frac{B}{2} + \frac{D}{2}\right) h^2 = 1 \\ \left(\frac{4A}{3} + \frac{B}{6} - \frac{D}{6}\right) h^3 = 0 \end{cases} .$$

Solving this system, we obtain

$$\begin{cases} A = 0 \\ B = \frac{1}{h^2} \\ C = \frac{-2}{h^2} \\ D = \frac{1}{h^2} \end{cases}. \tag{4.15}$$

hence we obtain again the formula

$$f''(x) \simeq \frac{f(x+h) - 2f(x) + f(x-h)}{2h^2}. \tag{4.16}$$

In order to obtain the error of approximation formula, one can substitute (4.15), into (4.14) to obtain

$$DD_h(x) \simeq f''(x) + \frac{1}{12h^2} h^4 f^{(4)}(x) + \cdots .$$

Therefore, the error of approximation can be estimated by

$$f''(x) - DD_h(x) \simeq -\frac{1}{12} h^2 f^{(4)}(x).$$

4.2 Derivative Formulae Using Interpolating Polynomials

We can use interpolating polynomials to approximate the derivative of a function f at given values of x. For this, let $p(x)$ be the polynomial that interpolates f at $n+1$ nodes: $x_0, x_1, \ldots, x_n$. That is

$$p(x_0) = f(x_0), p(x_1) = f(x_1), \ldots, p(x_n) = f(x_n). \tag{4.17}$$

Then, using the fact that

$$f(x) \simeq p(x) \tag{4.18}$$

we can derive the following

$$f'(x) \simeq p'(x). \tag{4.19}$$

In particular, when $n = 1$ we need two nodes, x_0, x_1, so we can compute the interpolating function $p(x)$. Then, we would obtain

$$p(x) = f(x_0) + f[x_0, x_1](x - x_0),$$

therefore

$$\begin{aligned} p'(x) &= f[x_0, x_1] \\ &= \frac{f(x_1) - f(x_0)}{x_1 - x_0}, \end{aligned} \tag{4.20}$$

hence

$$f'(x) \simeq \frac{f(x_1) - f(x_0)}{x_1 - x_0}. \tag{4.21}$$

There are different ways to choose x_0 and x_1, which will lead to different formulae.

If we choose $x_0 = x$, $x_1 = x + h$, we obtain

$$f'(x) \simeq \frac{f(x+h) - f(x)}{h} \tag{4.22}$$

which recovers formula (4.2) given on page 161. The error of approximation was already discussed there, hence we will not repeat it here.

If we choose $x_0 = x - h$, $x_1 = x + h$, we obtain

$$f'(x) \simeq \frac{f(x+h) - f(x-h)}{2h} \tag{4.23}$$

which recovers formula (4.11) given on page 164. The error of approximation for this formula was also discussed there, and we will not repeat it here.

A similar discussion can hold for $n = 2$, where we would have to choose three nodes. There we have

$$\begin{aligned} p(x) = {} & f(x_0) + f[x_0, x_1](x - x_0) \\ & + f[x_0, x_1, x_2](x - x_0)(x - x_1), \end{aligned}$$

therefore

$$p'(x) = f[x_0, x_1] + f[x_0, x_1, x_2](2x - x_0 - x_1), \tag{4.24}$$

hence

$$f'(x) \simeq f[x_0, x_1] + f[x_0, x_1, x_2](2x - x_0 - x_1). \tag{4.25}$$

If we choose $x_0 = x - h$, $x_1 = x, x_2 = x + h$, we obtain

$$\begin{aligned} f'(x) &\simeq \frac{f(x_1) - f(x_0)}{x_1 - x_0} + \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}(2x - x_0 - x_1) \\ &= \frac{f(x+h) - f(x-h)}{2h}, \end{aligned} \tag{4.26}$$

which again recovers formula (4.11) given on page 164.

If we choose $x_0 = x - h$, $x_1 = x + h$, we obtain

$$f'(x) \simeq \frac{f(x+h) - f(x-h)}{2h} \tag{4.27}$$

It can be shown (see [1, p. 235] and [3, p. 195]) that if f is $n+2$ continuous derivatives, then the error of approximation is given by

$$f'(x) - p'(x) = \frac{1}{(n+1)!}(x-x_0)(x-x_1)\dots(x-x_n)f^{(n+2)}(\xi) + \frac{1}{(n+1)!}\left[(x-x_0)(x-x_1)\dots(x-x_n)\right]' f^{(n+1)}(\xi),$$

for some ξ between the largest and the smallest of the $n+1$ nodes: $x_0, x_1, \dots, x_n$.

4.3 Exercises

1. Use formula (4.2) to approximate the first derivative of the function $f(x) = \sin x$, at $x = 1.5$, and assess the error of approximation.
2. Use formula (4.2) to approximate the first derivative of the function $f(x) = e^x$, at $x = 0$, and assess the error of approximation.
3. Use formula (4.2) to approximate the first derivative of the function $f(x) = \cos x$, at $x = \frac{\pi}{6}$, and assess the error of approximation.
4. Use formula (4.2) to approximate the first derivative of the function $f(x) = \cos x$, at $x = \pi$, and assess the error of approximation.
5. Use formula (4.2) to approximate the first derivative of the function $f(x) = \sin x$, at $x = \pi$, and assess the error of approximation.
6. Use formula (4.11) to approximate the first derivative of the function $f(x) = \sin x$, at $x = 1.5$, and assess the error of approximation.
7. Use formula (4.11) to approximate the first derivative of the function $f(x) = e^x$, at $x = 0$, and assess the error of approximation.
8. Use formula (4.11) to approximate the first derivative of the function $f(x) = \cos x$, at $x = \frac{\pi}{6}$, and assess the error of approximation.
9. Use formula (4.11) to approximate the first derivative of the function $f(x) = \cos x$, at $x = \pi$, and assess the error of approximation.
10. Use formula (4.11) to approximate the first derivative of the function $f(x) = \sin x$, at $x = \pi$, and assess the error of approximation.
11. Approximate the second derivative of the function $f(x) = \sin x$, at $x = \pi$, and assess the error of approximation.
12. Approximate the second derivative of the function $f(x) = \cos x$, at $x = \frac{\pi}{6}$, and assess the error of approximation.
13. Approximate the second derivative of the function $f(x) = \sin x$, at $x = 1.5$, and assess the error of approximation.
14. Derive the formula

$$f'(x) \simeq \frac{f(x+2h) - f(x-2h)}{4h},$$

and estimate the error of approximation in using it.

15. Derive the formula

$$f'(x) \simeq \frac{f(x+3h) - f(x-h)}{4h},$$

and estimate the error of approximation in using it.

16. Derive the formula

$$f'(x) \simeq \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h},$$

and estimate the error of approximation in using it.

4.4 Computer Exercises

1. Use formula (4.2) to approximate the first derivative of the function $f(x) = \sin x$, at $x = 1.5$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
2. Use formula (4.2) to approximate the first derivative of the function $f(x) = e^x$, at $x = 0$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
3. Use formula (4.2) to approximate the first derivative of the function $f(x) = \cos x$, at $x = \frac{\pi}{6}$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
4. Use formula (4.2) to approximate the first derivative of the function $f(x) = \cos x$, at $x = \pi$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
5. Use formula (4.2) to approximate the first derivative of the function $f(x) = \sin x$, at $x = \pi$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
6. Use formula (4.2) to approximate the first derivative of the function $f(x) = \arctan(x^2 + x + 1)$, at $x = 0$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
7. Use formula (4.11) to approximate the first derivative of the function $f(x) = \sin x$, at $x = 1.5$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
8. Use formula (4.11) to approximate the first derivative of the function $f(x) = e^x$, at $x = 0$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
9. Use formula (4.11) to approximate the first derivative of the function $f(x) = \cos x$, at $x = \frac{\pi}{6}$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
10. Use formula (4.11) to approximate the first derivative of the function $f(x) = \cos x$, at $x = \pi$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
11. Use formula (4.11) to approximate the first derivative of the function $f(x) = \sin x$, at $x = \pi$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
12. Use formula (4.11) to approximate the first derivative of the function $f(x) = \arctan(x^2 + x + 1)$, at $x = 0$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
13. Approximate the second derivative of the function $f(x) = \sin x$, at $x = \pi$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
14. Approximate the second derivative of the function $f(x) = \cos x$, at $x = \frac{\pi}{6}$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.
15. Approximate the second derivative of the function $f(x) = \sin x$, at $x = 1.5$, and assess the error of approximation. Print a table for values of $h = 0.1, 0.01, 0.001, \ldots, 0.000001$.

16. Given the following table of values

$$\begin{array}{c|c|c|c|c|c} x & 0.2 & 0.4 & 0.6 & 0.8 & 1.0 \\ \hline y & 7.11 & 10.21 & 20.98 & 32.91 & 1.12 \end{array},$$

estimate $f'(0.6)$, using $h = 0.2$. Estimate the error of approximation.

17. Given the following table of values

$$\begin{array}{c|c|c|c|c|c} x & 0.2 & 0.4 & 0.6 & 0.8 & 1.0 \\ \hline y & 7.11 & 10.21 & 20.98 & 32.91 & 1.12 \end{array},$$

estimate $f'(0.6)$, using $h = 0.4$. Estimate the error of approximation.

18. Given the following table of values

$$\begin{array}{c|c|c|c|c|c} x & 0.2 & 0.4 & 0.6 & 0.8 & 1.0 \\ \hline y & 7.11 & 10.21 & 20.98 & 32.91 & 1.12 \end{array},$$

estimate $f''(0.6)$, using $h = 0.2$. Estimate the error of approximation.

19. Given the following table of values

$$\begin{array}{c|c|c|c|c|c} x & 0.2 & 0.4 & 0.6 & 0.8 & 1.0 \\ \hline y & 7.11 & 10.21 & 20.98 & 32.91 & 1.12 \end{array},$$

estimate $f''(0.6)$, using $h = 0.4$. Estimate the error of approximation.

References

1. Atkinson, K., Han, W.: Elementary Numerical Analysis, 3rd edn. Wiley, New York (2003)
2. Blanco-Silva, F.J.: Learning SciPy for Numerical and Scientific Computing. Packt Publishing, Birmingham (2013)
3. Cheney, W., David Kincaid, D.: Numerical Mathematics and Computing, 7th edn. Brooks/Cole: Cengage Learning, Boston (2013)
4. Stein, W.A. et al.: Sage mathematics software (Version 6.3), The sage development team, Sage official webpage: http://www.sagemath.org (2014)
5. Sage cell server: http://sagecell.sagemath.org/
6. Stein, W.A.: Sage for power users, http://www.wstein.org/books/sagebook/sagebook.pdf (2012)
7. Sullivan, M., Miranda, K.: Calculus-Early Transcendentals. W. H. Freeman and Company, New York (2014)

Chapter 5
Numerical Integration

Definite integrals have many applications in several fields, including Physics, Mathematics, Probability and Statistics, Computer Science, Economics, and others. In this chapter we will see some numerical methods that can be used to approximate definite integrals. **Quadrature,** a Mathematical term used for the problems of calculating areas, will be used in the names of some of these methods.

Recall from Calculus (see, for example [10, p. 362–364]) the following result

Theorem 94 (Fundamental Theorem of Calculus). *Let f be a continuous function on $[a, b]$. Then*

$$\int_a^b f(x)dx = F(b) - F(a),$$

for any antiderivative F of f, on $[a, b]$.

By an **antiderivative** of a function f on $[a, b]$, we mean a function F that is differentiable on $[a, b]$, and $F'(x) = f(x)$ for every x in $[a, b]$.

Having this result, we can easily solve the following

Example 95. Find the value of the following definite integral:

$$\int_1^e \frac{1}{x}dx.$$

Solution. Let $f(x) = \frac{1}{x}$, for x in $[1, e]$. Then we can easily check that $F(x) = \ln x$ is an antiderivative of f on $[1, e]$, since $F'(x) = \frac{1}{x}$. Therefore, using the Fundamental Theorem of Calculus, we obtain

$$\begin{aligned}\int_1^e \frac{1}{x}dx &= \ln e - \ln 1 \\ &= 1 - 0 = 0.\end{aligned}$$

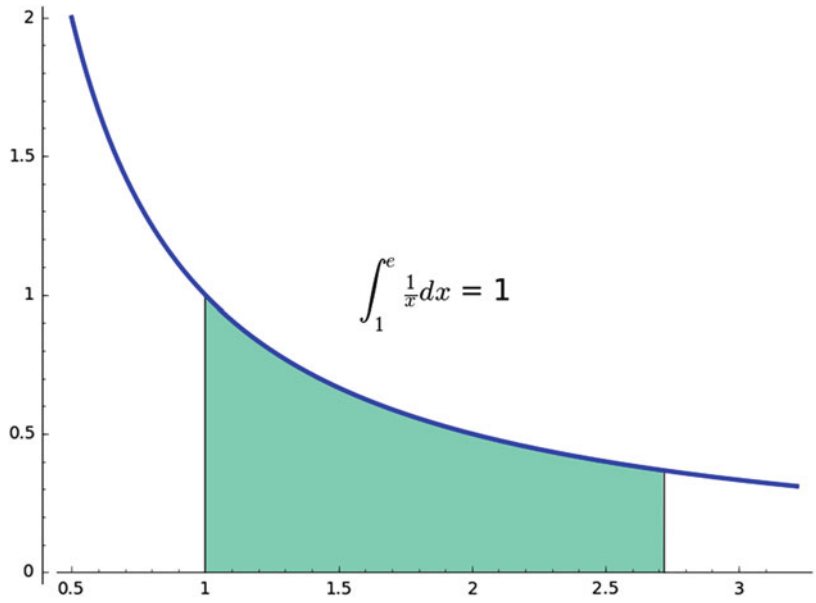
The original version of this chapter was revised. An erratum to this chapter can be found at DOI 10.1007/978-3-319-16739-8_8

Electronic supplementary material The online version of this chapter (doi: 978-3-319-16739-8_5) contains supplementary material, which is available to authorized users.

© Springer International Publishing Switzerland 2015

G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8_5

An interpretation of the *definite integral, for nonnegative functions*, is that it represents the *area under the curve* (between the function and the x-axis). So in our previous example, the region that is shaded has an area equal to 1.



The graph above was obtained using the following Sage code

```
#define the function to be used
f(x) = 1/x

#define the interval
a = 1
b = e

#graph the two horizontal lines
P = line([(a,0),(a,f(a))], color='black')
P += line([(b,0),(b,f(b))], color='black')

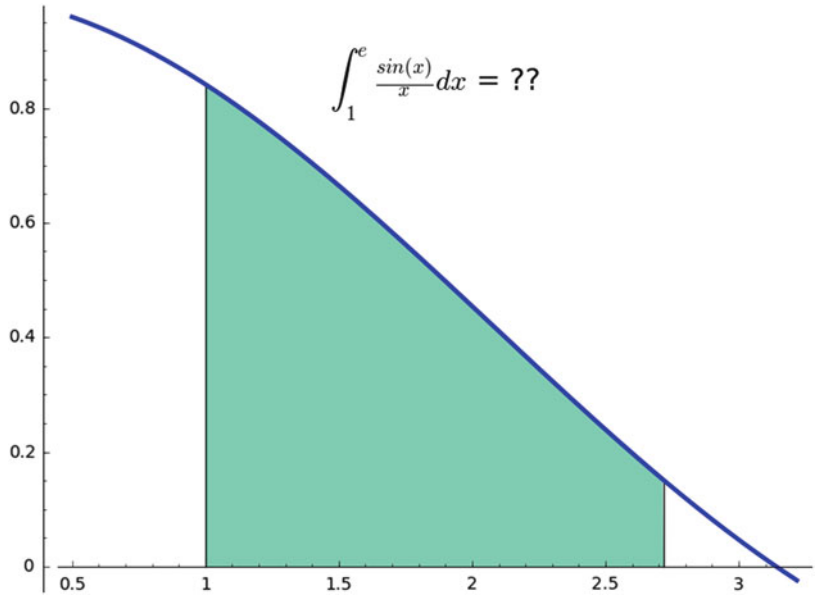
#make the shade between the graph and x-axis
#this will be a polygon with many points
P += polygon( [(a,0),(a,f(a))] +
              [(x, f(x)) for x in [a,a+0.005,..,b]] +
              [(b,0),(a,0)],
              rgbcolor=(0.5,0.8,0.7),aspect_ratio='automatic')

#draw the text above the graph
#NOTE THE USE OF LATEX CODE BELOW!!!
P += text("$\\int_{1}^e \\frac{1}{x} dx$ = 1",
( (a+b)/2, max(f(a),f(b)) ), fontsize=20, color='black')

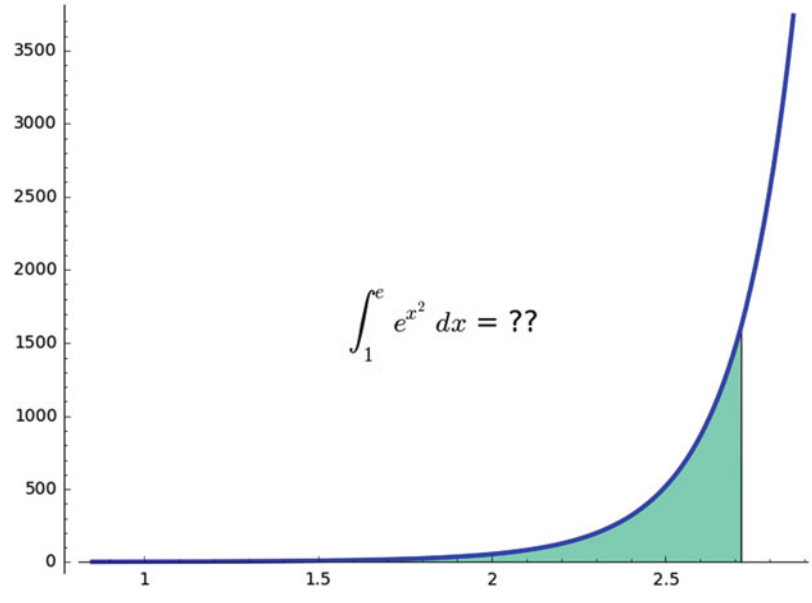
#draw the curve
P += plot(f, (x, a-0.5, b+0.5), thickness=3)
```

```
#show the plot
show(P)
```

In the previous example, if we replace the function $f(x) = \frac{1}{x}$ with $g(x) = \frac{\sin x}{x}$, or $h(x) = e^{x^2}$ then we obtain the following graphs:



and



We can see that those functions have a well-defined area under their curves, and hence the following expressions should be well defined

$$\int_1^e \frac{\sin x}{x} dx, \quad \text{and} \quad \int_1^e e^{x^2} dx.$$

But how could you compute them? Neither one of the functions $g(x) = \frac{\sin x}{x}$, and $h(x) = e^{x^2}$, have an antiderivative that can be expressed in terms of elementary functions. Therefore, we cannot simply apply the Fundamental Theorem of Calculus to find their values.

The answer to this question is the subject of the current chapter. We will see some formulae that will allow us to compute the values of the definite integral

$$\int_a^b f(x)dx$$

to any desired accuracy.

5.1 Trapezoidal Rule

5.1.1 Basic Trapezoidal Rule

The main idea of Trapezoidal Rule (see also [2, 5, 6, 10]) is to replace the function $f(x)$ with the linear function $p(x)$ that interpolates f at the nodes a and b (see formula (3.2), on page 118). From

$$\begin{aligned} f(x) &\simeq p(x) \\ &= \left(\frac{x-b}{a-b}\right) f(a) + \left(\frac{x-a}{b-a}\right) f(b) \end{aligned} \tag{5.1}$$

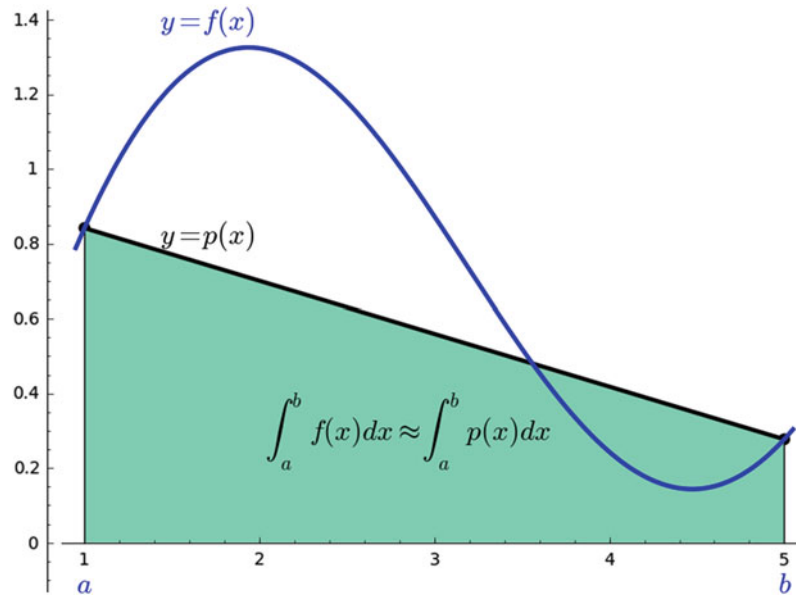
we obtain

$$\begin{aligned} \int_a^b f(x)dx &\simeq \int_a^b p(x)dx \\ &= \int_a^b \left[\left(\frac{x-b}{a-b}\right) f(a) + \left(\frac{x-a}{b-a}\right) f(b)\right] dx \\ &= (b-a)\left[\frac{f(a)+f(b)}{2}\right]. \end{aligned}$$

Hence, we call the **basic trapezoidal rule**, for the interval $[a, b]$ the formula:

$$\int_a^b f(x)dx \simeq (b-a)\left[\frac{f(a)+f(b)}{2}\right]. \tag{5.2}$$

We can illustrate this using the following graph.



As we can observe here, the area under the curve $f(x)$ is approximated with the area of a trapezoid, hence the name of the rule.

Let's see an example.

Example 96. Approximate the integral $\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx$ using the Basic Trapezoid Rule.

Solution. The previous graph is related to this problem, as it represents the function given here. Applying the formula (5.2), to $f(x) = \sin x - 1 + \sqrt{x}$, we obtain

$$\begin{aligned}\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx &\simeq (5-1)\left[\frac{f(5) + f(1)}{2}\right] \\ &\simeq 4.47445875057819.\end{aligned}$$

This particular example was chosen so we can compute the true value of the integral directly (by applying the Fundamental Theorem of Calculus):

$$\begin{aligned}\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx &= \left(-\cos x - x + \frac{2}{3}x\sqrt{x}\right)\Big|_1^5 \\ &\simeq 3.04353337873755.\end{aligned}$$

Or use the following Sage code (see [3, 7–9]) to get the same result:

```
integral(sin(x)-1+sqrt(x),x,1,5).n()
```

We obtain

```
3.04353337873755
```

Whichever the way we choose to compute it, we can see that the error of approximation using Basic Trapezoid Rule is -1.43092537184064, which is rather large.

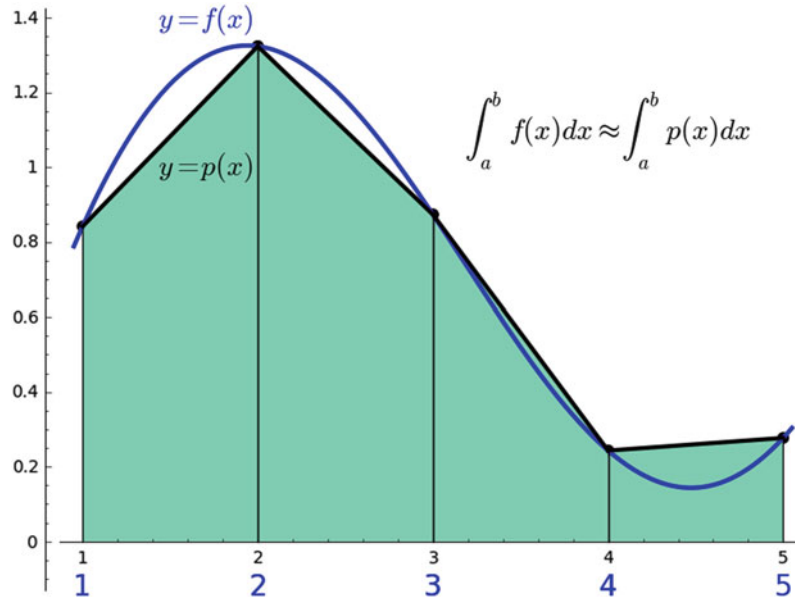
Next we'll see a technique that will allow us to improve the *Basic Trapezoidal Rule* and get an answer much more accurate value. The error can be made as small as needed, using appropriate parameters.

5.1.2 Composite Trapezoidal Rule

We should note that the *Basic Trapezoidal Rule* works best when the graph of $f(x)$ is linear or nearly linear. However, this is usually not the case. So the trick we'll do in here is to split the interval $[a, b]$ into several subintervals: $[a = x_0, x_1], [x_1, x_2], \ldots, [x_{n-1}, x_n = b]$ and then apply the Basic Trapezoidal Rule on each of these subintervals. This is what we call the **Composite Trapezoidal Rule**:

$$\int_a^b f(x)dx \simeq \sum_{k=1}^{n} (x_k - x_{k-1}) \left[\frac{f(x_k) + f(x_{k-1})}{2} \right]. \tag{5.3}$$

If we split the interval $[a, b]$ in subintervals that are small enough, then the function $f(x)$ will be approximately linear on each one, and hence we expect to get a much better estimate.



In general, the more nodes we use, the more accurate answers we should expect. When dealing with computers, there is a limit, of course, in how many nodes we can actually use, due to the propagation of errors.

Example 97. Approximate the integral $\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx$ using the Composite Trapezoid Rule, with five evenly spaced nodes.

Solution. The previous graph is related to this problem, as it represents the function given here. Using formula (5.3), for $f(x) = \sin x - 1 + \sqrt{x}$, we obtain

$$\begin{aligned}\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx &\simeq (2-1)\left[\frac{f(2)+f(1)}{2}\right] + (3-2)\left[\frac{f(3)+f(2)}{2}\right] \\ &\quad + (4-3)\left[\frac{f(4)+f(3)}{2}\right] + (5-4)\left[\frac{f(5)+f(4)}{2}\right] \\ &= \frac{1}{2}f(1) + f(2) + f(3) + f(4) + \frac{1}{2}f(5) \\ &\simeq 2.99918665334187,\end{aligned}$$

which is a much better estimate. The error of approximation is in this case about 0.0443467253956832, which is a great improvement.

When we choose the nodes, we can select them as either uniformly or nonuniformly spaced in $[a, b]$. The previous formula works with either case. In practice, however, when we are using the power of computers, we would usually prefer to use nodes that are evenly spaced, as it simplifies the algorithm we will be implementing next. For this, we are dividing the interval $[a, b]$ into n subintervals of equal length each,

$$h = \frac{b-a}{n}, \tag{5.4}$$

and obtain the following evenly spaced nodes

$$x_i = a + i * h, i = 0, 1, \ldots, n. \tag{5.5}$$

Then we obtain the following simplified formula called **Composite Trapezoidal Rule with evenly spaced nodes**:

$$\begin{aligned}\int_a^b f(x)dx &\simeq h\sum_{k=1}^{n}\left[\frac{f(x_k)+f(x_{k-1})}{2}\right] \qquad (5.6)\\ &= h\left[\frac{1}{2}f(x_0) + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + \frac{1}{2}f(x_n)\right] \\ &= h\left[\frac{1}{2}f(a) + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + \frac{1}{2}f(b)\right].\end{aligned}$$

5.1.3 Sage Code

Here we will implement Sage code to compute a definite integral using Composite Trapezoidal Rule with evenly spaced nodes in $[a, b]$. Then we will apply this code to find the answer to each of the following problems:

$$\int_1^e \frac{\sin x}{x}dx, \quad \int_0^1 e^{-x^2}dx, \quad \int_1^e e^{x^2}dx,$$

and compare these answers against the values given by the Sage Library function `integral()`.

Below, we give the Sage code that implements the formula (5.6)

```
#this function needs [a,b] and n, for n+1 nodes
def CompositeTrapezoidal(f, a, b, n):
    #compute the size of each subinterval
    h = (b-a)/n

    #here we add all the terms
    sum = 1/2*f(a)

    #add the values for all intermediate nodes
    for i in [1,2,..,n-1] :
        sum += f(a+i*h)

    #add the last node
    sum +=1/2*f(b)

    #multiply by h to complete the computation
    sum = sum*h

    #return the sum
    return sum
```

Let's use this code to solve some of the problems.

Example 98. Apply the previous Sage code to find the value of

$$\int_1^e \frac{\sin x}{x} dx,$$

using $n = 5$ subintervals.

Solution. For this, to the previous Sage code, we simply add

```
#define the function we will be using
f(x) = sin(x)/x

#call the function on [1,e], with 5+1 nodes
print CompositeTrapezoidal(f, 1, e, 5).n()
```

and obtain

```
0.874072951263462
```

Using the Sage Library function, `integral(sin(x)/x,x,1,e).n()` we obtain:

```
0.874957198780384
```

Example 99. Create a table of values obtained using the Composite Trapezoidal Rule, with $n = 1, 2, 4, 8, 16$ and 32, for the expression

$$\int_0^1 \frac{1}{1+x^2} dx,$$

and print out the error of approximation in each case.

Solution. For this, we use the following Sage code:

```
#define the function we will be using
f(x) = 1/(1+x^2)

#print the top part of the table
print 'trapezoidal' , '\t\t', 'error'
print '----------------------------------------'

#call the function several times
for power in [0,1,2,3,4, 5]:
    n = 2^power
    trapezoidal = CompositeTrapezoidal(f, 0, 1, n).n()
    error = (trapezoidal - integral(f(x) ,x,0,1)).n()
    print trapezoidal , '\t', error
```

to obtain

```
trapezoidal           error
----------------------------------------
0.750000000000000     -0.0353981633974483
0.775000000000000     -0.0103981633974483
0.782794117647059     -0.00260404575038942
0.784747123622772     -0.000651039774676065
0.785235403010347     -0.000162760387101057
0.785357473293744     -0.0000406901037046659
```

Example 100. Use the Composite Trapezoidal Rule, with $n = 32$, to find an approximate value for

$$\int_0^1 e^{-x^2} dx.$$

Solution. For this, we use the following Sage code:

```
#define the function we will be using
f(x) = e^(-x^2)

#set the value of n
n = 32

#call the function on [0,1], with n+1 nodes
CompositeTrapezoidal(f, 0, 1, n).n()
```

to obtain

```
0.746764254652294
```

Using the Sage Library function, `integral(f(x),x,0,1).n()` we obtain

```
0.746824132812427
```

Example 101. Use the Composite Trapezoidal Rule, with $n = 64$, to find an approximate value for

$$\int_1^e e^{x^2} dx.$$

Solution. For this, we use the following Sage code:

```
#define the function we will be using
f(x) = e^(x^2)

#set the value of n
n = 64

#call the function on [1,e], with n+1 nodes
CompositeTrapezoidal(f, 1, e, n).n()
```

and obtain

```
323.823079851799
```

Using the Sage Library function, `integral(f(x),x,1,e).n()` we obtain

```
323.295188678248
```

5.1.4 Error Bounds

Let us use the following notation for a function f defined on $[a, b]$:

$$I = \int_a^b f(x)dx$$

which will represent the definite integral of f on $[a, b]$ and

$$\begin{aligned} T &= h \sum_{k=1}^{n} \left[\frac{f(x_k) + f(x_{k-1})}{2} \right] \\ &= h \left[\frac{1}{2} f(a) + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + \frac{1}{2} f(b) \right], \end{aligned}$$

which is given by the Composite Trapezoidal Rule with $n + 1$ evenly spaced nodes

$$x_i = a + i * h, i = 0, 1, \ldots, n, \tag{5.7}$$

with

$$h = \frac{b-a}{n}. \tag{5.8}$$

We give next, without a proof, the following error estimation. The reader interested in the proof of this result can refer to [5, p. 206].

Theorem 102. *Let f be a function defined on $[a, b]$ that is twice differentiable, with f'' also continuous. Then the error of approximation is given by*

$$I - T = -\frac{(b-a)h^2}{12} f''(\xi), \tag{5.9}$$

for some ξ in (a, b).

In particular we have the following estimation for the error of approximation

Corollary 103. *Let f be a function defined on $[a,b]$ that is twice differentiable, with f'' also continuous, such that $|f''(x)| \leq M$, for all x in $[a,b]$. Then*

$$|I - T| \leq \frac{(b-a)Mh^2}{12}. \tag{5.10}$$

This last result shows that the Composite Trapezoidal Rule with evenly spaced nodes **converges quadratically**.

Another direct consequence of the previous theorem is

Corollary 104. *Let f be a linear function defined on $[a,b]$. Then the Trapezoid Rule gives us exact answers, that is*

$$I = T. \tag{5.11}$$

Example 105. How many evenly spaced nodes do we need to use in Composite Trapezoidal Rule, in order to find an approximate value for

$$\int_1^e e^{x^2} dx,$$

with an error less than 0.001?

Solution. We have $f(x) = e^{x^2}$, defined on $[1,e]$. Hence

$$f'(x) = 2xe^{x^2}$$

and

$$f''(x) = (2 + 4x^2)e^{x^2}$$

and therefore

$$\begin{aligned} \left|f''(x)\right| &\leq (2 + 4e^2)e^{e^2} \\ &< 51063.58783. \end{aligned}$$

Using formula (5.10), we have

$$\begin{aligned} |I - T| &\leq \frac{(b-a)Mh^2}{12} \\ &< \frac{51063.58783\,(e-1)^3}{12n^2} \\ &< \frac{21588.04287}{n^2} \end{aligned}$$

Therefore, from

$$\frac{21588.04287}{n^2} < 0.001$$

we obtain

$$n \geq 4647.$$

The answer to the question is therefore 4648 evenly spaced nodes would be sufficient.

Example 106. How many evenly spaced nodes do we need to use in Composite Trapezoidal Rule, in order to find an approximate value for

$$\int_0^1 e^{x^2} dx,$$

with an error less than 0.0001? Then estimate the value of $\int_0^1 e^{x^2} dx$.

Solution. We have $f(x) = e^{x^2}$, defined on $[0, 1]$. Hence

$$f'(x) = 2xe^{x^2}$$

and

$$f''(x) = (2 + 4x^2)e^{x^2}$$

and therefore

$$\left| f''(x) \right| \leq 6e.$$

Using formula (5.10), we have

$$\begin{aligned} |I - T| &\leq \frac{(b-a)Mh^2}{12} \\ &\leq \frac{e}{2n^2}. \end{aligned}$$

Therefore, from

$$\frac{e}{2n^2} < 0.0001$$

we obtain

$$n \geq 117,$$

which means we would need at least 117+1=118 evenly spaced nodes to make sure the error of approximation is less than 0.0001. Using the Sage code we developed earlier, we obtain the following estimation

$$\int_0^1 e^{x^2} dx \simeq 1.46268484119943.$$

Compare this to `1.46265174590718`, the value obtained from `integrate(e^(x^2), x, 0, 1).n()`.

5.1.5 Sage Library

If we want to utilize Sage libraries, we can use Sage code similar to the one in the next examples

Example 107. Find the value of

$$\int_1^e \frac{\sin x}{x} dx,$$

using $n = 5$ subintervals.

Solution. For this, one can use

```
#import the needed libraries
from scipy.integrate import simps
import scipy
import numpy

#define the function, the interval [a,b], and n
def f(x):
    return sin(x)/x
a=1.0
b=e
n=5

#define the n+1 evenly spaced nodes in [a,b]
x = numpy.array([a+i*(b-a)/n  for i in [0,1,..,n]])

#compute and output the Trapezoidal Rule
print  scipy.integrate.trapz(f(x), x).n()
```

to obtain

```
0.874072951263462
```

Example 108. Find the value of

$$\int_0^1 \frac{1}{1+x^2} dx,$$

using $n = 32$ subintervals.

Solution. For this, we can use the following Sage code:

```
#import the needed libraries
from scipy.integrate import simps
import scipy
import numpy

#define the function, the interval [a,b], and n
def f(x):
    return 1/(1+x^2)
a=0
```

```
b=1
n=32

#define the n+1 evenly spaced nodes in [a,b]
x = numpy.array([a+i*(b-a)/n  for i in [0,1,..,n]])

#compute and output the Trapezoidal Rule
print  scipy.integrate.trapz(f(x), x)
```

and obtain:

```
0.785357473294
```

5.2 Simpson's Rule

5.2.1 *Basic Simpson's Rule*

In Basic Trapezoidal Rule, we approximated the function $f(x)$ on $[a,b]$ by the linear function that interpolated f at the nodes a and b. Hence we used

$$f(x) \simeq \left(\frac{x-b}{a-b}\right) f(a) + \left(\frac{x-a}{b-a}\right) f(b).$$

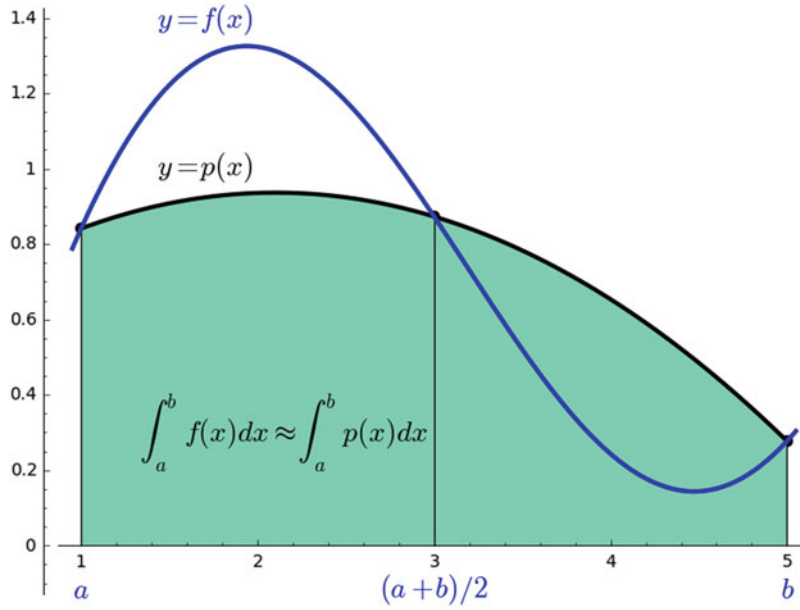
In Simpson's Rule, we will replace the function $f(x)$ with the quadratic interpolation function $p(x)$ that interpolates f at the nodes a, $\frac{a+b}{2}$, and b (see formula (3.3), on page 121):

$$\begin{aligned} f(x) &\simeq p(x) \\ &= \left(\frac{x-\frac{a+b}{2}}{a-\frac{a+b}{2}}\right)\left(\frac{x-b}{a-b}\right) f(a) \\ &\quad + \left(\frac{x-a}{\frac{a+b}{2}-a}\right)\left(\frac{x-b}{\frac{a+b}{2}-b}\right) f\left(\frac{a+b}{2}\right) \\ &\quad + \left(\frac{x-a}{b-a}\right)\left(\frac{x-\frac{a+b}{2}}{b-\frac{a+b}{2}}\right) f(b) \end{aligned} \tag{5.12}$$

hence we obtain the following **basic trapezoidal rule**, for the interval $[a,b]$:

$$\begin{aligned} \int_a^b f(x)dx &\simeq \int_a^b p(x)dx \\ &= \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]. \end{aligned} \tag{5.13}$$

We illustrate this means of an example.



Example 109. Approximate the integral $\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx$ using the Basic Simpson's Rule.

Solution. The above given graph is related to this problem, as it represents the function given here. Applying the formula (5.13), to $f(x) = \sin x - 1 + \sqrt{x}$, we obtain

$$\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx \simeq \frac{5-1}{6}\left[f\left(1\right) + 4f\left(\frac{1+5}{2}\right) + f\left(5\right)\right]$$
$$\simeq 3.07419863343968.$$

This particular example was chosen so we can compute true value of the integral directly (by applying the Fundamental Theorem of Calculus):

$$\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx = \left(-\cos x - x + \frac{2}{3}x\sqrt{x}\right)\Big|_1^5$$
$$\simeq 3.04353337873755.$$

As we can see that the error of approximation is approximately 0.0306652547021335, which is fairly good.

Next we'll see the *Composite Simpson's Rule* that will help us improve our estimation. As with the Trapezoid Rule, the error can be made as small as needed, using appropriate parameters. We'll also discuss this below, after we give the Sage implementation for our algorithm.

5.2.2 Composite Simpson's Rule

We should note from the way de derived Simpson's Rule that the more close to a linear or quadratic a function f is on a given interval $[a, b]$, the more accurate answers we obtain using Simpson's Rule. In particular, if f is a polynomial of degree at most two, then the Simpson's Rule will give an **exact** answer. In reality, it is exact also for cubic polynomials, but this will become clear once we analyze the error of approximation.

As before, it makes sense to divide the interval $[a, b]$ into a number of subintervals, $[a = x_0, x_2]$, $[x_2, x_4], \ldots, [x_{n-2}, x_n]$, and apply the Basic Simpson's Rule on each such subinterval. Note that here n must be even, and $x_1 = \frac{x_0+x_2}{2}, x_3 = \frac{x_2+x_4}{2}, \ldots, x_{n-1} = \frac{x_{n-2}+x_n}{2}$. Then we combine (add) the answers for each such subinterval to obtain an estimation for the entire interval. This is what will be called the **Composite Simpson's Rule**.

$$\begin{aligned}\int_a^b f(x)dx &= \int_{x_0}^{x_2} f(x)dx + \int_{x_2}^{x_4} f(x)dx + \cdots + \int_{x_{n-2}}^{x_n} f(x)dx \\ &\simeq \frac{x_0 - x_2}{6}\left[f\left(x_0\right) + 4f\left(x_1\right) + f\left(x_2\right)\right] \\ &\quad + \frac{x_2 - x_4}{6}\left[f\left(x_2\right) + 4f\left(x_3\right) + f\left(x_4\right)\right] \\ &\quad + \cdots \\ &\quad + \frac{x_{n-2} - x_n}{6}\left[f\left(x_{n-2}\right) + 4f\left(x_{n-1}\right) + f\left(x_n\right)\right].\end{aligned}$$

In particular, if the nodes are evenly spaced in $[a, b]$, that is

$$x_i = a + ih,$$

where

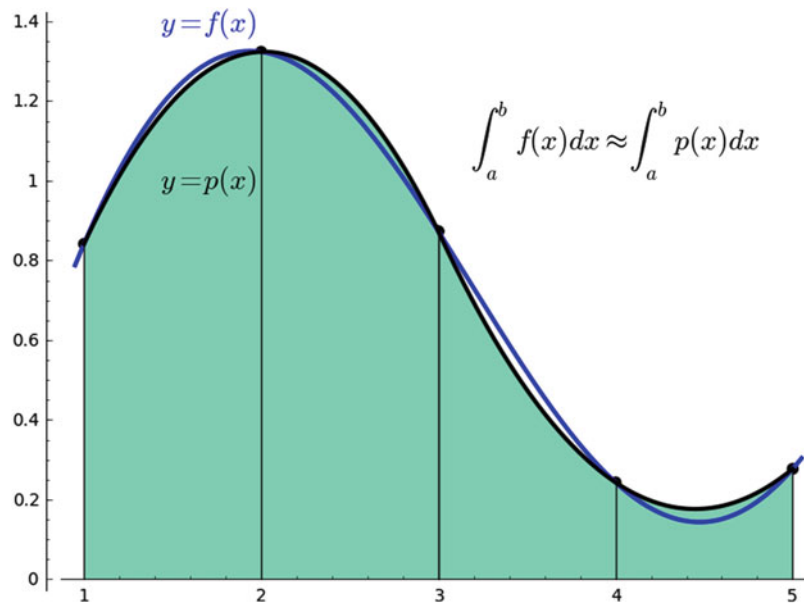
$$h = \frac{b-a}{n},$$

and n is even, then we obtain the following simplified formula

$$\begin{aligned}\int_a^b f(x)dx \simeq \frac{h}{3}&\left[f(a) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4)\right. \\ &\left. + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(b)\right],\end{aligned} \tag{5.14}$$

called the **Composite Simpson's Rule** with evenly spaced nodes.

Let's see an example.



Example 110. Approximate the integral $\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx$ using the Composite Simpson's Rule, with five evenly spaced nodes.

Solution. We have that $n = 4$. The above given graph is related to this problem, as it represents the function given here. Using formula (5.14), for $f(x) = \sin x - 1 + \sqrt{x}$, we obtain

$$\int_1^5 \left(\sin x - 1 + \sqrt{x}\right) dx \simeq \left(\frac{5-1}{12}\right) [f(1) + 4f(2) + 2f(3) + 4f(4) + f(5)]$$
$$\simeq 3.04393009815514,$$

which is a much better estimate than the one obtained earlier. The error of approximation is, in this case, about 0.000396719417593783, which is a significant improvement.

For completeness, we give below the Sage code used to create the previous graph

```
#define the function to be used
f(x) = sin(x)-1+sqrt(x)
a = 1
b = 5

#draw the curve f
P = plot(f, (x, a-0.05, b+0.05), thickness=3)

#draw the texts above the graph
P += text("$\\int_{a}^b f(x) dx \\approx  \\int_{a}^b p(x) dx$ ",
    ( 4.0 , 1.1), fontsize=20, color='black')
P += text("$y=f(x)$",(1.7,f(1.7)+0.1), fontsize=20, color='blue')
P += text("$y=p(x)$",(1.7,1.0), fontsize=20, color='black')
```

```
def BasicSimpson(f, aa,bb,x):
    #interpolation
    p(x)=  (((x-((aa+bb)/2))/(aa-((aa+bb)/2)))) * \
                    (((x-bb)/(aa-bb)))* f(aa) +(((x-aa)/
                    (((aa+bb)/2) \ -aa)))*(((x-bb)/
                    (((aa+bb)/2)-bb)))*f(((aa+bb)/2)) \
                    +(((x-aa)/(bb-aa))) \ *(((x-((aa+bb)/2)))/
                    (bb-((aa+bb)/2))))*f(bb)
    global P

    #graph the horizontal lines
    P += line([(aa,0),(aa,f(aa))], color='black')
    P += line([(bb,0),(bb,f(bb))], color='black')
    P += line([( (aa+bb)/2,0),((aa+bb)/2,f((aa+bb)/2))],
              color='black')

    #plot the interpolating function p(x)
    P += plot(p, (x, aa, bb), thickness=3, color = 'black')

    #make the shade - as a polygon with many points
    #use only one line for the following 4 lines of code!
    P += polygon( [(aa,0),(aa,p(aa))] +
                  [(x, p(x)) for x in [aa,aa+0.005,..,bb]] +
                  [(bb,0),(aa,0)], rgbcolor=(0.5,0.8,0.7),
                  aspect_ratio='automatic')

    P += list_plot([ (aa,f(aa)), (bb, f(bb)),
                ((aa+bb)/2, f((aa+bb)/2))],
                 color = 'black', size = 70)

#call the basic simpson on each subinterval
BasicSimpson(f, 1, 3, x)
BasicSimpson(f, 3, 5, x)

#show the plot
show(P)
```

5.2.3 *Sage Code*

Below we give the Sage implementation for the Composite Simpson's Rule we derived (5.14), on page 190.

```
def CompositeSimpson(f, a, b, n):
    #if n is odd return error
    if int(n)%2 == 1:
        return  "ERROR - n should be even!!!"

    #compute the size of each subinterval
    h = (b-a)/n
```

```
#here we add all the terms
sum = f(a)

#add the values for all intermediate nodes
for i in [1,2,..,n-1] :
    #odd i's are terms multiplied by 4
    if( i%2 == 1 ):
        sum += 4* f(a+i*h)
    else:
        sum += 2* f(a+i*h)

#add the last node
sum +=f(b)

#multiply by h to complete the computation
sum = sum*h/3

#return the sum
return sum.n()
```

This function returns an error message if n is odd. For even values of n it returns the Composite Simpson's Rule estimation for the $\int_a^b f(x)dx$.

Next, we use this code to solve the same problems that we solved in the previous section, where we used Trapezoidal Rule.

Example 111. Apply the previous Sage code to find the value of

$$\int_1^e \frac{\sin x}{x} dx,$$

using $n = 5$ subintervals.

Solution. Since n is odd, we obtain an error message. Hence the code

```
#define the function we will be using
f(x) = sin(x)/x

#call the function on [0,1], with 5+1 nodes
print CompositeSimpson(f, 1, e, 5)
```

produces the following error message

```
ERROR - n should be even!!!
```

Example 112. Apply the previous Sage code to find the value of

$$\int_1^e \frac{\sin x}{x} dx,$$

using $n = 4$ subintervals.

Solution. Adding the following code

```
#define the function we will be using
f(x) = sin(x)/x

#call the function on [0,1], with 5+1 nodes
print CompositeSimpson(f, 1, e, 4)
```

we obtain

```
0.874958585267728
```

Using the Sage Library function, `integral(sin(x)/x,x,1,e).n()` we obtained a very close value:

```
0.874957198780384
```

Example 113. Create a table of values obtained using the Composite Simpson's Rule, with $n = 2, 4, 8, 16$ and 32, for

$$\int_0^1 \frac{1}{1+x^2} dx,$$

and print out the error of approximation in each case.

Solution. For this, we could use the following Sage code:

```
#define the function we will be using
f(x) = 1/(1+x^2)

#print the top part of the table
print "simpson's rule" , '\t\t', 'error'
print '----------------------------------------'

#call the function several times
for power in [1,2,3,4, 5]:
    n = 2^power
    simpson = CompositeSimpson(f, 0, 1, n).n()
    error = (simpson - integral(f(x) ,x,0,1)).n()
    print simpson , '\t', error
```

to obtain:

```
simpson's rule        error
----------------------------------------
0.783333333333333     -0.00206483006411495
0.785392156862745     -6.00653470317347e-6
0.785398125614677     -3.77827715780654e-8
0.785398162806206     -5.91242721448282e-10
0.785398163388209     -9.23916498862809e-12
```

Please compare these results with the ones obtained using Trapezoidal Rule, on page 183.

Example 114. Use the Composite Simpson's Rule, with $n = 32$, to find an approximate value for

$$\int_0^1 e^{-x^2} dx.$$

Solution. For this, we could use the following Sage code:

```
#define the function we will be using
f(x) = e^(-x^2)

#set the value of n
n = 32

#call the function on [0,1], with n+1 nodes
CompositeSimpson(f, 0, 1, n)
```

and obtain:

```
0.746824140606985
```

Using the Sage Library function, `integral(f(x),x,0,1).n()` we obtain:

```
0.746824132812427
```

Example 115. Use the Composite Simpson's Rule, with $n = 64$, to find an approximate value for

$$\int_1^e e^{x^2} dx.$$

Solution. Using the following Sage code:

```
#define the function we will be using
f(x) = e^(x^2)

#set the value of n
n = 64

#call the function on [1,e], with n+1 nodes
CompositeSimpson(f, 1, e, n)
```

we obtain

```
323.296088136270
```

Using the Sage Library function, `integral(f(x),x,1,e).n()` we obtain:

```
323.295188678248
```

5.2.4 Error Bounds

In here, let us use the following notation

$$I = \int_a^b f(x)dx$$

and

$$S = \frac{h}{3}\left[f(a) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(b)\right], \tag{5.15}$$

where

$$x_i = a + i * h, i = 0, 1, \ldots, n, \tag{5.16}$$

with

$$h = \frac{b-a}{n}, \tag{5.17}$$

and n is even.

We give next, without a proof, the following error estimation. The reader interested in the proof of this result can refer to [5, p. 231].

Theorem 116. *Let f be a function defined on $[a, b]$ that is four times differentiable, with $f^{(4)}$ continuous. Then the error of approximation is given by*

$$I - T = -\frac{(b-a)h^4}{180} f^{(4)}(\xi), \tag{5.18}$$

for some ξ in (a, b).

In particular we have the following estimation for the error of approximation

Corollary 117. *Let f be a function defined on $[a, b]$ that is four differentiable, with $f^{(4)}$ continuous, such that $\left|f^{(4)}(x)\right| \leq M$, for all x in $[a, b]$. Then*

$$|I - T| \leq \frac{(b-a)Mh^4}{180}. \tag{5.19}$$

This last result shows that the Composite Simpson's Rule with evenly spaced nodes converges quartically.

Another direct consequence of the previous theorem is

Corollary 118. *Let f be a polynomial function of degree at most three, defined on $[a, b]$. Then the Simpson's Rule gives us exact answer, that is*

$$I = T. \tag{5.20}$$

Compare the results of the following examples with the corresponding ones obtained in the previous section.

Example 119. How many evenly spaced nodes do we need to use in Composite Simpson's Rule, in order to find an approximate value for

$$\int_1^e e^{x^2} dx,$$

with an error less than 0.001?

Solution. We have $f(x) = e^{x^2}$, defined on $[1, e]$. Hence

$$f'(x) = 2xe^{x^2},$$

$$f''(x) = (2 + 4x^2)e^{x^2},$$

$$f'''(x) = (12x + 8x^3)e^{x^2},$$

$$f^{(4)}(x) = (12 + 48x^2 + 16x^4)e^{x^2},$$

therefore

$$\begin{aligned} \left|f^{(4)}(x)\right| &\leq (12 + 48e^2 + 16e^4)e^{e^2} \\ &< 2,006,938. \end{aligned}$$

Using formula (5.19), we have

$$\begin{aligned} |I - T| &\leq \frac{(b-a)Mh^4}{180} \\ &< \frac{(2006938)\,(e-1)^5}{180n^4} \end{aligned}$$

Therefore, from

$$\frac{(2006938)\,(e-1)^5}{180n^4} < 0.001$$

we obtain

$$n \geq 114.$$

Example 120. How many evenly spaced nodes do we need to use in Composite Simpson's Rule, in order to find an approximate value for

$$\int_0^1 e^{x^2} dx,$$

with an error less than 0.0001? Then estimate the value of $\int_0^1 e^{x^2} dx$.

Solution. We have $f(x) = e^{x^2}$, defined on $[0, 1]$. Hence

$$f'(x) = 2xe^{x^2},$$

$$f''(x) = (2 + 4x^2)e^{x^2},$$

$$f'''(x) = (12x + 8x^3)e^{x^2},$$

$$f^{(4)}(x) = (12 + 48x^2 + 16x^4)e^{x^2},$$

therefore

$$\left|f^{(4)}(x)\right| \leq 76e.$$

Using formula (5.19), we have

$$\begin{aligned} |I - T| &\leq \frac{(b-a)Mh^4}{180} \\ &\leq \frac{19e}{45n^4}. \end{aligned}$$

Therefore, from

$$\frac{19e}{45n^4} < 0.0001$$

we obtain

$$n \geq 12,$$

which means we would need at least 12+1=13 evenly spaced nodes to make sure the error of approximation is less than 0.0001. Using the Sage code we developed earlier, we obtain the following estimation

$$\int_0^1 e^{x^2} dx \simeq 1.46266612641848.$$

Compare this to `1.46265174590718`, the value obtained from `integrate(e^(x^2), x, 0, 1).n()`.

5.2.5 *Sage Library*

If we want to utilize Sage libraries, we can use Sage code similar to the one given in the next example.

Example 121. Find the value of

$$\int_1^e \frac{\sin x}{x} dx,$$

using $n = 5$ subintervals.

Solution. For this, one can use

```
#import the needed libraries
from scipy.integrate import simps
import scipy
import numpy
```

```
#define the function, the interval [a,b], and n
def f(x):
    return sin(x)/x
a=1.0
b=e
n=5
#define the n+1 evenly spaced nodes in [a,b]
x = numpy.array([a+i*(b-a)/n  for i in [0,1,..,n]])

#compute and output the Simpson's Rule
print  scipy.integrate.simps(f(x), x).n()
```

to obtain

```
0.874782324470474
```

Example 122. Find the value of

$$\int_0^1 \frac{1}{1+x^2}dx,$$

using $n = 32$ subintervals.

Solution. For this, we can use the following Sage code:

```
#import the needed libraries
from scipy.integrate import simps
import scipy
import numpy

#define the function, the interval [a,b], and n
def f(x):
    return 1/(1+x^2)
a=0
b=1
n=32

#define the n+1 evenly spaced nodes in [a,b]
x = numpy.array([a+i*(b-a)/n  for i in [0,1,..,n]])

#compute and output the Simpson's Rule
print  scipy.integrate.simps(f(x), x)
```

to obtain

```
0.785398163388
```

5.3 Using the Method of Undetermined Coefficients

Applying the Method of Undetermined Coefficients we used on page 167, we can find alternative ways to obtain new and existing integration rules. In particular, we will see alternative ways to derive the same formulae given above: the Trapezoidal Rule and the Simpson's Rule.

5.3.1 Basic Trapezoidal Rule Using the Method of Undetermined Coefficients

For example, if we want to find a rule that approximates $\int_a^b f(x)dx$ using only the nodes a and b, then we would need to find A, B such that

$$\int_a^b f(x)dx \simeq Af(a) + Bf(b). \tag{5.21}$$

If we want this rule to be exact for all linear functions, then (by the linearity of the integral), it is sufficient that the rule is exact for $f(x) = 1$, and $f(x) = x$. Therefore

$$\int_a^b 1dx = A \cdot 1 + B \cdot 1,$$

and

$$\int_a^b xdx = A \cdot a + B \cdot b.$$

This leads to

$$\begin{cases} A + B = b - a \\ Aa + Bb = \frac{b^2 - a^2}{2} \end{cases},$$

hence

$$\begin{cases} A = \frac{b-a}{2} \\ B = \frac{b-a}{2} \end{cases},$$

giving us the same formula (5.2), given on page 178.

5.3.2 Basic Simpson's Rule Using the Method of Undetermined Coefficients

Next, let's try to find a rule that approximates $\int_a^b f(x)dx$ using only the nodes a, $\frac{a+b}{2}$, and b. We are trying to find A, B, and C such that

$$\int_a^b f(x)dx \simeq Af(a) + Bf\left(\frac{a+b}{2}\right) + Cf(b). \tag{5.22}$$

Here, we will try to find a rule that is exact for all polynomials up to degree two. Hence, using again the linearity of integrals, we will solve for A, B, C, such that the approximation given above is exact for $f(x) = 1$, $f(x) = x$, and $f(x) = x^2$. Then

$$\int_a^b 1dx = A \cdot 1 + B \cdot 1 + C \cdot 1,$$

$$\int_a^b xdx = A \cdot a + B \cdot \frac{a+b}{2} + C \cdot b,$$

and

$$\int_a^b x^2 dx = A \cdot a^2 + B \cdot \left(\frac{a+b}{2}\right)^2 + C \cdot b^2.$$

This leads to

$$\begin{cases} A + B + C = b - a \\ aA + \frac{a+b}{2} B + bC = \frac{b^2 - a^2}{2} \\ a^2 A + \left(\frac{a+b}{2}\right)^2 B + b^2 C = \frac{b^3 - a^3}{3} \end{cases},$$

hence

$$\begin{cases} A = \frac{b-a}{6} \\ B = \frac{4(b-a)}{6} \\ C = \frac{b-a}{6} \end{cases},$$

giving us the same formula (5.13), given on page 188.

5.4 Richardson Extrapolation

We present here the idea of Richardson Extrapolation and provide a motivation for using it. Then, in the next section, we'll see how we can apply Richardson Extrapolation to the Trapezoidal Rule, in order to obtain a more accurate approximation formula, Romberg Method. Please note that the idea of Richardson Extrapolation can be easily applied to other approximation techniques too, and not only to numerical integration. In particular, it was already used on page 164, although this was not mentioned explicitly.

We have seen on page 184 that we have the following error of approximation in the case of Trapezoidal Rule

$$I - T = \mathcal{O}(h^2),$$

and

$$I - S = \mathcal{O}(h^4).$$

for the Simpson's Rule (see page 196). Recall that h is the size of each of the n subintervals of $[a, b]$. The same discussion can be done, similarly, based on n, instead of h.

In general, let's suppose we are using some approximation method, $M = M_h$, whose error is of the form

$$I - M_h = \mathcal{O}(h^p).$$

Let's be a little more precise and assume the error of approximation looks something like

$$I - M_h = a_p h^p + a_q h^q + a_r h^r + \dots \tag{5.23}$$

where $r > q > p$.

If we apply the same method for a smaller value of h (here, for simplicity, let's look at $\frac{h}{2}$), we obtain

$$\begin{aligned} I - M_{h/2} &= a_p \left(\frac{h}{2}\right)^p + a_q \left(\frac{h}{2}\right)^q + a_r \left(\frac{h}{2}\right)^r + \cdots \\ &= \frac{a_p}{2^p} h^p + \frac{a_q}{2^q} h^q + \frac{a_r}{2^r} h^r + \cdots . \end{aligned} \tag{5.24}$$

Therefore, we can linearly combine (5.23) and (5.24) to obtain a more accurate formula. We can use the following linear combination, to cancel the h^p term:

$$\begin{aligned} (I - M_h) - 2^p \left(I - M_{h/2}\right) &= \left(a_p h^p + a_q h^q + a_r h^r + \cdots\right) \\ &\quad -2^p \left(\frac{a_p}{2^p} h^p + \frac{a_q}{2^q} h^q + \frac{a_r}{2^r} h^r + \cdots\right) \\ &= \left(1 - \frac{1}{2^{q-p}}\right) a_q h^q + \left(1 - \frac{1}{2^{r-p}}\right) a_r h^r + \cdots , \end{aligned}$$

therefore

$$I - \left(\frac{2^p}{2^p - 1} M_{h/2} - \frac{1}{2^p - 1} M_h\right) = \overline{a}_q h^q + \overline{a}_r h^r + \cdots ,$$

where

$$\begin{aligned} \overline{a}_q &= \frac{1}{(1 - 2^p)} \left(1 - \frac{1}{2^{q-p}}\right) a_q , \\ \overline{a}_r &= \frac{1}{(1 - 2^p)} \left(1 - \frac{1}{2^{r-p}}\right) a_r , \\ &\cdots \end{aligned}$$

Hence, the following linear combination of two approximations

$$\overline{M} = \left(\frac{2^p}{2^p - 1} M_{h/2} - \frac{1}{2^p - 1} M_h\right), \tag{5.25}$$

gives us a better estimate,

$$I - \overline{M} = \overline{a}_q h^q + \overline{a}_r h^r + \cdots .$$

We can repeat this process on the values of $\overline{M}$, to obtain even better estimates

$$I - \overline{\overline{M}} = \overline{\overline{a}}_r h^r + \cdots,$$

and so on.

5.5 Romberg Method

5.5.1 Motivation

We can apply the Richardson Extrapolation technique to the Trapezoidal Rule, in order to improve the error of approximation. For Trapezoidal Rule we had:

$$I - T = -\frac{(b-a)h^2}{12} f''(\xi),$$

hence

$$I - T = a_2 h^2 + a_3 h^3 + a_4 h^4 + \cdots.$$

In this section, we'll assume that f has as many derivatives as needed. And a consequence of this is the following **Euler–MacLaurin Formula**

$$I - T = a_2 h^2 + a_4 h^4 + a_6 h^6 + \cdots.$$

Using (5.25), for $p = 2$, we obtain the values

$$\begin{aligned} \overline{M} &= \left(\frac{2^2}{2^2 - 1} M_{h/2} - \frac{1}{2^2 - 1} M_h \right) \\ &= \frac{4}{4-1} M_{h/2} - \frac{1}{4-1} M_h \end{aligned} \tag{5.26}$$

whose error of approximation is given by

$$I - \overline{M} = \overline{a}_4 h^4 + \overline{a}_6 h^6 + \cdots,$$

hence they represent improved estimations.

If we apply this step again on the values of $\overline{M}$'s, now with $p = 4$, from (5.25), we would the values

$$\overline{\overline{M}} = \left(\frac{2^4}{2^4 - 1} \overline{M}_{h/2} - \frac{1}{2^4 - 1} \overline{M}_h \right) \tag{5.27}$$

whose error of approximation is given by

$$I - \overline{\overline{M}} = \overline{a}_6 h^6 + \ldots,$$

and so on.

In general, for $p = k$, we would use

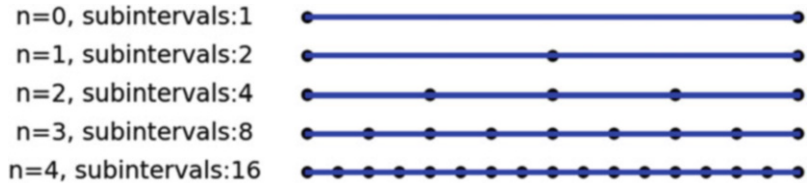
$$\overline{\overline{M}} = \left(\frac{2^{2k}}{2^{2k}-1} \overline{M}_{h/2} - \frac{1}{2^{2k}-1} \overline{M}_h \right) \tag{5.28}$$

whose error of approximation is given by

$$I - \overline{\overline{M}} = \overline{a}_{2k+2} h^{2k+2} + \ldots, \tag{5.29}$$

Hence we obtained the following algorithm (called the **Romberg Integration Method**) for estimating $\int_a^b f(x)dx$.

Step 1:
We start with using the Basic Trapezoidal Rule on the interval $[a, b]$, to obtain $R(0, 0)$.
Then, using the Composite Trapezoidal Rule on 2^1 equal subintervals, we obtain $R(1, 0)$.
Then, halving each subinterval, we use the Composite Trapezoidal Rule on 2^2 equal subintervals, to obtain $R(2, 0)$.
We keep on going, in the same fashion, several times, and obtain $R(n, 0)$.



This step can be further improved by using a recursive version of the Trapezoidal Rule. In order to simplify the presentation of this algorithm, we did not include this step here.

Step 2:
Using the Richardson Extrapolation technique, and formula (5.26)

$$R(k, 1) = \frac{2^2}{2^2-1} R(k, 0) - \frac{2^2}{2^2-1} R(k-1, 0), \text{ for } k = 1, 2, \ldots, n.$$

Step 3:
Using the Richardson Extrapolation technique, and formula (5.26)

$$R(k, 2) = \frac{2^4}{2^4-1} R(k, 1) - \frac{1}{2^4-1} R(k-1, 1), \text{ for } k = 2, \ldots, n.$$

and so on

Step n:
Using the Richardson Extrapolation technique, and formula (5.26)

$$R(n, n) = \frac{2^{2n}}{2^{2n}-1} R(n, n-1) - \frac{1}{2^{2n}-1} R(n-1, n-1).$$

5.5.2 Algorithm

The Romberg Integration Method is described as obtaining a triangular array of numbers:

$$\begin{array}{cccccc}
R(0,0) & & & & & \\
R(1,0) & R(1,1) & & & & \\
R(2,0) & R(2,1) & R(2,2) & & & \\
\ldots & \ldots & \ldots & \ldots & & \\
R(n-1,0) & R(n-1,1) & R(n-1,2) & \ldots & R(n-1,n-1) & \\
R(n,0) & R(n,1) & R(n,2) & \ldots & R(n,n-1) & R(n,n)
\end{array}$$

(each value being a better and better estimation of $\int_a^b f(x)dx$).

These numbers are obtained as follows

$$R(i,0)$$

are given by the applying the Trapezoidal Rule on 2^i equal subintervals of $[a,b]$, $i = 0,1,\ldots,n$, and

$$R(i,j) = \frac{2^{2i}}{2^{2i}-1}R(i,j-1) - \frac{1}{2^{2i}-1}R(i-1,j-1), \tag{5.30}$$

for $i = 1,2,\ldots,n$, and $j = 1,2,\ldots,i$.

At the end of this, $R(n,n)$ will represent our best estimate:

$$\int_a^b f(x)dx \simeq R(n,n).$$

5.5.3 Sage Code

We present below our Sage implementation of the previous algorithm.

```
def CompositeTrapezoidal(f, a, b, n):
    #include this with your code!

def RombergAlgorithm(f, a,b,n):
    #create an 'empty' n+1 by n+1 array
    R = [[0 for i in range(n+1)] for i in range(n+1)]

    #compute the first column, the R(i,0)
    for row in [0,1,..,n]:
        R[row][0] = CompositeTrapezoidal(f, a, b, 2^row)

    #compute the remaining columns
    for column in [1,2,..,n]:
        for row in [1,2,..,n]:
            if(row>=column):
                R[row][column] = ((2^(2*row))/(2^(2*row)-1))*
     R[row][column-1] -(1/(2^(2*row)-1))*R[row-1][column-1]
```

```
    #print the array
    for row in [0,1,..,n]:
        print [ (R[row][column]).n()  for column in  [0,1,..,row] ]
```

Let's use this code to solve some problems.

Example 123. Apply the previous Sage code to estimate

$$\int_1^e \frac{\sin x}{x} dx,$$

using Romberg Method with $n = 5$.

Solution. For this, to the previous Sage code, we simply add

```
#define the function we will be using
f(x) = sin(x)/x

#define the parameters and call the function
n = 5
a = 1
b = e
RombergAlgorithm(f, a, b, n)
```

we obtain

[0.852773800639980]
[0.869425981076183, 0.874976707888250]
[0.873575434219842, 0.873852064429419, 0.873777088198830]
[0.874611824389402, 0.874628275027014, 0.874640595830151, 0.874654302300489]
[0.874870859381198, 0.874871875204695, 0.874872830499510, 0.874873741223703,
0.874874601768500]
[0.874935614193412, 0.874935677492349, 0.874935739860177, 0.874935801355153,
0.874935862019993, 0.874935921902937]

If we change the default precision in the `print` statement of the `CompositeTrapezoidal()` method, and replace `.n()` by `.n(30)`, we obtain:

```
[0.85277380]
[0.86942598, 0.87497671]
[0.87357544, 0.87385207, 0.87377709]
[0.87461182, 0.87462828, 0.87464060, 0.87465430]
[0.87487086, 0.87487187, 0.87487283, 0.87487374, 0.87487460]
[0.87493561, 0.87493568, 0.87493574, 0.87493580, 0.87493586,
   0.87493592]
```

Hence

$$\int_1^e \frac{\sin x}{x} dx \simeq 0.874935921902937.$$

Using the Sage Library function, `integral(sin(x)/x,x,1,e).n()` we obtain:

```
0.874957198780384
```

Example 124. Apply Romberg Method with $n = 3$ to estimate

$$\int_0^1 \frac{1}{1+x^2} dx.$$

Solution. For this, we use the following Sage code:

```
#define the function
f(x) = 1/(1+x^2)
#define the parameters and call the function
n = 3
a = 0
b = 1
RombergAlgorithm(f, a, b, n)
```

to obtain

```
[0.750000000000000]
[0.775000000000000, 0.783333333333333]
[0.782794117647059, 0.783313725490196, 0.783312418300654]
[0.784747123622772, 0.784778123717625, 0.784801368133933,
   0.784825002258271]
```

Therefore we conclude that

$$\int_0^1 \frac{1}{1+x^2} dx \simeq 0.784825002258271.$$

Using the Sage Library function, `integral(1/(1+x^2),x,0,1).n()` we obtain:

```
0.785398163397448
```

Example 125. Modify the Sage code for the Romberg Method, so it only prints the last value from table. Then use it to estimate

$$\int_0^1 e^{-x^2} dx,$$

with $n = 7$.

Solution. For this, we only need to modify the last two statements of the `RombergAlgorithm()` into :

```
#print only the last value of the array
print ' Romberg Method Estimate = ' , (R[n][n]).n()
```

and add the following

```
#define the function
f(x) = e^(-x^2)
#define the parameters and call the function
RombergAlgorithm(f, 0, 1, 7)
```

to obtain

```
 Romberg Method Estimate =  0.746820395325466
```

Using the Sage Library function, `integral(f(x),x,0,1).n()` we obtain:

```
0.746824132812427
```

Example 126. Use the previous Sage code, with $n = 8$, to estimate

$$\int_1^e e^{x^2} dx.$$

Solution. For this, we use the following Sage code:

```
#define the function we will be using
f(x) = e^(x^2)
#call the function
RombergAlgorithm(f, 1, e, 8)
```

to obtain

```
Romberg Method Estimate =  323.328183013859
```

Using the Sage Library function, `integral(f(x),x,1,e).n()` we obtain

```
323.295188678248
```

5.5.4 Error Bounds

From (5.28) and (5.29), given on page 204 we can see that

$$I - R(n,n) = \overline{a}_{2n+2} h^{2n+2} + \cdots = \mathcal{O}\left(h^{2n+2}\right). \tag{5.31}$$

Hence, with some extra work on the values obtained using Simpson's Rule, we can obtain a much more accurate approximation.

5.6 Newton–Cotes Formulae

Most of the numerical integration formulae have the following form

$$\int_a^b f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1) + \cdots + w_n f(x_n). \tag{5.32}$$

One way to obtain the weights $w_0, w_1, \ldots, w_n$ is by approximating $f(x)$ with a polynomial $p(x)$ that interpolates f at the nodes $x_0, x_1, \ldots, x_n$. Hence, using (3.7) and (3.5) we would obtain

$$\begin{aligned}
\int_a^b f(x)dx &\simeq \int_a^b p(x) \\
&= \int_a^b \left[f(x_0)L_0(x) + f(x_1)L_1(x) + \cdots + f(x_n)L_n(x)\right] dx \\
&= \left[\int_a^b L_0(x)dx\right] f(x_0) + \cdots + \left[\int_a^b L_n(x)dx\right] f(x_n)
\end{aligned}$$

giving us

$$w_0 = \int_a^b L_0(x)dx, w_1 = \int_a^b L_1(x)dx, \ldots, w_n = \int_a^b L_n(x)dx.$$

If the nodes $x_0, x_1, \ldots, x_n$ are **evenly spaced** (that is, $x_i = x_0 + i * h$, with $h = \frac{(x_n - x_0)}{n}$, for $i = 0, 1, \ldots, n$), then the formula (5.32) is called a **Newton–Cotes formula.** There are two types of Newton–Cotes formulae: **closed** (if the nodes include $x_0 = a$, and $x_n = b$), and **open** (if they do not include $x_0 = a$, and $x_n = b$). Without a proof, we give, below, some of these formulae. Interested reader may also refer to [1, See Section 25.4, p. 885], for a more comprehensive list. For some ξ between a and b we have:

5.6.1 Closed Newton–Cotes Formulae

Trapezoid Rule ($n = 1$):

$$\int_a^b f(x)dx = (b-a)\left[\frac{f(a)+f(b)}{2}\right] - \frac{(b-a)^3}{12} f''(\xi).$$

Simpson's Rule ($n = 2$):

$$\int_a^b f(x)dx = \frac{(b-a)}{6}[f(a) + 4f(x_1) + f(b)] - \frac{(b-a)^5}{2880} f^{(4)}(\xi).$$

Simpson's 3/8 rule ($n = 3$):

$$\int_a^b f(x)dx = \frac{(b-a)}{8}[f(a) + 3f(x_1) + 3f(x_2) + f(b)] - \frac{(b-a)^5}{6480} f^{(4)}(\xi).$$

Boole's rule ($n = 4$):

$$\int_a^b f(x)dx = \frac{(b-a)}{90}[7f(a) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(b)] - \frac{(b-a)^7}{1935360} f^{(6)}(\xi).$$

5.6.2 Open Newton–Cotes Formulae

Midpoint Rule ($n = 2$):

$$\int_a^b f(x)dx = (b-a) f(x_1) + \frac{(b-a)^3}{24} f''(\xi).$$

Two-Point Newton–Cotes Open Rule ($n = 3$):

$$\int_a^b f(x)dx = \frac{(b-a)}{2}[f(x_1) + f(x_2)] + \frac{(b-a)^3}{108}f''(\xi).$$

Three-Point Newton–Cotes Open Rule ($n = 4$):

$$\int_a^b f(x)dx = \frac{(b-a)}{3}[2f(x_1) - f(x_2) + 2f(x_3)] + \frac{7(b-a)^5}{23040}f^{(4)}(\xi).$$

Four-Point Newton–Cotes Open Rule ($n = 5$):

$$\int_a^b f(x)dx = \frac{(b-a)}{24}[11f(x_1) + f(x_2) + f(x_3) + 11f(x_4)] + \frac{19(b-a)^5}{90000}f^{(4)}(\xi).$$

5.7 Gauss Quadrature Formula

5.7.1 A Brief Outline

In the previous formulae, we fixed the nodes $x_0, x_1, \ldots, x_n$ to be evenly spaced in the interval $[a, b]$, and searched for appropriate coefficients $w_0, w_1, \ldots, w_n$ to obtain a quadrature formula of the form

$$\int_a^b f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1) + \cdots + w_n f(x_n). \tag{5.33}$$

To obtain the Gauss Quadrature Formula, we will not hold these nodes, evenly distributed. This flexibility will allow us to obtain more accurate quadrature formulae that are exact for polynomials of higher degrees. We will employ the *Method of Undetermined Coefficients* technique in order to obtain these results.

As we have seen in one of the previous sections, the Trapezoidal Rule has the form

$$\int_a^b f(x)dx \simeq w_0 f(a) + w_1 f(b), \tag{5.34}$$

and one way to find the weights w_0 and w_1 (See the Subsection "*Using the Method of Undetermined Coefficients*") was to impose that the formula (5.34) is exact for $f(x) = 1$ and x. The nodes $x_0 = a$ and $x_1 = b$ were fixed.

By not fixing the values of the nodes to $x_0 = a$ and $x_1 = b$, one can obtain a higher degree of accuracy. For example, we can find nodes x_0, x_1 and weights w_0, w_1 such that

$$\int_a^b f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1), \tag{5.35}$$

is exact for $f(x) = 1, x, x^2$, and x^3. Then, by linearity of the integrals, the formula will be exact for all polynomials of degree at most 3. See the example given below.

Similarly, the Simpson's Rule has the form

$$\int_a^b f(x)dx \simeq w_0 f(a) + w_1 f(\frac{a+b}{2}) + w_2 f(b), \tag{5.36}$$

and the weights w_0, w_1, w_2 were chosen such that the formula (5.36) is exact for $f(x) = 1$, $f(x) = x$, and x^2. Here, the nodes $x_0 = a, x_1 = \frac{a+b}{2}$ and $x_2 = b$ were fixed.

By not fixing the values of the nodes to $x_0 = a, x_1 = \frac{a+b}{2}$, and $x_2 = b$, we can expect to obtain a higher degree of accuracy. For example, we can find the nodes x_0, x_1, x_2 and weights w_0, w_1, w_2 such that

$$\int_a^b f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1) + w_2 f(x_2), \tag{5.37}$$

is exact for $f(x) = 1, x, x^2, x^3, x^4$, and x^5. Then, by linearity of the integrals, the formula will be exact for all polynomials of degree at most 5. See the example given below.

To simplify our discussion, we will first only consider integrals of the form $\int_{-1}^1 f(x)dx$. Then, using the change of variables

$$t = \frac{a+b}{2} + \frac{b-a}{2}x \tag{5.38}$$

we obtain a formula applicable to any interval $[a, b]$

$$\int_a^b f(t)dt = \frac{b-a}{2}\int_{-1}^1 f\left(\frac{a+b}{2} + \frac{b-a}{2}x\right)dx$$

and hence we will replace a formula obtained on $[-1, 1]$

$$\int_{-1}^1 f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1) + \cdots + w_n f(x_n),$$

with one applicable on $[a, b]$

$$\int_a^b f(x)dx \simeq \frac{b-a}{2}\left[w_0 f\left(\frac{a+b}{2} + \frac{b-a}{2}x_0\right) + \cdots + w_n f\left(\frac{a+b}{2} + \frac{b-a}{2}x_n\right)\right]. \tag{5.39}$$

5.7.2 *Algorithm by Example*

Example 127. Determine x_0, x_1, w_0, w_1, such that

$$\int_{-1}^1 f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1) \tag{5.40}$$

is exact for $f(x) = 1$, x, x^2, and x^3. Then obtain the corresponding quadrature formula for $\int_a^b f(x)dx$.

Solution. Knowing that (5.40) is exact for $f(x) = 1, x, x^2$, and x^3 we obtain the following equations

$$\int_{-1}^{1} 1dx = w_0 \cdot 1 + w_1 \cdot 1$$
$$\int_{-1}^{1} xdx = w_0 \cdot x_0 + w_1 \cdot x_1$$
$$\int_{-1}^{1} x^2dx = w_0 \cdot x_0^2 + w_1 \cdot x_1^2$$
$$\int_{-1}^{1} x^3dx = w_0 \cdot x_0^3 + w_1 \cdot x_1^3$$

hence solving

$$\begin{cases} w_0 + w_1 = 2 \\ w_0x_0 + w_1x_1 = 0 \\ w_0x_0^2 + w_1x_1^2 = \frac{2}{3} \\ w_0x_0^3 + w_1x_1^3 = 0 \end{cases}.$$

we obtain

$$w_0 = w_1 = 1$$
$$x_0 = -\frac{1}{\sqrt{3}}, x_1 = \frac{1}{\sqrt{3}}$$

giving us the following Gaussian Quadrature Formula

$$\int_{-1}^{1} f(x)dx \simeq f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right). \tag{5.41}$$

Using the change of variable given in (5.39), we obtain the following Gaussian Quadrature Formula that works for any interval $[a, b]$:

$$\int_a^b f(x)dx \simeq \frac{b-a}{2}\left[f\left(\frac{a+b}{2} - \frac{b-a}{2}\cdot\frac{1}{\sqrt{3}}\right) + f\left(\frac{a+b}{2} + \frac{b-a}{2}\cdot\frac{1}{\sqrt{3}}\right)\right].$$

Example 128. Determine $x_0, x_1, x_2, w_0, w_1, w_2$, such that

$$\int_{-1}^{1} f(x)dx \simeq w_0f(x_0) + w_1f(x_1) + w_2f(x_2) \tag{5.42}$$

is exact for $f(x) = 1$, x, x^2, x^3, x^4, and x^5. Then obtain the corresponding quadrature formula for $\int_a^b f(x)dx$.

Solution. Knowing that (5.40) is exact for $f(x) = 1, x, x^2, x^3, x^4$, and x^5 we obtain the following equations

$$\begin{aligned}
\int_{-1}^{1} 1dx &= w_0 \cdot 1 + w_1 \cdot 1 + w_2 \cdot 1 \\
\int_{-1}^{1} xdx &= w_0 \cdot x_0 + w_1 \cdot x_1 + w_2 \cdot x_2 \\
\int_{-1}^{1} x^2dx &= w_0 \cdot x_0^2 + w_1 \cdot x_1^2 + w_2 \cdot x_2^2 \\
\int_{-1}^{1} x^3dx &= w_0 \cdot x_0^3 + w_1 \cdot x_1^3 + w_2 \cdot x_2^3 \\
\int_{-1}^{1} x^4dx &= w_0 \cdot x_0^4 + w_1 \cdot x_1^4 + w_2 \cdot x_2^4 \\
\int_{-1}^{1} x^5dx &= w_0 \cdot x_0^5 + w_1 \cdot x_1^5 + w_2 \cdot x_2^5
\end{aligned}$$

hence solving

$$\begin{cases}
w_0 + w_1 + w_2 = 2 \\
w_0x_0 + w_1x_1 + w_2x_2 = 0 \\
w_0x_0^2 + w_1x_1^2 + w_2x_2^2 = \frac{2}{3} \\
w_0x_0^3 + w_1x_1^3 + w_2x_2^3 = 0 \\
w_0x_0^4 + w_1x_1^4 + w_2x_2^4 = \frac{2}{5} \\
w_0x_0^5 + w_1x_1^5 + w_2x_2^5 = 0
\end{cases}.$$

we obtain

$$\begin{aligned}
w_0 &= \frac{5}{9} = w_2, w_1 = \frac{8}{9} \\
x_0 &= -\sqrt{\frac{3}{5}} = -x_2, x_1 = 0
\end{aligned}$$

giving us the following Gaussian Quadrature Formula

$$\int_{-1}^{1} f(x)dx \simeq \frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\right). \tag{5.43}$$

Using the change of variable given in (5.39), we obtain the following Gaussian Quadrature Formula that works for any interval $[a, b]$:

$$\begin{aligned}
\int_a^b f(x)dx \simeq \frac{b-a}{2} \Bigg[&\frac{5}{9} f\left(\frac{a+b}{2} - \frac{b-a}{2} \cdot \sqrt{\frac{3}{5}}\right) \\
&+ \frac{8}{9} f\left(\frac{a+b}{2}\right) + \frac{5}{9} f\left(\frac{a+b}{2} - \frac{b-a}{2} \cdot \sqrt{\frac{3}{5}}\right) \Bigg].
\end{aligned} \tag{5.44}$$

5.7.3 The General Algorithm

The previous two examples suggest us the following technique: for a fixed n, using the *Method of Undetermined Coefficients*, we are looking to find the nodes $x_0, x_1, \ldots, x_n$ and the weights $w_0, w_1, \cdots, w_n$ such that

$$\int_{-1}^{1} f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1) + \cdots + w_n f(x_n)$$

is exact for each of the monomials $1, x, x^2, \ldots, x^{2n+1}$. Hence, the formula will be exact for any polynomial of degree up to $2n + 1$.

Hence, we need to solve for $x_0, x_1, \ldots, x_n$ and $w_0, w_1, \ldots, w_n$ from

$$\begin{cases} w_0 + w_1 + \cdots + w_n = 2 \\ w_0 x_0 + w_1 x_1 + \cdots + w_n x_n = 0 \\ w_0 x_0^2 + w_1 x_1^2 + \cdots + w_n x_n^2 = \frac{2}{3} \\ w_0 x_0^3 + w_1 x_1^3 + \cdots + w_n x_n^3 = 0 \\ \cdots \\ w_0 x_0^{2n} + w_1 x_1^{2n} + \cdots + w_n x_n^{2n} = \frac{2}{2n+1} \\ w_0 x_0^{2n+1} + w_1 x_1^{2n+1} + \cdots + w_n x_n^{2n+1} = 0 \end{cases} \tag{5.45}$$

Once we solve this system, by a change of variables, we obtain the corresponding **Gauss Quadrature Formula** for the interval $[a, b]$:

$$\int_a^b f(x)dx \simeq \frac{b-a}{2}\left[w_0 f\left(\frac{a+b}{2} + \frac{b-a}{2}x_0\right) + \cdots + w_n f\left(\frac{a+b}{2} + \frac{b-a}{2}x_n\right)\right]. \tag{5.46}$$

As you can see from the previous two examples, the system (5.45) will get increasingly complicated, as n increases. For this reason, in practice, alternative ways are being used. For example, it can be shown that $x_0, x_1, \ldots, x_n$ are the solutions of the **Legendre polynomials** $P_{n+1}(x)$, where

$$\begin{aligned} P_0(x) &= 1, P_1(x) = x, \text{ and} \\ P_{n+1}(x) &= \frac{2n+1}{n+1}xP_n(x) - \frac{n}{n+1}P_{n-1}(x), \text{ for } n = 1, 2, \ldots. \end{aligned} \tag{5.47}$$

From (5.47) and (5.45) we can obtain the following list of nodes and weights

n	Nodes x_i	Weights w_i
1	$-\frac{1}{\sqrt{3}}$	1
	$\frac{1}{\sqrt{3}}$	1
2	$-\sqrt{\frac{3}{5}}$	$\frac{5}{9}$
	0	$\frac{8}{9}$
	$\sqrt{\frac{3}{5}}$	$\frac{5}{9}$
3	$-\sqrt{\frac{3}{7}+\frac{2}{7}\sqrt{\frac{6}{5}}}$	$\frac{1}{2}-\frac{\sqrt{30}}{36}$
	$-\sqrt{\frac{3}{7}-\frac{2}{7}\sqrt{\frac{6}{5}}}$	$\frac{1}{2}+\frac{\sqrt{30}}{36}$
	$\sqrt{\frac{3}{7}-\frac{2}{7}\sqrt{\frac{6}{5}}}$	$\frac{1}{2}+\frac{\sqrt{30}}{36}$
	$\sqrt{\frac{3}{7}+\frac{2}{7}\sqrt{\frac{6}{5}}}$	$\frac{1}{2}-\frac{\sqrt{30}}{36}$
4	$-\frac{1}{3}\sqrt{5+2\sqrt{\frac{10}{7}}}$	$\frac{161}{450}-\frac{13\sqrt{70}}{900}$
	$-\frac{1}{3}\sqrt{5-2\sqrt{\frac{10}{7}}}$	$\frac{161}{450}+\frac{13\sqrt{70}}{900}$
	0	$\frac{128}{225}$
	$\frac{1}{3}\sqrt{5-2\sqrt{\frac{10}{7}}}$	$\frac{161}{450}+\frac{13\sqrt{70}}{900}$
	$\frac{1}{3}\sqrt{5+2\sqrt{\frac{10}{7}}}$	$\frac{161}{450}-\frac{13\sqrt{70}}{900}$

(5.48)

As one can easily notice, although Gauss Quadrature Formulae are exact for higher degree polynomials, and therefore we expect more accurate answers in less steps, these steps themselves are more complex and require more work.

Example 129. Use Gaussian Quadrature to estimate $\int_1^5(\sin x - 1 + \sqrt{x})dx$.

Solution. For $n = 1$, we can use the following Gaussian Quadrature Formula:

$$\int_1^5 f(x)dx \simeq 2\left[f\left(3-\frac{2}{\sqrt{3}}\right)+f\left(3+\frac{2}{\sqrt{3}}\right)\right].$$

Hence, using for example the following Sage code

```
f(x) = sin(x)-1+sqrt(x)
( 2*( f(3-(2/sqrt(3))) + f(3+(2/sqrt(3))))).n()
```

we obtain

$$\int_1^5(\sin x - 1 + \sqrt{x})dx \simeq 3.02161108118542,$$

which is a pretty good answer. Compare this with 4.47445875057819, obtained using Trapezoid Method, and the true value of 3.04353337873755.

For $n = 2$, we can use

$$\int_1^5 f(x)dx \simeq 2\left[\frac{5}{9}f\left(3-2\cdot\sqrt{\frac{3}{5}}\right)+\frac{8}{9}f(3)+\frac{5}{9}f\left(3-2\cdot\sqrt{\frac{3}{5}}\right)\right].$$

Hence, using for example the following Sage code

```
f(x) = sin(x)-1+sqrt(x)
( 2*( f(3-(2/sqrt(3))) + f(3+(2/sqrt(3))))).n()
```

we obtain

$$\int_1^5 (\sin x - 1 + \sqrt{x})dx \simeq 3.04505358424349.$$

5.7.4 Sage Code

We give, below, Sage implementation for the Gauss Quadrature, based on (5.47) and (5.48).

For $n = 1$:

```
def GaussQuad1(f, a, b):
    w0 = 1
    w1 = 1
    x0 = -1/sqrt(3)
    x1 = 1/sqrt(3)
    return (b-a)/2*( w0* f( (a+b)/2+(b-a)/2*x0)  +
                     w1* f( (a+b)/2+(b-a)/2*x1) )
```

For $n = 2$:

```
def GaussQuad2(f, a, b):
    w0 = 5/9
    w1 = 8/9
    w2 = 5/9
    x0 = -sqrt(3/5)
    x1 = 0
    x2 = sqrt(3/5)
    return (b-a)/2*( w0* f( (a+b)/2+(b-a)/2*x0)  +
                     w1*f( (a+b)/2+(b-a)/2*x1)+
                     w2*f( (a+b)/2+(b-a)/2*x2) )
```

Similar code can be written for higher values of n.

Example 130. Estimate the value of $\int_0^1 \frac{1}{1+x^2}dx$.

Solution. Using the previous Sage code, and

```
f(x) =  1/(1+x^2)
#we print two Gauss Quadratures
GaussQuad1(f,0,1).n(), GaussQuad2(f,0,1).n()
```

we obtain

```
(0.786885245901639, 0.785267034990792)
```

Therefore we conclude that

$$\int_0^1 \frac{1}{1+x^2}dx \approx 0.786885245901639$$

and

$$\int_0^1 \frac{1}{1+x^2}dx \approx 0.785267034990792.$$

Example 131. Estimate the value of $\int_1^e e^{x^2}dx$.

Solution. Using the previous Sage code, and

```
f(x) =  e^(x^2)
#we print Gauss Quadratures and the exact answer
print GaussQuad1(f,0,1).n()
print GaussQuad2(f,0,1).n()
print integral(f(x),x,0,1).n()
```

we obtain

1.45416788923913
1.46240971147732
1.46265174590718

5.8 An Application to Materials Science and Engineering

Fick's second law of diffusion is given by the following (*second order differential equation*):

$$\frac{\partial C}{\partial t} = D\frac{\partial^2 C}{\partial x^2}.$$

Under certain (*boundary*) conditions, this can be solved and the solution has the form

$$\frac{C_x - C_0}{C_s - C_0} = 1 - \text{erf}\left(\frac{x}{2\sqrt{Dt}}\right),$$

with C_s surface concentration, which remains constant, C_0 the initial concentration, and

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}}\int_0^x e^{-t^2}dt$$

called the **error function**. See [4] for more details.

Assuming $x = 4 \cdot 10^{-3}$ cm, $C_0 = 0$, and that the surface concentration $C_s = 7 \cdot 10^{18}$ cm^{-3} is held constant, find the donor concentration C_x of germanium if diffusion is carried out for two hours at 790 °C. Use $D_{790°C} = 4.8 \cdot 10^{-11}$ cm^2/s.

Solution. We have

$$\frac{C_x - 0}{C_s - 0} = 1 - \text{erf}\left(\frac{4 \cdot 10^{-3}}{2\sqrt{(4.8 \cdot 10^{-11})\,(2 \cdot 3600)}}\right)$$

hence the donor concentration of germanium is

$$C_x = 7 \cdot 10^{18} \cdot \left(1 - \text{erf}\left(\frac{4 \cdot 10^{-3}}{2\sqrt{(4.8 \cdot 10^{-11})\,(2 \cdot 3600)}}\right)\right).$$

Using the following Sage code:

```
def CompositeSimpson(f, a, b, n):
    #GIVEN EARLIER IN THIS CHAPTER

#define the function f(x)
f(x) = e^-(x^2)

#use Composite Simpson Method to compute
#the error function using 100+1 nodes
def erf(xx):
    return  (2/sqrt(pi) * CompositeSimpson(f, 0, xx, 100))

#display the answer
print "answer = ", 7*10^18*(1-
        erf(4*10^-3/(2*sqrt(4.8*(10^(-11))*2*3600)))).n()
```

we obtain that

```
answer =  1.04992815120886e13
```

hence

$$C_x = 1.0499 \times 10^{13}/\text{cm}^3.$$

5.9 Exercises

1. Using the Composite Trapezoidal Rule, and the nodes $\left\{0, \frac{1}{6}, \frac{1}{3}, \frac{1}{2}, 1\right\}$, find an approximate value for

$$\int_0^1 e^{\cos \pi x} dx.$$

   Estimate the error.
2. Using the Composite Trapezoidal Rule, and the nodes $\left\{0, \frac{\pi}{6}, \frac{\pi}{3}, \frac{\pi}{2}, \pi\right\}$, find an approximate value for

$$\int_0^\pi \frac{\sin x}{x} dx.$$

   Estimate the error.

3. Using the Composite Trapezoidal Rule, with evenly spaced nodes, and $n = 3$, find an approximate value for

$$\int_0^1 e^{-x^2} dx.$$

Estimate the error.

4. Using the Composite Simpson's Rule, with evenly spaced nodes, and $n = 2$, find an approximate value for

$$\int_0^1 e^{-x^2} dx.$$

Estimate the error.

5. In the Romberg Method, what will the value of $R(6, 3)$ be, if $R(6, 2) = 10$ and $R(5, 2) = 8$?
6. In the Romberg Method, what will the value of $R(5, 5)$ be, if $R(5, 4) = -10$ and $R(4, 4) = 8$?
7. Use Romberg Method, to estimate $\int_0^1 e^{-x^2} dx$ by $R(1, 1)$.
8. Using the Romberg Method, with $n = 4$, to estimate

$$\int_0^1 e^{-x^2} dx.$$

9. Using the Composite Trapezoidal Rule, with evenly spaced nodes, and $n = 3$, find an approximate value for

$$\int_0^1 \frac{\sin x}{x} dx.$$

Estimate the error.

10. Using the Composite Simpson's Rule, with evenly spaced nodes, and $n = 2$, find an approximate value for

$$\int_0^1 \frac{\sin x}{x} dx.$$

Estimate the error.

11. Using the Romberg Method, with $n = 4$, to estimate

$$\int_0^1 \frac{\sin x}{x} dx.$$

12. Using the Composite Trapezoidal Rule, with evenly spaced nodes, and $n = 2$, find an approximate value for

$$\int_{-1}^1 e^{\cos \pi x} dx.$$

Estimate the error.

13. Using the Composite Trapezoidal Rule, with evenly spaced nodes, and $n = 4$, find an approximate value for

$$\int_{-1}^{1} e^{\cos \pi x} dx.$$

14. Using the Romberg Method, with $n = 4$, to estimate

$$\int_{-1}^{1} e^{\cos \pi x} dx.$$

15. Using the Composite Trapezoidal Rule, with evenly spaced nodes, and $n = 4$, find an approximate value for

$$\int_{0}^{1} \frac{1}{1+x^2} dx.$$

Estimate the error.

16. Using the Composite Trapezoidal Rule, with evenly spaced nodes, and $n = 4$, find an approximate value for

$$\int_{0}^{1} \arctan x dx.$$

Estimate the error.

17. Using the Composite Simpson's Rule, with evenly spaced nodes, and $n = 4$, find an approximate value for

$$\int_{0}^{1} \arctan x dx.$$

Estimate the error.

18. Using the Composite Trapezoidal Rule, and the table

x	0.2	0.4	0.6	0.8	1.0
y	7.11	10.21	20.98	32.91	1.12

estimate $\int_{0.2}^{1} f(x)dx$.

19. Using the Composite Simpson's Rule, and the table

x	0.2	0.4	0.6	0.8	1.0
y	7.11	10.21	20.98	32.91	1.12

estimate $\int_{0.2}^{1} f(x)dx$.

20. Using the Composite Trapezoidal Rule, and the table

x	0	0.4	0.7	0.8	1.0
y	7	10	20	32	1

estimate $\int_{0}^{1} f(x)dx$.

21. How many evenly spaced nodes do one need to use in the Composite Trapezoidal Rule in order to estimate

$$\int_{-1}^{1} e^{-x^2} dx$$

with error less than 0.0001?

22. How many evenly spaced nodes do one need to use in the Composite Trapezoidal Rule in order to estimate

$$\int_{-1}^{1} e^{\cos^2(\pi x)} dx$$

with error less than 0.0001?

23. How many evenly spaced nodes do one need to use in the Composite Trapezoidal Rule in order to estimate

$$\int_{1}^{\pi} \frac{\sin(x)}{x} dx$$

with error less than 0.0001?

24. Estimate $\int_{-2}^{2} e^{x^3} dx$, using the nodes and weights given in (5.48), for $n = 1$.
25. Estimate $\int_{0}^{2} \frac{\sin x}{x} dx$, using the nodes and weights given in (5.48), for $n = 2$.
26. Estimate $\int_{-2}^{2} e^{x^2} dx$, using the nodes and weights given in (5.48), for $n = 3$.
27. Estimate $\int_{-2}^{2} e^{-x^3} dx$, using the nodes and weights given in (5.48), for $n = 4$.
28. Estimate $\int_{-1}^{1} e^{\cos \pi x} dx$, using the nodes and weights given in (5.48), for $n = 1$.
29. Estimate $\int_{-1}^{1} e^{\cos \pi x} dx$, using the nodes and weights given in (5.48), for $n = 2$.
30. Determine the weights w_0, and w_1 such that the estimation

$$\int_{0}^{4} f(x)dx \simeq w_0 f(0) + w_1 f(4)$$

is exact for all polynomials of as large as possible degree. What is that degree?

31. Determine the weights w_0, and w_1, and the nodes x_0, and x_1 such that the estimation

$$\int_{0}^{4} f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1)$$

is exact for all polynomials of as large as possible degree. What is that degree?

32. Determine the weights w_0, and w_1 such that the estimation

$$\int_{0}^{1} \sqrt{x} f(x)dx \simeq w_0 f(0) + w_1 f(1)$$

is exact for all polynomials of as large as possible degree. What is that degree?

33. Determine the weight w_0, and the node x_0, such that the estimation

$$\int_{0}^{1} \sqrt{x} f(x)dx \simeq w_0 f(x_0)$$

is exact for all polynomials of as large as possible degree. What is that degree?

34. Determine the weights w_0, and w_1, and the nodes x_0, and x_1 such that the estimation

$$\int_0^1 \sqrt{x} f(x)dx \simeq w_0 f(x_0) + w_1 f(x_1)$$

is exact for all polynomials of as large as possible degree. What is that degree?

35. Determine the weights w_0, and w_1 such that the estimation

$$\int_a^b f(x)dx \simeq w_0 f(a) + w_1 f'(b)$$

is exact for all polynomials of as large as possible degree. What is that degree?

5.10 Computer Exercises

1. Create a table of values obtained using the Composite Trapezoidal Rule, with $n = 1, 2, 4, 8, 16$ and 32, for

$$\int_0^1 e^{-x^2} dx.$$

Also output the estimated errors, and the actual errors in each case.

2. Create a table of values obtained using the Composite Simpson's Rule, with $n = 2, 4, 8, 16$ and 32, for

$$\int_0^1 e^{-x^2} dx.$$

Also output the estimated errors, and the actual errors in each case.

3. Create a table of values obtained using the Composite Trapezoidal Rule, with $n = 1, 2, 4, 8, 16$ and 32, for

$$\int_0^1 \frac{\sin x}{x} dx.$$

Also output the estimated errors, and the actual errors in each case.

4. Create a table of values obtained using the Composite Simpson's Rule, with $n = 2, 4, 8, 16$ and 32, for

$$\int_0^1 \frac{\sin x}{x} dx.$$

Also output the estimated errors, and the actual errors in each case.

5. Create a table of values obtained using the Composite Trapezoidal Rule, with $n = 1, 2, 4, 8, 16$ and 32, for

$$\int_{-1}^1 e^{\cos \pi x} dx.$$

Also output the estimated errors.

6. Create a table of values obtained using the Composite Simpson's Rule, with $n = 1, 2, 4, 8, 16$ and 32, for

$$\int_{-1}^{1} e^{\cos \pi x} dx.$$

Also output the estimated errors.

7. Create Sage code that will use the Composite Trapezoidal Rule for nodes that may not necessarily be evenly spaced in $[a, b]$. Then use it to compute

$$\int_{0}^{1} \frac{\sin x}{x} dx.$$

for the following nodes $\{1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, 0\}$.

8. Create Sage code that will use the Composite Trapezoidal Rule for nodes that may not necessarily be evenly spaced in $[a, b]$. Then use it to compute

$$\int_{-1}^{1} e^{x^2} dx.$$

using Chebyshev's nodes given in formula (3.20), on page 149. Apply this for $n = 8, 16, 32$ and 64.

9. Create Sage code that will use the Composite Trapezoidal Rule for nodes that may not necessarily be evenly spaced in $[a, b]$. Then use it to compute

$$\int_{-1}^{1} e^{-x^2} dx.$$

using Chebyshev's nodes given in formula (3.20), on page 149. Apply this for $n = 8, 16, 32$ and 64.

10. Using Trapezoidal Rule, estimate

$$\int_{-1}^{1} e^{-x^2} dx.$$

with an error less than 0.000001.

11. Using Simpson's Rule, estimate

$$\int_{-1}^{1} e^{-x^2} dx.$$

with an error less than 0.000001

12. Using Trapezoidal Rule, estimate

$$\int_{-1}^{1} e^{\cos \pi x} dx.$$

with an error less than 0.000001.

13. Using Simpson's Rule, estimate

$$\int_{-1}^{1} e^{\cos \pi x} dx.$$

with an error less than 0.000001.

14. Using the Romberg Method, with $n = 8$, to estimate

$$\int_{-1}^{1} e^{\cos \pi x} dx.$$

15. Using Trapezoidal Rule, estimate

$$\int_{0}^{2} e^{x^2} dx.$$

with an error less than 0.000001.

16. Using Simpson's Rule, estimate

$$\int_{0}^{2} e^{x^2} dx.$$

with an error less than 0.000001.

17. Using the Romberg Method, with $n = 16$, to estimate

$$\int_{0}^{2} e^{x^2} dx.$$

18. Estimate $\int_0^2 e^{x^2} dx$, using the nodes and weights given in (5.48), for $n = 1$.
19. Estimate $\int_0^2 e^{x^2} dx$, using the nodes and weights given in (5.48), for $n = 2$.
20. Estimate $\int_0^2 e^{x^2} dx$, using the nodes and weights given in (5.48), for $n = 3$.
21. Estimate $\int_0^2 e^{x^2} dx$, using the nodes and weights given in (5.48), for $n = 4$.
22. Estimate $\int_0^{\pi} e^{-\sin^2 x} dx$, using the nodes and weights given in (5.48), for $n = 3$.
23. Estimate $\int_0^1 \frac{\sin x}{x} dx$, using the nodes and weights given in (5.48), for $n = 3$.
24. Estimate $\int_0^1 \frac{1}{\sqrt{\sin x}} dx$, using the nodes and weights given in (5.48), for $n = 3$.

References

1. Abramowitz, M., Stegun, I.A. (eds.): Handbook of Mathematical Functions with Formulae, Graphs, and Mathematical Tables. Dover, New York (1972)
2. Atkinson, K., Han, W.: Elementary Numerical Analysis, 3rd edn. Wiley, New York (2003)
3. Blanco-Silva, F.J.: Learning SciPy for Numerical and Scientific Computing. Packt Publishing, Birmingham (2013)
4. Callister, W.D.: Materials Science and Engineering, 6th edn. Wiley, New York (2003)
5. Cheney, W., Kincaid, D.: Numerical Mathematics and Computing, 7th edn. Brooks/Cole: Cengage Learning, Boston, MA (2013)

6. Epperson, J.F.: An Introduction to Numerical Methods and Analysis, revised edition. Wiley, Hoboken (2007)
7. Stein, W.A., et al.: Sage mathematics software (Version 6.3), The sage development team, Sage official webpage: http://www.sagemath.org (2014)
8. Sage cell server: http://sagecell.sagemath.org/
9. Stein, W.A.: Sage for power users, http://www.wstein.org/books/sagebook/sagebook.pdf (2012)
10. Sullivan, M., Miranda K.: Calculus - Early Transcendentals. W. H. Freeman and Company, New York (2014)

Chapter 6
Spline Interpolation

Given the set of $n+1$ data points,

$$\begin{array}{c|c|c|c|c} x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \end{array}, \text{ with } x_0 < x_1 < \cdots < x_n,$$

we have seen how we can obtain a polynomial function of degree (at most) n, $p(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$ that interpolates these points. That is:

$$p(x_0) = y_0, p(x_1) = y_1, \dots, p(x_n) = y_n.$$

This, however, has a major drawback: the polynomial can have a very high degree (up to n) and hence, the interpolating function can oscillate too much. The oscillation may be quite wild even when all the y-values of the data set given are essentially constant.

Let's see this challenge through an example. Consider the following set of data points

$$\{(-3, 1), (2, 1), (4, 1.01), (5, 1), (16, 1), (40, 1)\}.$$

Notice that all the data points, but one, have a y-value equal to 1. Even that exceptional data point has a y-value of 1.01, which is fairly close to 1. Therefore, one would normally expect that all $y - values$ (including those that correspond to $x = 30$, or $x = 35$), be somewhere close to 1. Also, the value 1.01 could easily be the result of rounding, or an entry error.

We give next the polynomial of interpolation:

$$\begin{aligned} p(x) \simeq & -(1.65343915344 \cdot 10^{-6})x^5 + (9.92063492063 \cdot 10^{-5})x^4 \\ & -0.00141038359788x^3 + 0.00316468253968x^2 \\ & +0.014417989418x + 0.968253968254 \end{aligned}$$

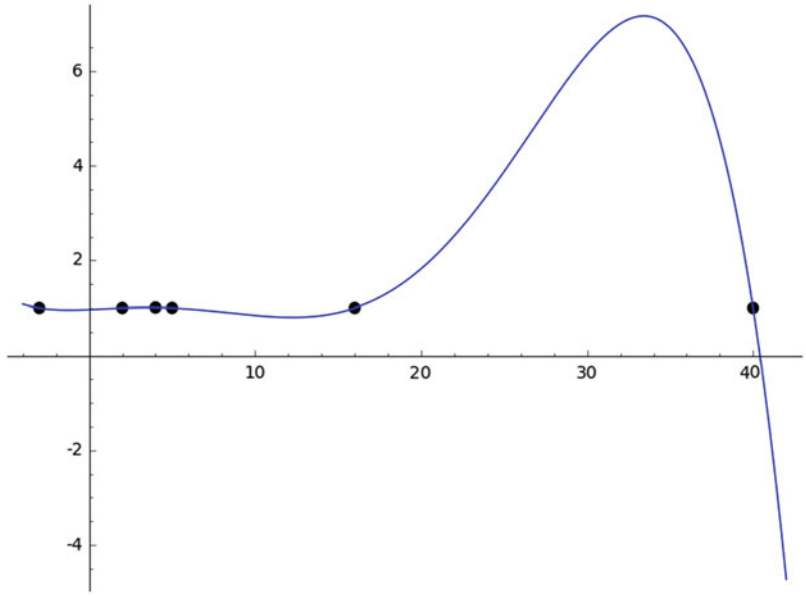
The original version of this chapter was revised. An erratum to this chapter can be found at DOI 10.1007/978-3-319-16739-8_8

Electronic supplementary material The online version of this chapter (doi: 978-3-319-16739-8_6) contains supplementary material, which is available to authorized users.

© Springer International Publishing Switzerland 2015

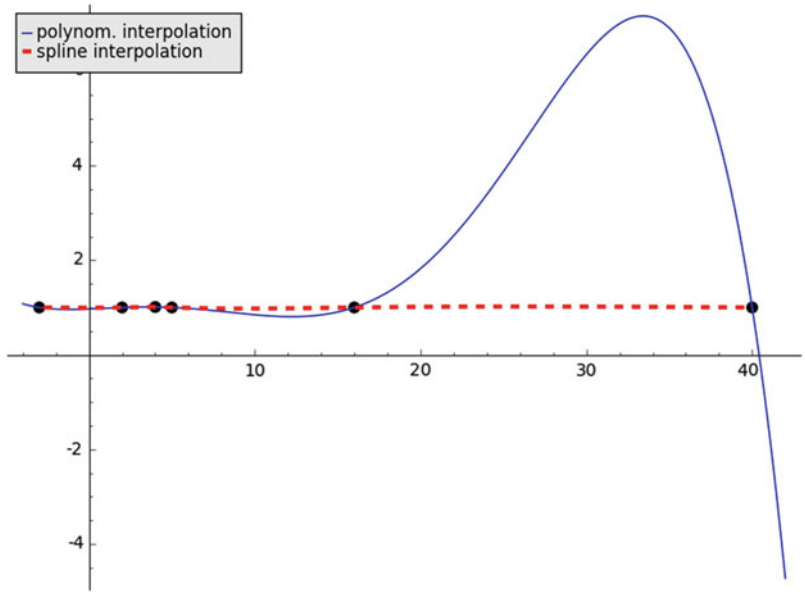
G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8_6

and the graph of the polynomial of interpolation $p(x)$:



Notice that $p(35)$ is somewhere around 7, way different than 1.

Ideally, we would love to obtain a curve that more closely follows the datapoints. One that is smooth, interpolates the datapoints, and does not oscillate that much. Something that looks like:

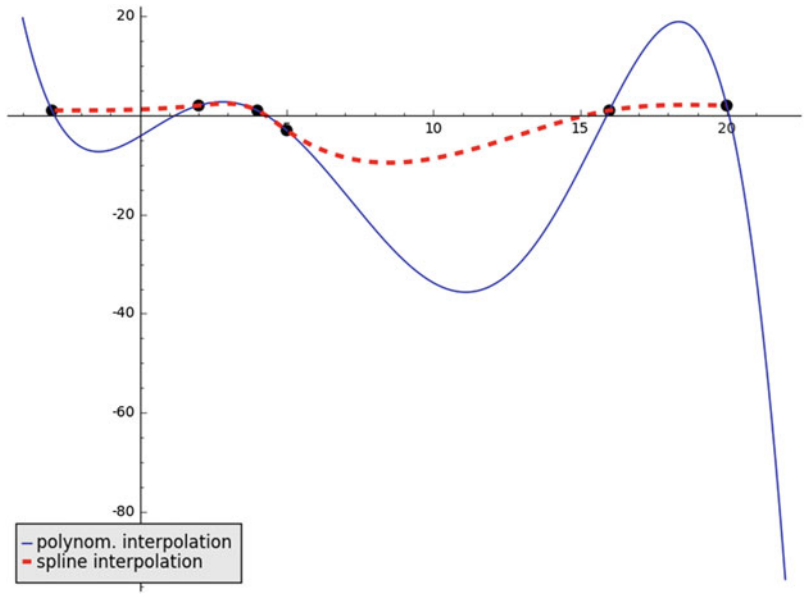


Here is another example where polynomial interpolations are not performing very well and an alternative is very desirable. Consider the following set of data points

$$\{(-3,1),(2,2),(4,1),(5,-3),(16,1),(20,2)\}.$$

The corresponding polynomial interpolation oscillates quite wildly. We would like to control, if possible, this oscillation.

As we will see below, we can control this oscillation by limiting the degree of the (piecewise-defined) polynomial.



In this chapter, we will learn about an alternative to interpolating polynomials, namely **spline functions** (see also [1, 3, 4]). These are piece-wise defined functions, whose pieces are polynomials joined together, to form functions with certain smoothness properties (such as *continuity*). Not only that we can limit the degree of the polynomial, for each piece, but this technique will also help us attenuate the effect of small error that may be contained in the table of data points.

Spline functions are extensively used in several areas, including aircraft industry, automotive industry, computer graphics, geometric modeling, and so on. As we mentioned earlier in this book, most image and video compression/decompression algorithms use interpolation. Using various sampling techniques, we can compress images ("lossy" compression), and then, using interpolation techniques, we can decompress them. Image restoration, digital sounds, GPS coordination, video games, etc. all use interpolation techniques. Interpolation techniques help us make the transition from discrete into continuous, and in particular, from digital into analog.

Because of the nice properties of the spline interpolations, they are the most used in practice, and are preferred more than the polynomial interpolations.

6.1 Linear Splines

The simplest spline function is the **linear spline** (also called **polygonal line**), that is obtained by connecting every two adjacent nodes by a straight line.

The **linear spline function** $LS(x)$ for a given the set of $n+1$ data points,

$$\begin{array}{|c|c|c|c|c|} \hline x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \\ \hline \end{array},$$

is a function that satisfies the following requirements:

1. It interpolates the given table. That is: $LS(x_0) = y_0, LS(x_1) = y_1, \dots, LS(x_n) = y_n$.
2. Smoothness: $LS(x)$ is continuous on $[x_0, x_n]$.
3. On each of the subintervals $[x_{i-1}, x_i]$, $i = 1, 2, \dots, n$, $LS(x)$ is a linear function.

That is

$$LS(x) = \begin{cases} a_1 x + b_1, & \text{for } x \in [x_0, x_1] \\ a_2 x + b_2, & \text{for } x \in [x_1, x_2] \\ \dots & \\ a_n x + b_n, & \text{for } x \in [x_{n-1}, x_n] \end{cases}.$$

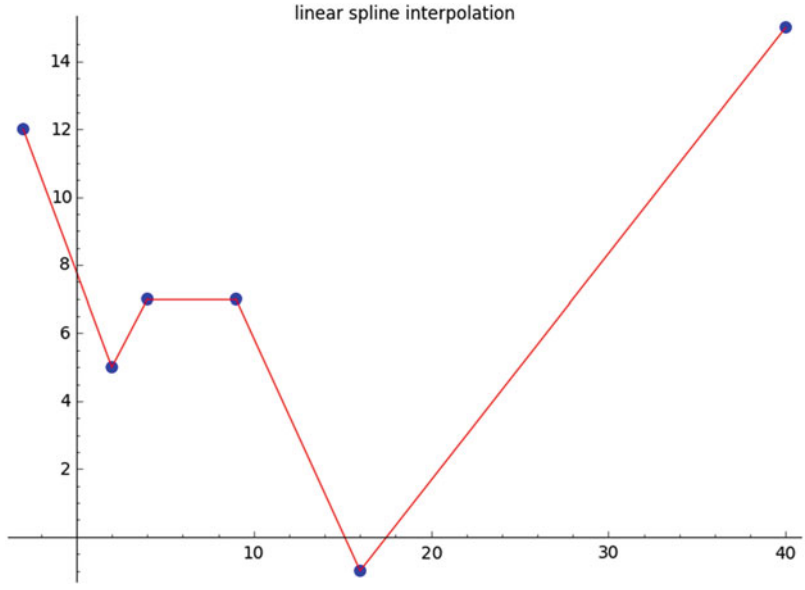
Please note that requirement 2. is a direct consequence of requirements 1 and 3. We included it here, to help make the transition to higher order spline functions a little easier.

To find the coefficients a_1, b_1, we need to find the linear function $y = a_1 x + b_1$ that interpolates (x_0, y_0) and (x_1, y_1). Therefore, we compute $a_1 = \frac{y_1 - y_0}{x_1 - x_0}$, and then $b_1 = y_1 - a_1 x_1$. We do a similar work for $a_2, b_2, \dots, a_n, b_n$.

For example, let's consider the following set of data points

$$\{(-3, 12), (2, 5), (4, 7), (9, 7), (16, -1), (40, 15)\}.$$

Then, its corresponding *linear spline* looks like



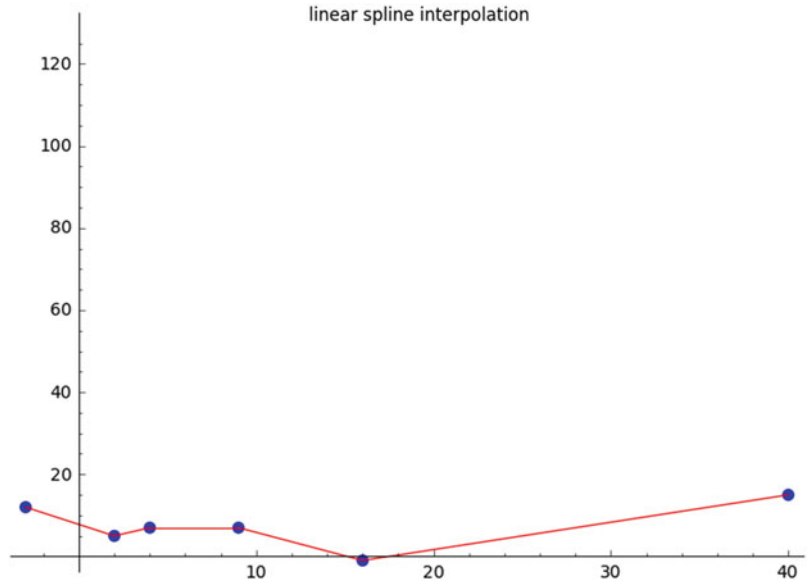
and can be written as

$$LS(x) = \begin{cases} \frac{39-7x}{5}, \text{ for } x \in [-3, 2] \\ x + 3, \text{ for } x \in [2, 4] \\ 7, \text{ for } x \in [4, 9] \\ \frac{121-8x}{7}, \text{ for } x \in [9, 16] \\ \frac{2x-35}{3}, \text{ for } x \in [16, 40] \end{cases}.$$

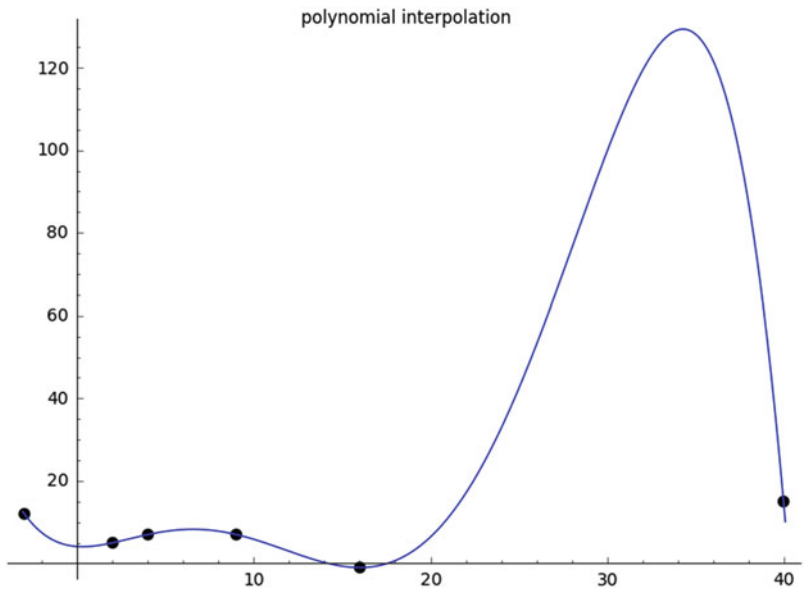
As one can easily notice, linear splines are continuous functions, but they are usually not smooth. They are not differentiable. If smoothness is essential, we should use quadratic or even cubic splines (defined in the next sections).

Before we go forward, it would be worth making a comparison between the linear spline functions versus polynomial interpolation applied to the same set of data points.

First, we scaled the y-axis of the previous graph, for a more objective comparison



and next we show the graph of the polynomial interpolation for the same set of data points:



Please note how the spline function follows the points more closely, without having any large spikes.

6.1.1 Sage Code

We give, below, the Sage code (see [2, 5–7]) we used to graph the above linear spline.

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#we compute n from the size of the array
n = len(points)-1

#compute and draw the linear spline
def DrawLinearSpline():
    #plot the points
    p = list_plot( points, color = 'blue', size = 70)

    #plot each branch, one by one
    for i in [1,..,n]:
        #first, find the constant in y=ax+b
        slope = (points[i][1]-points[i-1][1])/(points[i][0]
                          -points[i-1][0])
        yIntercept = points[i][1]-slope*points[i][0]

        #then plot the y=ax+b on the interval [xi-1, xi]
        p += plot(slope*x+yIntercept,
                  (x,points[i-1][0],points[i][0]),
                  color = 'red',
                  title = 'linear spline interpolation')
```

```
    #display the plot
    p.show()
#call the function
DrawLinearSpline()
```

Next, we'll write a Sage function that can be used to evaluate the Linear Spline function for a given value of x. For example, we may wish to compute $LS(1.34)$.

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#we compute n from the size of the array
n = len(points)-1

#computes the value of  LS(x)
#it assumes x is inside the interval [x0,xn]!
def ComputeLinearSpline(x):
    #search for the subinterval [xi-1,xi] where x belongs
    i = 0
    while x>points[i][0]:
        #if x is too large,
        #then go to the next subinterval
        i = i+1

    slope = (points[i][1]-points[i-1][1])/(points[i][0]
                                          -points[i-1][0])
    yIntercept = points[i][1]-slope*points[i][0]

    return slope*x+yIntercept
#call the function
ComputeLinearSpline(1.34)
```

We obtain the following output

```
5.92400000000000
```

6.1.2 Sage Library

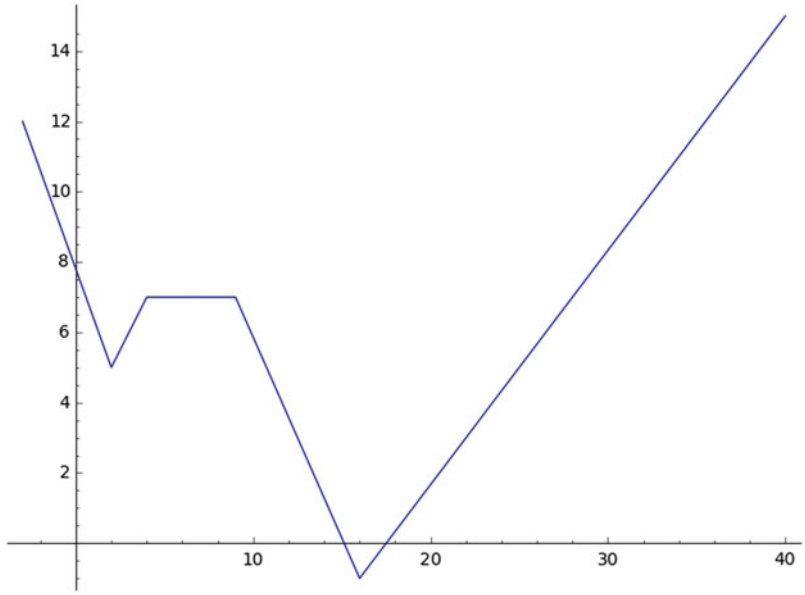
If we want to draw the linear spline function using Sage Library, we can use code similar to

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#draw the linear spline

line(points)
```

to obtain



6.2 Quadratic Splines

As we have seen in the previous section, linear splines, although they are continuous, they usually are not very smooth. In particular, they are not everywhere differentiable inside the domain $[x_0, x_n]$. We can improve this by using quadratic functions in place of linear functions.

The **quadratic spline function** $QS(x)$ for a given the set of $n + 1$ data points,

$$\begin{array}{|c|c|c|c|c|} \hline x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \\ \hline \end{array},$$

is a function that satisfies the following requirements:

1. Interpolation: $QS(x_0) = y_0, QS(x_1) = y_1, \dots, QS(x_n) = y_n$.
2. Smoothness: $QS(x)$ and $QS'(x)$ are both continuous on $[x_0, x_n]$
3. On each of the subintervals $[x_{i-1}, x_i]$, $i = 1, 2, \dots, n$, $QS(x)$ is a polynomial of degree at most two.

That is

$$QS(x) = \begin{cases} a_1x^2 + b_1x + c_1, \text{ for } x \in [x_0, x_1] \\ a_2x^2 + b_2x + c_2, \text{ for } x \in [x_1, x_2] \\ \dots \\ a_nx^2 + b_nx + c_n, \text{ for } x \in [x_{n-1}, x_n] \end{cases}.$$

Please note that there are $3n$ coefficients to determine, and the requirements 1–3 only give us $2n + (n - 1) = 3n - 1$ conditions. Therefore, to uniquely determine a quadratic spline function, one needs an additional condition. Standard examples of such conditions are: $QS'(x_0) = 0$, or $QS'(x_n) = 0$, or $QS''(x_0) = 0$, etc.

To solve for $QS(x)$, we need to find $a_1, b_1, c_1, \ldots, a_n, b_n, c_n$. Let's use the following notation:

$$\begin{aligned} QS_1(x) &= a_1x^2 + b_1x + c_1 \\ QS_2(x) &= a_2x^2 + b_2x + c_2 \\ &\ldots \\ QS_n(x) &= a_nx^2 + b_nx + c_n \\ QS'(x_0) &= z_0, QS'(x_1) = z_1, \ldots, QS'(x_n) = z_n. \end{aligned}$$

Then, from $QS_i(x_{i-1}) = y_{i-1}, QS_i'(x_{i-1}) = z_{i-1}$, and $QS_i'(x_i) = z_i$,it can be shown that

$$QS_i(x) = \frac{z_i - z_{i-1}}{2(x_i - x_{i-1})}(x - x_{i-1})^2 + z_{i-1}(x - x_{i-1}) + y_{i-1}. \tag{6.1}$$

Since $QS_i(x_i) = y_i$, we obtain

$$y_i = \frac{z_i - z_{i-1}}{2(x_i - x_{i-1})}(x_i - x_{i-1})^2 + z_{i-1}(x_i - x_{i-1}) + y_{i-1},$$

hence

$$z_i = 2\frac{(y_i - y_{i-1})}{(x_i - x_{i-1})} - z_{i-1} \tag{6.2}$$

for all $i = 1, 2, \ldots, n$. This gives us the following algorithm to compute $QS(x)$:

6.2.1 Algorithm

We give, below, the algorithm for finding a Quadratic Spline for a given set of data points. To uniquely determine such a spline function, will use the following additional condition: $QS'(x_0) = 0$. In this case, the quadratic spline is called **natural quadratic spline**.

Step 1: using (6.2) we find the values for $z_1, z_2, \ldots, z_n$.
Step 2: using (6.1) we find each of $QS_1(x)$, $QS_2(x)$, $\ldots$, $QS_n(x)$ and hence find $QS(x)$.

Example 132. Find the Quadratic Spline function for the following set of data points

$$\{(-3, 12), (2, 5), (4, 7), (9, 7), (16, -1), (40, 15)\}.$$

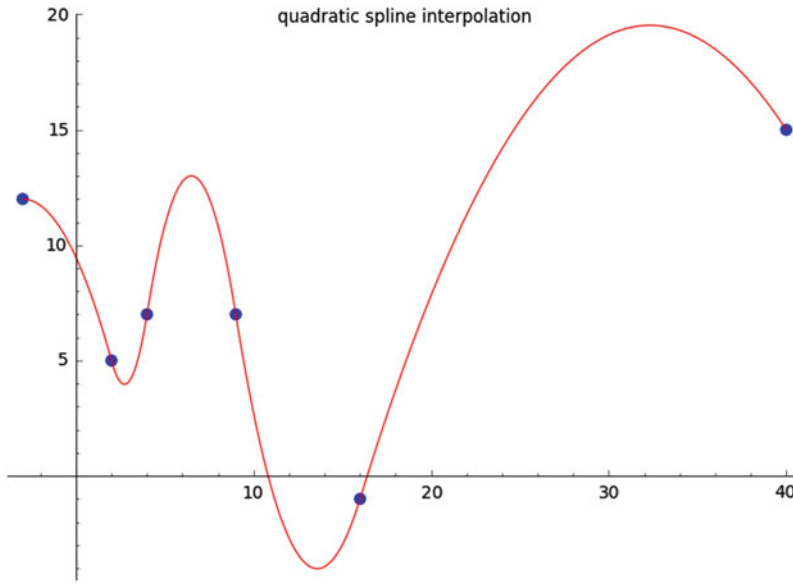
Solution. Step 1: we need to find z_1, z_2, z_3, z_4, and z_5. Using (6.2) we obtain

$$z_1 = 2\frac{(y_1 - y_0)}{(x_1 - x_0)} - z_0 = -\frac{14}{5}$$
$$z_2 = 2\frac{(y_2 - y_1)}{(x_2 - x_1)} - z_1 = \frac{24}{5}$$
$$z_3 = 2\frac{(y_3 - y_2)}{(x_3 - x_2)} - z_2 = -\frac{24}{5}$$
$$z_4 = 2\frac{(y_4 - y_3)}{(x_4 - x_3)} - z_3 = \frac{88}{35}$$
$$z_5 = 2\frac{(y_5 - y_4)}{(x_5 - x_4)} - z_4 = -\frac{124}{105}$$

Step 2: using (6.1) we obtain

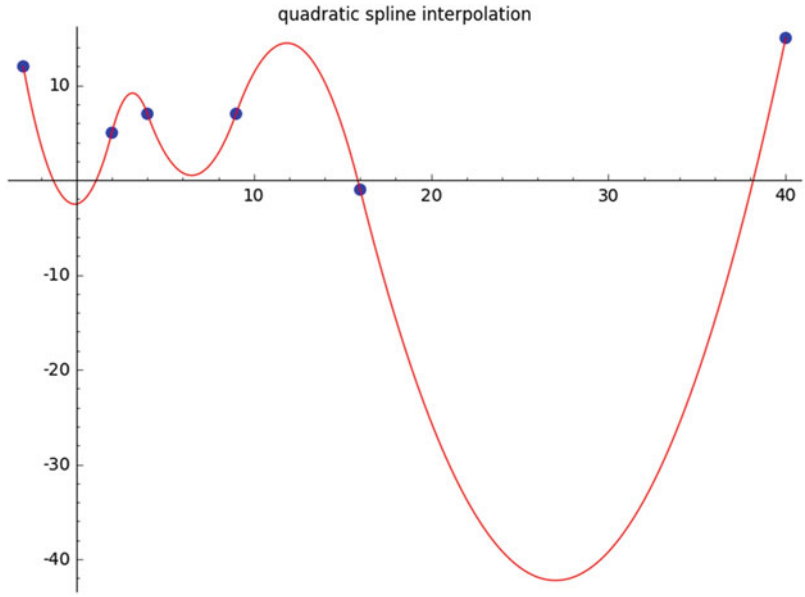
$$QS(x) = \begin{cases} -\frac{7}{25}x^2 - \frac{42}{25}x + \frac{237}{25}, & \text{for } x \in [-3, 2] \\ \frac{19}{10}x^2 - \frac{52}{5}x + \frac{91}{5}, & \text{for } x \in [2, 4] \\ -\frac{24}{25}x^2 + \frac{312}{25}x - \frac{689}{25}, & \text{for } x \in [4, 9] \\ \frac{128}{245}x^2 - \frac{696}{49}x + \frac{22667}{245}, & \text{for } x \in [9, 16] \\ -\frac{97}{1260}x^2 + \frac{224}{45}x - \frac{3839}{63}, & \text{for } x \in [16, 40] \end{cases}.$$

The corresponding graph is



Compare this to the linear spline obtained on page 230.

Note: If we change the additional condition $QS'(x_0) = 0$ into $QS'(x_0) = -10$ we obtain the following graph of the Quadratic Spline Function, which is another Quadratic Spline function that interpolates the same given data set.



6.2.2 Sage Code

We give, below, the Sage code we used to graph the above quadratic spline. It also contains code to generate the quadratic spline.

```
#the array of data points given

points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#we compute n from the size of the array
n = len(points)-1

#compute and draw the quadratic spline
def DrawQuadraticSpline():
    #plot the points
    p = list_plot( points, color = 'blue', size = 70)

    #we use this extra condition z0=0
    oldZ=0

    #plot each branch, one by one
    for i in [1,..,n]:
        #first, find the z's
        newZ = 2*(((points[i][1]-points[i-1][1]))/
                  ((points[i][0]-points[i-1][0])))-oldZ

        #find  the corresponding branch
        QS(x)=((newZ-oldZ)/(2*(points[i][0]-points[i-1][0])))* \
              (x-points[i-1][0])^2+oldZ*(x-points[i-1][0])     \
              +points[i-1][1]
```

```
        #save newZ as oldZ, to make room for next z
        oldZ=newZ

        #then plot the branch in the interval [xi-1, xi]
        p += plot(QS(x),
                  (x,points[i-1][0],points[i][0]),
                  color = 'red',
                  title = 'quadratic spline interpolation')
    #display the plot
    p.show()
#call the function
DrawQuadraticSpline()
```

The following Sage code will compute and display $QS(x)$

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#we compute n from the size of the array
n = len(points)-1

#compute and draw the quadratic spline
def DisplayQuadraticSpline():
    #we use this extra condition z0=0
    oldZ=0

    #plot each branch, one by one
    for i in [1,..,n]:
        #first, find the z's
        newZ = 2*(((points[i][1]-points[i-1][1]))
               /((points[i][0]-points[i-1][0])))-oldZ

        #find  the corresponding branch
        QS(x)=((newZ-oldZ)/(2*(points[i][0]-points[i-1][0]))) \
              *(x-points[i-1][0])^2+oldZ*(x-points[i-1][0])   \
              +points[i-1][1]
        print 'QS'+str(i)+'(x)=',  QS(x).full_simplify()

        #save newZ as oldZ, to make room for next z
        oldZ=newZ
#call the function
DisplayQuadraticSpline()
```

and obtain

```
QS1(x)= -7/25*x^2 - 42/25*x + 237/25
QS2(x)= 19/10*x^2 - 52/5*x + 91/5
QS3(x)= -24/25*x^2 + 312/25*x - 689/25
QS4(x)= 128/245*x^2 - 696/49*x + 22667/245
QS5(x)= -97/1260*x^2 + 224/45*x - 3839/63
```

Next, we'll write a Sage function that can be used to evaluate the Quadratic Spline function for a given value of x. For example, we may wish to compute $QS(1.34)$.

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#we compute n from the size of the array
n = len(points)-1

#computes the value of  QS(x)
#it assumes x is inside the interval [x0,xn]!
def ComputeQuadraticSpline(xx):
    #search for the subinterval [xi-1,xi] where x belongs
    i = 0

    #we use this extra condition z0=0
    oldZ=0

    #skip the intervals that are below x
    #but compute the z's
    while xx>points[i][0]:
        i = i+1

        #first, find the z's
        newZ = 2*(((points[i][1]-points[i-1][1]))/
                  ((points[i][0]-points[i-1][0])))-oldZ

        #save newZ as oldZ, to make room for next z
        oldZ=newZ

    #compute zi
    newZ = 2*(((points[i][1]-points[i-1][1]))/
              ((points[i][0]-points[i-1][0])))-oldZ

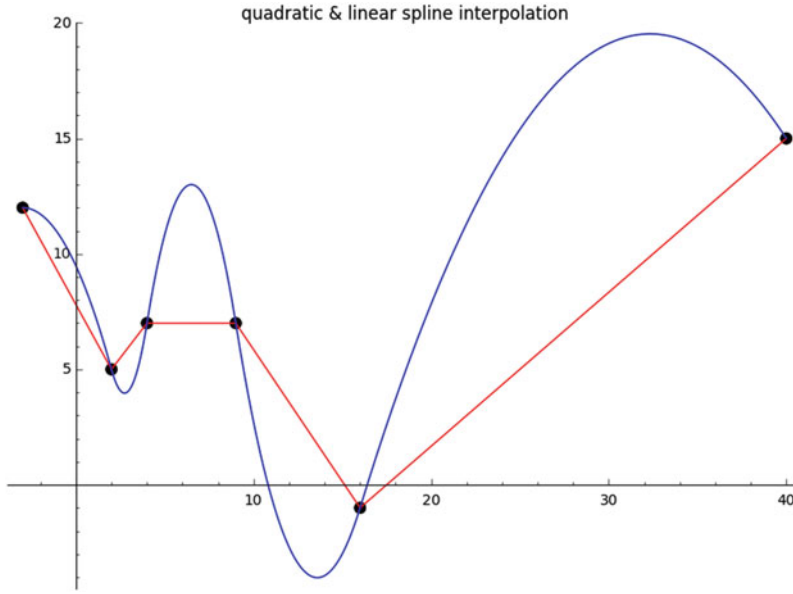
    #find  the corresponding branch QSi
    QS(x)=((newZ-oldZ)/(2*(points[i][0]-points[i-1][0]))) \
           *(x-points[i-1][0])^2+oldZ*(x-points[i-1][0])  \
           +points[i-1][1]

    #display the plot
    return QS(xx)
#call the function
ComputeQuadraticSpline(1.34)
```

We obtain the following output

```
5.12196800000000
```

Putting side by side the linear and the quadratic spline functions we obtain



6.3 Natural Cubic Splines

In applications, it is common to require that the first and second derivatives of the approximating function to be also continuous. Neither the linear spline nor the quadratic spline have this smoothness property. For this we would need higher order spline functions. In this section we will present the Natural Cubic Spline functions.

The **cubic spline function** $CS(x)$ for a given the set of $n+1$ data points,

$$\begin{array}{|c|c|c|c|c|} \hline x_0 & x_1 & x_2 & \dots & x_n \\ \hline y_0 & y_1 & y_0 & \dots & y_n \\ \hline \end{array},$$

is a function that satisfies the following requirements:

1. Interpolation: $CS(x_0) = y_0, CS(x_1) = y_1, \dots, CS(x_n) = y_n$.
2. Smoothness: $CS(x)$, $CS'(x)$ and $CS''(x)$ are all continuous on $[x_0, x_n]$
3. On each of the subintervals $[x_{i-1}, x_i]$, $i = 1, 2, \dots, n$, $CS(x)$ is a polynomial of degree at most three.

That is

$$CS(x) = \begin{cases} a_1x^3 + b_1x^2 + c_1x + d_1, & \text{for } x \in [x_0, x_1] \\ a_2x^3 + b_2x^2 + c_2x + d_2, & \text{for } x \in [x_1, x_2] \\ \dots & \\ a_nx^3 + b_nx^2 + c_nx + d_n, & \text{for } x \in [x_{n-1}, x_n] \end{cases}.$$

In order to uniquely determine a Cubic Spine function, two extra conditions need to be added. If we impose the extra conditions

$$CS''(x_0) = 0, \text{ and } CS''(x_n) = 0, \tag{6.3}$$

then we obtain what is called **Natural Cubic Splines.**

If the y values are coming from a function f, $y = f(x)$, and instead of (6.3), we impose the extra conditions

$$CS'(x_0) = f'(x_0),\ CS'(x_n) = f'(x_n), \tag{6.4}$$

then we obtain what is called **Complete Cubic Splines.**

In this section we will only focus on Natural Cubic Spline Functions. Before we present the general algorithm to find the natural cubic spline for a given set of data points, let's start with a simple example.

Example 133. Find the natural cubic spline function for the following set of data points $\{(-1, 2), (0, 5), (2, -1)\}$.

Solution. To solve for $CS(x)$, we need to find $a_1, b_1, c_1, d_1, a_2, b_2, c_2, d_2$, such that the function

$$CS(x) = \begin{cases} CS_1(x) = a_1x^3 + b_1x^2 + c_1x + d_1, \text{ for } x \in [-1, 0] \\ CS_2(x) = a_2x^3 + b_2x^2 + c_2x + d_2, \text{ for } x \in [0, 1] \end{cases},$$

satisfy the Natural Cubic Spline properties. That is:

$$\begin{cases} CS_1(-1) = 2 \\ CS_1(0) = 5 \\ CS_2(0) = 5 \\ CS_2(2) = -1 \\ CS_1'(0) = CS_2'(0) \\ CS_1''(0) = CS_2''(0) \\ CS_1''(-1) = 0 \\ CS_2''(2) = 0 \end{cases}$$

therefore

$$\begin{cases} -a_1 + b_1 - c_1 + d_1 = 2 \\ d_1 = 5 \\ d_2 = 5 \\ 8a_2 + 4b_2 + 2c_2 + d_2 = -1 \\ c_1 = c_2 \\ 2b_1 = 2b_2 \\ -6a_1 + 2b_1 = 0 \\ 12a_2 + 2b_2 = 0 \end{cases},$$

obtaining

$$CS(x) = \begin{cases} -x^3 - 3x^2 + x + 5, \text{ for } x \in [-1, 0] \\ 0.5x^3 - 3x^2 + x + 5, \text{ for } x \in [0, 1] \end{cases}.$$

We describe, below, an algorithm that can be used to solve for the Natural Cubic Spline Function. For this, let's use the following notation:

$$\begin{aligned}
CS_1(x) &= a_1x^3 + b_1x^2 + c_1x + d_1,\\
CS_2(x) &= a_2x^3 + b_2x^2 + c_2x + d_2,\\
&\dots\\
CS_n(x) &= a_nx^3 + b_nx^2 + c_nx + d_n,\\
CS'(x_0) &= z_0, CS'(x_1) = z_1, \dots, CS'(x_n) = z_n,\\
CS''(x_0) &= t_0, CS''(x_1) = t_1, \dots, CS''(x_n) = t_n.
\end{aligned}$$

Since $CS_i(x)$ is a cubic polynomial on each subinterval $[x_{i-1}, x_i]$, $i = 1, 2, \dots, n$, we have that $CS_i''(x)$ is a linear function on each of these subintervals. Using the continuity of $CS''(x)$, on $[x_0, x_n]$, and the interpolation conditions, we have that $CS_i''(x_{i-1}) = t_{i-1}$ and $CS_i''(x_i) = t_i$, hence using linear interpolation (see for example (3.2), on page 118) we obtain

$$CS_i''(x) = \left(\frac{x - x_i}{x_{i-1} - x_i}\right) \cdot t_{i-1} + \left(\frac{x - x_{i-1}}{x_i - x_{i-1}}\right) \cdot t_i,\ i = 1, 2, \dots, n. \tag{6.5}$$

Then, by integrating twice, we get

$$CS_i(x) = (x - x_i)^3 \frac{t_{i-1}}{6\,(x_{i-1} - x_i)} + (x - x_{i-1})^3 \frac{t_i}{6\,(x_i - x_{i-1})} + C_i x + D_i, \tag{6.6}$$

for $i = 1, 2, \dots, n$, with C_i, D_i constants that have to be determined.

Using the interpolation conditions and continuity of $CS(x)$, we have that $CS_i(x_{i-1}) = y_{i-1}$ and $CS_i(x_i) = y_i$ $i = 1, 2, \dots, n$, hence solving for C_i and D_i from these conditions we obtain

$$\begin{aligned}
C_i &= \frac{(y_i - y_{i-1})}{(x_i - x_{i-1})} - (x_i - x_{i-1}) \frac{(t_i - t_{i-1})}{6}\\
D_i &= y_{i-1} + \frac{(x_i - x_{i-1})}{6} (t_i x_{i-1} - x_i t_{i-1}) - \frac{(y_i - y_{i-1})}{(x_i - x_{i-1})} x_{i-1}
\end{aligned}$$

therefore, after simplifying we get

$$\begin{aligned}
CS_i(x) = &\frac{t_{i-1}\,(x_i - x)^3 + t_i\,(x - x_{i-1})^3}{6\,(x_i - x_{i-1})}\\
&+ \frac{(x - x_{i-1})\,y_i + y_{i-1}\,(x_i - x)}{(x_i - x_{i-1})}\\
&- \frac{(x_i - x_{i-1})}{6} \left[t_i\,(x - x_{i-1}) + t_{i-1}\,(x_i - x)\right]
\end{aligned} \tag{6.7}$$

for $i = 1, 2, \dots, n$.

The only missing piece of the puzzle is to find the values of $t_i, i = 1, 2, \ldots, n-1$. To find these values we will use the continuity of $CS'(x)$. Hence from

$$CS'_i(x_i) = CS'_{i+1}(x_i)$$

for $i = 1, 2, \ldots, n-1$, and $t_0 = t_n = 0$, we obtain the following tri-diagonal system

$$\begin{cases} t_0 = 0 \\ t_{i-1}(x_i - x_{i-1}) + 2t_i(x_{i+1} - x_{i-1}) + t_{i+1}(x_{i+1} - x_i) \\ \qquad = 6\left(\frac{y_{i+1}-y_i}{x_{i+1}-x_i} - \frac{y_i - y_{i-1}}{x_i - x_{i-1}}\right) \\ t_n = 0 \end{cases}, \tag{6.8}$$

for $i = 1, 2, \ldots, n-1$. Then solving it for $t_0, t_1, \ldots, t_n$, and using these in (6.7), we find the Natural Cubic Spline function for our given set of data.

This gives us the following computational algorithm for $CS(x)$:

6.3.1 Algorithm

We give, below, the algorithm for finding a Natural Cubic Spline for a given set of data points. First we need to solve the system (6.8), to find $t_1, t_2, \ldots, t_{n-1}$. Then, substituting them, and $t_0 = t_n = 0$, into (6.7) we solve the problem.

Step 1: solve the system (6.8).
Step 2: using the answer to Step 1, we will find each of $CS_1(x), CS_2(x), \ldots, CS_n(x)$ and hence find $CS(x)$.

Example 134. Find the Natural Cubic Spline function for the following set of data points

$$\{(-3, 12), (2, 5), (4, 7), (9, 7), (16, -1), (40, 15)\}.$$

Solution. Step 1: we need to find t_1, t_2, t_3, and t_4. Using (6.8) we obtain

$$\begin{cases} t_0 = 0 \\ t_0(x_1 - x_0) + 2t_1(x_2 - x_0) + t_2(x_2 - x_1) = 6\left(\frac{y_2-y_1}{x_2-x_1} - \frac{y_1-y_0}{x_1-x_0}\right) \\ t_1(x_2 - x_1) + 2t_2(x_3 - x_1) + t_3(x_3 - x_2) = 6\left(\frac{y_3-y_2}{x_3-x_2} - \frac{y_2-y_1}{x_2-x_1}\right) \\ t_2(x_3 - x_2) + 2t_3(x_4 - x_2) + t_4(x_4 - x_3) = 6\left(\frac{y_4-y_3}{x_4-x_3} - \frac{y_3-y_2}{x_3-x_2}\right) \\ t_3(x_4 - x_3) + 2t_4(x_5 - x_3) + t_5(x_5 - x_4) = 6\left(\frac{y_5-y_4}{x_5-x_4} - \frac{y_4-y_3}{x_4-x_3}\right) \\ t_5 = 0 \end{cases}, \tag{6.9}$$

therefore

$$\begin{cases} t_0 = 0 \\ 14t_1 + 2t_2 = \frac{72}{5} \\ 2t_1 + 14t_2 + 5t_3 = -6 \\ 5t_2 + 24t_3 + 7t_4 = \frac{-48}{7} \\ 7t_3 + 62t_4 = \frac{76}{7} \\ t_5 = 0 \end{cases}, \tag{6.10}$$

hence

$$\begin{cases} t_0 = 0 \\ t_1 = \frac{2450341}{2227645} \\ t_2 = -\frac{159049}{318235} \\ t_3 = -\frac{107190}{445529} \\ t_4 = \frac{90121}{445529} \\ t_5 = 0 \end{cases}. \tag{6.11}$$

If you want, you can use Sage to compute the previous system. You can use, for example, the following code

```
var('t1,t2,t3,t4')
solve( (14*t1+2*t2== 72/5,
        2*t1+14*t2+5*t3==-6,
        5*t2+24*t3+7*t4==-48/7,
        7*t3+62*t4==76/7),
      t1, t2, t3, t4)
```

to obtain

```
[[t1 == (2450341/2227645), t2 == (-159049/318235),
  t3 == (-107190/445529), t4 == (90121/445529)]]
```

Then

Step 2: using (6.7) we obtain

$$CS(x) = \begin{cases} \frac{2450341}{66829350}x^3 + \frac{7351023}{22276450}x^2 - \frac{44330204}{33414675}x + \frac{67275427}{11138225}, & \text{for } x \in [-3, 2] \\ -\frac{890921}{6682935}x^3 + \frac{1202805}{891058}x^2 - \frac{22497502}{6682935}x + \frac{16484299}{2227645}, & \text{for } x \in [2, 4] \\ \frac{577393}{66829350}x^3 - \frac{7876287}{22276450}x^2 + \frac{115190962}{33414675}x - \frac{18768937}{11138225}, & \text{for } x \in [4, 9] \\ \frac{197311}{18712218}x^3 - \frac{2526129}{6237406}x^2 + \frac{36583846}{9356109}x - \frac{9585679}{3118703}, & \text{for } x \in [9, 16] \\ -\frac{90121}{64156176}x^3 + \frac{450605}{2673174}x^2 - \frac{2346530}{445529}x + \frac{183929215}{4009761}, & \text{for } x \in [16, 40] \end{cases}.$$

One can also use Sage to make these computations:

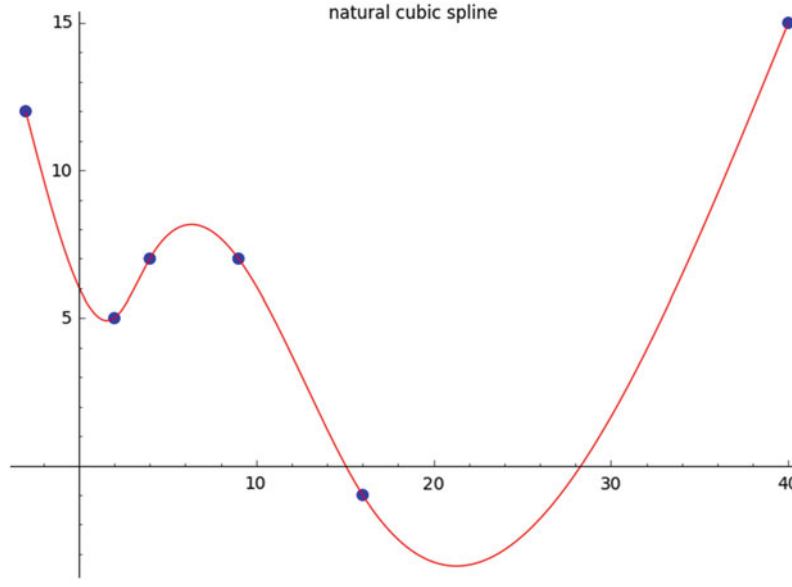
```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]
 #we compute n from the size of the array
n = len(points)-1
```

```
t = [0, 2450341/2227645, -159049/318235,
        -107190/445529, 90121/445529, 0]
for i in [1,2,..,n]:
    #compute each branch
    CS(x) = ((t[i-1]*(points[i][0]-x)^3+t[i]*(x-points[i-1][0])^3) \
      /(6*(points[i][0]-points[i-1][0])))                          \
      +(((x-points[i-1][0])*points[i][1]+points[i-1][1]            \
      *(points[i][0]-x))                                           \
      /((points[i][0]-points[i-1][0])))                            \
      -(((points[i][0]-points[i-1][0]))/6)*(t[i]*(x-points[i-1][0]))\
        +t[i-1]*(points[i][0]-x))
    print "CS"+str(i)+"(x)=", CS(x).full_simplify()
```

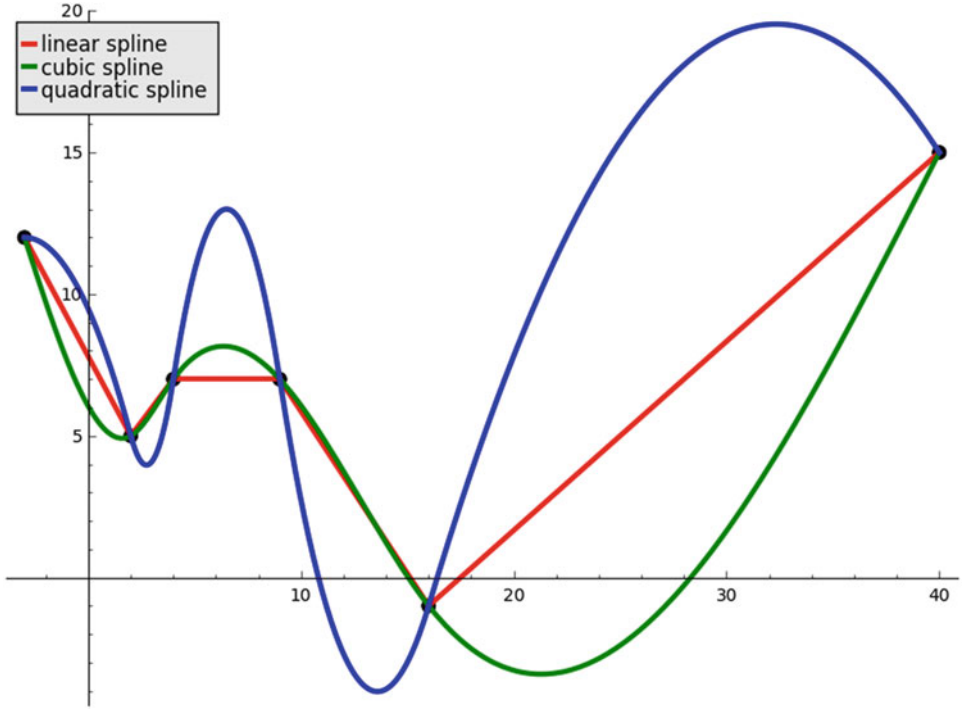
and obtain:

```
CS1(x)= 2450341/66829350*x^3 + 7351023/22276450*x^2
   - 44330204/33414675*x + 67275427/11138225
CS2(x)= -890921/6682935*x^3 + 1202805/891058*x^2
   - 22497502/6682935*x + 16484299/2227645
CS3(x)= 577393/66829350*x^3 - 7876287/22276450*x^2
   + 115190962/33414675*x - 18768937/11138225
CS4(x)= 197311/18712218*x^3 - 2526129/6237406*x^2
   + 36583846/9356109*x - 9585679/3118703
CS5(x)= -90121/64156176*x^3 + 450605/2673174*x^2
   - 2346530/445529*x + 183929215/4009761
```

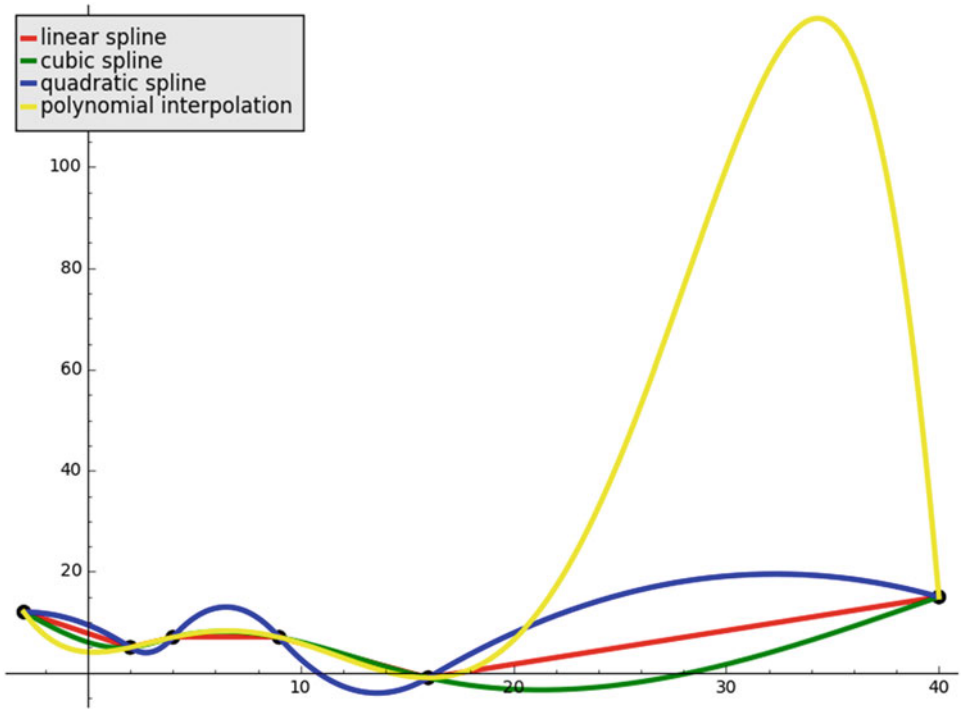
The corresponding graph for the Natural Cubic Spline is given by



Compare this to the linear and quadratic splines obtained earlier:



And if we include the polynomial interpolation, we obtain



6.3.2 Sage Code

We will see next how to implement a Sage function that computes the Natural Cubic Spline for a given data set.

It can be shown (see [3, pp. 269–270]) that the system (6.8) can be solved in two steps as follows:

Step 1: compute the following intermediate values

$$u_i = \begin{cases} 2(x_2 - x_0), \text{ for } i = 1 \\ 2\,(x_{i+1} - x_{i-1}) - \frac{(x_i - x_{i-1})^2}{u_{i-1}}, \text{ for } i = 2, 3, \ldots, n-1 \end{cases} \tag{6.12}$$

$$v_i = \begin{cases} 6\left(\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_2 - x_0}\right), \text{ for } i = 1 \\ 6\left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i} - \frac{y_i - y_{i-1}}{x_i - x_{i-1}}\right) - \frac{(x_i - x_{i-1})v_{i-1}}{u_{i-1}}, \text{ for } i = 2, 3, \ldots, n-1 \end{cases}.$$

Step 2: compute the values of t_i

$$\begin{aligned} t_0 &= 0, t_n = 0 \\ t_i &= \frac{v_i - (x_{i+1} - x_i)\,t_{i+1}}{u_i}, \text{ for } i = n-1, n-2, \ldots, 1. \end{aligned} \tag{6.13}$$

Once you obtain the values of $t_0, t_1, t_2, \ldots, t_n$ you can substitute them in (6.7) to get $CS(x)$.

We give, below, the Sage implementation that uses (6.12), (6.13) and (6.7) to draw $CS(x)$. We will apply this to the data set given in the previous example.

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#we compute n from the size of the array
n = len(points)-1
def DrawNaturalCubicSpline():
    #plot the points
    p = list_plot( points, color = 'blue', size = 70)

    #compute the value of t_i's
    t = [0  for i in [0,1,..,n]]
    u = [0  for i in [0,1,..,n]]
    v = [0  for i in [0,1,..,n]]
    u[1] = 2*((points[2][0]-points[0][0]))
    v[1] = 6*(((points[2][1]-points[1][1])      \
              /((points[2][0]-points[1][0])))   \
              -((points[1][1]-points[0][1])     \
              /((points[1][0]-points[0][0]))))

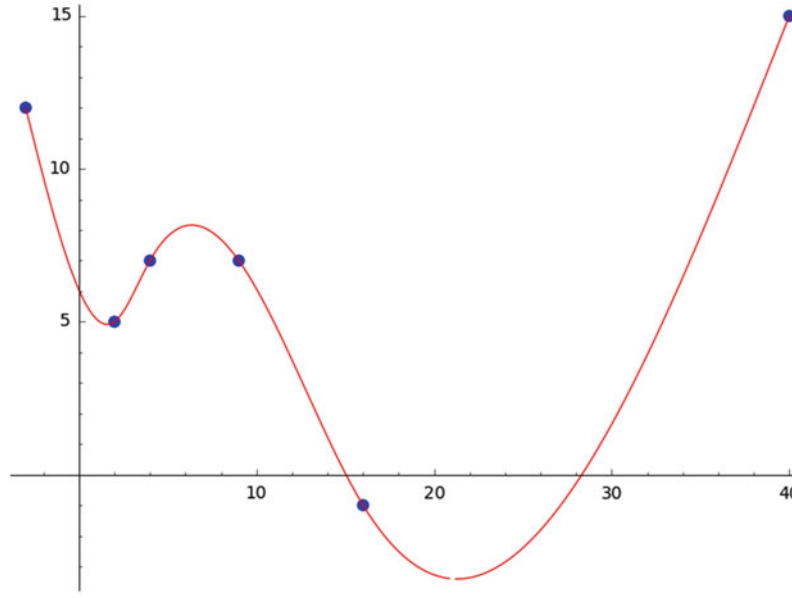
    for i in [2,3,..,n-1]:
        u[i]=2*(points[i+1][0]-points[i-1][0]) \
               -((points[i][0]-points[i-1][0])^2)/(u[i-1])
        v[i]=6*((points[i+1][1]-points[i][1])   \
               /(points[i+1][0]-points[i][0])   \
```

```
                -(points[i][1]-points[i-1][1])   \
                /(points[i][0]-points[i-1][0])) \
                -((points[i][0]-points[i-1][0]))*v[i-1]/u[i-1]

    for i in [n-1,n-2,..,1]:
        t[i]=( v[i] - (points[i+1][0]-points[i][0])*t[i+1] )/u[i]

    for i in [1,2,..,n]:
    #compute each branch
        CS(x) = ((t[i-1]*(points[i][0]-x)^3          \
                  +t[i]*(x-points[i-1][0])^3)/       \
                  (6*(points[i][0]-points[i-1][0]))) \
                +(((x-points[i-1][0])*points[i][1]   \
                +points[i-1][1]*(points[i][0]-x))    \
                /((points[i][0]-points[i-1][0])))    \
                -(((points[i][0]-points[i-1][0]))/6) \
                *(t[i]*(x-points[i-1][0])+t[i-1]*(points[i][0]-x))
        p += plot(CS(x), (x,points[i-1][0],points[i][0]), color
          = 'red')
     #display the plot
    p.show()
#call the function
DrawNaturalCubicSpline()
```

We obtained



If you want to calculate and display the pieces of the Natural Cubic Spline associated with the set of data, then you can use Sage code similar to:

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#we compute n from the size of the array
n = len(points)-1
def DisplayNaturalCubicSpline():
    #compute the value of t_i's
    t = [0  for i in [0,1,..,n]]
    u = [0  for i in [0,1,..,n]]
    v = [0  for i in [0,1,..,n]]
    u[1] = 2*((points[2][0]-points[0][0]))
    v[1] = 6*(((points[2][1]-points[1][1])       \
            /((points[2][0]-points[1][0])))      \
            - ((points[1][1]-points[0][1])       \
            /((points[1][0]-points[0][0]))))

    for i in [2,3,..,n-1]:
        u[i]=2*(points[i+1][0]-points[i-1][0])   \
            -((points[i][0]-points[i-1][0])^2)/(u[i-1])
        v[i]=6*((points[i+1][1]-points[i][1])    \
             /(points[i+1][0]-points[i][0])      \
            -(points[i][1]-points[i-1][1])       \
             /(points[i][0]-points[i-1][0]))     \
            -((points[i][0]-points[i-1][0]))*v[i-1]/u[i-1]

    for i in [n-1,n-2,..,1]:
        t[i]=( v[i] - (points[i+1][0]-points[i][0])*t[i+1] )/u[i]

    for i in [1,2,..,n]:
    #compute each branch
        CS(x) = ((t[i-1]*(points[i][0]-x)^3+t[i]*(x-points[i-1]
          [0])^3)  \
               /(6*(points[i][0]-points[i-1][0])))    \
               +(((x-points[i-1][0])*points[i][1]     \
               +points[i-1][1]*(points[i][0]-x))      \
               /((points[i][0]-points[i-1][0])))      \
               -(((points[i][0]-points[i-1][0]))/6)   \
               *(t[i]*(x-points[i-1][0])+t[i-1]*(points[i][0]-x))
        print "CS"+str(i)+"(x)=" , CS(x).full_simplify()
#call the function
DisplayNaturalCubicSpline()
```

to get again, the same answer as the one obtained in the example we solved earlier:

```
CS1(x)= 2450341/66829350*x^3 + 7351023/22276450*x^2
   - 44330204/33414675*x + 67275427/11138225
CS2(x)= -890921/6682935*x^3 + 1202805/891058*x^2
   - 22497502/6682935*x + 16484299/2227645
CS3(x)= 577393/66829350*x^3 - 7876287/22276450*x^2
   + 115190962/33414675*x - 18768937/11138225
CS4(x)= 197311/18712218*x^3 - 2526129/6237406*x^2
   + 36583846/9356109*x - 9585679/3118703
```

```
CS5(x)= -90121/64156176*x^3 + 450605/2673174*x^2
   - 2346530/445529*x + 183929215/4009761
```

6.3.3 Sage Library

One can also use the following Sage Library function to compute the Natural Cubic Spline

```
spline(points)
```

If you only wish to evaluate the Natural Cubic Spline function at $x = 1.34$, for the data set

$$\{(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)\},$$

then you would use the following code

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#obtain the natural cubic  spline function
cs = spline(points)

#compute and output the value at x=1.34
print cs(1.34)
```

to obtain

```
4.9430659023
```

To plot the graph of the Natural Cubic Spline given in the previous example, we could use the following code

```
#the array of data points given
points = [(-3,12),(2,5),(4,7),(9,7),(16,-1),(40,15)]

#we compute n from the size of the array
n = len(points)-1

#obtain the natural cubic  spline function
s = spline(points)

#plot the points
p = list_plot( points, color = 'blue', size = 70)

#plot the spline
p += plot(s, (x,points[0][0],points[n][0]),
                  color = 'red',
                  title = 'natural cubic spline')
p.show()
```

Example 135. Graph the natural cubic spline function that interpolates the function $f(x) = \sin x$ at 11 evenly spaced points in $[0, 4\pi]$. Also include the graph of the function $f(x)$.

Solution. Using the code

```
#the array of data points given
f(x) = sin(x)
```

```
points = [(0+i*4*pi/10,f(0+i*4*pi/10)) for i in [0,1,2,..,10]]

#we compute n from the size of the array
n = len(points)-1

#obtain the natural cubic  spline function
s = spline(points)

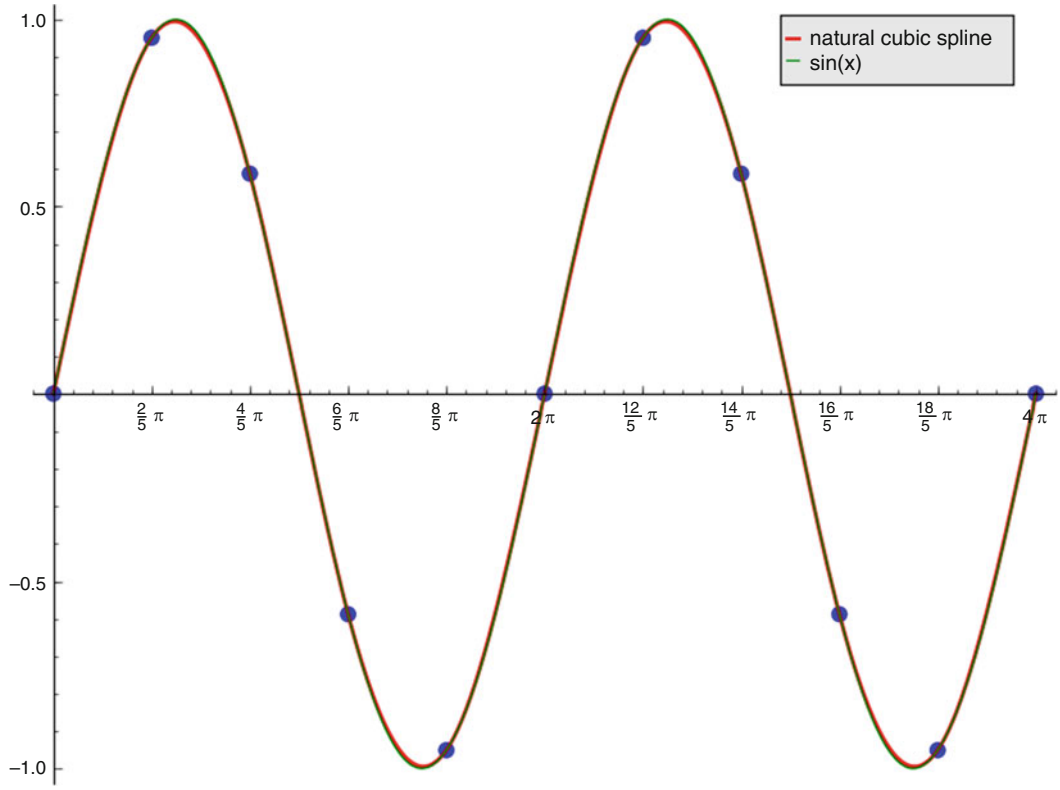
#plot the points
p = list_plot( points, color = 'blue', size = 70)

#plot the spline
p += plot(s, (x,points[0][0],points[n][0]),
                  color = 'red', thickness = 2,
                  legend_label = 'natural cubic spline')

#plot the function f
p += plot(f(x), (x,points[0][0],points[n][0]),
                  color = 'green',
                  legend_label = 'sin(x)')

p.show(ticks=4*pi/10, tick_formatter=pi)
```

we obtain



Example 136. Graph the natural cubic spline function and the polynomial function that interpolate the function $f(x) = |x|$ at 10 evenly spaced points in $[-1.6, 2]$. Also include the graph of the function $f(x)$.

Solution. Using the code

```
#the array of data points given
f(x) = abs(x)
points = [( -2+i*4/10,f(-2+i*4/10)) for i in [1,2,..,10]]

#we compute n from the size of the array
n = len(points)-1

#obtain the natural cubic  spline function
s = spline(points)

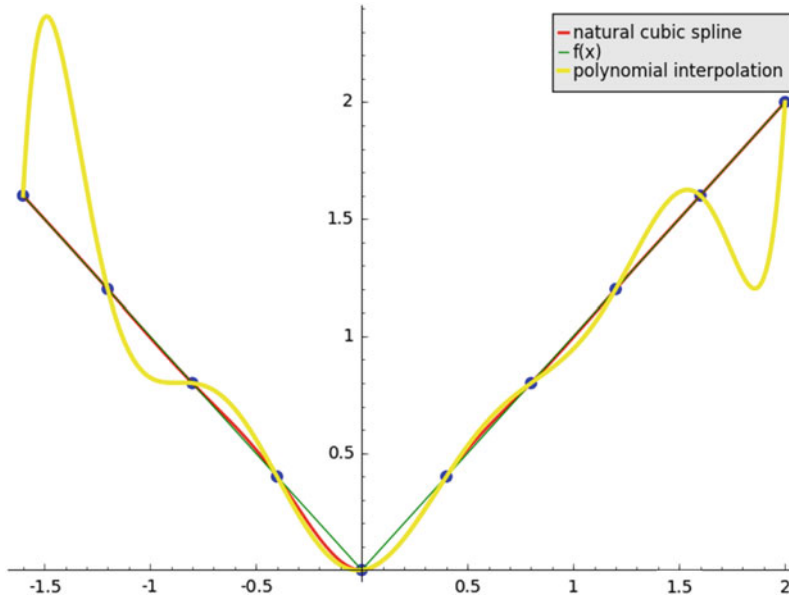
#plot the points
p = list_plot( points, color = 'blue', size = 70)

#plotting the cubic spline
p += plot(s, (x,points[0][0],points[n][0]),
                  color = 'red', thickness = 2,
                  legend_label = 'natural cubic spline')

#plotting the function f
p += plot(f(x), (x,points[0][0],points[n][0]),
                  color = 'green',
                  legend_label = 'f(x)')

#plotting the polynomial interpolation
R = PolynomialRing(QQ, 'xx')
pol(xx) = R.lagrange_polynomial(points)
p +=plot(pol(xx),  (xx,points[0][0], points[n][0]),
         color = 'yellow', thickness = 3,
         legend_label =   "polynomial interpolation")
p.show()
```

we obtain



6.4 An Application to Computer Science

Font creation, Animation Paths, Shape Modeling, Tessellation, Spline Images, and Vector Graphics are just a few examples of the wide range of applications of spline functions in Computer Science.

In particular, PostScript fonts use cubic splines, and TrueType fonts use quadratic. Storing each character using splines allows us to easily scale, stretch, or even rotate their display, while maintaining their smooth appearance.

The interested reader is invited to obtain more details by visiting the following websites: http://fontforge.org and http://apple.com.

Let us develop our own code that will make use of spline functions to display characters using our own font specification. For the sake of simplicity our system will contain very limited and simple characters, and hence some details that would normally be used in a real font system are omitted here. In order to specify our font, we will only need to specify the set of points that described the character we want to be displayed. Then, we will let Sage use spline functions to do the rest, connect those points to obtain the intended character.

As an example, let's say we want to display a letter such as a calligraphic "A". How can we do that? First, we obtain a set of points (using the two dimensions) that describes the letter with the desired level of fidelity. For our example, we could use:

t	0	1	2	3	4	5	6
x	0.2	1.0	2.0	3.0	4.0	5.0	6.0
y	0.5	0.0	2.7	5.5	2.7	0.0	0.5

,

and for the middle part,

t	0	1	2	3
x	1.5	2.5	3.5	4.5
y	2.5	2.7	2.3	2.5

.

Note: for two-dimensional curves, we can use two cubic spline functions together to form a parametric representation of such a curve that can turn and twist. In our example, we will use t as the parametric variable. For this, we could, for example, use the following Sage code

```
points1 = [(0,0.2),(1,1.0),(2,2.0),(3,3.0),(4,4.0),
           (5, 5.0), (6, 6.0)]
points2 = [(0,0.5),(1,0.0),(2,2.7),(3,5.5),(4,2.7),
           (5, 0.0), (6, 0.5)]

n = len(points1)-1
points = [(points1[i][1],points2[i][1]) for i in range(len(points1))]
p = list_plot(points, color = 'red', size = 70)
pol1 = spline(points1)
pol2 = spline(points2)
p+=parametric_plot((pol1,pol2),(x,0.0,points1[len(points1)-1][0]))

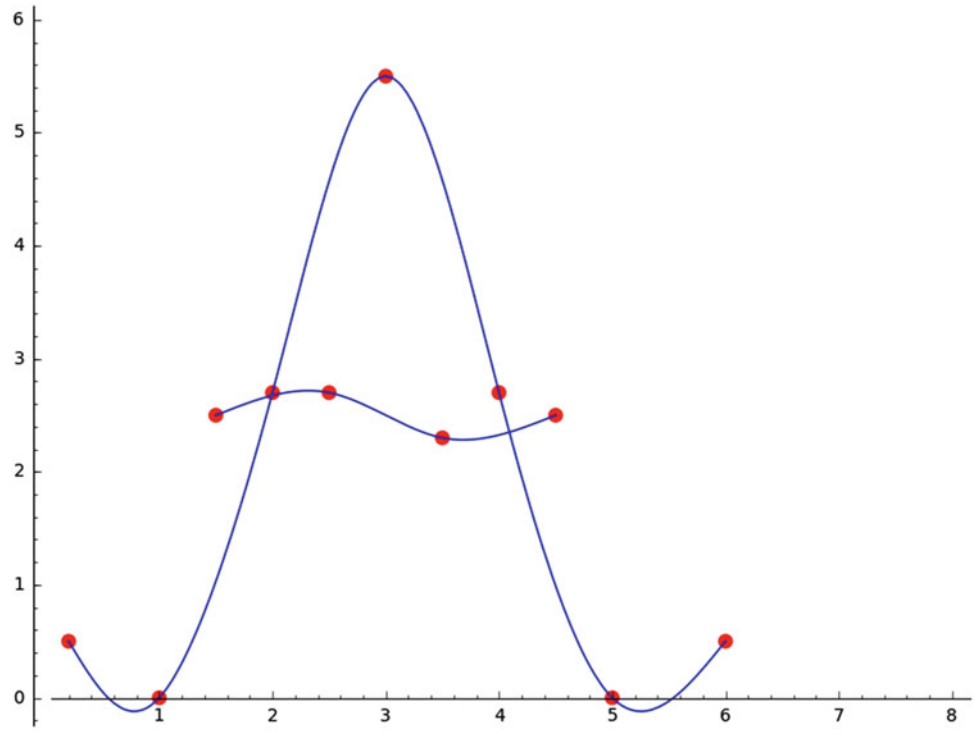
points1 = [(0,1.5),(1,2.5),(2,3.5),(3,4.5)]
points2 = [(0,2.5),(1,2.7),(2,2.3),(3,2.5)]

n = len(points1)-1
points = [(points1[i][1],points2[i][1]) for i in range(len(points1))]
p += list_plot(points, color = 'red', size = 70)
pol1 = spline(points1)
pol2 = spline(points2)

p+=parametric_plot((pol1,pol2) , (x, 0.0, points1[len(points1)-1][0]))

p.show(xmax=8,ymax=6)
```

and obtain the following image

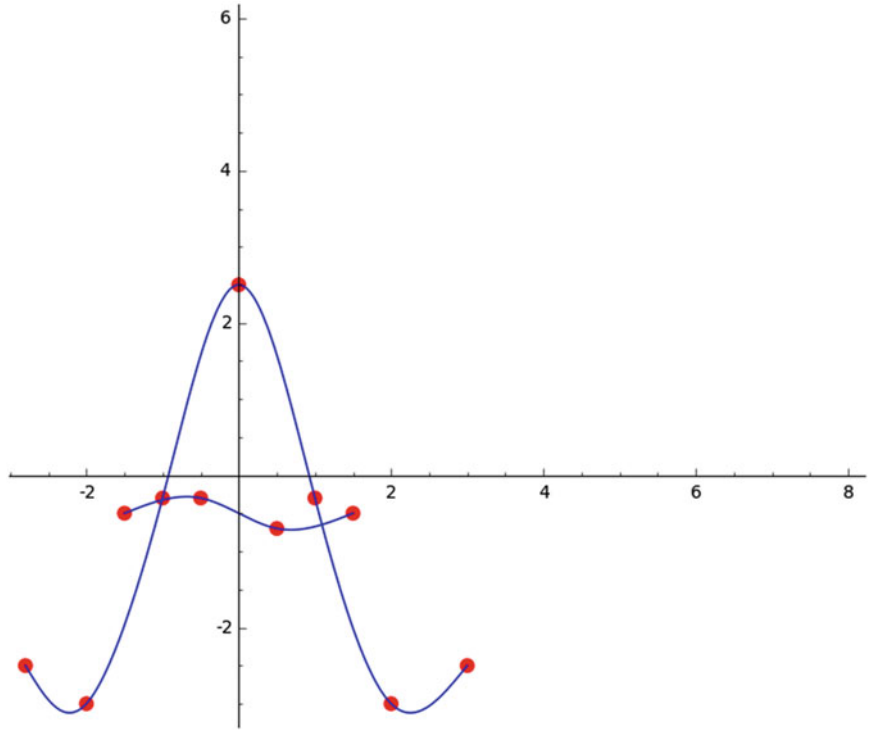


To see the power of splines, we can center the coordinates given above so that the origin is somewhere in the center of the character. Then, we can multiply the x coordinates (for a horizontal stretch) or the y-coordinates (for a vertical stretch), or both (for a change in size). Splines are used in some fonts (such as TrueType fonts) rather than image copy of each characters in order to avoid pixilation.

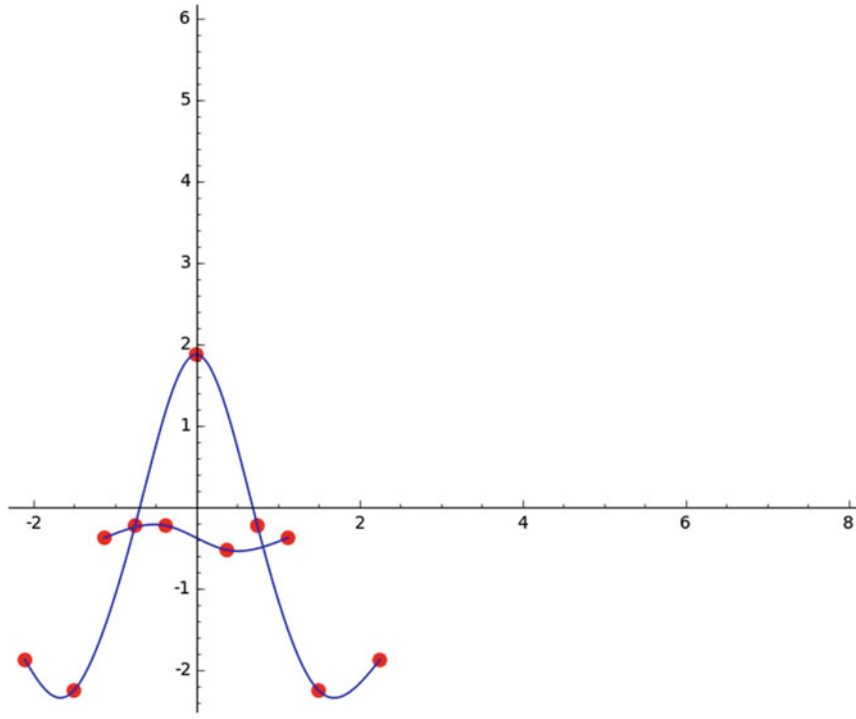
Using the following Sage code:

```
sX = 1 #used for horizontal stretch
sY = 1 #used for  vertical  stretch
points1 = [(0,-2.8*sX),(1,-2.0*sX),(2,-1.0*sX),(3,0.0*sX),
           (4,1.0*sX), (5, 2.0*sX), (6, 3.0*sX)]
points2 = [(0,-2.5*sY),(1,-3.0*sY),(2,-0.3*sY),(3,2.5*sY),
           (4,-0.3*sY),(5,-3.0*sY), (6,-2.5*sY)]
n = len(points1)-1
points = [(points1[i][1],points2[i][1]) for i in range(len(points1))]
p = list_plot(points, color = 'red', size = 70)
pol1 = spline(points1)
pol2 = spline(points2)
p+=parametric_plot((pol1,pol2),(x,0.0,points1[len(points1)-1][0]))
points1 = [(0,-1.5*sX),(1,-0.5*sX),(2,0.5*sX),(3,1.5*sX)]
points2 = [(0,-0.5*sY),(1,-0.3*sY),(2,-0.7*sY),(3,-0.5*sY)]
n = len(points1)-1
points = [(points1[i][1],points2[i][1]) for i in range(len(points1))]
p += list_plot(points, color = 'red', size = 70)
pol1 = spline(points1)
pol2 = spline(points2)
p+=parametric_plot((pol1,pol2) , (x, 0.0, points1[len(points1)-1][0]))
p.save('s.png',xmax=8,ymax=6)
```

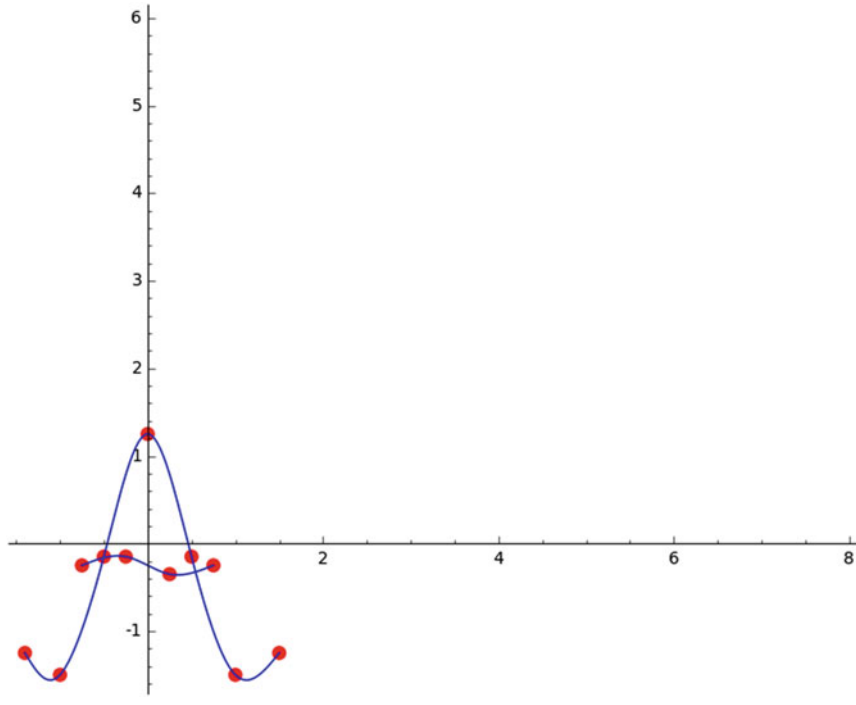
one obtains:



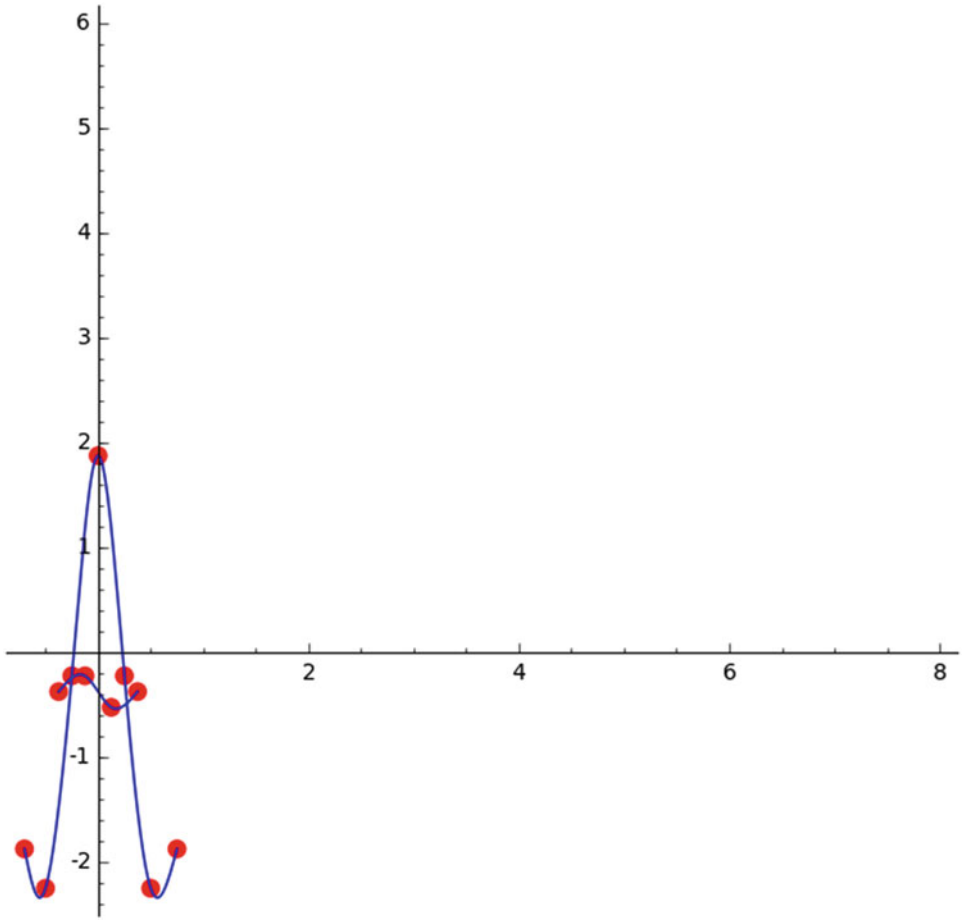
Changing the values of sX to 0.75 and sY to 0.75 we obtain:



Changing the values of sX to 0.5 and sY to 0.5 we obtain:



Changing the values of sX to 0.25 and sY to 0.75 we obtain:



We challenge you to adapt the previous example using Sage interacts.

6.5 Exercises

1. Find the linear spline function that interpolates the following set of data points, then graph it

$$\{(-3,1),(2,5),(4,7),(9,7),(16,1),(20,5)\}.$$

2. Find the linear spline function that interpolates the following set of data points, then graph it

$$\{(-3,1),(2,1),(4,1.01),(5,1),(16,1),(40,1)\}.$$

3. Find the linear spline function that interpolates the following set of data points, then graph it

$$\{(0,7),(1,10),(3,2),(4,5),(8,0)\}.$$

4. Find the quadratic spline function that interpolates the following set of data points, then graph it

$$\{(-3,1),(2,5),(4,7),(9,7),(16,1),(20,5)\}.$$

5. Find the quadratic spline function that interpolates the following set of data points, then graph it

$$\{(-3,1),(2,1),(4,1.01),(5,1),(16,1),(40,1)\}.$$

6. Find the quadratic spline function that interpolates the following set of data points, then graph it

$$\{(0,7),(1,10),(3,2),(4,5),(8,0)\}.$$

7. Find the natural cubic spline function that interpolates the following set of data points, then graph it

$$\{(-3,1),(2,5),(4,7),(9,7)\}.$$

8. Find the natural cubic spline function that interpolates the following set of data points, then graph it

$$\{(-3,1),(4,1.01),(5,1)\}.$$

9. Find the natural cubic spline function that interpolates the following set of data points, then graph it

$$\{(3,2),(4,5),(8,0)\}.$$

10. Find the cubic spline function that satisfies the following set of conditions, then graph it: $CS(0) = 0, CS(1) = 1, CS(2) = 2, CS'(0) = 0, CS''(2) = 3$.
11. For what values of C, the following function

$$\begin{cases} 2x^3 - 6x^2 + 8x + 6, & \text{for } x \in [-1,2] \\ -2x^3 + 18x^2 - 40x + C, & \text{for } x \in [2,3] \end{cases}$$

is a cubic spline? Can it be a natural cubic spline?
12. Find the values of C, and D, such that

$$\begin{cases} 2x^3 + 6x^2 + x + C, & \text{for } x \in [-1,2] \\ -2x^3 + 18x^2 + Dx + 3, & \text{for } x \in [2,3] \end{cases}$$

is a cubic spline function. Can it also be a natural cubic spline?
13. Do there exist A, B, C, and D, such that

$$\begin{cases} Ax^3 + 6x^2 + Bx + 7, & \text{for } x \in [-1,2] \\ Cx^3 + Dx^2 + x + 3, & \text{for } x \in [2,3] \end{cases}$$

is a natural cubic spline function?

14. Do there exist A, B, C, and D, such that

$$\begin{cases} 6x^2 + Ax + B, \text{ for } x \in [1,2] \\ Cx^3 + Dx^2 + x, \text{ for } x \in [2,3] \end{cases}$$

is a natural cubic spline function?

15. Find the natural cubic spline that interpolates the function $f(x) = x^4$, using the nodes $x = 0, 1, 2$.
16. Find the natural cubic spline that interpolates the function $f(x) = \sin x$, using the nodes $x = 0, \frac{\pi}{6}, \frac{\pi}{2}$.
17. Find the natural cubic spline that interpolates the function $f(x) = \sin x$, using the nodes $x = 0, \frac{\pi}{6}, \frac{\pi}{2}, \pi$.

6.6 Computer Exercises

1. Use Sage to graph the linear spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(-2,2), (-1,1), (0,0.5), (1,1), (2,2), (3,3.5)\}.$$

2. Use Sage to graph the linear spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(1,1), (2,0.5), (3,0.333), (4,0.25), (5,0.20)\}.$$

3. Use Sage to graph the linear spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(0,2.5), (1,0.5), (2,0.5), (3,2), (4,2), (5,1.5), (6,0)\}.$$

4. Use Sage to graph the linear spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(0,7), (1,10), (3,2), (4,5), (8,0)\}.$$

5. Use Sage to graph the quadratic spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(-2,2), (-1,1), (0,0.5), (1,1), (2,2), (3,3.5)\}.$$

6. Use Sage to graph the quadratic spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(1,1),(2,0.5),(3,0.333),(4,0.25),(5,0.20)\}.$$

7. Use Sage to graph the quadratic spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(0,2.5),(1,0.5),(2,0.5),(3,2),(4,2),(5,1.5),(6,0)\}.$$

8. Use Sage to graph the quadratic spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(0,7),(1,10),(3,2),(4,5),(8,0)\}.$$

9. Use Sage to graph the natural cubic spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(-2,2),(-1,1),(0,0.5),(1,1),(2,2),(3,3.5)\}.$$

10. Use Sage to graph the natural cubic spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(1,1),(2,0.5),(3,0.333),(4,0.25),(5,0.20)\}.$$

11. Use Sage to graph the natural cubic spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(0,2.5),(1,0.5),(2,0.5),(3,2),(4,2),(5,1.5),(6,0)\}.$$

12. Use Sage to graph the natural cubic spline function that interpolates the following set of data points. Then compare this graph with the one obtained from the polynomial interpolation for the same set of points.

$$\{(0,7),(1,10),(3,2),(4,5),(8,0)\}.$$

13. Use Sage to construct and graph the natural cubic spline function that interpolates the function $f(x) = \frac{1}{x}$ at 10 evenly spaced points in $[0.5, 3]$. Then compare this graph with the one obtained from interpolating the same set of points using polynomial interpolation.
14. Use Sage to construct and graph the natural cubic spline function that interpolates the function $f(x) = \sin x$ at 10 evenly spaced points in $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$. Then compare this graph with the one obtained from interpolating the same set of points using polynomial interpolation.

References

1. Atkinson, K., Han, W.: Elementary Numerical Analysis, 3rd edn. Wiley, New York (2003)
2. Blanco-Silva, F.J.: Learning SciPy for Numerical and Scientific Computing. Packt Publishing, Birmingham/Mumbai (2013)
3. Cheney, W., Kincaid, D.: Numerical Mathematics and Computing, 7th edn. Brooks/Cole: Cengage Learning, Boston (2013)
4. Epperson, J.F.: An Introduction to Numerical Methods and Analysis, revised edition. Wiley, New York (2007)
5. Sage Cell Server: http://sagecell.sagemath.org/
6. Stein, W.A., et al.: Sage mathematics software (Version 6.3). The Sage development team. Sage official webpage: http://www.sagemath.org (2014). Retrieved 2014
7. Stein, W.A.: Sage for power users. http://www.wstein.org/books/sagebook/sagebook.pdf (2012)

Chapter 7
Numerical Methods for Differential Equations

7.1 Introduction

An equation that involves an unknown function, y and one or more of its derivatives is called a **differential equation**. In this chapter, we will consider the case when $y = y(x)$ is a function of x. In this case, the differential equation is called an **ordinary differential equation** (or **ODE).** The case when the function y is a function of two or more independent variable is called a **partially differential equation (or PDE)** (see also [3, 4, 10]).

In this chapter, we will mainly consider ordinary differential equations. Examples of these are

$$\begin{aligned} y' &= 2x + 3\sin(x) + e^x \\ y' &= 2y + 3\sin(x) + e^x \\ y' &= \frac{2x}{1 + e^{-\sqrt{x^2 - \sin x}}} + 3\sin\left(\left|\sin x + \cosh x^5\right|\right) \\ \cos(y') - e^{|y|} &= \ln(1 + x^2) + 7. \end{aligned}$$

Some differential equations can be solved easily. For example

$$y' = 2x + 3\sin(x) + e^x$$

can be solved by integrating both sides. In this case we would obtain

$$y = x^2 - 3\cos(x) + e^x + C,$$

where C can be any constant. Hence this equation has an infinite number of solutions. If we specify an extra condition like

$$y(0) = 7$$

The original version of this chapter was revised. An erratum to this chapter can be found at DOI 10.1007/978-3-319-16739-8_8

Electronic supplementary material The online version of this chapter (doi: 978-3-319-16739-8_7) contains supplementary material, which is available to authorized users.

© Springer International Publishing Switzerland 2015

G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8_7

then we obtain that $C = 9$ hence the desired solution is

$$y = x^2 - 3\cos(x) + e^x + 9.$$

One can also solve this equation using the following Sage code (see [2, 6–8])

```
x = var('x')
y = function('y', x)
de =   diff(y,x) == 2*x+3*sin(x)+e^(x)
desolve(de, y)
```

and obtain the same answer:

```
x^2 + _C - 3*cos(x) + e^x
```

If we want to find the particular solution that satisfies the extra condition $y(0) = 7$, then Sage code like

```
x = var('x')
y = function('y', x)
de =   diff(y,x) == 2*x+3*sin(x)+e^(x)
desolve(de, y, ics=[0,7])
```

would give us the solution:

```
x^2 - 3*cos(x) + e^x + 9
```

Other types of equations may be solved, using some tricks or recipes that are explained in any standard Differential Equations Course. For example

$$y' = 2y + 3\sin(x) + e^x$$

can be solved by multiplying both sides by e^{-2x}

$$\left(ye^{-2x}\right)' = 3\sin(x)e^{-2x} + e^{-x}$$

and then integrating both sides, hence obtaining

$$y = -\frac{3}{5}\left(\cos(x) + 2\sin(x)\right) - e^x + Ce^{2x}.$$

Again, we could use the following Sage code

```
x = var('x')
y = function('y', x)
de =   diff(y,x) == 2*y+3*sin(x)+e^(x)
desolve(de, y).full_simplify()
```

to obtain the solution:

```
_C*e^(2*x) - 3/5*cos(x) - e^x - 6/5*sin(x)
```

There are differential equations that can be solved, in theory, but in practice they may be too complicated and hence not feasible to be used in computations. Also, some differential equations simply cannot be solved, or no solutions are known at this time. Try, for example, solving the following equations

$$y' = \frac{2x}{1 + e^{-\sqrt{x^2 - \sin x}}} + 3\sin\left(\left|\sin x + \cosh x^5\right|\right)$$

$$y' = 2\sin(y) * \cos(x) + y.$$

In this chapter we will focus on **initial-value problems for a first-order differential equation**, of the form

$$\begin{cases} y' = f(x, y), \text{ for } x \in [a, b] \\ y(a) \text{ - given} \end{cases}. \tag{7.1}$$

By y' we mean $\frac{dy}{dx}$.

For example,

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases}.$$

can be solved by finding the value of C, such that $y = -\frac{3}{5}(\cos(x) + 2\sin(x)) - e^x + Ce^{2x}$ satisfies the initial-value condition $y(0) = 7$. Therefore

$$7 = -\frac{3}{5}(\cos(0) + 2\sin(0)) - e^0 + Ce^0$$

hence

$$C = \frac{43}{5},$$

giving us the solution

$$y = -\frac{3}{5}(\cos(x) + 2\sin(x)) - e^x + \frac{43}{5}e^{2x}.$$

To include an initial condition of the form $y(a) = y_0$, when using Sage, you should use the following option `ics=[a,y0]` inside the `desolve` method. Therefore, the equation

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases}. \tag{7.2}$$

can be solved using the code:

```
x = var('x')
y = function('y', x)
desolve( diff(y,x) == 2*y+3*sin(x)+e^(x), y, ics=[0,7])
```

obtaining

```
-3/5*cos(x) + 43/5*e^(2*x) - e^x - 6/5*sin(x)
```

Let us discuss, briefly, the analytic interpretation of the solution of a first-order differential equation (7.1). The graph of the solution y, at any point (x, y), is a tangent line whose slope is given by $f(x, y)$. These lines can be represented graphically by short line segments having the prescribed values. Such a graphical representation is called a **directional field**. We can use Sage to create such diagrams. For example, we can draw the solutions of

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases},$$

and

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 4 \end{cases},$$

using the following Sage code

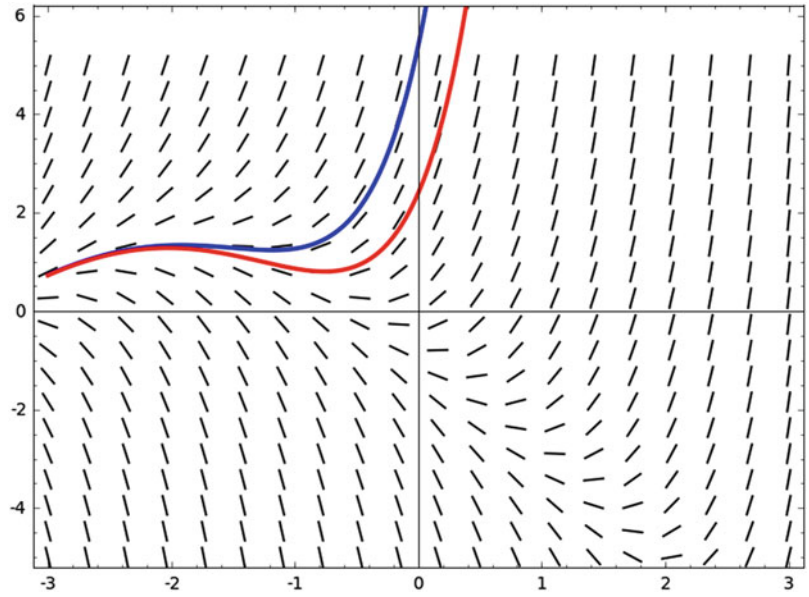
```
var('x,y')
#plotting the directional field
df = plot_slope_field(2*y+3*sin(x)+e^(x), (x,-3,3), (y,-5,5))

#plotting the solution with C = 7
sol1 = plot(-(3/5)*(cos(x)+2*sin(x))-e^(x)+7*e^(2*x),
            (x,-3,3), thickness = 3)

#plotting the solution with C = 4
sol2 = plot(-(3/5)*(cos(x)+2*sin(x))-e^(x)+4*e^(2*x),
            (x,-3,3), thickness = 3, color = 'red')

show(df+sol1+sol2, ymax=6)
```

to obtain

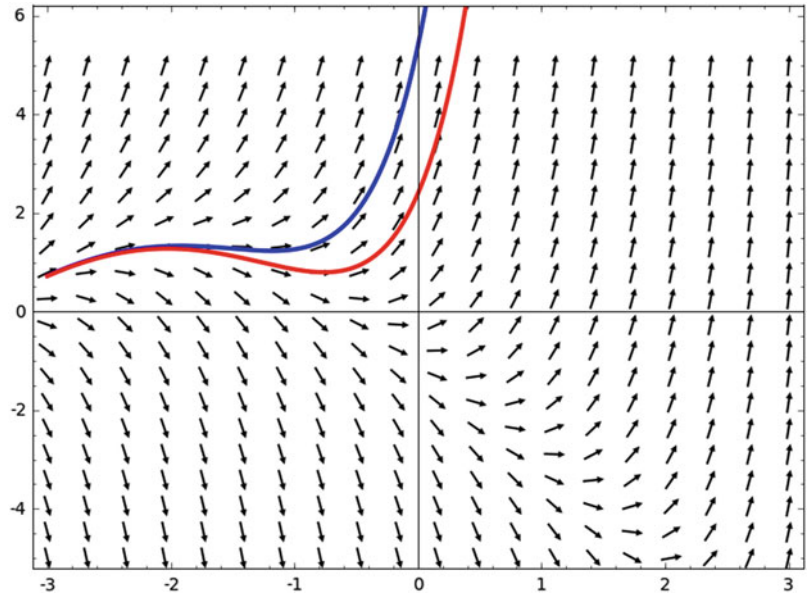


If we include the options `headaxislength = 3, headlength = 3` like in the code below:

```
df = plot_slope_field(2*y+3*sin(x)+e^(x), (x,-3,3), (y,-5,5),
                      headaxislength = 3, headlength = 3)
```

we obtain

Before we describe the numerical methods of solving an initial value problem of first-order derivatives, it is worth mentioning the following result (see [3, p. 304]): under certain conditions, (7.1) has a unique solution.



Theorem 137 (Uniqueness of Initial-Value Problems). *If both* $f(x, y)$ *and* $\frac{\partial f(x,y)}{\partial y}$ *are continuous around* $(a, y(a))$ *(including at* $(a, y(a))$*) then there is a unique continuous solution to the initial-value problem given by (7.1).*

7.2 Euler's Method

Most numerical methods for solving ordinary differential equations of the type

$$\begin{cases} y' = f(x, y), \text{ for } x \in [a, b] \\ y(a) = y_0 \end{cases}. \tag{7.3}$$

will come in the form of a table of values, for the nodes $x_0 = a, x_1, \ldots, x_n = b$. If one needs to know an approximate value of the solution y for other values of x than the ones in the table, interpolation techniques can be used.

One of the simplest, although not very efficient, numerical methods for solving initial-value problems of first-order derivatives is **Euler's Method**. It is based on the Taylor series formula, with $n = 1$ (see (1.29), on page 54):

$$f(x + h) \simeq f(x) + hf'(x). \tag{7.4}$$

From it, we derive

$$y(x + h) = y(x) + hy'(x)$$

hence we will use the following formula

$$y(x + h) = y(x) + hf(x, y). \tag{7.5}$$

7.2.1 Algorithm

We will create a table of values for y, at the nodes

$$x_0 = a, x_1 = a + h, x_2 = a + 2h, \ldots, x_n = b, \tag{7.6}$$

where

$$h = \frac{b-a}{n}. \tag{7.7}$$

Using (7.3) and (7.5) we have

$$\begin{aligned} y(x_0) &= y_0, \\ y(x_i) &= y(x_{i-1}) + hf(x_{i-1}, y(x_{i-1})), \text{ for } i = 1, 2, \ldots, n, \end{aligned} \tag{7.8}$$

hence obtaining

x	y
x_0	y_0
x_1	$y(x_1)$
...	...
x_n	$y(x_n)$

(7.9)

Example 138. Using Euler's method, compute an approximate value for $y(1)$, where y is the solution for the differential equation

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases}.$$

using $n = 5$ steps.

Solution. We have that $f(x, y) = 2y + 3\sin(x) + e^x$, and $a = 0, b = 1$. Then, using (7.6), (7.7), and (7.8) we obtain

$$\begin{aligned} y(0) &= 7 \\ y(0.2) &= 7 + 0.2f(0, 7) \\ &= 10 \\ y(0.4) &= 10 + 0.2f(0.2, 10) \\ &\simeq 14.3634821501091 \\ y(0.6) &= 14.3634821501091 + 0.2f(0.4, 14.3634821501091) \\ &\simeq 20.6408909550662 \\ y(0.8) &= 20.6408909550662 + 0.2f(0.6, 20.6408909550662) \\ &\simeq 29.6004565812077 \end{aligned}$$

$$\begin{aligned} y(1) &= 29.6004565812077 + 0.2\, f(0.8, 29.6004565812077) \\ &\simeq 42.3161610539290. \end{aligned}$$

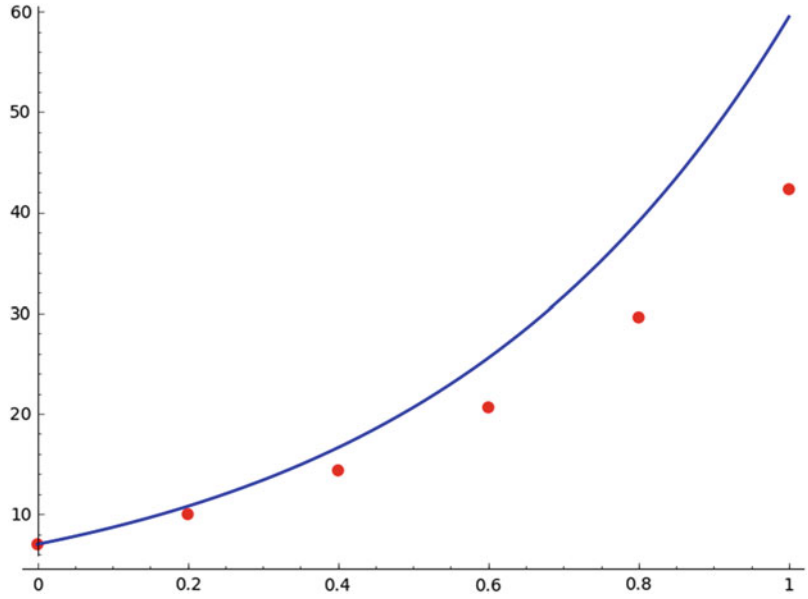
Hence $y(1) \simeq 42.3161610539290$.

Compare these values with the ones obtained from using the actual solution

$$y = -\frac{3}{5}(\cos(x) + 2\sin(x)) - e^x + \frac{43}{5}e^{2x}$$

$$\begin{aligned} y(0) &= 7 \\ y(0.2) &= 10.7818464978959 \\ y(0.4) &= 16.6278886802218 \\ y(0.6) &= 25.5581143981239 \\ y(0.8) &= 39.0916865866178 \\ y(1) &= 59.4936540570542 \end{aligned}$$

If we plot the exact solution and the table obtained above, we would get



Please note how the errors accumulate and they increases with each new point. They are getting further and further away from the exact solution. This phenomenon will make more sense once we discuss the error bounds. This will be discussed below in Sect. 7.2.5.

As we will also see later using Sage, a larger value of n will give us a better estimate.

7.2.2 Sage Code

We can use the following Sage code to implement the Euler's Method Algorithm presented in the previous subsection.

```
def PrintEuler(a,b,n,f, y0):
    #set up the initial values
    h = (b-a)/n
    xi = a
    yi = y0

    #print the first line of the table
    print "y("+str(a.n()) +") = ", y0.n()

    for i in [1,2,..,n]:
        #compute the next value of yi
        yi= yi+h*f(xi,yi)

        #prepare for the next x
        xi = xi + h

        #print the next line of the table
        print "y("+str(xi.n()) +") = ", yi.n()
```

Example 139. Using Euler's method, compute an approximate value for $y(1)$, where y is the solution for the differential equation

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases}.$$

using $n = 100$ steps.

Solution. We can use the following code

```
def PrintEuler(a,b,n,f, y0):
    #copy here its implementation!!!

#set up the problem
a = 0; b = 1; n = 100
f(x,y) = 2*y+3*sin(x)+e^(x)
y0 = 7

#call the function for Euler's Method
PrintEuler(a,b,n,f, y0)
```

to obtain

```
y(0.000000000000000) =  7.00000000000000
y(0.0100000000000000) =  7.15000000000000
y(0.0200000000000000) =  7.30340049667087
y(0.0300000000000000) =  7.46027048000535
y(0.0400000000000000) =  7.62068029995107
y(0.0500000000000000) =  7.78470169371761
```

```
y(0.0600000000000000) =  7.95240781363385
y(0.0700000000000000) =  8.12387325556636
y(0.0800000000000000) =  8.29917408791036
y(0.0900000000000000) =  8.47838788116439
y(0.100000000000000) =  8.66159373810067
y(0.110000000000000) =  8.84887232454284
y(0.120000000000000) =  9.04030590076340
y(0.130000000000000) =  9.23597835351313
y(0.140000000000000) =  9.43597522869523
y(0.150000000000000) =  9.64038376469704
y(0.160000000000000) =  9.84929292639247
y(0.170000000000000) =  10.0627934398287
y(0.180000000000000) =  10.2809778276104
y(0.190000000000000) =  10.5039404449966
y(0.200000000000000) =  10.7317775167225
y(0.210000000000000) =  10.9645871745624
y(0.220000000000000) =  11.2024694956485
    [ ...    OMITTED    ...]
y(0.890000000000000) =  46.3695278742487
y(0.900000000000000) =  47.3445818806723
y(0.910000000000000) =  48.3395693566862
y(0.920000000000000) =  49.3548890813445
y(0.930000000000000) =  50.3909478154718
y(0.940000000000000) =  51.4481604617839
y(0.950000000000000) =  52.5269502282151
y(0.960000000000000) =  53.6277487945162
y(0.970000000000000) =  54.7509964821898
y(0.980000000000000) =  55.8971424278272
y(0.990000000000000) =  57.0666447599179
y(1.00000000000000) =  58.2599707791977
```

Hence we conclude

$$y(1) \simeq 58.2599707791977$$

which is pretty close to the actual solution

$$y(1) = 59.4936540570542.$$

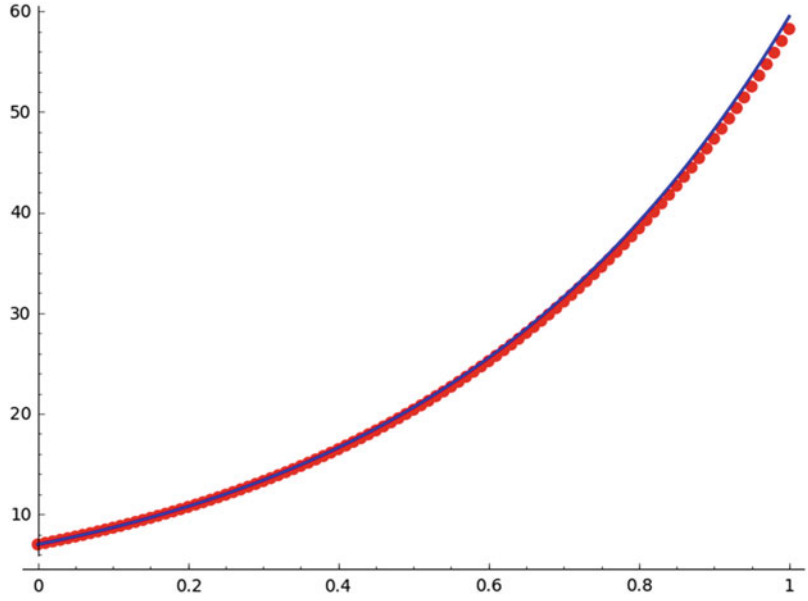
On the next page a graph containing the exact solution and the table obtained above using the Euler's Method.

Using $n = 400$, we obtain $y(1) \simeq 59.1798634022700$, and for $n = 1{,}000$, we obtain $y(1) \simeq 59.3676993862066$.

Below, we give the Sage code used to generate the graphs given in subsections 7.2.1 and 7.2.2.

```
def PlotEuler(a,b,n,f, y0):
    #set up the initial values
    h = (b-a)/n
    xi = a
    yi = y0

    #print the first line of the table
```




```
    EulerTable = [(xi, y0)]

    for i in [1,2,..,n]:
        #compute the next value of yi
        yi= yi+h*f(xi,yi)

        #prepare for the next x
        xi = xi + h

        #compute the next node, add it to the table
        EulerTable.append((xi.n(),yi.n()))
    #plot the points obtained from Euler's Method
    p = list_plot (EulerTable, color='red', size = 70)

    #compute the actual solution
    yy = function('yy',x)
    ActualSol(x) = desolve( diff(yy,x) == f(x,yy), yy, ics=[a,y0])

    #plot the actual solution
    p+=plot(ActualSol(x), x,a,a+n*h,color = 'blue', thickness = 2)
    p.show()

#set up the problem
a = 0; b = 1; n = 5
f(x,y) = 2*y+3*sin(x)+e^(x)
y0 = 7

#call the function for Euler's Method
PlotEuler(a,b,n,f, y0)
```

7.2.3 Sage Library

To solve the previous problem, one can also use the Sage Library. For example

```
x,y=PolynomialRing(QQ,2,"xy").gens()
a =0
y0=7
b =1
h =0.01
eulers_method(2*y+3*sin(x)+e^(x),a,y0,h,b)
```

would return the following

x	y	h*f(x,y)
0	7	0.150000000000000
0.0100000000000000	7.15000000000000	0.153400496670867
0.0200000000000000	7.30340049667087	0.156869983334485
0.0300000000000000	7.46027048000535	0.160409819945717
0.0400000000000000	7.62068029995107	0.164021393766544
0.0500000000000000	7.78470169371761	0.167706119916233
0.0600000000000000	7.95240781363385	0.171465441932514
0.0700000000000000	8.12387325556636	0.175300832343995
0.0800000000000000	8.29917408791036	0.179213793254032
0.0900000000000000	8.47838788116439	0.183205856936280
0.100000000000000	8.66159373810067	0.187278586442175
0.110000000000000	8.84887232454284	0.191433576220561
0.120000000000000	9.04030590076340	0.195672452749729
0.130000000000000	9.23597835351313	0.199996875182100
0.140000000000000	9.43597522869523	0.204408536001804
0.150000000000000	9.64038376469703	0.208909161695432
0.160000000000000	9.84929292639247	0.213500513436195
0.170000000000000	10.0627934398287	0.218184387781787
0.180000000000000	10.2809778276104	0.222962617386202
0.190000000000000	10.5039404449966	0.227837071725800
0.200000000000000	10.7317775167224	0.232809657839903
0.210000000000000	10.9645871745624	0.237882321086198
0.220000000000000	11.2024694956485	0.243057045911271
0.230000000000000	11.4455265415598	0.248335856636547
0.240000000000000	11.6938623981964	0.253720818259955
0.250000000000000	11.9475832164563	0.259214037273640
0.260000000000000	12.2067972537300	0.264817662498022
0.270000000000000	12.4716149162280	0.270533885932557
0.280000000000000	12.7421488021605	0.276364943623509
0.290000000000000	13.0185137457841	0.282313116549081
0.300000000000000	13.3008268623331	0.288380731522263
0.310000000000000	13.5892075938554	0.294570162111733
0.320000000000000	13.8837777559671	0.300883829581186
0.330000000000000	14.1846615855483	0.307324203847450
0.340000000000000	14.4919857893958	0.313893804457776
0.350000000000000	14.8058795938535	0.320595201586667
0.360000000000000	15.1264747954402	0.327431017052660
0.370000000000000	15.4539058124929	0.334403925355439
0.380000000000000	15.7883097378483	0.341516654733698

```
0.390000000000000     16.1298263925820    0.348771988244161
0.400000000000000     16.4785983808262    0.356172764862196
0.410000000000000     16.8347711456884    0.363721880604428
0.420000000000000     17.1984930262928    0.371422289673829
      [...      OMITTED      ...]
0.930000000000001     50.3909478154718     1.05721264631213
0.940000000000001     51.4481604617839     1.07878976643112
0.950000000000001     52.5269502282151     1.10079856630114
0.960000000000001     53.6277487945162     1.12324768767359
0.970000000000001     54.7509964821898     1.14614594563749
0.980000000000001     55.8971424278273     1.16950233209060
0.990000000000001     57.0666447599179     1.19332601927987
1.00000000000000     58.2599707791977     1.21762636341278
```

If we include the option `algorithm = 'none'` option, like in the following Sage code:

```
x,y=PolynomialRing(QQ,2,"xy").gens()
a =0
y0=7
b =1
h =0.01
eulers_method(2*y+3*sin(x)+e^(x),a,y0,h,b, algorithm = 'none')
```

then we obtain:

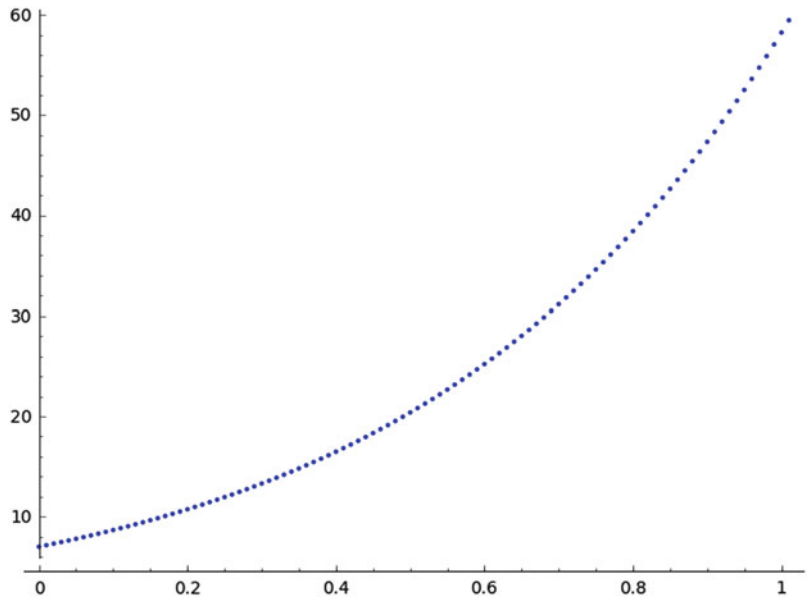
```
[[0, 7],
 [0.0100000000000000, 7.15000000000000],
 [0.0200000000000000, 7.30340049667087],
 [0.0300000000000000, 7.46027048000535],
 [0.0400000000000000, 7.62068029995107],
 [0.0500000000000000, 7.78470169371761],
 [0.0600000000000000, 7.95240781363385],
 [0.0700000000000000, 8.12387325556636],
 [0.0800000000000000, 8.29917408791036],
 [0.0900000000000000, 8.47838788116439],
 [0.100000000000000, 8.66159373810067],
 [0.110000000000000, 8.84887232454284],
 [0.120000000000000, 9.04030590076340],
 [0.130000000000000, 9.23597835351313],
 [0.140000000000000, 9.43597522869523],
 [0.150000000000000, 9.64038376469703],
 [0.160000000000000, 9.84929292639247],
 [0.170000000000000, 10.0627934398287],
 [0.180000000000000, 10.2809778276104],
 [0.190000000000000, 10.5039404449966],
 [0.200000000000000, 10.7317775167224],
 [0.210000000000000, 10.9645871745624],
 [0.220000000000000, 11.2024694956485],
 [0.230000000000000, 11.4455265415598],
 [0.240000000000000, 11.6938623981964],
 [0.250000000000000, 11.9475832164563],
 [0.260000000000000, 12.2067972537300],
 [0.270000000000000, 12.4716149162280],
 [0.280000000000000, 12.7421488021605],
```

```
[0.290000000000000, 13.0185137457841],
[0.300000000000000, 13.3008268623331],
[0.310000000000000, 13.5892075938554],
[0.320000000000000, 13.8837777559671],
[0.330000000000000, 14.1846615855483],
[0.340000000000000, 14.4919857893958],
     [...       OMITTED       ...]
[0.890000000000001, 46.3695278742487],
[0.900000000000001, 47.3445818806724],
[0.910000000000001, 48.3395693566862],
[0.920000000000001, 49.3548890813445],
[0.930000000000001, 50.3909478154718],
[0.940000000000001, 51.4481604617839],
[0.950000000000001, 52.5269502282151],
[0.960000000000001, 53.6277487945162],
[0.970000000000001, 54.7509964821898],
[0.980000000000001, 55.8971424278273],
[0.990000000000001, 57.0666447599179],
[1.00000000000000, 58.2599707791977],
[1.01000000000000, 59.4775971426105]]
```

To plot the points obtained from calling the `eulers_method()` we could use the following code:

```
x,y=PolynomialRing(QQ,2,"xy").gens()
a =0
y0=7
b =1
h =0.01
points = eulers_method(2*y+3*sin(x)+e^(x),a,y0,h,b, algorithm =
'none') list_plot(points)
```

to obtain



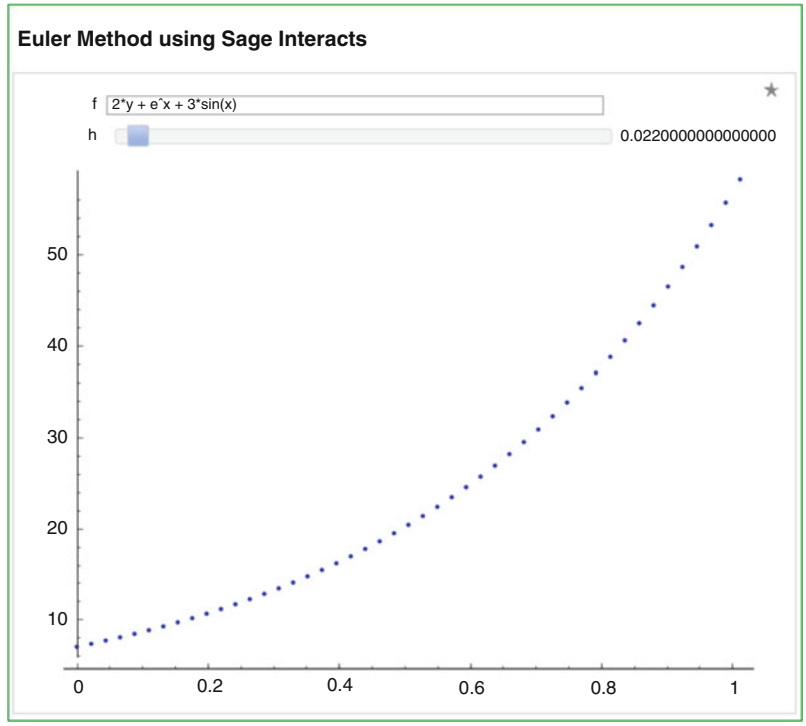
7.2.4 Sage Interacts

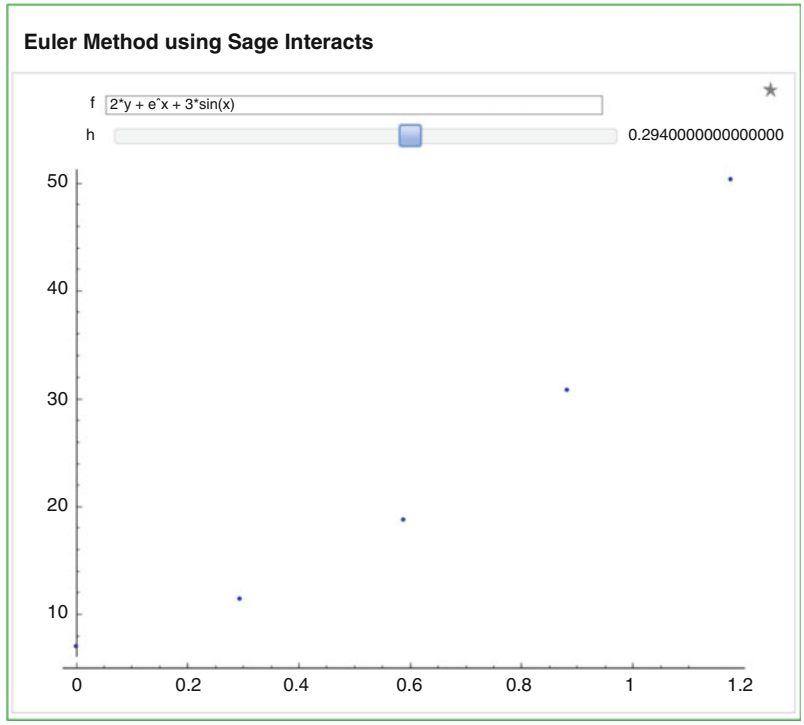
Below, we put the previous Sage code into a Sage interact:

```
html('<h2>Euler Method using Sage Interacts</h2>')
var('x y')

@interact
def taylor_through_Sage_interaction(
          f = input_box(default= 2*y+3*sin(x)+e^(x)),
          h = slider(0.001,0.5,0.001,0.25) ):
    x,y=PolynomialRing(QQ,2,"xy").gens()
    a =0
    y0=7
    b =1
    points = eulers_method(f,a,y0,h,b, algorithm = 'none')
    p = list_plot(points)
    show(p)
```

and obtained
and





7.2.5 *Error Bounds*

There are two error sources we should be concerned about, and they both are independent from the use of computers in this process (which introduces yet another type of errors). They are called the **truncation error**, and the **propagated error**. Let us denote by $\overline{y}$ the true, exact solution.

The **truncation error** is coming from using the Taylor formula with $n = 1$ (see (7.4)) at each step.

$$\overline{y}(x_{i+1}) = \overline{y}(x_i) + hf(x_i, \overline{y}(x_i)) + E_T$$

If f is two times continuously differentiable (see also (1.29), on page 54) then this error is

$$\begin{aligned} E_T &= \frac{f''(\xi)}{2}h^2, \text{ with } x_n \leq \xi \leq x_{n+1} \\ &\simeq \mathcal{O}(h^2). \end{aligned}$$

Hence, each time we approximated $y(x_{i+1})$ by $y(x_i) + hf(x_i, y(x_i))$, we committed a truncation error.

The **propagated error** comes from the accumulated errors given by the truncation errors obtained at each step. When computing $y(x_{i+1})$, we used $y(x_i)$ which already contains errors due truncation errors. It can be shown (see [1, p. 387]) that at each step, this error is

$$\begin{aligned} E_P &= f(x_i, \overline{y}(x_i)) - f(x_i, y(x_i)) \\ &\simeq \mathcal{O}(h). \end{aligned}$$

Hence, overall, as $h \to 0$ (or equivalently, as $n \to \infty$) we have a linear convergence.

$$\overline{y}(x_i) - y(x_i) = \mathcal{O}(h).$$

In our previous example, when we used Euler's Method to solve

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases},$$

we had the following errors:

Case $n = 5$, hence $h = 0.2$:

```
y(0.000000000000000) =  7.00000000000000  actual error =  0
y(0.200000000000000) =  10.0000000000000  actual error =  0.781846497895935
y(0.400000000000000) =  14.3634821501091  actual error =  2.26440653011277
y(0.600000000000000) =  20.6408909550661  actual error =  4.91722344305780
y(0.800000000000000) =  29.6004565812077  actual error =  9.49123000541007
y(1.00000000000000) =  42.3161610539290  actual error =  17.1774930031252
```

Note how the errors accumulate as you go down the list of computations!

Case $n = 10$, hence $h = 0.1$:

```
y(0.000000000000000) =  7.00000000000000   actual error =  0
y(0.100000000000000) =  8.50000000000000   actual error =  0.182090202958805
y(0.200000000000000) =  10.3404671168016   actual error =  0.441379381094323
y(0.300000000000000) =  12.5903016152165   actual error =  0.802235119096930
y(0.400000000000000) =  15.3320038810158   actual error =  1.29588479920607
y(0.500000000000000) =  18.6644126296756   actual error =  1.96222964091275
y(0.600000000000000) =  22.7059949442620   actual error =  2.85211945386190
y(0.700000000000000) =  27.5987985551720   actual error =  4.03020191516578
y(0.800000000000000) =  33.5131988431248   actual error =  5.57848774349302
y(0.900000000000000) =  40.6535995318688   actual error =  7.60080727841018
y(1.00000000000000) =  49.2652778222465   actual error =  10.2283762348076
```

and when $n = 20$, hence $h = 0.05$:

```
y(0.000000000000000) =  7.00000000000000   actual error =  0
y(0.0500000000000000) =  7.75000000000000   actual error =  0.0439736397127519
y(0.100000000000000) =  8.58506043020940   actual error =  0.0970297727494013
y(0.150000000000000) =  9.51380003163115   actual error =  0.160563065064250
y(0.200000000000000) =  10.5456874668017   actual error =  0.236159031094215
y(0.250000000000000) =  11.6911267510092   actual error =  0.325618556192389
y(0.300000000000000) =  12.9615512908326   actual error =  0.430985443480764
y(0.350000000000000) =  14.3695273912939   actual error =  0.554577347703969
y(0.400000000000000) =  15.9288681789713   actual error =  0.699020501250567
y(0.450000000000000) =  17.6547589830968   actual error =  0.867288685017754
y(0.500000000000000) =  19.5638953207976   actual error =  1.06274694979076
y(0.550000000000000) =  21.6746347472030   actual error =  1.28920065291640
y(0.600000000000000) =  24.0071639571563   actual error =  1.55095044096764
y(0.650000000000000) =  26.5836826639007   actual error =  1.85285388259820
y(0.700000000000000) =  29.4286059326019   actual error =  2.20039453773590
y(0.750000000000000) =  32.5687868143213   actual error =  2.59975934062112
y(0.800000000000000) =  36.0337613105875   actual error =  3.05792527603027
y(0.850000000000000) =  39.8560179017058   actual error =  3.58275644152276
y(0.900000000000000) =  44.0712940952438   actual error =  4.18311271503526
y(0.950000000000000) =  48.7189026967701   actual error =  4.86897138809019
y(1.00000000000000) =  53.8420907751313   actual error =  5.65156328192288
```

7.3 Taylor Series Method

Euler's Method is just a particular case of a more general algorithm, called Taylor Series Method.

In Euler's Method, we used Taylor series formula, with $N = 1$ (see (1.29), on page 54) to derive

$$y(x+h) \simeq y(x) + hy'(x)$$

hence

$$y(x+h) = y(x) + hf(x,y). \tag{7.10}$$

We can use Taylor series with N larger than one. For example, we can go all the way up to $N = 5$ to obtain more accurate formulae.

Let us see how **Taylor Series Method** works for $N = 5$. From (1.29), on page 54) we have

$$y(x+h) \simeq y(x) + hy'(x) + \frac{h^2}{2}y''(x) + \frac{h^3}{3!}y'''(x) + \frac{h^4}{4!}y^{(4)}(x) + \frac{h^5}{5!}y^{(5)}(x), \tag{7.11}$$

giving us the following algorithm:

7.3.1 Algorithm

We will create a table of values for y, at the nodes

$$x_0 = a, x_1 = a + h, x_2 = a + 2h, \ldots, x_n = b, \tag{7.12}$$

where

$$h = \frac{b-a}{n}. \tag{7.13}$$

Using (7.3) and (7.5) we have

$$\begin{aligned} y(x_0) &= y_0, \\ y(x_{i+1}) &= y(x_i) + y'(x_i) + \frac{h^2}{2}y''(x_i) + \frac{h^3}{3!}y'''(x_i) \\ &\quad + \frac{h^4}{4!}y^{(4)}(x_i) + \frac{h^5}{5!}y^{(5)}f(x_i), \\ \text{for } i &= 1, 2, \ldots, n, \end{aligned} \tag{7.14}$$

hence obtaining the table

x	y
x_0	y_0
x_1	$y(x_1)$
...	...
x_n	$y(x_n)$

(7.15)

Example 140. Using Taylor Series method, with $N = 5$, compute an approximate value for $y(1)$, with y the solution for the following differential equation

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases}.$$

using $n = 5$ steps.

Solution. We have that $a = 0, b = 1$, and

$$\begin{aligned} y' &= 2y + 3\sin(x) + e^x \\ y'' &= 2y' + 3\cos(x) + e^x \\ y''' &= 2y'' - 3\sin(x) + e^x \\ y^{(4)} &= 2y''' - 3\cos(x) + e^x \\ y^{(5)} &= 2y^{(4)} + 3\sin(x) + e^x \end{aligned}$$

using (7.12), (7.13), and (7.14) we obtain

$$\begin{aligned} y(0) &= 7 \\ y(0.2) &= 10.757288 \\ y(0.4) &= 16.5759433020146 \\ y(0.6) &= 25.4741965479400 \\ y(0.8) &= 38.9661958471361 \\ y(1) &= 59.3063724625059 \end{aligned}$$

Hence $y(1) \simeq 42.3161610539290$.

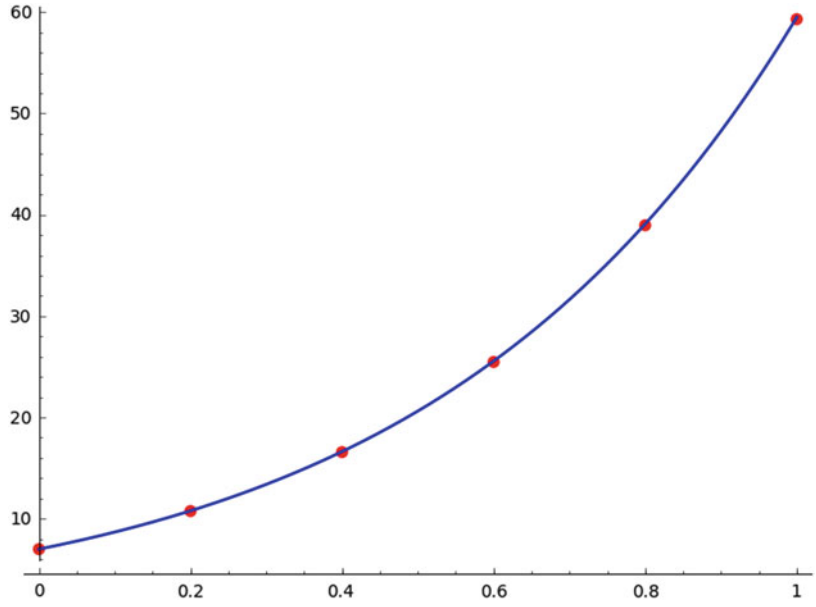
Compare these values with the ones obtained from using the actual solution

$$y = -\frac{3}{5}(\cos(x) + 2\sin(x)) - e^x + \frac{43}{5}e^{2x}$$

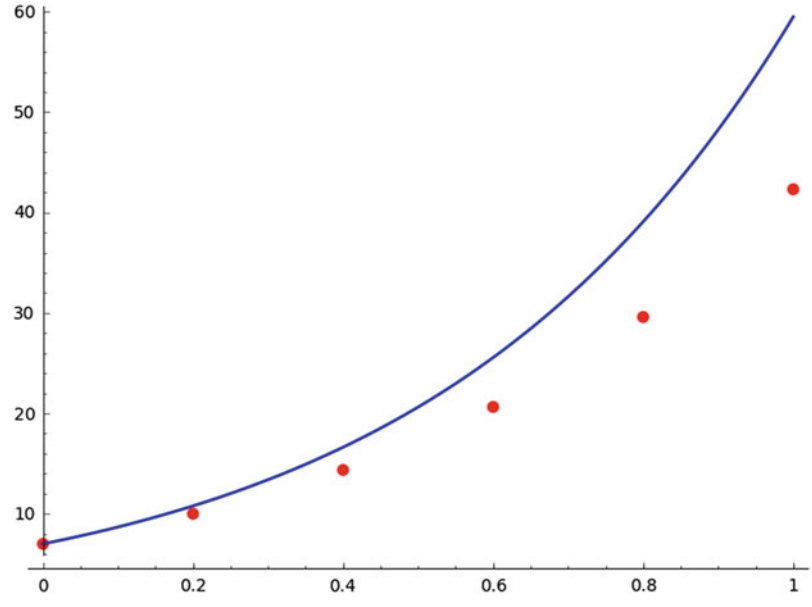
$$\begin{aligned} y(0) &= 7 \\ y(0.2) &= 10.7818464978959 \end{aligned}$$

$$\begin{aligned} y(0.4) &= 16.6278886802218 \\ y(0.6) &= 25.5581143981239 \\ y(0.8) &= 39.0916865866178 \\ y(1) &= 59.4936540570542 \end{aligned}$$

If we plot the exact solution and the table obtained above, we would get



Notice what a great improvement compared to Euler's Method that gave us the following plot:



7.3.2 Sage Code

This part is left as an exercise.

Example 141. Using Taylor's method, with $N = 5$, compute an approximate value for $y(1)$ for the following differential equation

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases}.$$

using $n = 5$ steps. Also give the error of approximation for each node.

Solution. The Sage code is left as a homework. We obtain

```
y(0.000000000000000) = 7.00000000000000   error = 0.000000000000000
y(0.200000000000000) = 10.7572880000000   error = 0.0245584978959342
y(0.400000000000000) = 16.5759433020146   error = 0.0519453782072922
y(0.600000000000000) = 25.4741965479400   error = 0.0839178501839517
y(0.800000000000000) = 38.9661958471361   error = 0.125490739481717
y(1.00000000000000)  = 59.3063724625059   error = 0.187281594548264
```

Example 142. Using Taylor's method, with $N = 5$, compute an approximate value for $y(1)$ for the following differential equation

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases}.$$

using $n = 100$ steps. Also give the error of approximation for each node.

Solution. The Sage code is left as a homework. We obtain

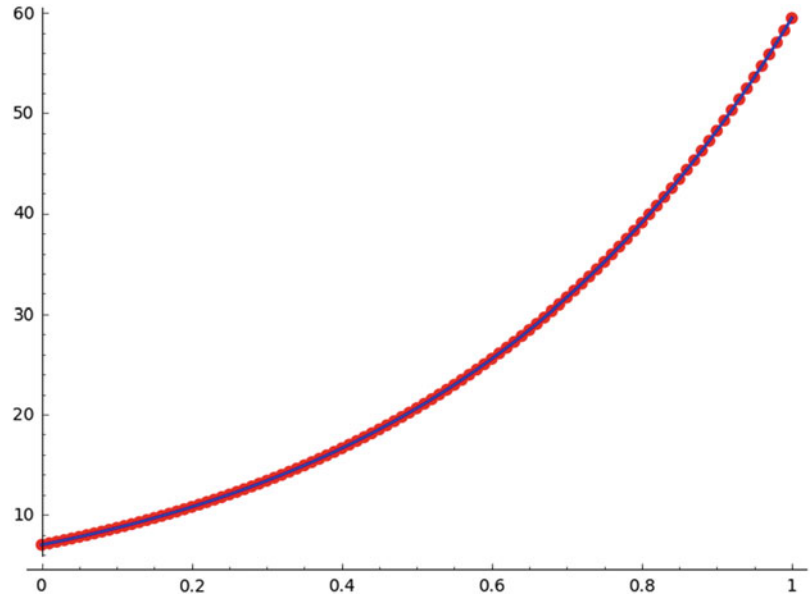
```
y(0.000000000000000) = 7.00000000000000   error = 0.000000000000000
y(0.0100000000000000) = 7.15166105396500   error = 0.0000505029299312509
y(0.0200000000000000) = 7.30679189783291   error = 0.000101016162930123
y(0.0300000000000000) = 7.46546361552791   error = 0.000151530316279569
y(0.0400000000000000) = 7.62774870660780   error = 0.000202036183907572
y(0.0500000000000000) = 7.79372111496462   error = 0.000252524748135086
y(0.0600000000000000) = 7.96345625810909   error = 0.000302987191725723
y(0.0700000000000000) = 8.13703105705068   error = 0.000353414910254091
y(0.0800000000000000) = 8.31452396678522   error = 0.000403799524804738
y(0.0900000000000000) = 8.49601500740261   error = 0.000454132894933568
y(0.100000000000000) = 8.68158579582678   error = 0.000504407132023155
y(0.110000000000000) = 8.87131957820105   error = 0.000554614612922499
y(0.120000000000000) = 9.06530126293170   error = 0.000604747993978805
y(0.130000000000000) = 9.26361745440318   error = 0.000654800225330376
y(0.140000000000000) = 9.46635648737840   error = 0.000704764565665039
y(0.150000000000000) = 9.67360846209816   error = 0.000754634597242565
y(0.160000000000000) = 9.88546528009366   error = 0.000804404241290690
y(0.170000000000000) = 10.1020206807265   error = 0.000854067773913503
y(0.180000000000000) = 10.3233702784711   error = 0.000903619842153147
y(0.190000000000000) = 10.5496116009543   error = 0.000953055480623743
y(0.200000000000000) = 10.7808441277675   error = 0.00100237012846160
y(0.210000000000000) = 11.0171693300670   error = 0.00105155964671011
y(0.220000000000000) = 11.2586907109786   error = 0.00110062033608682
y(0.230000000000000) = 11.5055138468224   error = 0.00114954895519548
```

```
y(0.240000000000000) = 11.7577464291745   error = 0.00119834273917008
y(0.250000000000000) = 12.0154983077828   error = 0.00124699941869721
y(0.260000000000000) = 12.2788815343538   error = 0.00129551723958921
y(0.270000000000000) = 12.5480104072277   error = 0.00134389498271226
y(0.280000000000000) = 12.8230015169612   error = 0.00139213198441090
y(0.290000000000000) = 13.1039737928343   error = 0.00144022815743161
y(0.300000000000000) = 13.3910485503011   error = 0.00148818401228912
y(0.310000000000000) = 13.6843495394037   error = 0.00153600067909099
      [...      OMITTED      ...]
y(0.850000000000000) = 43.4344605207091   error = 0.00431382251949236
y(0.860000000000000) = 44.3585244195329   error = 0.00438612316260789
y(0.870000000000000) = 45.3016916335822   error = 0.00446029509808016
y(0.880000000000000) = 46.2643481117555   error = 0.00453643249167290
y(0.890000000000000) = 47.2468876029601   error = 0.00461463265753442
y(0.900000000000000) = 48.2497118141456   error = 0.00469499613341662
y(0.910000000000000) = 49.2732305715357   error = 0.00477762675683974
y(0.920000000000000) = 50.3178619851199   error = 0.00486263174334284
y(0.930000000000000) = 51.3840326164738   error = 0.00495012176595783
y(0.940000000000000) = 52.4721776499729   error = 0.00504021103691904
y(0.950000000000000) = 53.5827410674708   error = 0.00513301738945549
y(0.960000000000000) = 54.7161758265093   error = 0.00522866236334968
y(0.970000000000000) = 55.8729440421336   error = 0.00532727129045996
y(0.980000000000000) = 57.0535171723850   error = 0.00542897338254633
y(0.990000000000000) = 58.2583762075448   error = 0.00553390182098651
y(1.00000000000000) = 59.4880118632060   error = 0.00564219384817183
```

Hence, $y(1) \simeq 59.4880118632060$
Using $n = 200$, we obtain

```
y(1.00000000000000) = 59.4908900310364 error = 0.00276402601785891
```

We can plot the values in the previous table to obtain the following graph



where the continuous line represents the actual solution, and the dots represent the estimates obtained using Taylor Series Method. Compare this to the graph we obtained from the Euler's Method.

7.4 Runge–Kutta's Method

One of the most popular methods for solving initial-value problems is Runge–Kutta's Method. It is designed to imitate the Taylor Series Method but avoid the need to compute the time-consuming higher-order derivatives. These techniques were named after the German mathematicians Carl Runge and Martin Wilhelm Kutta around 1900.

Although the Runge–Kutta family of methods is very rich, we will only present the Runge–Kutta Method of Order 2, and the Runge–Kutta Method of Order 4 (also called the "classic" or "original" Runge–Kutta method).

7.4.1 Runge–Kutta Method of Order 2

This method will need two function evaluations

$$k_1 = hf(x, y)$$
$$k_2 = hf(x + \alpha h, y + \beta k_1)$$

and then the formula will be given by adding a linear combination of them to the value of $y(x)$ to obtain the value at $y(x + h)$

$$y(x + h) = y(x) + \gamma k_1 + \delta k_2, \tag{7.16}$$

with $\alpha, \beta, \gamma, \delta$—constants to be determined next.

Note: if we choose $\gamma = 1, \delta = 0$, then we obtain again the Euler's Method. But we can do better than that.

The constants $\alpha, \beta, \gamma, \delta$ will be determined so that (7.16) will be as accurate as possible. Using Taylor Series we obtain

$$y(x + h) = y(x) + hy'(x) + \frac{h^2}{2} y''(x) + \mathcal{O}(h^3) \tag{7.17}$$

and

$$f(x + \alpha h, y + \beta h f(x, y)) = f(x, y) + \alpha h \frac{\partial}{\partial x} f(x, y) + \beta h f(x, y) \frac{\partial}{\partial y} f(x, y) + \mathcal{O}(h^2) \tag{7.18}$$

hence, from (7.16), (7.17), (7.18), and using $y'(x) = f(x, y)$, we obtain

$$y(x) + hy'(x) + \frac{h^2}{2} y''(x) + \mathcal{O}(h^3) = y(x) + (\gamma + \delta) hf(x, y) + \delta \alpha h^2 \frac{\partial}{\partial x} f(x, y)$$
$$+ \delta \beta h^2 f(x, y) \frac{\partial}{\partial y} f(x, y) + \mathcal{O}(h^3)$$

From here, we deduce

$$\begin{cases} \gamma + \delta = 1 \\ \delta\alpha = \frac{1}{2} \\ \delta\beta = \frac{1}{2} \end{cases}.$$

This system has infinitely many solutions. Some of the most common solutions are

$$\begin{cases} \alpha = 1 \\ \beta = 1 \\ \gamma = \frac{1}{2} \\ \delta = \frac{1}{2} \end{cases} \text{ also called Heun's method,}$$

$$\begin{cases} \alpha = \frac{2}{3} \\ \beta = \frac{2}{3} \\ \gamma = \frac{1}{4} \\ \delta = \frac{3}{4} \end{cases} \tag{7.19}$$

and

$$\begin{cases} \alpha = \frac{1}{2} \\ \beta = \frac{1}{2} \\ \gamma = 0 \\ \delta = 1 \end{cases}, \text{ also called Midpoint method.}$$

From the way we derived this formula, see (7.17), we can see that the truncation error has order $\mathcal{O}(h^3)$.

7.4.2 Runge–Kutta Method of Order 4

This method will need four function evaluations

$$\begin{aligned} k_1 &= hf(x, y) \\ k_2 &= hf\left(x + \frac{1}{2}h, y + \frac{1}{2}k_1\right) \\ k_3 &= hf\left(x + \frac{1}{2}h, y + \frac{1}{2}k_2\right) \\ k_4 &= hf\left(x + h, y + k_3\right) \end{aligned}$$

and then the formula will be given by adding the following linear combination of them to the value of $y(x)$ to obtain the value at $y(x + h)$:

$$y(x + h) = y(x) + \frac{1}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{7.20}$$

In this case, it can be shown that the truncation error has order $\mathcal{O}(h^5)$, while the total accumulated error has order $\mathcal{O}(h^4)$.

7.4.3 Sage Code

We present, below, the Sage implementation for each of the Runge–Kutta Methods presented above, applied to the initial-value problem

$$\begin{cases} y' = 2y + 3\sin(x) + e^x \\ y(0) = 7 \end{cases}. \tag{7.21}$$

Example 143. Using Heun's method (Runge–Kutta's Method of order 2), compute an approximate value for $y(1)$ for the differential equation (7.21) using $n = 5$ steps. Also give the error of approximation for each node.

Solution. Using the Sage code

```
def HeunMethod(a,b,n,f, y0):
    h = (b-a)/n
    xi = a
    yi = y0

    #compute the actual solution
    yy = function('yy',x)
    TrueAnswer(x) = desolve( diff(yy,x) == f(x,yy), yy, ics=[a,y0])

    #print the first line
    print "y("+str(xi.n()) +") = ", yi.n() , "\terror = ", \
                                    (TrueAnswer(xi)-yi).n()

    #apply the method to compute each line in the table
    for i in [1,2,..,n]:
        k1 = h*f(xi,yi)
        k2 = h*f(xi+h,yi+k1)

        yi =yi + k1/2+k2/2
        xi=xi+h
        print "y("+str(xi.n()) +") = ", yi.n() , "\terror = ", \
                                    (TrueAnswer(xi)-yi).n()

#set up the problem
a = 0; b = 1; n = 5
f(x,y) = 2*y+3*sin(x)+e^(x)
y0 = 7

#call the function for Heun's Method
HeunMethod(a,b,n,f, y0)
```

we obtain

```
y(0.000000000000000) = 7.00000000000000   error = 0.000000000000000
y(0.200000000000000) = 10.6817410750545   error = 0.100105422841400
y(0.400000000000000) = 16.3294222686138   error = 0.298466411608058
y(0.600000000000000) = 24.8915607410454   error = 0.666553657078574
y(0.800000000000000) = 37.7695172877468   error = 1.32216929887095
y(1.00000000000000) = 57.0360203523203   error = 2.45763370473384
```

Example 144. Using Midpoint method (Runge–Kutta's Method of order 2), compute an approximate value for $y(1)$ for the differential equation (7.21) using $n = 5$ steps. Also give the error of approximation for each node.

Solution. Using the Sage code

```
def MidpointMethod(a,b,n,f, y0):
    h = (b-a)/n
    xi = a
    yi = y0

    #compute the actual solution
    yy = function('yy',x)
    TrueAnswer(x) = desolve( diff(yy,x) == f(x,yy), yy, ics=[a,y0])

    #print the first line
    print "y("+str(xi.n()) +") = ", yi.n() , "\terror = ", \
                                    (TrueAnswer(xi)-yi).n()

    #apply the method to compute each line in the table
    for i in [1,2,..,n]:
        k1 = h*f(xi,yi)
        k2 = h*f(xi+h/2,yi+k1/2)

        yi =yi+k2
        xi=xi+h
        print "y("+str(xi.n()) +") = ", yi.n() , "\terror = ", \
                                    (TrueAnswer(xi)-yi).n()

#set up the problem
a = 0; b = 1; n = 5
f(x,y) = 2*y+3*sin(x)+e^(x)
y0 = 7

#call the function for Midpoint Method
MidpointMethod(a,b,n,f, y0)
```

we obtain

```
y(0.000000000000000) = 7.00000000000000   error = 0.000000000000000
y(0.200000000000000) = 10.6809342336032   error = 0.100912264292709
y(0.400000000000000) = 16.3277629812666   error = 0.300125698955248
y(0.600000000000000) = 24.8888919785598   error = 0.669222419564151
y(0.800000000000000) = 37.7654831309282   error = 1.32620345568956
y(1.00000000000000)  = 57.0299361698293   error = 2.46371788722488
```

Example 145. Using Runge–Kutta's Method of order 4, compute an approximate value for $y(1)$ for the differential equation (7.21) using $n = 5$ steps. Also give the error of approximation for each node.

Solution. Using the Sage code

```
def RungeKuttaOrder4(a,b,n,f, y0):
    h = (b-a)/n
    xi = a
    yi = y0

    #compute the actual solution
```

```
    yy = function('yy',x)
    TrueAnswer(x) = desolve( diff(yy,x) == f(x,yy), yy, ics=[a,y0])

    print "y("+str(xi.n()) +")=", yi.n() , ", error=", \
                            (TrueAnswer(xi)-yi).n()


    #apply the method
    for i in [1,2,..,n]:
        k1 = h*f(xi,yi)
        k2 = h*f(xi+h/2,yi+k1/2)
        k3 = h*f(xi+h/2,yi+k2/2)
        k4 = h*f(xi+h  ,yi+k3  )

        yi =yi+(k1+2*k2+2*k3+k4)/6
        xi=xi+h
        print "y("+str(xi.n()) +")=", yi.n() , ", error=", \
                            (TrueAnswer(xi)-yi).n()


#set up the problem
a = 0; b = 1; n = 5
f(x,y) = 2*y+3*sin(x)+e^(x)
y0 = 7


#call the function for RungeKutta's Method of order 4
RungeKuttaOrder4(a,b,n,f, y0)
```

we obtain

```
y(0.000000000000000)= 7.00000000000000, error= 0.000000000000000
y(0.200000000000000)= 10.7810735350155, error= 0.000772962880466022
y(0.400000000000000)= 16.6255751950963, error= 0.00231348512556406
y(0.600000000000000)= 25.5529272101886, error= 0.00518718793531631
y(0.800000000000000)= 39.0813557670383, error= 0.0103308195794800
y(1.00000000000000) = 59.4743733524585, error= 0.0192807045956751
```

7.5 Systems of Ordinary Differential Equations

7.5.1 Taylor Series Method

Taylor Series Method can be used to solve not only an initial-value problem of an ordinary differential equation, but also systems of such equations. Suppose we are trying to solve the following system

$$\begin{cases} y_1'(x) = f_1(x, y_1, y_2, \ldots, y_m) \\ y_2'(x) = f_2(x, y_1, y_2, \ldots, y_m) \\ \qquad\qquad \ldots \\ y_m'(x) = f_m(x, y_1, y_2, \ldots, y_m) \end{cases},$$

with the initial conditions

$$\begin{cases} y_1(a) = y_1^0 \\ y_2(a) = y_2^0 \\ \quad \cdots \\ y_m(a) = y_m^0 \end{cases}.$$

Then, we can use Taylor Polynomial of Order n :

$$Y(x+h) \simeq Y(x) + hY'(x) + \frac{h^2}{2}Y''(x) + \cdots + \frac{h^n}{n!}Y^{(n)}(x)$$

to obtain an approximation for the next iteration.

Example 146. For the system of differential equations

$$\begin{cases} y'(x) = 2y + 3\sin(x) + e^x \\ z'(x) = 4z + \cos(z) - x^2 \\ y(1) = 7, z(1) = 1.2 \end{cases}$$

find the values of $y(2)$, and $z(2)$, using Taylor series Method of order $N = 4$, with $n = 5$ steps.

Solution. We have

$$\begin{cases} y'(x) = 2y + 3\sin(x) + e^x \\ z'(x) = 4z + \cos(z) - x^2 \end{cases}$$

Therefore

$$\begin{cases} y''(x) = 2y' + 3\cos(x) + e^x \\ z''(x) = 4z' - \sin(z)z' - 2x \end{cases},$$

$$\begin{cases} y'''(x) = 2y'' - 3\sin(x) + e^x \\ z'''(x) = 4z'' - \cos(z)\,(z')^2 - \sin(z)z'' - 2 \end{cases},$$

and

$$\begin{cases} y^{(4)}(x) = 2y''' - 3\cos(x) + e^x \\ z^{(4)}(x) = 4z''' + \sin(z)\,(z')^3 - 3\cos(z)z'z'' - \sin(z)z''' \end{cases},$$

then, starting with

$$\begin{cases} y(1) = 7 \\ z(1) = 1.2 \end{cases},$$

and using

$$Y(x+h) \simeq Y(x) + hY'(x) + \frac{h^2}{2}Y''(x) + \frac{h^3}{6}Y'''(x) + \frac{h^4}{24}Y^{(4)}(x),$$

we have

$$\begin{cases} y(x_{i+1}) \simeq y(x_i) + hy'(x_i) + \frac{h^2}{2}y''(x_i) + \frac{h^3}{6}y'''(x_i) + \frac{h^4}{24}y^{(4)}(x_i) \\ z(x_{i+1}) \simeq z(x_i) + hz'(x_i) + \frac{h^2}{2}z''(x_i) + \frac{h^3}{6}z'''(x_i) + \frac{h^4}{24}z^{(4)}(x_i) \end{cases},$$

for $i = 0, 1, 2, 3, 4$, hence we obtain the following

$$\begin{cases} y(1.2) = 2.28606910353070 \\ z(1.2) = 0.596892537878381 \end{cases}$$

$$\begin{cases} y(1.4) = 4.75699820755308 \\ z(1.4) = -1.24492429778426 \end{cases}$$

$$\begin{cases} y(1.6) = 18.3631941512904 \\ z(1.6) = -6.52269434184452 \end{cases}$$

$$\begin{cases} y(1.8) = 1217.15058952107 \\ z(1.8) = -593.064833163811 \end{cases}$$

$$\begin{cases} y(2) = 17617714583.4531 \\ z(2) = -6304669145.88529 \end{cases}.$$

We can also implement this in Sage as follows:

```
def PrintSystemTaylorN3(a,b,n,yp,y2p,y3p,zp,z2p,z3p,y0, z0):
    #set up the initial values
    h = (b-a)/n
    xi = a
    yi = y0
    zi = z0
    #print the first line of the table
    print "y("+str(xi.n()) +") = ", yi.n()
    print "   and z("+str(xi.n()) +") = ", zi.n()

    for i in [1,2,..,n]:
        #compute the next value of yi
        new_yi= yi+h*yp(xi,yi,zi)+h^2/2*y2p(xi,yi,zi)+h^3/6*y3p(xi,yi,zi)
        new_zi= zi+h*zp(xi,yi,zi)+h^2/2*z2p(xi,yi,zi)+h^3/6*z3p(xi,yi,zi)
        yi = new_yi
        zi = new_zi

        #prepare for the next x
        xi = xi + h

        #print the next line of the table
        print "y("+str(xi.n()) +") = ", yi.n()
        print "   and z("+str(xi.n()) +") = ", zi.n()
#set up the problem
a = 1; b = 2; n = 5

#the functions y',y'' and y'''
#the functions z',z'' and z'''
def yp(x,y,z):
    return y^2+z+x^3
```

```
def zp(x,y,z):
    return e^(z)+cos(y)-(y+z)^2

def y2p(x,y,z) :
    return 2 * y* yp(x,y,z)+zp(x,y,z)+3*x^2

def z2p(x,y,z) :
    return e^(z)*zp(x,y,z)-sin(y)*yp(x,y,z)          \
               -2*(y+z)*(yp(x,y,z)+zp(x,y,z))

def y3p(x,y,z) :
    return 2*((yp(x,y,z))^2+y*y2p(x,y,z))+z2p(x,y,z)+6*x

def z3p(x,y,z) :
    return e^(z)*((zp(x,y,z))^2+z2p(x,y,z))-         \
          (cos(y)*(yp(x,y,z))^2+sin(y)*y2p(x,y,z))-  \
          2*((yp(x,y,z)+zp(x,y,z))^2+                \
          (y+z)*(y2p(x,y,z)+z2p(x,y,z)))

y0 = 1.2
z0 = 1.7
 #call the function for Euler's Method
PrintSystemTaylorN3(a,b,n,yp,y2p,y3p,zp,z2p,z3p,y0, z0)
```

to obtain

```
y(1.00000000000000) =  1.20000000000000
   and z(1.00000000000000) =  1.70000000000000
y(1.20000000000000) =  2.28606910353070
   and z(1.20000000000000) =  0.596892537878381
y(1.40000000000000) =  4.75699820755308
   and z(1.40000000000000) =  -1.24492429778426
y(1.60000000000000) =  18.3631941512904
   and z(1.60000000000000) =  -6.52269434184452
y(1.80000000000000) =  1217.15058952107
   and z(1.80000000000000) =  -593.064833163811
y(2.00000000000000) =  1.76177145834531e10
   and z(2.00000000000000) =  -6.30466914588529e9
```

Example 147. For the system of differential equations

$$\begin{cases} y'(x) = y^2 + \ln(z) + x^3 \\ z'(x) = e^z + \cos(y) - (y+z)^2 \\ y(1) = 1.2, z(1) = 1.7 \end{cases}$$

find the values of $y(2)$, and $z(2)$, using Taylor series Method of order $N = 3$, with $n = 5$ steps.

Solution. We have

$$\begin{cases} y'(x) = y^2 + z + x^3 \\ z'(x) = e^z + \cos(y) - (y+z)^2 \end{cases}$$

Therefore

$$\begin{cases} y''(x) = 2yy' + z' + 3x^2 \\ z''(x) = e^z z' - \sin(y)y' - 2(y+z)(y'+z') \end{cases},$$

and

$$\begin{cases} y'''(x) = 2\left((y')^2 + yy''\right) + z'' + 6x \\ z'''(x) = e^z\left((z')^2 + z''\right) - \left(\cos(y)\,(y')^2 + \sin(y)\cdot y''\right) \\ \qquad -2\left((y'+z')^2 + (y+z)(y''+z'')\right) \end{cases},$$

then, starting with

$$\begin{cases} y(1) = 1.2 \\ z(1) = 1.7 \end{cases},$$

and using

$$Y(x+h) \simeq Y(x) + hY'(x) + \frac{h^2}{2}Y''(x) + \frac{h^3}{6}Y'''(x),$$

we have

$$\begin{cases} y(x_{i+1}) \simeq y(x_i) + hy'(x_i) + \frac{h^2}{2}y''(x_i) + \frac{h^3}{6}y'''(x_i) \\ z(x_{i+1}) \simeq z(x_i) + hz'(x_i) + \frac{h^2}{2}z''(x_i) + \frac{h^3}{6}z'''(x_i) \end{cases},$$

for $i = 1, 2, 3, 4, 5$, hence we obtain the following

$$\begin{cases} y(1.2) = 2.28606910353070 \\ z(1.2) = 0.596892537878381 \end{cases}$$

$$\begin{cases} y(1.4) = 4.75699820755308 \\ z(1.4) = -1.24492429778426 \end{cases}$$

$$\begin{cases} y(1.6) = 18.3631941512904 \\ z(1.6) = -6.52269434184452 \end{cases}$$

$$\begin{cases} y(1.8) = 1217.15058952107 \\ z(1.8) = -593.064833163811 \end{cases}$$

$$\begin{cases} y(2) = 17617714583.4531 \\ z(2) = -6304669145.88529 \end{cases}.$$

We can also implement this in Sage as follows:

```
def PrintSystemTaylorN2(a,b,n,yp,y2p,y3p,zp,z2p,z3p,y0, z0):
    #set up the initial values
    h = (b-a)/n
    xi = a
    yi = y0
    zi = z0
    #print the first line of the table
    print "y("+str(xi.n()) +") = ", yi.n()
```

```
    print "   and z("+str(xi.n()) +") = ", zi.n()

    for i in [1,2,..,n]:
        #compute the next value of yi
        new_yi= yi+h*yp(xi,yi,zi)+h^2/2*y2p(xi,yi,zi)+h^3/6*y3p(xi,yi,zi)
        new_zi= zi+h*zp(xi,yi,zi)+h^2/2*z2p(xi,yi,zi)+h^3/6*z3p(xi,yi,zi)
        yi = new_yi
        zi = new_zi

        #prepare for the next x
        xi = xi + h

        #print the next line of the table
        print "y("+str(xi.n()) +") = ", yi.n()
        print "   and z("+str(xi.n()) +") = ", zi.n()
#set up the problem
a = 1; b = 2; n = 5

#the functions y',y'' and y'''
#the functions z',z'' and z'''
def yp(x,y,z):
    return y^2+z+x^3

def zp(x,y,z):
    return e^(z)+cos(y)-(y+z)^2

def y2p(x,y,z) :
    return 2 * y* yp(x,y,z)+zp(x,y,z)+3*x^2

def z2p(x,y,z) :
    return e^(z)*zp(x,y,z)-sin(y)*yp(x,y,z)           \
               -2*(y+z)*(yp(x,y,z)+zp(x,y,z))

def y3p(x,y,z) :
    return 2*((yp(x,y,z))^2+y*y2p(x,y,z))+z2p(x,y,z)+6*x

def z3p(x,y,z) :
    return e^(z)*((zp(x,y,z))^2+z2p(x,y,z))-          \
          (cos(y)*(yp(x,y,z))^2+sin(y)*y2p(x,y,z))-   \
          2*((yp(x,y,z)+zp(x,y,z))^2+                 \
          (y+z)*(y2p(x,y,z)+z2p(x,y,z)))

y0 = 1.2
z0 = 1.7
 #call the function for Euler's Method
PrintSystemTaylorN2(a,b,n,yp,y2p,y3p,zp,z2p,z3p,y0, z0)
```

to obtain

```
y(1.00000000000000) =  1.20000000000000
   and z(1.00000000000000) =  1.70000000000000
y(1.20000000000000) =  2.28606910353070
   and z(1.20000000000000) =  0.596892537878381
y(1.40000000000000) =  4.75699820755308
   and z(1.40000000000000) =  -1.24492429778426
```

```
y(1.60000000000000) =  18.3631941512904
   and z(1.60000000000000) =  -6.52269434184452
y(1.80000000000000) =  1217.15058952107
   and z(1.80000000000000) =  -593.064833163811
y(2.00000000000000) =  1.76177145834531e10
   and z(2.00000000000000) =  -6.30466914588529e9
```

7.6 Higher-Order Ordinary Differential Equations

In this section we will present a way to solve initial-value problems for higher-order differential equations, by using systems of Ordinary Differential Equations.

To solve an equation of the form

$$\begin{cases} y^{(n)}(x) = f\left(x, y, y', y'', \ldots, y^{(n-1)}\right) \\ y(a), y'(a), y''(a), \ldots, y^{(n-1)}(a) \text{ given} \end{cases}, \tag{7.22}$$

we could first turn it into a system of first order differential equations, by defining the new variables

$$y_1 = y, y_2 = y', \ldots, y_n = y^{(n-1)},$$

therefore (7.22) becoming

$$\begin{cases} y_1' = y_2 \\ y_2' = y_3 \\ \ldots \\ y_{n-1}' = y_n \\ y_n'(x) = f\left(x, y_1, y_2, y_3, \ldots, y_n\right) \end{cases},$$

with the given initial conditions:

$$\begin{cases} y_1(a) = y(a) \\ y_2(a) = y'(a) \\ y_3(a) = y''(a) \\ \ldots, \\ y_n(a) = y^{(n-1)}(a) \end{cases},$$

and then we could simply apply the techniques we have already seen in the previous section.

Example 148. For the following higher order differential equation

$$\begin{cases} y'''(x) = 2y' + 3\sin(y) + e^{y''} + e^x \\ y(1) = 7, \\ y'(1) = 1.2, \\ y''(1) = -2 \end{cases}$$

find the value of the solution y at $x = 1.5$, using Taylor series Method of order $N = 2$, with $n = 5$ steps.

Solution. We need to find $y(1.5)$.

First, we turn the given problem into a system of first order differential equations. By defining the new variables

$$y_1 = y, y_2 = y', y_3 = y'',$$

we obtain

$$\begin{cases} y_1' = y_2 \\ y_2' = y_3 \\ y_3' = f\,(x, y_1, y_2, y_3) = 2y_2 + 3\sin(y_1) + e^{y_3} + e^x \end{cases},$$

with the given initial conditions:

$$\begin{cases} y_1(1) = 7 \\ y_2(1) = 1.2 \\ y_3(1) = -2 \end{cases}.$$

Then, we can use Taylor's Series Method of order $N = 2$, to solve it. For this, we will need to compute

$$\begin{cases} y_1''(x) = y_2'(x) \\ y_2''(x) = y_3'(x) \\ y_3''(x) = 2y_2'(x) + 3\cos(y_1(x)) \cdot y_1'(x) + y_3'(x)e^{y_3(x)} + e^x \end{cases}$$

Therefore using

$$Y(x+h) \simeq Y(x) + hY'(x) + \frac{h^2}{2}Y''(x),$$

we have

$$\begin{cases} y_1(x_{i+1}) \simeq y_1(x_i) + hy_1'(x_i) + \frac{h^2}{2}y_1''(x_i) \\ y_2(x_{i+1}) \simeq y_2(x_i) + hy_2'(x_i) + \frac{h^2}{2}y_2''(x_i) \\ y_3(x_{i+1}) \simeq y_3(x_i) + hy_3'(x_i) + \frac{h^2}{2}y_3''(x_i) \end{cases},$$

for $i = 0, 1, 2, 3, 4$, hence we obtain the following

$$\begin{cases} y_1(1.1) = 7.11 \\ y_2(1.1) = 1.03612288453926 \\ y_3(1.1) = -1.26549195868388 \end{cases}$$

$$\begin{cases} y_1(1.2) = 7.20728482866051 \\ y_2(1.2) = 0.947402919205352 \\ y_3(1.2) = -0.485344501650709 \end{cases}$$

$$\begin{cases} y_1(1.3) = 7.29959839807279 \\ y_2(1.3) = 0.939991685974457 \\ y_3(1.3) = 0.382740618947083 \end{cases}$$

$$\begin{cases} y_1(1.4) = 7.39551126976497 \\ y_2(1.4) = 1.02609701549618 \\ y_3(1.4) = 1.43909717879495 \end{cases}$$

$$\begin{cases} y_1(1.5) = 7.50531645720857 \\ y_2(1.5) = 1.23507909629775 \\ y_3(1.5) = 3.05642611369023 \end{cases},$$

and conclude that

$$y(1.5) \simeq y_1(1.5) = 7.50531645720857.$$

If we let Sage do the work for $n = 10$, we obtain:

```
y1(1.00000000000000) =  7.00000000000000
        y2(1.00000000000000) =  1.20000000000000
        y3(1.00000000000000) =  -2.00000000000000
y1(1.05000000000000) =  7.05750000000000
        y2(1.05000000000000) =  1.10903072113481
        y3(1.05000000000000) =  -1.63575856697467
y1(1.10000000000000) =  7.11090683784802
        y2(1.10000000000000) =  1.03645303783972
        y3(1.10000000000000) =  -1.26309872542805
y1(1.15000000000000) =  7.16115061633322
        y2(1.15000000000000) =  0.982759381368820
        y3(1.15000000000000) =  -0.878745078880483
y1(1.20000000000000) =  7.20919015405306
        y2(1.20000000000000) =  0.948631300008566
        y3(1.20000000000000) =  -0.478199612804905
y1(1.25000000000000) =  7.25602396953748
        y2(1.25000000000000) =  0.935015010996149
        y3(1.25000000000000) =  -0.0549781662309876
y1(1.30000000000000) =  7.30270599737950
        y2(1.30000000000000) =  0.943249027734137
        y3(1.30000000000000) =  0.400933616499468
y1(1.35000000000000) =  7.35036961578684
        y2(1.35000000000000) =  0.975301438649356
        y3(1.35000000000000) =  0.906531634991313
y1(1.40000000000000) =  7.40026785226304
        y2(1.40000000000000) =  1.03426712636244
        y3(1.40000000000000) =  1.49471562794438
y1(1.45000000000000) =  7.45384960311609
        y2(1.45000000000000) =  1.12560075704766
        y3(1.45000000000000) =  2.24312971247186
y1(1.50000000000000) =  7.51293355310907
        y2(1.50000000000000) =  1.26113239330543
        y3(1.50000000000000) =  3.41097550740710
```

concluding

$$y(1.5) \simeq y_1(1.5) = 7.51293355310907.$$

For $n = 100$, we obtain

$$y(1.5) \simeq y_1(1.5) = 7.51612031631261.$$

We leave the Sage implementation as a homework.

7.6.1 Sage Library

Instead of Taylor's Method, one can use other methods, to get more accurate answers.

Next, we show how we can use Sage library function to solve the previous example using Runge–Kutta Method of order 4.

Example 149. For the following higher order differential equation

$$\begin{cases} y'''(x) = 2y' + 3\sin(y) + e^{y''} + e^x \\ y(1) = 7, \\ y'(1) = 1.2, \\ y''(1) = -2 \end{cases}$$

find the value of the solution y at $x = 1.5$, using Runge–Kutta Method of order 4, with $n = 5$ steps.

Solution. We need to find $y(1.5)$. First, we turn the given problem into a system of first order differential equations. By defining the new variables

$$y_1 = y, y_2 = y', y_3 = y'',$$

we obtain

$$\begin{cases} y_1' = y_2 \\ y_2' = y_3 \\ y_3' = f\,(x, y_1, y_2, y_3) = 2y_2 + 3\sin(y_1) + e^{y_3} + e^x \end{cases},$$

with the given initial conditions:

$$\begin{cases} y_1(1) = 7 \\ y_2(1) = 1.2 \\ y_3(1) = -2 \end{cases}.$$

Next, we use the following Sage code to solve this problem:

```
#include the library
from Sage.calculus.desolvers import desolve_system_rk4

#declare the variables
var('y1 y2 y3 x')

#solve the system of equations using Runge Kuta 4
P=desolve_system_rk4(
         #specify the right hand side of each equation
         [y2,  y3,  2*y2*3*sin(y1)+e^(y3)+e^x],
```

```
            #specify the dependent variables
            [y1, y2,  y3],

            #specify the initial values for x, y1,y2,y3
            ics=[1, 7, 1.2, -2],

            #specify the independent variable
            ivar=x,

            #specify the value of h
            step = 0.1,

            #specify the value of b
            end_points=1.5)

#print all the values
# either use [ [x,y1,y2, y3] for x,y1,y2, y3 in P]
# or  simply use:
print(P)
```

and obtain

```
[[1, 7, 1.20000000000000, -2],
 [1.1, 7.11127216554, 1.03834128834, -1.22807586556],
 [1.2, 7.21029897501, 0.955852822027, -0.410207404994],
 [1.3, 7.30531028447, 0.95985149834, 0.518468982343],
 [1.4, 7.4057306258, 1.06892608707, 1.75113974135],
 [1.5, 7.52461476811, 1.35759828593, 5.7648996391]]
```

Hence we conclude

$$y(1.5) \simeq y_1(1.5) = 7.52461476811.$$

If we change $n = 50$, hence $h = 0.01$, we obtain:

```
[[1, 7, 1.20000000000000, -2],
 [1.01, 7.01190126496, 1.18037959255, -1.92404169641],
 [1.02, 7.02361012775, 1.16151996837, -1.84784253549],
 [1.03, 7.03513420835, 1.14342356623, -1.77139589307],
 [1.04, 7.0464811515, 1.12609290858, -1.69469166277],
 [1.05, 7.05765862774, 1.10953063627, -1.61771627584],
 [1.06, 7.06867433476, 1.09373954312, -1.54045270493],
 [1.07, 7.07953599907, 1.07872261056, -1.46288045015],
 [1.08, 7.09025137811, 1.06448304251, -1.38497550587],
 [1.09, 7.1008282626, 1.05102430071, -1.30671030586],
 [1.1, 7.11127447931, 1.03835014067, -1.22805364465],
 [1.11, 7.12159789421, 1.02646464857, -1.14897057208],
 [1.12, 7.13180641593, 1.01537227939, -1.06942225765],
 [1.13, 7.14190799966, 1.00507789648, -0.98936582072],
 [1.14, 7.15191065142, 0.995586813129, -0.908754121657],
 [1.15, 7.16182243283, 0.986904836333, -0.827535508081],
```

```
[1.16, 7.17165146621, 0.979038313434, -0.745653509266],
[1.17, 7.18140594028, 0.971994182066, -0.663046470102],
[1.18, 7.1910941163, 0.965780024127, -0.579647114217],
[1.19, 7.20072433482, 0.960404124513, -0.495382023506],
[1.2, 7.21030502301, 0.955875535545, -0.410171018281],
[1.21, 7.21984470271, 0.952204148173, -0.323926418521],
[1.22, 7.22935199917, 0.949400771255, -0.236552161772],
[1.23, 7.2388356507, 0.947477220504, -0.147942746998],
[1.24, 7.24830451921, 0.946446419024, -0.0579819654381],
[1.25, 7.25776760185, 0.946322511799, 0.033458631224],
[1.26, 7.26723404386, 0.947120997082, 0.126521589462],
[1.27, 7.27671315286, 0.948858878337, 0.221365822576],
[1.28, 7.28621441475, 0.951554841356, 0.318169398871],
[1.29, 7.29574751157, 0.95522946245, 0.417132994742],
[1.3, 7.30532234156, 0.959905455282, 0.518484208057],
[1.31, 7.31494904201, 0.965607966205, 0.622482998007],
[1.32, 7.32463801528, 0.972364931107, 0.729428619127],
[1.33, 7.33439995883, 0.980207511144, 0.8396685656],
[1.34, 7.34424590003, 0.989170630917, 0.953610262929],
[1.35, 7.35418723707, 0.999293651583, 1.07173658036],
[1.36, 7.36423578738, 1.01062122456, 1.19462676118],
[1.37, 7.37440384583, 1.02320439131, 1.32298520609],
[1.38, 7.38470425547, 1.03710202528, 1.45768192651],
[1.39, 7.39515049503, 1.05238276109, 1.59981084187],
[1.4, 7.40575678902, 1.06912763648, 1.7507762793],
[1.41, 7.41653824923, 1.08743381097, 1.91242581026],
[1.42, 7.42751106123, 1.10741997371, 2.08726281938],
[1.43, 7.4386927376, 1.12923452623, 2.27880417267],
[1.44, 7.45010247475, 1.15306858993, 2.49222094238],
[1.45, 7.46176167984, 1.17917803067, 2.73558257185],
[1.46, 7.47369479863, 1.20792402021, 3.02254873422],
[1.47, 7.48593073249, 1.2398571056, 3.37916388986],
[1.48, 7.49850559653, 1.2759260398, 3.86571421988],
[1.49, 7.51146941271, 1.31819567565, 4.68894162633],
[1.5, 7.52491343636, 1.37601034736, 11.3245065041]]
```

and conclude

$$y(1.5) \simeq y_1(1.5) = 7.52491343636.$$

7.7 Systems of Higher-Order Differential Equations

This section is similar to the previous one, in the sense that we can reduce a system of higher-order differential equations to a system of first order differential equations.

If we are given to solve a system of higher-order differential equations of the form

$$\begin{cases} y^{(n)}(x) = f\left(x, y, y', y'', \ldots, y^{(n-1)}, z, z', z'', \ldots, z^{(m-1)}\right) \\ z^{(m)}(x) = g\left(x, y, y', y'', \ldots, y^{(n-1)}, z, z', z'', \ldots, z^{(m-1)}\right) \end{cases}, \tag{7.23}$$

with

$$\begin{cases} y(a), y'(a), y''(a), \ldots, y^{(n-1)}(a) \\ z(a), z'(a), z''(a), \ldots, z^{(m-1)}(a) \end{cases}$$

all given, we first turn it into a system of first order differential equations, by defining the new variables

$$\begin{aligned} y_1 = y, y_2 = y', \ldots, y_n = y^{(n-1)}, \\ y_{n+1} = z, y_{n+2} = z', \ldots, y_{n+m} = z^{(m-1)} \end{aligned}$$

therefore (7.23) becomes

$$\begin{cases} y_1' = y_2 \\ y_2' = y_3 \\ \ldots \\ y_{n-1}' = y_n \\ y_n' = f\,(x, y_1, y_2, y_3, \ldots, y_{n+m}) \\ y_{n+1}' = y_{n+2} \\ y_{n+2}' = y_{n+3} \\ \ldots \\ y_{n+m-1}' = y_{n+m} \\ y_{n+m}' = g\,(x, y_1, y_2, y_3, \ldots, y_{n+m}) \end{cases},$$

with the given initial conditions:

$$\begin{cases} y_1(a) = y(a) \\ y_2(a) = y'(a) \\ y_3(a) = y''(a) \\ \ldots, \\ y_n(a) = y^{(n-1)}(a) \\ y_{n+1}(a) = z(a) \\ y_{n+2}(a) = z'(a) \\ \ldots, \\ y_{n+m}(a) = z^{(m-1)}(a) \end{cases},$$

and then we can apply the techniques we have already seen in the previous two sections.

NOTE: A similar work can be done for a system with more than two higher-order differential equations.

Example 150. For the following system of higher order differential equations

$$\begin{cases} y'''(x) = 3\sin(y) + e^{y''} + z' - z^4 + e^x \\ z''(x) = z' - y^4 + \cos(x) \\ y(1) = -2, \\ y'(1) = -1, \\ y''(1) = 1 \\ z(1) = -2 \\ z'(1) = -1 \end{cases}$$

find the values of the solutions y, and z, at $x = 1.5$, using Taylor series Method of order $N = 2$, with $n = 5$ steps.

Solution. We need to find $y(1.5)$, and $z(1.5)$.

First, we turn the given problem into a system of first order differential equations, by defining the new variables

$$y_1 = y, y_2 = y', y_3 = y'', y_4 = z, y_5 = z'$$

obtaining

$$\begin{cases} y_1' = y_2 \\ y_2' = y_3 \\ y_3' = f\ (x, y_1, y_2, y_3, y_4, y_4) = 3\sin(y_1) + e^{y_3} + y_5 - y_4^4 + e^x \\ y_4' = y_5 \\ y_5' = y_5 - y_1^4 + \cos(x) \end{cases},$$

with the given initial conditions:

$$\begin{cases} y_1(1) = -2 \\ y_2(1) = -1 \\ y_3(1) = 1 \\ y_4(1) = -2 \\ y_5(1) = -1 \end{cases}.$$

Then, we can use Taylor's Series Method of order, let's say, $N = 2$, to solve it. For this, we will need

$$\begin{cases} y_1'' = y_2' \\ y_2'' = y_3' \\ y_3'' = 3\cos(y_1)y_1' + y_3'e^{y_3} + y_5' - 4y_4^3y_4' + e^x \\ y_4'' = y_5' \\ y_5'' = y_5' - 4y_1^3y_1' - \sin(x). \end{cases}$$

Therefore using

$$Y(x + h) \simeq Y(x) + hY'(x) + \frac{h^2}{2}Y''(x),$$

we have

$$\begin{cases} y_1(x_{i+1}) \simeq y_1(x_i) + hy_1'(x_i) + \frac{h^2}{2}y_1''(x_i) \\ y_2(x_{i+1}) \simeq y_2(x_i) + hy_2'(x_i) + \frac{h^2}{2}y_2''(x_i) \\ y_3(x_{i+1}) \simeq y_3(x_i) + hy_3'(x_i) + \frac{h^2}{2}y_3''(x_i) \\ y_4(x_{i+1}) \simeq y_4(x_i) + hy_4'(x_i) + \frac{h^2}{2}y_4''(x_i) \\ y_5(x_{i+1}) \simeq y_5(x_i) + hy_5'(x_i) + \frac{h^2}{2}y_5''(x_i) \end{cases},$$

for $i = 0, 1, 2, 3, 4$, hence using Sage we obtain the following results

```
y1(1.00000000000000) =  -2.00000000000000
        y2(1.00000000000000) =  -1.00000000000000
        y3(1.00000000000000) =  1.00000000000000
y4(1.00000000000000) =  -2.00000000000000
        y5(1.00000000000000) =  -1.00000000000000
----------------------------------------
y1(1.10000000000000) =  -2.09500000000000
        y2(1.10000000000000) =  -0.971456643117795
        y3(1.10000000000000) =  -0.525837033645837
y4(1.10000000000000) =  -2.18229848847066
        y5(1.10000000000000) =  -2.89247561280789
----------------------------------------
y1(1.20000000000000) =  -2.19477484948001
        y2(1.20000000000000) =  -1.14691597429669
        y3(1.20000000000000) =  -2.54094212601509
y4(1.20000000000000) =  -2.58005814946148
        y5(1.20000000000000) =  -5.35433676347818
----------------------------------------
y1(1.30000000000000) =  -2.32217115753975
        y2(1.30000000000000) =  -1.64451900892177
        y3(1.30000000000000) =  -5.70545237672101
y4(1.30000000000000) =  -3.25647093080549
        y5(1.30000000000000) =  -8.56206920818365
----------------------------------------
y1(1.40000000000000) =  -2.51515032031554
        y2(1.40000000000000) =  -2.81276075182632
        y3(1.40000000000000) =  -11.8994744363292
y4(1.40000000000000) =  -4.29954484614262
        y5(1.40000000000000) =  -12.9029557410429
----------------------------------------
y1(1.50000000000000) =  -2.85592376767981
        y2(1.50000000000000) =  -5.76441779205738
        y3(1.50000000000000) =  -26.8319860156146
y4(1.50000000000000) =  -5.85359554976254
        y5(1.50000000000000) =  -19.3418058178558
```

and conclude that:

$$y(1.5) \simeq y_1(1.5) = -2.85592376767981$$
$$z(1.5) \simeq y_4(1.5) = -5.85359554976254.$$

If we let Sage do the work for $n = 10$, we obtain:

```
y1(1.00000000000000) =  -2.00000000000000
        y2(1.00000000000000) =  -1.00000000000000
        y3(1.00000000000000) =  1.00000000000000
y4(1.00000000000000) =  -2.00000000000000
        y5(1.00000000000000) =  -1.00000000000000
----------------------------------------
```

```
y1(1.05000000000000) =  -2.04875000000000
        y2(1.05000000000000) =  -0.967864160779449
        y3(1.05000000000000) =  0.261257525999567
y4(1.05000000000000) =  -2.07057462211767
        y5(1.05000000000000) =  -1.88461134555527
----------------------------------------
y1(1.10000000000000) =  -2.09681663613147
        y2(1.10000000000000) =  -0.978267504157610
        y3(1.10000000000000) =  -0.642728655180003
y4(1.10000000000000) =  -2.18856145227679
        y5(1.10000000000000) =  -2.90131753818579
----------------------------------------
y1(1.15000000000000) =  -2.14653342215833
        y2(1.15000000000000) =  -1.04153884698139
        y3(1.15000000000000) =  -1.77405379796278
y4(1.15000000000000) =  -2.36085003513012
        y5(1.15000000000000) =  -4.06365547688984
----------------------------------------
y1(1.20000000000000) =  -2.20082793175485
        y2(1.20000000000000) =  -1.17313822807305
        y3(1.20000000000000) =  -3.25487352984490
y4(1.20000000000000) =  -2.59513930725166
        y5(1.20000000000000) =  -5.39166893452345
----------------------------------------
y1(1.25000000000000) =  -2.26355343507081
        y2(1.25000000000000) =  -1.39814918446889
        y3(1.25000000000000) =  -5.30567061500437
y4(1.25000000000000) =  -2.90033549692836
        y5(1.25000000000000) =  -6.91548486496673
----------------------------------------
y1(1.30000000000000) =  -2.34009298256301
        y2(1.30000000000000) =  -1.75904456298052
        y3(1.30000000000000) =  -8.32037134494797
y4(1.30000000000000) =  -3.28717498934296
        y5(1.30000000000000) =  -8.68142295705742
----------------------------------------
y1(1.35000000000000) =  -2.43844567489322
        y2(1.35000000000000) =  -2.32997104630311
        y3(1.35000000000000) =  -13.0137467347423
y4(1.35000000000000) =  -3.76924724379470
        y5(1.35000000000000) =  -10.7633785553386
----------------------------------------
y1(1.40000000000000) =  -2.57121141062680
        y2(1.40000000000000) =  -3.24402219702593
        y3(1.40000000000000) =  -20.7122636643571
y4(1.40000000000000) =  -4.36479053870162
        y5(1.40000000000000) =  -13.2858589814962
----------------------------------------
y1(1.45000000000000) =  -2.75930285005855
        y2(1.45000000000000) =  -4.74689316089311
```

```
        y3(1.45000000000000) =  -33.9343436346622
y4(1.45000000000000) =  -5.10011212131771
        y5(1.45000000000000) =  -16.4749840780874
----------------------------------------
y1(1.50000000000000) =  -3.03906543764653
        y2(1.50000000000000) =  -7.30599859438698
        y3(1.50000000000000) =  -57.5730973206327
y4(1.50000000000000) =  -6.01676595541085
        y5(1.50000000000000) =  -20.7839438492649
```

concluding

$$y(1.5) \simeq y_1(1.5) = -3.03906543764653,$$
$$z(1.5) \simeq y_4(1.5) = -6.01676595541085.$$

For $n = 100$, we obtain

$$y(1.5) \simeq y_1(1.5) = -3.30881013044170,$$
$$z(1.5) \simeq y_4(1.5) = -6.17561463102781.$$

We leave the Sage implementation as a homework.

7.7.1 Sage Library

Next, we show the solution to the previous problem using Sage library function for Runge–Kutta Method of order 4.

Example 151. For the following system of higher order differential equations

$$\begin{cases} y'''(x) = 3\sin(y) + e^{y''} + z' - z^4 + e^x \\ z''(x) = z' - y^4 + \cos(x) \\ y(1) = -2, \\ y'(1) = -1, \\ y''(1) = 1 \\ z(1) = -2 \\ z'(1) = -1 \end{cases}$$

find the values of the solutions y, and z, at $x = 1.5$, using Runge–Kutta's Method of order 4, with $n = 5$ steps.

Solution. We need to find $y(1.5)$, and $z(1.5)$.

First, we turn the given problem into a system of first order differential equations, by defining the new variables

$$y_1 = y, y_2 = y', y_3 = y'', y_4 = z, y_5 = z'$$

obtaining

$$\begin{cases} y_1' = y_2 \\ y_2' = y_3 \\ y_3' = f\,(x, y_1, y_2, y_3, y_4, y_4) = 3\sin(y_1) + e^{y_3} + y_5 - y_4^4 + e^x \\ y_4' = y_5 \\ y_5' = y_5 - y_1^4 + \cos(x) \end{cases},$$

with the given initial conditions:

$$\begin{cases} y_1(1) = -2 \\ y_2(1) = -1 \\ y_3(1) = 1 \\ y_4(1) = -2 \\ y_5(1) = -1 \end{cases}.$$

Next, we use the following Sage code to solve this problem:

```
#include the library
from sage.calculus.desolvers import desolve_system_rk4

#declare the variables
var('y1 y2 y3 y4 y5 x')

#solve the system of equations using Runge Kuta 4
P=desolve_system_rk4(
         #specify the right hand side of each equation
         [y2,  y3,  3*sin(y1)+e^(y3)+y5-y4^4+e^x,  y5,  y5-y1^4+cos(x)],

         #specify the dependent variables
         [y1, y2,  y3, y4, y5],

         #specify the initial values for x, y1,y2,y3,y4,y5
         ics=[1, -2, -1, 1, -2, -1],

         #specify the independent variable
         ivar=x,

         #specify the value of h
         step = 0.1,

         #specify the value of b
         end_points=1.5)

#print all the values
#  use  [ [x,y1, y2, y3, y4, y5] for x,y1,y2,y3,y4,y5 in P]
#  or use:
P
```

and obtain

```
[[1, -2, -1, 1, -2, -1],
 [1.1,
  -2.09768745142,
  -0.986702389997,
  -0.917932774628,
  -2.19078813367,
  -2.90550714494],
 [1.2,
  -2.20579333122,
  -1.235557241,
  -4.48335702511,
  -2.60089031308,
  -5.41273066194],
 [1.3,
  -2.36209135572,
  -2.02523868598,
  -12.5863904403,
  -3.30117637398,
  -8.77701427303],
 [1.4,
  -2.65508219847,
  -4.21179591097,
  -35.5661928443,
  -4.40665753611,
  -13.7313684517],
 [1.5,
  -3.3436380293,
  -10.8505196966,
  -115.680182878,
  -6.18228660237,
  -23.213247068]]
```

Hence we conclude

$$y(1.5) \simeq y_1(1.5) = -3.3436380293,$$
$$z(1.5) \simeq y_4(1.5) = -6.18228660237.$$

If we use $n = 100$, hence using a `step = 0.005`, we would obtain

$$y(1.5) \simeq y_1(1.5) = -3.34816700595,$$
$$z(1.5) \simeq y_4(1.5) = -6.19246861798.$$

7.8 An Application to Biology

In [4, p. 353] the author gives the following tumor growth model, based on the work given in [5]:

$$\begin{cases} R'(t) = -\frac{S_i R}{3} + \frac{2\lambda\sigma}{\mu R + \sqrt{(\mu R)^2 + 4\sigma}} \\ R(0) = a \end{cases}, \tag{7.24}$$

where $R(t)$ is the radius of a tumor that is assumed to be spherical, S_i the death rate of the cells at the core of the tumor, σ is the nutrient level, and μ and λ are scale parameters.

Assuming that $S_i = 0.90$, $\sigma = 0.05$, $a = 0.25$, and that $\mu = 1$ and $\lambda = 1$, find the radius of the tumor $R(2)$.

Solution. Using the assumptions given, we need to solve for $R(2)$ from the following Initial Value Problem

$$\begin{cases} R'(t) = -0.3R + \frac{0.1}{R + \sqrt{R^2 + 0.2}} \\ R(0) = 0.25 \end{cases}.$$

For this, we can use Sage code similar to the one below

```
def PrintEuler(a,b,n,f, y0):
    #GIVEN EARLIER IN THIS CHAPTER!

#set up the problem
a = 0; b = 2; n = 100

#our R is the y,  and t is the x
f(x,y) = -0.3*y+0.1/(y + sqrt(y*y+0.2))
y0 = 0.25

#call the function for Euler's Method
PrintEuler(a,b,n,f, y0)
```

and obtain that at time $t = 2$, the tumor's radius is $R(2) = 0.319065833427810$.
Using similar code, one can obtain that $R(8) = 0.356036825570526$.

7.9 Exercises

1. Using Euler's Method, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y \\ y(0) = 1 \end{cases}$$

using $n = 5$ steps.

2. Using Euler's Method, compute an approximate value for $y(7)$ for the differential equation

$$\begin{cases} y' = -y \\ y(6) = 10 \end{cases}$$

using $n = 5$ steps.

3. Using Euler's Method, compute an approximate value for $y(10)$ for the differential equation

$$\begin{cases} y' = -y^2 + 4x - \cos(y) \\ y(6) = 10 \end{cases}$$

using $n = 5$ steps.
4. Find the formulae for the Taylor Series Method with $N = 4$.
5. Find the formulae for the Taylor Series Method with $N = 6$.
6. Using Runge–Kutta's Method of order 2, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y^2 + 4x - \cos(y) \\ y(6) = 10 \end{cases}$$

using $n = 5$ steps. Also give the error of approximation for each node.
7. Using Runge–Kutta's Method of order 4, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y^2 + 4x - \cos(y) \\ y(6) = 10 \end{cases}$$

using $n = 5$ steps. Also give the error of approximation for each node.
8. Using Runge–Kutta's Method of order 2, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y \\ y(0) = 1 \end{cases}$$

using $n = 5$ steps. Also give the error of approximation for each node.
9. Using Runge–Kutta's Method of order 4, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y \\ y(0) = 1 \end{cases}$$

using $n = 5$ steps. Also give the error of approximation for each node.
10. For the following higher order differential equation

$$\begin{cases} y'''(x) = \cos(2y) + \sin(y') - e^{y''} + x^5 \\ y(1) = 4, \\ y'(1) = 7, \\ y''(1) = -2 \end{cases}$$

find the value of the solution y at $x = 1.5$, using Taylor series Method of order $N = 2$, with $n = 5$ steps.
11. For the following higher order differential equation

$$\begin{cases} y'''(x) = \cos(2y) + 3y' - e^{y''} + x^2 \\ y(0) = 4, \\ y'(0) = 7, \\ y''(0) = -2 \end{cases}$$

find the value of the solution y at $x = 2$, using Taylor series Method of order $N = 3$, with $n = 4$ steps.

12. For the following system of higher order differential equations

$$\begin{cases} y'''(x) = y'' - z' + e^z - x \\ z'' = z - y - 4(z')^2 + (y')^3 + 6y'' + 3x \\ y(0) = -2, \\ y'(0) = 7, \\ y''(0) = 5 \\ z(0) = 2 \\ z'(0) = -4 \end{cases}$$

find the values of y, and z at $x = 1.5$, using Taylor series Method of order $N = 3$, with $n = 5$ steps.

7.10 Computer Exercises

1. Using Euler's method, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y \\ y(0) = 1 \end{cases}$$

using $n = 100$ steps. Then graph its true solution and the table of values obtained from Euler's Method.
2. Using Euler's method, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y \\ y(0) = 1 \end{cases}$$

using $n = 100$ steps. Output the table of values. In this table include a column computing the error of approximation for each node.
3. Using Euler's method, compute an approximate value for $y(10)$ for the differential equation

$$\begin{cases} y' = \arctan x \\ y(0) = 0 \end{cases}$$

using $n = 100$ steps. Then graph its true solution and the table of values obtained from Euler's Method.
4. Using Euler's method, compute an approximate value for $y(5)$ for the differential equation

$$\begin{cases} y' = \frac{2x}{1+x^2} - 2y^2 \\ y(0) = 0 \end{cases}$$

using $n = 100$ steps. Then graph its true solution and the table of values obtained from Euler's Method.

5. Using Euler's method, compute an approximate value for $y(4)$ for the differential equation

$$\begin{cases} y' = \frac{x^4}{y} \\ y(0) = 1 \end{cases}$$

using $n = 100$ steps. Then graph the table of values obtained from Euler's Method. Find an approximate value for $y(\pi)$.

6. Using Euler's method, compute an approximate value for $y(4)$ for the differential equation

$$\begin{cases} y' = \frac{2x}{1+e^{-\sqrt{x^2-\sin x}}} + 3\sin\left(\left|\sin x + \cosh x^5\right|\right) \\ y(0) = 1 \end{cases}$$

using $n = 100$ steps. Then graph the table of values obtained from Euler's Method.

7. Using Euler's method, compute an approximate value for $y(5)$ for the differential equation

$$\begin{cases} \cos(y') - e^{|y|} = \ln(1+x^2) + 7 \\ y(3) = 1 \end{cases}$$

using $n = 100$ steps. Then graph the table of values obtained from Euler's Method.

8. Write the Sage code implementation of the Taylor Series Method with $N = 4$. Then apply it to find $y(2)$ from the following differential equation

$$\begin{cases} y' = \frac{2x}{1+x^2} - 2y^2 \\ y(0) = 0 \end{cases}$$

with $n = 10$.

9. Write the Sage code implementation of the Taylor Series Method with $N = 5$. Then apply it to find $y(2)$ from the following differential equation

$$\begin{cases} y' = \frac{2x}{1+x^2} - 2y^2 \\ y(0) = 0 \end{cases}$$

with $n = 50$.

10. Implement the Sage code for Runge–Kutta's Method of order 2 given by (7.19), on page 285.

11. Using Runge–Kutta's Method of order 2, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y^2 + 4x - \cos(y) \\ y(6) = 10 \end{cases}$$

using $n = 100$ steps. Also give the error of approximation for each node. Then plot the obtained values.

12. Using Runge–Kutta's Method of order 4, compute an approximate value for $y(1)$ for the differential equation

$$\begin{cases} y' = -y^2 + 4x - \cos(y) \\ y(6) = 10 \end{cases}$$

using $n = 100$ steps. Also give the error of approximation for each node. Then plot the obtained values.

13. Implement the Sage solution for the problem 148, given on page 294.
14. Implement Sage code for the following higher order differential equation

$$\begin{cases} y'''(x) = \cos(2y) + \sin(y') - e^{y''} + x^5 \\ y(1) = 4, \\ y'(1) = 7, \\ y''(1) = -2 \end{cases}$$

to find the value of the solution y at $x = 1.5$, using Taylor series Method of order $N = 2$, with $n = 50$ steps.

15. Implement Sage code for the following higher order differential equation

$$\begin{cases} y'''(x) = \cos(2y) + 3y' - e^{y''} + x^2 \\ y(0) = 4, \\ y'(0) = 7, \\ y''(0) = -2 \end{cases}$$

to find the value of the solution y at $x = 2$, using Taylor series Method of order $N = 3$, with $n = 50$ steps.

16. Implement the Sage solution for the problem 150, given on page 300.
17. Implement Sage code for the following system of higher order differential equations

$$\begin{cases} y'''(x) = y'' - z' + e^z - x \\ z'' = z - y - 4(z')^2 + (y')^3 + 6y'' + 3x \\ y(0) = -2, \\ y'(0) = 7, \\ y''(0) = 5 \\ z(0) = 2 \\ z'(0) = -4 \end{cases}$$

to find the values of y, and z at $x = 1.5$, using Taylor series Method of order $N = 3$, with $n = 50$ steps.

References

1. Atkinson, K., Han, W.: Elementary Numerical Analysis, 3rd edn. Wiley, New York (2003)
2. Blanco-Silva, F.J.: Learning SciPy for Numerical and Scientific Computing. Packt Publishing, Birmingham/Mumbai (2013)
3. Cheney, W., Kincaid, D.: Numerical Mathematics and Computing, 7th edn. Brooks/Cole: Cengage Learning, Boston (2013)
4. Epperson, J.F.: An Introduction to Numerical Methods and Analysis, revised edition. Wiley, Hoboken (2007)
5. Greenspan, H.P.: On the growth and stability of cell cultures and solid tumors. J. Theor. Biol. **56**(1), 229–242 (1976)
6. Rivlin, T.J.: An Introduction to the Approximation of Functions. Dover, New York (2003)
7. Sage Cell Server: http://sagecell.sagemath.org/. Retrieved 2014
8. Stein, W.A., et al.: Sage mathematics software (Version 6.3). The sage development team. Sage Official Webpage: http://www.sagemath.org (2014)
9. Stein, W.A.: Sage for power users. http://www.wstein.org/books/sagebook/sagebook.pdf (2012)
10. Sullivan, M., Miranda, K.: Calculus: Early Transcendentals. W. H. Freeman and Company, New York (2014)

Erratum

Numerical Analysis Using Sage

George A. Anastassiou and Razvan A. Mezei

© Springer International Publishing Switzerland 2015
G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8

DOI 10.1007/978-3-319-16739-8_8

Unfortunately, there was an error on the Copyright page with an extra blurb line stating that the book can be downloaded from http://extras.springer.com. Electronic Supplementary Materials can be found on SpringerLink instead.

Which now has been removed.

The online version of the original book can be found at http://dx.doi.org/10.1007/978-3-319-16739-8

© Springer International Publishing Switzerland 2016

G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8_8

Index

A
Absolute Error Formula, 20
Accuracy, 21
Accurate to n Decimal Places, 21
Accurate to n Significant Digits, 21
Alternating Series Theorem, 56
Antiderivative, 175

B
Bankers' Rounding, 24
Basic Simpson's Rule, 188
Basic Trapezoidal Rule, 178
Big O Notation, 54
Bisection Method, 65
Body of a Function, 24

C
Central Difference Formula, 164
Chebyshev Nodes, 149
Chopped to n Digits, 23
Complete Cubic Splines, 241
Composite Simpson's Rule, 190
Composite Trapezoidal Rule, 180
Composite Trapezoidal Rule with Evenly Spaced Nodes, 181
Cubic Equation, 65
Cubic Spline Function, 240

D
Definite Integral, 176
Derivative of a Function, 161
Differential Equation, 263
Directional Field, 265
Divided Differences, 134

E
Error Formula, 20
Euler's Method, 267
Euler-MacLaurin Formula, 203
Evenly Spaced Nodes, 209

F
Fixed Point, 101
Fixed-Point Iteration, 101
Fundamental Theorem of Approximation Theorem, 37
Fundamental Theorem of Calculus, 175

G
Gaussian Quadrature Formula, 210

H
Header of the Function, 24
Heun's Method, 285
Higher-Order Differential Equations, 294

I
Initial-Value Problems, 265
Interacts, 44
Intermediate Value Theorem, 66
Interpolation, 117
Interpolation Polynomial of Degree n, 123

L
Lagrange Basis Polynomials, 123
Lagrange Formula, 123
Legendre Polynomials, 214
Linear Equation, 65
Linear Interpolation, 118
Linear Spline Function, 230
Loop, 26

M
Maclaurin Series, 38
Mean-Value Theorem, 53

© Springer International Publishing Switzerland 2015

G.A. Anastassiou, R. Mezei, *Numerical Analysis Using Sage*, Springer Undergraduate Texts in Mathematics and Technology, DOI 10.1007/978-3-319-16739-8

Method of Undetermined Coefficients, 167
Midpoint Method, 285

N
Natural Cubic Splines, 241
Natural Quadratic Spline, 235
Newton Algorithm, 129
Newton Formula, 129
Newton Interpolation Formula, 129
Newton Method, 76
Newton-Cotes Formula, 209
Newton-Raphson's Method, 76
Newton's Method Theorem, 82
Node, 117
Numerical Derivative, 161

O
Ordinary Differential Equation, 263

P
Partially Differential Equation, 263
Polygonal Line, 230
Polynomial, 37
Polynomial Interpolation, 117
Polynomial of Degree at most n, 118
Polynomial of Degree n, 118
Precision, 21
Print Method, 25
Propagated Error, 277

Q
Quadratic Equation, 65
Quadratic Function, 120
Quadratic Interpolation, 120
quadratic spline function, 234
Quadrature, 175

R
Range Method, 25
Relative Error Formula, 20
Richardson Extrapolation, 201
Romberg Integration Method, 205
Root of an Equation, 65
Rounded to n Digits, 23
Round-to-Even , 24
Round-to-Odd, 24
Runge Function, 144
Runge-Kutta's Method, 284
Runge's Phenomenon, 149

S
Sage Math, 2
Sage Interacts, 44
Secant Method, 90
Solution of an Equation, 65

T
Tab Space, 39
Taylor Polynomial, 37
Taylor Series, 37
Taylor Series Method, 279
Taylor's Theorem with Remainder, 48
Truncation Error, 277

U
Unbiased Rounding, 24

W
Weierstrass Approximation Theorem, 37

Z
Zero of an Equation, 65